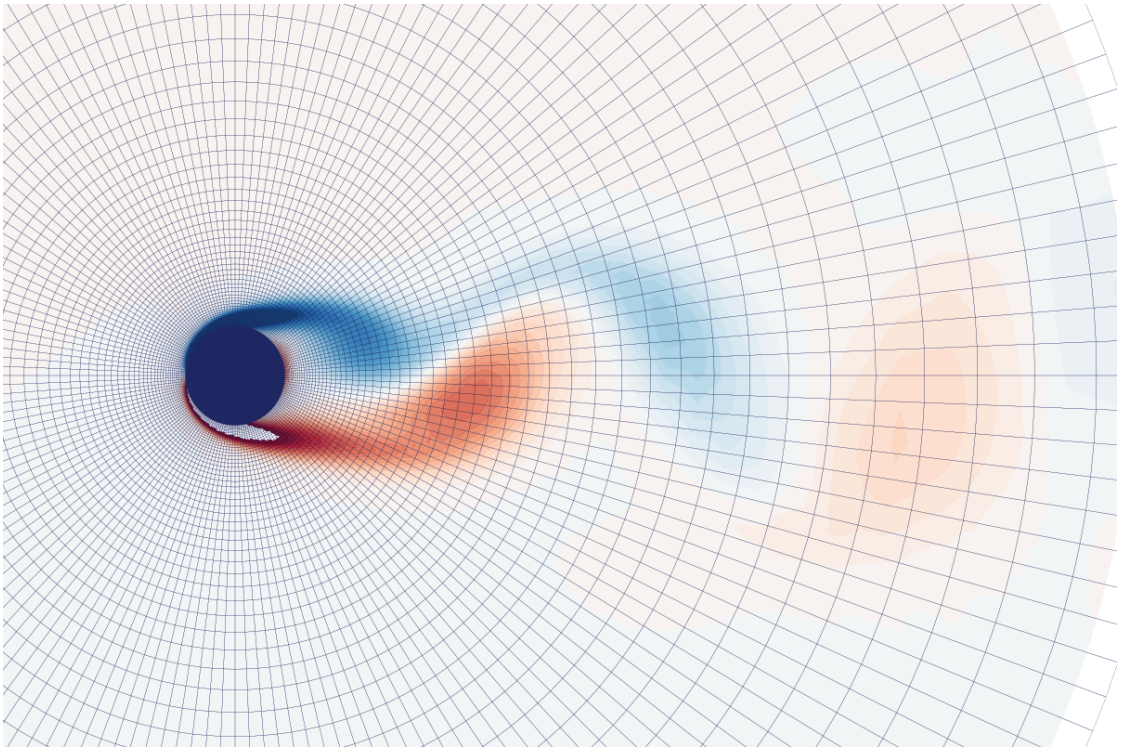
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

Il metodo dei volumi finiti e OpenFOAM

Dai primi passi all’uso consapevole: discretizzazione, algoritmi, turbolenza e applicazioni aeronautiche



Monografia didattica di fluidodinamica numerica
Modulo 5: la pratica con un codice professionale

Prof. Ing. Biagio Raucci

Casi eseguiti con OpenFOAM 12; strumenti di supporto in Python

Indice

Presentazione **ix**

I Fondamenti **1**

1 Primi passi: installare, eseguire, leggere **2**
1.1 Che cos'è OpenFOAM . . . 2
1.2 Installare . . . 2
1.3 La struttura di un caso . . . 3
1.4 Il primo caso: la cavità con coperchio mobile . . . 4
1.4.1 I file del caso . . . 4
1.4.2 Eseguire . . . 6
1.4.3 Leggere il log . . . 7
1.4.4 I risultati . . . 8
1.5 ParaView e i dati in Python . . . 8
1.6 Approfondimenti . . . 9
1.6.1 Il tempo e le cartelle . . . 9
1.6.2 Quando il caso non parte . . . 10
1.6.3 Un tutorial al giorno . . . 10

2 Le equazioni di conservazione e il metodo dei volumi finiti **11**
2.1 Perché i volumi finiti . . . 11
2.2 L'equazione generale di trasporto . . . 11
2.3 Il processo di discretizzazione . . . 12
2.4 Le proprietà di una buona discretizzazione . . . 14
2.5 Come OpenFOAM scrive le equazioni . . . 14
2.6 Lettura guidata: `icoFoam.C` . . . 15
2.7 Approfondimenti . . . 17
2.7.1 Volumi finiti centrati nelle celle e nei vertici . . . 17
2.7.2 Che cosa significa "secondo ordine" . . . 17
2.7.3 La forma non conservativa e i suoi rischi . . . 18

3 La mesh ai volumi finiti **19**

3.1 Dalla geometria alle celle . . . 19
3.2 La rappresentazione `polyMesh` . . . 19
3.3 Indici di qualità e `checkMesh` . . . 21
3.4 `blockMesh`: mesh strutturate a blocchi . . . 22
3.5 Tre mesh con `blockMesh` . . . 23
3.6 `snappyHexMesh`: geometrie arbitrarie . . . 28
3.7 Approfondimenti . . . 28
3.7.1 L'indirizzamento *ldu* e la matrice . . . 28
3.7.2 Altri generatori e la conversione . . . 29
3.7.3 Mesh dinamiche e raffinamento adattivo . . . 29

II Discretizzazione: `fvSchemes` e `fvSolution` 30

4 Gradienti e termine diffusivo 31
4.1 Il termine diffusivo su una mesh ortogonale . . . 31
4.2 Il calcolo del gradiente . . . 32
4.3 Mesh non ortogonali: la correzione . . . 32
4.4 Esperimento: conduzione su mesh ortogonali e distorte . . . 33
4.5 Approfondimenti . . . 39
4.5.1 Diffusività discontinua e interpolazione armonica . . . 39
4.5.2 Diffusione anisotropa . . . 40
4.5.3 Il gradiente alle pareti e lo sforzo . . . 40

5 Il termine convettivo e gli schemi ad alta risoluzione 41
5.1 Il problema . . . 41
5.2 Schemi di ordine superiore . . . 42
5.3 Limitatezza: NVF, CBC e TVD . . . 42
5.4 La sintassi di `divSchemes` . . . 44
5.5 Esperimento: il gradino obliquo . . . 45
5.6 Approfondimenti . . . 48
5.6.1 Il limitatore sulle mesh non strutturate e il gradiente . . . 48
5.6.2 I limitatori nei calcoli stazionari: la convergenza che si ferma . . . 48
5.6.3 Trasporto e dispersione: l'equazione modificata . . . 48

6 Tempo, sorgenti, sottorilassamento e residui 49
6.1 Il termine instazionario . . . 49
6.2 Esperimento: l'ordine nel tempo . . . 50
6.3 Termini sorgente: `fvModels` e `fvConstraints` . . . 51
6.4 Sottorilassamento . . . 52
6.5 I residui: che cosa misura OpenFOAM . . . 52
6.6 Approfondimenti . . . 53

6.6.1 Passi temporali locali per lo stazionario . . . 53
6.6.2 Condizioni iniziali e il primo passo di `backward` . . . 53
6.6.3 Il transitorio come strumento di convergenza . . . 53

7 La soluzione dei sistemi lineari 55
7.1 Il problema . . . 55
7.2 I metodi iterativi di OpenFOAM . . . 55
7.3 Esperimento: i solutori per la pressione . . . 57
7.4 Approfondimenti . . . 59
7.4.1 Perché la pressione costa più di tutto . . . 59
7.4.2 Il parallelo e i solutori . . . 59
7.4.3 Solutori accoppiati e GPU . . . 59

III Algoritmi e modelli 60

8 L'accoppiamento fra pressione e velocità 61
8.1 La difficoltà . . . 61
8.2 L'equazione della pressione su griglia collocata . . . 62
8.3 SIMPLE, SIMPLEC, PISO, PIMPLE . . . 62
8.4 Lettura guidata: `correctPressure.C` . . . 64
8.5 Esperimenti sulla cavità a $Re = 100$ . . . 65
8.6 Approfondimenti . . . 68
8.6.1 Le condizioni al contorno della pressione . . . 68
8.6.2 Il termine `ddtCorr` e la coerenza temporale . . . 68
8.6.3 Comprimibile e multifase: la stessa idea . . . 69

9 La turbolenza: modelli RANS e trattamento di parete 70
9.1 Dalle equazioni istantanee a quelle mediate . . . 70
9.2 La parete: legge universale e funzioni di parete . . . 71
9.3 Esperimento 1: la lastra piana . . . 73
9.4 Esperimento 2: il gradino a valle . . . 74
9.5 Approfondimenti . . . 75
9.5.1 Transizione e modelli a basso Reynolds . . . 75
9.5.2 LES e DES . . . 76
9.5.3 Sulla scelta del modello . . . 76

10 Flussi comprimibili 77
10.1 Che cosa cambia . . . 77
10.2 Esperimento 1: il tubo d'urto . . . 78
10.3 Esperimento 2: l'urto obliquo . . . 81
10.4 Approfondimenti . . . 82

10.4.1 L'equazione della pressione comprimibile . . . 82
10.4.2 Il problema di Riemann e i solutori approssimati . . . 82
10.4.3 Mach basso e precondizionamento . . . 82

11 Le condizioni al contorno 84
11.1 Come funziona una condizione al contorno . . . 84
11.2 Il catalogo ragionato . . . 85
11.3 Esperimento: il condotto di Poiseuille con profilo codificato . . . 86
11.4 Approfondimenti . . . 88
11.4.1 Condizioni al contorno e ordine di accuratezza . . . 88
11.4.2 Patch, gruppi e espressioni regolari . . . 88
11.4.3 Le condizioni al contorno nel calcolo parallelo e nelle mesh mobili . . . 89

12 Dentro il codice: leggere e scrivere OpenFOAM 90
12.1 Perché aprire i sorgenti . . . 90
12.2 Le classi che si incontrano sempre . . . 91
12.3 Leggere un solutore: `trasportoFoam` . . . 93
12.4 Scrivere una condizione al contorno . . . 96
12.5 Esperimento: i due programmi alla prova . . . 99
12.6 Approfondimenti . . . 102
12.6.1 Eseguire con il debugger e gli interruttori di debug . . . 102
12.6.2 Le differenze fra le versioni . . . 102
12.6.3 Una *function object* propria . . . 102

IV Applicazioni aeronautiche 103

13 Il profilo NACA 0012: polare e convergenza di griglia 104
13.1 Il caso di riferimento dell'aerodinamica numerica . . . 104
13.2 Impostazione del caso . . . 105
13.3 Risultati . . . 110
13.4 Che cosa insegna questo caso . . . 112
13.5 Approfondimenti . . . 114
13.5.1 La correzione di vortice per il contorno esterno . . . 114
13.5.2 Transizione e stallo . . . 114

14 Un transitorio: la scia di von Kármán 115
14.1 Perché un calcolo transitorio . . . 115
14.2 Esperimento: cilindro a $Re = 100$ . . . 116
14.3 Approfondimenti . . . 119
14.3.1 Da Re 100 a Re 3900: quando il 2D non basta . . . 119
14.3.2 Flussi instazionari su profili . . . 119

15 Un caso tridimensionale: la semiala con `snappyHexMesh` in parallelo 120
15.1 Dal 2D al 3D: che cosa cambia davvero . . . 120
15.2 Esperimento: la semiala NACA 0012 . . . 122
15.3 Approfondimenti . . . 127
15.3.1 Mesh ibride e `snappyHexMesh` su geometrie complesse . . . 127
15.3.2 Verso i calcoli industriali . . . 127

16 Verifica, validazione e buone pratiche: dal problema al caso 128
16.1 Tre domande distinte . . . 128
16.2 Il catalogo delle *function object* . . . 129
16.3 Dal problema al caso: una lista di controllo . . . 131
16.4 Approfondimenti . . . 132
16.4.1 Incertezza numerica e incertezza del modello . . . 132
16.4.2 Automazione e analisi parametrica . . . 132

A Tabelle di riferimento, strumenti e istruzioni 133
A.1 I dizionari in una pagina . . . 133
A.2 Le due linee di sviluppo: corrispondenze . . . 133
A.3 Glossario delle utility . . . 134
A.4 Gli strumenti Python . . . 135
A.5 Le letture dai sorgenti . . . 151
A.6 Come riprodurre i calcoli . . . 152

B Esercizi svolti 156
B.1 Esercizio 1 – La cavità a numero di Reynolds diverso . . . 156
B.2 Esercizio 2 – La gradazione da una prima cella assegnata . . . 157
B.3 Esercizio 3 – Non ortogonalità e skewness di una faccia . . . 157
B.4 Esercizio 4 – Il flusso diffusivo con la correzione non ortogonale . . . 158
B.5 Esercizio 5 – Péclet di cella e scelta dello schema . . . 158
B.6 Esercizio 6 – Il passo temporale per la scia del cilindro . . . 159
B.7 Esercizio 7 – Il rilassamento implicito su un'equazione . . . 160
B.8 Esercizio 8 – Il residuo normalizzato di OpenFOAM . . . 160
B.9 Esercizio 9 – Scegliere il solutore lineare . . . 161
B.10 Esercizio 10 – Un passo di SIMPLE su tre celle . . . 162
B.11 Esercizio 11 – La prima cella per un dato y^+ . . . 162
B.12 Esercizio 12 – I valori turbolenti d'ingresso . . . 163
B.13 Esercizio 13 – L'urto obliquo a Mach 3 . . . 164
B.14 Esercizio 14 – Un ingresso pulsante con `codedFixedValue` . . . 164
B.15 Esercizio 15 – Uno studio parametrico con `foamDictionary` . . . 165
B.16 Esercizio 16 – `forceCoeffs` con l'incidenza . . . 166
B.17 Esercizio 17 – L'indice di convergenza di griglia . . . 167

B.18 Esercizio 18 – Frequenza da un segnale 167
B.19 Esercizio 19 – Quanti processori 168
B.20 Esercizio 20 – Dal problema al caso 168

Bibliografia **170**

Elenco dei listati

1.1 c01_cavita: 0/U 4
1.2 c01_cavita: 0/p 5
1.3 c01_cavita: constant/physicalProperties 5
1.4 c01_cavita: system/controlDict 5
1.5 c01_cavita: Allrun 7
2.1 Lettura guidata – icoFoam.C 15
3.1 c02_canale: blockMeshDict 23
3.2 c02_cilindro: blockMeshDict 25
3.3 c02_tubo: blockMeshDict 26
4.1 c04_diffusione: system/fvSchemes 34
4.2 c04_diffusione/genera.py 34
5.1 c05_convezione: system/controlDict 45
5.2 c05_convezione: 0/s 46
8.1 Lettura guidata – correctPressure.C 64
8.2 c08_cavita_re100: system/fvSolution 66
9.1 c09_lastra: 0/nut 73
10.1 c11_tubo_urto: constant/physicalProperties 79
10.2 c11_tubo_urto: system/fvSchemes 79
11.1 c13_poiseuille: 0/U 87
12.1 c14_codice: trasportoFoam/trasportoFoam.C 93
12.2 c14_codice: trasportoFoam/createFields.H 94
12.3 c14_codice: la classe profiloParabolico, intestazione 96
12.4 c14_codice: la classe profiloParabolico, costruttore 97
12.5 c14_codice: la classe profiloParabolico, updateCoeffs 98
12.6 c14_codice: Allrun 100
12.7 c14_codice: casoProfilo/0/U 100
13.1 c15_naca0012: 0/U 105
13.2 c15_naca0012: system/forzeDict 105
13.3 c15_naca0012/genera.py 106
14.1 c16_cilindro: system/controlDict 116
14.2 c16_cilindro: system/fvSolution 117
15.1 c17_ala: Allrun 122

15.2 `c17_ala`: `system/snappyHexMeshDict` (1) 122
15.3 `c17_ala`: `system/snappyHexMeshDict` (2) 124
15.4 `wingstl.py` 124
A.1 `foamcase.py` 136
A.2 `foamlog.py` 144
A.3 `nacamesh.py` 147
A.4 `foamgci.py` 150

Presentazione

Di che cosa parla questa monografia

Le prime quattro monografie del corso hanno costruito, programma dopo programma, i metodi della fluidodinamica numerica: differenze finite, schemi espliciti e impliciti, metodi iterativi per l'equazione di Poisson, l'accoppiamento fra pressione e velocità su griglia sfalsata, la formulazione in vorticità e funzione di corrente. Ogni programma era scritto da noi, in poche decine di righe di Python, e proprio per questo era trasparente: si vedeva ogni coefficiente. Ma un'ala, un'automobile, una turbina non si calcolano con un programma di cento righe su una griglia cartesiana: servono mesh di milioni di celle di forma arbitraria, modelli di turbolenza, calcolo parallelo, e anni di lavoro di validazione. Servono, cioè, un *codice* e la capacità di usarlo *consapevolmente*, sapendo che cosa fa sotto ogni parola chiave.

Questa monografia insegna **OpenFOAM**, il codice di fluidodinamica numerica libero e open source più diffuso al mondo, con un'impostazione precisa: non un manuale di comandi, ma un percorso in cui ogni scelta che l'utente compie nei dizionari del caso (`fvSchemes`, `fvSolution`, le condizioni al contorno, il modello di turbolenza) è ricondotta alla teoria del *metodo dei volumi finiti* che la giustifica e verificata con un esperimento numerico. Il testo che ne detta l'ossatura è *The Finite Volume Method in Computational Fluid Dynamics* di Moukalled, Mangani e Darwish [1], che è a sua volta costruito in questo modo: fondamenti, discretizzazione termine per termine, algoritmi, applicazioni, con il codice sempre accanto alla formula. Dove quel testo mostra il codice C++ di OpenFOAM e il suo gemello didattico in MATLAB (uFVM), questa monografia mostra i dizionari dei casi, i risultati dei calcoli eseguiti, e piccoli strumenti in Python che leggono la mesh e i campi *esattamente* come fa il codice, per rendere visibile ciò che altrimenti resterebbe chiuso nei file binari di un post-processore. Python, che il Modulo 0 ha insegnato, è qui l'assistente; la protagonista è OpenFOAM.

Il percorso parte da zero (installare, eseguire il primo caso, leggere il log) e arriva a ciò che serve per dirsi smaliziati: scegliere e giustificare gli schemi, giudicare una mesh, impostare un calcolo turbolento con la risoluzione di parete adeguata, trattare un flusso comprimibile, scrivere una condizione al contorno e un solutore propri, calcolare la polare di un profilo e confrontarla con l'esperimento, stimare l'incertezza numerica di un risultato. Tutti i casi sono stati eseguiti con OpenFOAM 12 (la versione della OpenFOAM Foundation, `https://openfoam.org`) durante la stesura, e i numeri riportati sono quelli ottenuti; i casi completi, pronti da eseguire, sono scaricabili dai QR nel margine.

Come è organizzata

Parte I – Fondamenti.

Cap. 1: che cos'è OpenFOAM, come si installa, la struttura di un caso, la cavità con

coperchio mobile dal primo comando alla figura, la lettura del log. Cap. 2: le equazioni di conservazione in forma integrale, l'equazione semi-discretizzata, le proprietà di una buona discretizzazione, la corrispondenza fra ogni termine e gli operatori `fvm`/`fvc`; lettura di `icoFoam.C`. Cap. 3: la mesh poliedrica (`polyMesh`), le grandezze geometriche e gli indici di qualità, `blockMesh` nei dettagli, `snappyHexMesh`, `checkMesh`; un lettore della mesh in Python.

Parte II – Discretizzazione.
Un capitolo per ogni famiglia di voci di `fvSchemes` e `fvSolution`, con un esperimento numerico che misura l'effetto di ogni scelta: gradienti e termine diffusivo con la correzione non ortogonale (Cap. 4); termine convettivo, schemi upwind, centrati, di ordine superiore e TVD (Cap. 5); termine instazionario, sorgenti, sottorilassamento e residui (Cap. 6); solutori lineari e precondizionatori (Cap. 7).

Parte III – Algoritmi e modelli.
L'accoppiamento pressione–velocità su griglia collocata, SIMPLE, PISO e PIMPLE, l'interpolazione di Rhie–Chow, con la cavità confrontata con i dati di Ghia (Cap. 8); la turbolenza RANS, i modelli k–ε, k–ω SST e Spalart–Allmaras, le funzioni di parete e lo y^+ (Cap. 9); i flussi comprimibili con il tubo d'urto e l'urto obliquo (Cap. 10); il catalogo ragionato delle condizioni al contorno e la scrittura di una condizione propria (Cap. 11); dentro il codice: come si scrive, si compila e si usa un solutore (Cap. 12).

Parte IV – Applicazioni aeronautiche.
Il profilo NACA 0012 con la mesh a C generata da Python, la polare e il C_p confrontati con l'esperimento, lo studio di convergenza di griglia (Cap. 13); il cilindro e la scia di von Kármán, numero di Strouhal e coefficienti di forza nel tempo (Cap. 14); un caso tridimensionale con `snappyHexMesh` e il calcolo parallelo (Cap. 15); verifica, validazione, incertezza numerica e buone pratiche: dal problema al caso (Cap. 16).

Appendici.
Le tabelle di riferimento dei dizionari (parole chiave di `controlDict`, `fvSchemes`, `fvSolution`, condizioni al contorno), il glossario delle utility, i listati degli strumenti Python e le istruzioni per riprodurre tutto; i venti esercizi svolti.

I riquadri colorati hanno il significato consueto del corso: **teoria** (viola), **fasi** (blu: procedure passo per passo), **ipotesi e convenzioni** (giallo), **dati** (grigio: i dati dei casi), **risultati** (verde: esiti dei calcoli e loro verifica), **esempi** (arancio: letture guidate e applicazioni), **avvisi** (rosso: errori frequenti). I listati dei dizionari di OpenFOAM hanno la barra laterale blu; i comandi da terminale la barra arancione; l'uscita dei programmi è su fondo grigio.

I casi e gli strumenti

Ogni capitolo ha uno o più **casi** OpenFOAM, cartelle complete con gli script `Allrun` e `Allclean`, numerati `c01_cavita`, `c02_canale`, `...`; i QR nel margine accanto ai listati portano al file corrispondente in rete, e il QR accanto ai dati del caso all'archivio dell'intera cartella. Gli **strumenti Python** (`foamcase.py`, che legge mesh e campi; `foamlog.py`, che legge il log; `nacamesh.py`, che genera la mesh a C; `foamgci.py`, che calcola l'indice di convergenza di griglia; e gli script che producono le figure) stanno nella cartella `codice/` e usano soltanto NumPy, SciPy e Matplotlib. Le figure della monografia sono tutte prodotte da questi script a partire dai risultati dei casi, comprese quelle della semiala tridimensionale:

Python legge anche la mesh poliedrica di `snappyHexMesh`. Tutto è disponibile all'indirizzo `https://braucci.github.io/CFD/monografia5/`.

Versione di OpenFOAM e convenzioni

I casi usano la sintassi di **OpenFOAM 12** (OpenFOAM Foundation, luglio 2024): applicazione unica `foamRun` con il *modulo* solutore indicato in `controlDict` (`solver incompressibleFluid`, `fluid`, `shockFluid`), file `constant/physicalProperties` e `constant/momentumTransport`. I nomi storici dei solutori (`simpleFoam`, `pimpleFoam`, `rhoCentralFoam`, ...) restano disponibili come comandi che rimandano al modulo corrispondente, e sono indicati nel testo perché il lettore li ritroverà nella letteratura e nell'altra linea di sviluppo del codice (OpenFOAM ESI, `https://openfoam.com`, versioni v2xxx), le cui differenze di sintassi sono raccolte nell'Appendice A.2. I comandi da terminale sono dati per Linux (Ubuntu) e valgono identici in WSL su Windows e nel contenitore Docker su macOS.

Parte I

Fondamenti

Capitolo 1

Primi passi: installare, eseguire, leggere

1.1 Che cos'è OpenFOAM

OpenFOAM (*Open Field Operation and Manipulation*) è una libreria C++ per la soluzione di equazioni alle derivate parziali con il metodo dei volumi finiti su mesh poliedriche, distribuita con licenza GPL insieme a un centinaio di programmi (*applicazioni*) che la usano: solutori per flussi incomprimibili, comprimibili, multifase, reattivi, con trasferimento di calore, e utility per generare e manipolare mesh, impostare i campi, elaborare i risultati. Nasce all'Imperial College negli anni novanta (Jasak, Weller) e dal 2004 è open source; oggi ne esistono due linee di sviluppo, quella della OpenFOAM Foundation (`https://openfoam.org`, versioni numerate 10, 11, 12, 13) e quella di ESI–OpenCFD (`https://openfoam.com`, versioni v2306, v2312, v2406, ...), con la stessa architettura e piccole differenze di sintassi. Questa monografia usa la prima; l'Appendice A.2 elenca le differenze.

Tre caratteristiche lo distinguono da un codice commerciale e giustificano lo sforzo di impararlo. Non ha un'interfaccia grafica: un caso è una cartella di file di testo, e si lavora dal terminale; è più scomodo il primo giorno e molto più potente dal secondo, perché tutto è riproducibile, confrontabile e automatizzabile. Il codice sorgente è disponibile e leggibile: quando un manuale non dice che cosa fa una parola chiave, lo dice il codice, e la sintassi con cui le equazioni sono scritte (Cap. 2) è vicina alla matematica. È estensibile: una condizione al contorno o un solutore nuovi sono poche decine di righe compilate in un minuto (Capp. 11 e 12).

1.2 Installare

Installazione di OpenFOAM 12

Ubuntu (20.04–24.04) e derivate.
Dal repository della Foundation:

```
sudo sh -c "wget -O - https://dl.openfoam.org/gpg.key >
    /etc/apt/trusted.gpg.d/openfoam.asc"
sudo add-apt-repository http://dl.openfoam.org/ubuntu
sudo apt update
sudo apt -y install openfoam12
```

Il pacchetto installa in `/opt/openfoam12` anche ParaView. Per rendere disponibili i comandi in ogni terminale si aggiunge alla fine di `~/.bashrc` la riga `source`

`/opt/openfoam12/etc/bashrc`.

Windows.
Si installa WSL (*Windows Subsystem for Linux*) con Ubuntu dal Microsoft Store, e dentro WSL si procede come sopra; la cartella di Windows è visibile da Linux in `/mnt/c/`. ParaView si installa in Windows e apre i casi della cartella condivisa.

macOS.
La Foundation distribuisce lo script `openfoam12-macos`, che avvia un contenitore Docker con OpenFOAM e condivide la cartella corrente; serve Docker Desktop. In alternativa, una macchina virtuale Ubuntu.

Dai sorgenti.
Su qualunque Linux: `git clone https://github.com/OpenFOAM/OpenFOAM-12`, `source OpenFOAM-12/etc/bashrc`, `./Allwmake -j`; richiede il compilatore, `flex`, OpenMPI e alcune ore. È la strada seguita per i casi di questa monografia, e la sola che consenta di modificare il codice.

Per verificare: `foamRun -help` stampa le opzioni; `echo $FOAM_TUTORIALS` indica dove sono i tutorial della distribuzione.

1.3 La struttura di un caso

Le tre cartelle

Un caso OpenFOAM è una cartella con tre sotto-cartelle obbligatorie:

`0/` le condizioni iniziali e al contorno, un file per campo (`U`, `p`, `T`, `k`, `nut`, ...). Ogni file ha le `dimensions`, l'`internalField` (il valore iniziale nelle celle) e il `boundaryField`, con una voce per ogni patch della mesh: il `type` della condizione (`fixedValue`, `zeroGradient`, `noSlip`, `empty`, ...) e i suoi parametri. La cartella si chiama `0` perché è l'istante iniziale: il solutore scriverà i risultati in cartelle `0.1`, `0.2`, ... (o `100`, `200` per le iterazioni di un calcolo stazionario).

`constant/`
ciò che non cambia nel tempo: la mesh in `polyMesh/` (Cap. 3) e le proprietà fisiche: `physicalProperties` (viscosità, modello di fluido), `momentumTransport` (laminare o turbolento e con quale modello), ed eventuali `g`, `dynamicMeshDict`, `fvModels`.

`system/`
il controllo del calcolo: `controlDict` (solutore, istante iniziale e finale, passo temporale, frequenza di scrittura, *function objects*), `fvSchemes` (gli schemi di discretizzazione di ogni termine: Parte II), `fvSolution` (solutori lineari, tolleranze, algoritmo di accoppiamento, sottorilassamento), `blockMeshDict` e gli altri dizionari delle utility.

Tutti i file sono *dizionari* con la stessa sintassi: coppie `parola valore;`, sotto-dizionari fra graffe, liste fra tonde, commenti `//` e `/* */`, una intestazione `FoamFile` che dichiara il tipo di contenuto. Le parole chiave sono sensibili alle maiuscole; un punto e virgola

mancante è l'errore più frequente del principiante e il messaggio d'errore indica file e riga.

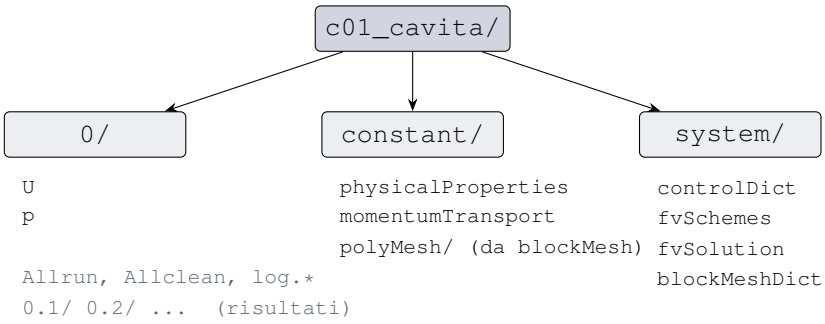


Figura 1.1: La struttura del caso `c01_cavita`. In grigio ciò che compare dopo l'esecuzione.

1.4 Il primo caso: la cavità con coperchio mobile

Il problema è quello del Modulo 4: una cavità quadrata di lato $L = 0{,}1$ m piena di fluido con $\nu = 0{,}01$ m^2/s, il coperchio che si muove a $U = 1$ m/s, numero di Reynolds $UL/\nu = 10$; flusso piano, laminare, instazionario a partire dal fluido in quiete, integrato fino a $t = 0{,}5$ s con $\Delta t = 0{,}005$ s su 20×20 celle. È il tutorial storico di OpenFOAM, e lo useremo come filo conduttore anche nei Capp. 7 e 8.

c01_cavita.zip

Dati del caso `c01_cavita`

$L = 0{,}1$ m, $U = 1$ m/s, $\nu = 0{,}01$ m^2/s ($Re = 10$); mesh $20 \times 20 \times 1$; $\Delta t = 0{,}005$ s, $t_{\text{fin}} = 0{,}5$ s, scrittura ogni 20 passi (0,1 s); Co $= U\Delta t/h = 1$ sul coperchio; algoritmo PISO con 2 correttori; solutore `PCG` con precondizionatore `DIC` per p, `smoothSolver` per **U**; `momentumTransport` con `simulationType laminar`.

1.4.1 I file del caso

c01_cavita/0/U

```
8  FoamFile
9  {
10     version     2.0;
11     format      ascii;
12     class       volVectorField;
13     object      U;
14 }
15 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
16
17 dimensions      [0 1 -1 0 0 0 0];
18
19 internalField   uniform (0 0 0);
20
21 boundaryField
22 {
23     movingWall
24     {
25         type            fixedValue;
```

```
        value           uniform (1 0 0);
    }

    fixedWalls
    {
        type            noSlip;
    }

    frontAndBack
    {
        type            empty;
    }
```

Listato 1.1: Le condizioni iniziali e al contorno della velocità: coperchio a velocità imposta, pareti con aderenza, piani frontale e posteriore `empty`.

```

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    movingWall
    {
        type            zeroGradient;
    }

    fixedWalls
    {
        type            zeroGradient;
    }

    frontAndBack
    {
        type            empty;
    }
```

c01_cavita/0/p

Listato 1.2: La pressione (cinematica, m^2/s^2): gradiente nullo su tutte le pareti.

```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

viscosityModel  constant;

nu              0.01 [m^2/s];
```

c01_cavita/constant/physicalProperties

Listato 1.3: Le proprietà fisiche: viscosità cinematica costante.

```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

application     foamRun;

solver          incompressibleFluid;

startFrom       startTime;

startTime       0;

stopAt          endTime;

```

c01_cavita/system/controlDict

```
endTime         0.5;

deltaT          0.005;

writeControl    timeStep;

writeInterval   20;

purgeWrite      0;

writeFormat     ascii;

writePrecision  6;

writeCompression off;

timeFormat      general;

timePrecision   6;
```

Listato 1.4: Il controllo del calcolo: solutore, tempi, scrittura.

Leggere i dizionari

In `0/U` le dimensioni `[0 1 -1 0 0 0 0]` sono m/s; `internalField uniform (0 0 0)` è il fluido in quiete; il patch `movingWall` ha `fixedValue` con `value uniform (1 0 0)`, le pareti ferme `noSlip` (equivalente a `fixedValue (0 0 0)`), i piani `frontAndBack empty`. In `0/p` tutte le pareti hanno `zeroGradient`: la pressione non ha condizioni fisiche alle pareti e la sua equazione (Cap. 8) richiede $\partial p/\partial n = 0$ dove la velocità è imposta; poiché nessun patch fissa un valore, `fvSolution` indica una cella di riferimento (`pRefCell 0; pRefValue 0;`). In `controlDict`, `application foamRun` con `solver incompressibleFluid` seleziona il modulo per flussi incomprimibili (laminari o turbolenti, stazionari o no: è `momentumTransport` e `fvSchemes` a decidere); `deltaT 0.005` ed `endTime 0.5` (il transitorio si esaurisce molto prima); `writeControl timeStep` con `writeInterval 20` scrive ogni 20 passi; `writeFormat ascii` produce file leggibili (anche da `foamcase.py`); `runTimeModifiable true` rilegge i dizionari a ogni passo, così si può cambiare `endTime` durante il calcolo.

1.4.2 Eseguire

Dal terminale alla figura

```
cd casi/c01_cavita
blockMesh                    # genera constant/polyMesh da system/blockMeshDict
checkMesh                    # controlla la mesh (400 celle, Mesh OK)
foamRun                      # esegue il solutore indicato in controlDict
    (circa un secondo)
foamRun > log.foamRun 2>&1 # lo stesso, salvando il log su file (sempre
    consigliato)
ls                           # compaiono 0.1 0.2 0.3 0.4 0.5
paraFoam                     # apre ParaView sul caso
python3 ../../codice/fig01_cavita.py   # le figure della monografia da Python
```

Lo script `Allrun` di ogni caso esegue questa sequenza salvando i log; `Allclean` riporta

la cartella allo stato iniziale. Le utility si eseguono *dalla cartella del caso* oppure con l'opzione `-case percorso`.

```
#!/bin/bash
# Caso c01_cavita: cavita' con coperchio mobile, Re = 10 (tutorial storico di OpenFOAM)
cd "${0%/*}" || exit 1
blockMesh  > log.blockMesh 2>&1          # la mesh
checkMesh  > log.checkMesh 2>&1          # la sua verifica
foamRun    > log.foamRun 2>&1            # il solutore indicato in controlDict (incompressibleFluid)
foamPostProcess -func "mag(U)" -latestTime > log.foamPostProcess 2>&1   # un campo derivato, a calcolo finito
grep -c "^Time = " log.foamRun | xargs echo "passi temporali eseguiti:"
tail -n 3 log.foamRun
```

c01_cavita/Allrun

Listato 1.5: Lo script che esegue il caso e ne salva i log.

1.4.3 Leggere il log

Il log è la prima cosa da guardare, prima di qualunque figura. Per ogni passo temporale il solutore stampa il numero di Courant, poi per ogni equazione risolta il solutore lineare usato, il *residuo iniziale* (quanto l'equazione era violata prima di risolverla, normalizzato come nel Cap. 6), il residuo finale e il numero di iterazioni; poi gli errori di continuità e il tempo di calcolo. Un passo della cavità:

```
Courant Number mean: 0.222158 max: 0.852134
Time = 0.5

smoothSolver:  Solving for Ux, Initial residual = 2.3091e-07, Final residual = 2.3091e-07, No Iterations 0
smoothSolver:  Solving for Uy, Initial residual = 5.0684e-07, Final residual = 5.0684e-07, No Iterations 0
DICPCG:  Solving for p, Initial residual = 1.0127e-06, Final residual = 1.0127e-06, No Iterations 0
time step continuity errors : sum local = 9.57e-09, global = -1.31e-18, cumulative = 1.17e-17
DICPCG:  Solving for p, Initial residual = 8.6e-07, Final residual = 8.6e-07, No Iterations 0
time step continuity errors : sum local = 9.66e-09, global = 1.13e-18, cumulative = 1.28e-17
ExecutionTime = 0.52 s  ClockTime = 1 s
```

Che cosa dice questo passo

Il numero di Courant massimo è 0,85 (nella cella sotto il coperchio: $U\Delta t/h = 1 \cdot 0{,}005/0{,}005 = 1$ con la velocità di cella leggermente inferiore a U). I residui iniziali di U_x, U_y e p sono dell'ordine di 10^{-7}–10^{-6} e il solutore lineare non fa nemmeno un'iterazione (`No Iterations 0`): il campo non cambia più da un passo all'altro, cioè il transitorio è esaurito e si è raggiunto lo stato stazionario. L'equazione di p compare due volte perché PISO ha `nCorrectors 2`. Gli errori di continuità, $\sim 10^{-9}$ locale e $\sim 10^{-18}$ globale, dicono che il campo di velocità corretto è solenoidale alla precisione di macchina. Lo strumento `foamlog.py` estrae questi numeri da tutto il log e li disegna (Fig. 1.3): il transitorio, con residui iniziali che partono da 1 e scendono di sei ordini di grandezza in

0,3 s, è la storia del moto che si instaura.

1.4.4 I risultati

Risultati del caso `c01_cavita`

- Mesh: 400 celle, 1640 facce (760 interne), 882 punti; `checkMesh`: *Mesh OK*, non ortogonalità nulla.
- Calcolo: 100 passi temporali in 0,14 s; numero di Courant massimo 0,850; errore di continuità cumulativo $-1{,}4 \times 10^{-18}$.
- A $t = 0{,}5$ s i residui iniziali sono $2{,}3 \times 10^{-7}$ (U_x), $5{,}0 \times 10^{-7}$ (U_y), $8{,}8 \times 10^{-7}$ (p): stato stazionario raggiunto.
- Velocità minima (controcorrente) sulla mezzeria: $u_{\min}/U = -0{,}2005$ in $y/L = 0{,}53$ su 20×20, $u_{\min}/U = -0{,}2070$ su 80×80: la mesh del tutorial sottostima il ricircolo del 3%, come si vede nel profilo di Fig. 1.2.
- Pressione cinematica: minima -4,491 e massima 4,960 $\mathrm{m}^2/\mathrm{s}^2$ negli spigoli superiori (dove il coperchio "strappa" e comprime il fluido); nella cella centrale $p = -0{,}0010$.

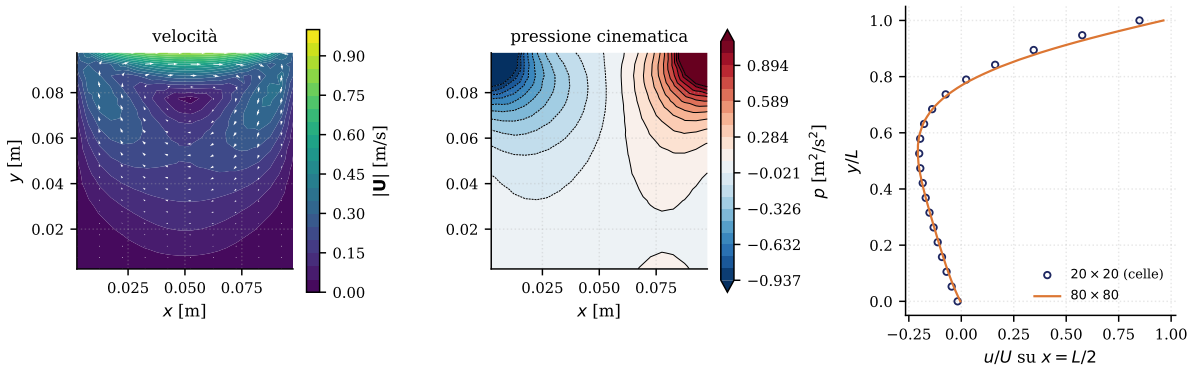


Figura 1.2: La cavità a $Re = 10$ a $t = 0{,}5$ s, letta con `foamcase.py`: modulo della velocità e vettori (a sinistra), pressione cinematica (al centro), profilo di u sulla mezzeria verticale confrontato con la soluzione su mesh 80×80 (a destra).

1.5 ParaView e i dati in Python

`paraFoam` apre ParaView con il caso caricato: nel pannello *Properties* si scelgono i campi e i patch, con *Apply* si visualizza; i filtri più usati sono *Slice* (una sezione), *Glyph* (vettori), *Stream Tracer* (linee di corrente), *Plot Over Line* (un profilo lungo un segmento), *Contour* (isosuperfici); la barra del tempo scorre le cartelle temporali. In alternativa `foamToVTK` converte i risultati in file VTK leggibili da ParaView senza il lettore di OpenFOAM, e da Python con la libreria `pyvista`. Per le figure di questa monografia si è seguita una terza via, coerente con il Modulo 0: `foamcase.py` legge direttamente i file ASCII del caso (mesh e campi) e Matplotlib disegna. È il modo di avere figure riproducibili e di *vedere* che cosa c'è nei file: un `volVectorField` è una lista di 400 terne, una per cella, nell'ordine delle celle della mesh; i valori di contorno stanno nel `boundaryField`, e il valore di una grandezza su una parete con `zeroGradient` è quello della cella adiacente.

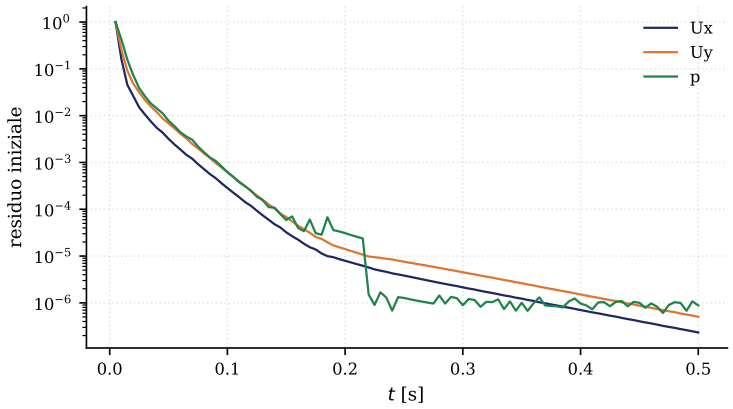


Figura 1.3: Residui iniziali di U_x, U_y e p in funzione del tempo per la cavità, estratti dal log con `foamlog.py`: il transitorio si esaurisce in circa 0,3 s.

Le utility che si usano ogni giorno

`foamListTimes`
elenca le cartelle temporali; `foamListTimes -rm` le cancella tranne `0`.

`foamDictionary`
legge e modifica i dizionari da riga di comando: `foamDictionary -entry endTime -set 1 system/controlDict`; indispensabile negli script che fanno studi parametrici (Cap. 13).

`foamPostProcess`
esegue sui risultati salvati una *function object*: `foamPostProcess -func "mag(U)"`, `-func vorticity`, `-func "graphUniform(...)"`; la stessa cosa si ottiene *durante* il calcolo elencando le funzioni in `controlDict` (Cap. 16).

`foamCloneCase`
copia un caso senza i risultati; `foamCleanCase` lo ripulisce; `foamGet` copia nel caso un dizionario di esempio dalla distribuzione (`foamGet decomposeParDict`).

`foamInfo`
cerca nella documentazione e nei sorgenti: `foamInfo fixedValue` mostra la descrizione della condizione al contorno e i tutorial che la usano.

`foamLog`
estrae i residui dal log in file di testo (ciò che fa `foamlog.py`).

1.6 Approfondimenti

1.6.1 Il tempo e le cartelle

Il solutore scrive i campi negli istanti stabiliti da `writeControl`: `timeStep` ogni `writeInterval` passi; `runTime` ogni `writeInterval` secondi (richiede che il passo divida l'intervallo); `adjustableRunTime` come il precedente ma adattando il passo per cadere esattamente sugli istanti di scrittura (con `adjustTimeStep yes` e `maxCo`, Cap. 6). `purgeWrite n` conserva solo le ultime n cartelle, utile nei calcoli lunghi. `startFrom latestTime`

riprende un calcolo interrotto dall'ultima cartella scritta: insieme a `runTimeModifiable` permette di allungare un calcolo che non è ancora stazionario senza ripartire da zero. Nei calcoli stazionari (Cap. 8) il "tempo" è il contatore delle iterazioni e `deltaT 1`.

1.6.2 Quando il caso non parte

I messaggi di OpenFOAM sono prolissi ma precisi; le cause più comuni. `keyword ... is undefined`: manca una voce in un dizionario, e il messaggio dice quale e in quale file: se è in `fvSchemes`, il solutore ha incontrato un termine per cui non è stato indicato lo schema. `Cannot find patchField entry for ...`: un patch della mesh non ha la condizione al contorno in un file di `0/`, oppure il nome è scritto diversamente. `incompatible dimensions`: due grandezze sommate con dimensioni diverse, spesso p cinematica contro assoluta. `Continuity error cannot be removed by adjusting the outflow`: nessun patch permette al fluido di uscire (tutte le velocità imposte, nessuna pressione fissata). `Floating point exception`: divergenza, quasi sempre per passo temporale troppo grande o mesh con celle degeneri; si guardano gli ultimi residui nel log e `checkMesh`.

1.6.3 Un tutorial al giorno

La distribuzione contiene centinaia di casi in `$FOAM_TUTORIALS`, organizzati per modulo solutore (`incompressibleFluid/`, `fluid/`, `shockFluid/`, `incompressibleVoF/`, ...). Sono la documentazione più aggiornata: ogni condizione al contorno, ogni schema, ogni function object è usato in almeno uno di essi (`foamInfo` li trova). Copiarne uno vicino al proprio problema (`foamCloneCase`) e adattarlo è il modo normale di impostare un caso nuovo, e il Cap. 16 lo fa passo per passo.

Capitolo 2

Le equazioni di conservazione e il metodo dei volumi finiti

2.1 Perché i volumi finiti

I Moduli 1–4 hanno risolto le equazioni della fluidodinamica con le *differenze finite*: si parte dalla forma differenziale, si sostituiscono le derivate con rapporti incrementali su una griglia strutturata e si ottiene un sistema algebrico nei valori nodali. Il metodo funziona benissimo su domini semplici, ma incontra due limiti quando il dominio è un'ala, una gondola o un'automobile: la griglia deve essere strutturata (o trasformata con coordinate curvilinee, come nel Modulo 4 per il cilindro), e la *conservazione* delle grandezze fisiche non è garantita per costruzione: lo schema può creare o distruggere massa a livello discreto, e lo si scopre solo a posteriori.

Il **metodo dei volumi finiti** (FVM) rovescia il punto di partenza: non la forma differenziale ma la forma *integrale* delle leggi di conservazione, applicata a ciascuna delle celle (*volumi di controllo*) in cui il dominio è suddiviso. Il flusso che esce da una cella attraverso una faccia è esattamente il flusso che entra nella cella adiacente: la conservazione è garantita cella per cella e quindi globalmente, qualunque sia la forma delle celle. È per questo che tutti i codici industriali (OpenFOAM, Fluent, STAR-CCM+, CFX) sono ai volumi finiti su mesh *poliedriche* non strutturate. Questo capitolo ricava la forma integrale delle equazioni e la sua discretizzazione nel modo in cui il testo di riferimento [1] la presenta (Capp. 3 e 5), e mostra come ogni termine diventi una riga di codice in OpenFOAM.

2.2 L'equazione generale di trasporto

Dalla legge di conservazione all'equazione di trasporto

Sia ϕ una grandezza scalare per unità di massa (la componente u della velocità, l'entalpia, la concentrazione di una specie, l'energia cinetica turbolenta). Il *teorema del trasporto di Reynolds* dice che la variazione nel tempo della quantità $\int_V \rho\phi \, \mathrm{d}V$ contenuta in un volume materiale è uguale alla variazione in un volume di controllo fisso V più il flusso netto convettivo attraverso la sua superficie ∂V:

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{V_m} \rho\phi \, \mathrm{d}V = \int_V \frac{\partial(\rho\phi)}{\partial t}\, \mathrm{d}V + \oint_{\partial V} \rho\phi \, \mathbf{v}\cdot \mathrm{d}\mathbf{S}. \tag{2.1}$$

Se la grandezza ϕ è trasportata anche per diffusione (flusso $-\Gamma^\phi \nabla\phi$, legge di Fourier o di Fick) ed è prodotta o distrutta da una sorgente per unità di volume Q^ϕ, la legge di conservazione per il volume di controllo è

$$\underbrace{\int_V \frac{\partial(\rho\phi)}{\partial t}\,\mathrm{d}V}_{\text{accumulo}} + \underbrace{\oint_{\partial V} \rho\phi\,\mathbf{v}\cdot\mathrm{d}\mathbf{S}}_{\text{convezione}} = \underbrace{\oint_{\partial V} \Gamma^\phi \nabla\phi\cdot\mathrm{d}\mathbf{S}}_{\text{diffusione}} + \underbrace{\int_V Q^\phi\,\mathrm{d}V}_{\text{sorgente}}. \tag{2.2}$$

Applicando il teorema della divergenza ai due integrali di superficie e facendo tendere V a zero si ottiene la forma differenziale *conservativa*

$$\frac{\partial(\rho\phi)}{\partial t} + \nabla\cdot(\rho\mathbf{v}\phi) = \nabla\cdot(\Gamma^\phi\nabla\phi) + Q^\phi, \tag{2.3}$$

che è il modello di *tutte* le equazioni della fluidodinamica: con $\phi = 1$, $\Gamma = 0$, $Q = 0$ è la continuità; con $\phi = u_i$, $\Gamma = \mu$ e $Q = -\partial p/\partial x_i + \dots$ è la quantità di moto; con $\phi = h$ è l'energia; con $\phi = k$, ε, ω sono le equazioni dei modelli di turbolenza. Il metodo dei volumi finiti discretizza la (2.2) una volta per tutte, e ogni equazione specifica è un caso particolare: è esattamente così che OpenFOAM è organizzato.

Le equazioni di Navier–Stokes nella forma che OpenFOAM risolve

Per un fluido newtoniano incomprimibile a proprietà costanti, dividendo per ρ e introducendo la *pressione cinematica* p/ρ (che OpenFOAM chiama ancora `p` e misura in $\mathrm{m}^2/\mathrm{s}^2$):

$$\nabla\cdot\mathbf{U} = 0, \tag{2.4}$$

$$\frac{\partial\mathbf{U}}{\partial t} + \nabla\cdot(\mathbf{U}\mathbf{U}) = -\nabla p + \nabla\cdot(\nu_{\mathrm{eff}}\nabla\mathbf{U}) + \nabla\cdot\left(\nu_{\mathrm{eff}}(\nabla\mathbf{U})^{\mathrm{T}}\right), \tag{2.5}$$

dove $\nu_{\mathrm{eff}} = \nu + \nu_t$ include la viscosità turbolenta dei modelli RANS (Cap. 9); l'ultimo termine, nullo per ν costante e campo solenoidale, è conservato perché non lo è più con ν_t variabile. Nel codice la (2.5) si legge, riga per riga, `fvm::ddt(U) + fvm::div(phi, U) + momentumTransport->divDevSigma(U) == -fvc::grad(p)`: il termine convettivo è scritto in forma conservativa $\nabla\cdot(\mathbf{U}\mathbf{U})$ e non $(\mathbf{U}\cdot\nabla)\mathbf{U}$, e il flusso di massa $\phi_f = \mathbf{U}_f\cdot\mathbf{S}_f$ sulle facce è un campo a sé (`phi`), che è l'oggetto conservato.

2.3 Il processo di discretizzazione

Il testo di riferimento descrive la soluzione numerica come una catena di quattro passi (Cap. 4): (I) il modello geometrico e fisico (dominio, equazioni, condizioni al contorno); (II) la discretizzazione del dominio in celle (la *mesh*, Cap. 3); (III) la discretizzazione delle equazioni, che trasforma la (2.2) in un sistema algebrico; (IV) la soluzione del sistema (Capp. 7 e 8). In OpenFOAM i quattro passi hanno un posto preciso: (I) è il contenuto di `0/` e `constant/`, (II) è `constant/polyMesh` prodotto da `blockMesh` o `snappyHexMesh`, (III) è governato da `system/fvSchemes`, (IV) da `system/fvSolution`.

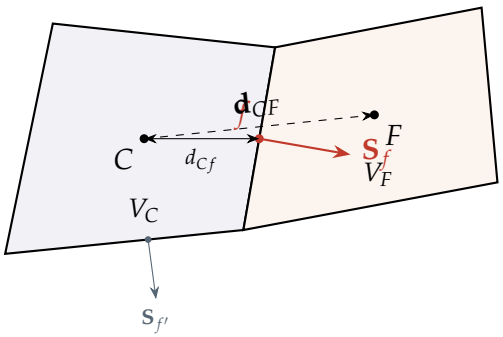


Figura 2.1: Due celle adiacenti C (*owner*) e F (*neighbour*) che condividono la faccia f: il vettore area $\mathbf{S}_f$ è normale alla faccia, ha modulo pari all'area ed è orientato da C verso F; $\mathbf{d}_{CF}$ unisce i centroidi. Quando $\mathbf{S}_f$ e $\mathbf{d}_{CF}$ non sono paralleli la mesh è *non ortogonale*.

L'equazione semi-discretizzata

Si integri la (2.2) sulla cella C di volume V_C (Fig. 2.1), trascurando per ora il termine instazionario. La superficie della cella è l'unione delle sue facce piane f, quindi gli integrali di superficie sono somme di integrali sulle facce; su ogni faccia l'integrale è approssimato con la *regola del punto medio*, cioè con il valore dell'integrando nel centroide f per l'area:

$$\sum_{f\sim\mathrm{nb}(C)} (\rho\mathbf{v}\phi)_f \cdot \mathbf{S}_f = \sum_{f\sim\mathrm{nb}(C)} (\Gamma^\phi \nabla\phi)_f \cdot \mathbf{S}_f + Q_C^\phi V_C. \tag{2.6}$$

La regola del punto medio è del secondo ordine: l'errore sull'integrale di faccia è $\mathcal{O}(h^2)$ se il valore è preso *nel centroide* della faccia, e lo stesso vale per il termine di volume valutato nel centroide della cella. Questo è il motivo per cui OpenFOAM, come vedremo nel Cap. 3, calcola con cura i centroidi di facce e celle. Il *flusso di massa* attraverso la faccia è $\dot{m}_f = (\rho\mathbf{v})_f \cdot \mathbf{S}_f$ (positivo se esce da C); con esso il termine convettivo è $\sum_f \dot{m}_f \phi_f$ e quello diffusivo $\sum_f \Gamma_f (\nabla\phi)_f \cdot \mathbf{S}_f$.

Linearizzazione e forma algebrica

Il passo decisivo è esprimere i valori di faccia ϕ_f e $(\nabla\phi)_f$ in funzione dei valori nelle celle ϕ_C e ϕ_F: sono le scelte dei Capp. 4 e 5, cioè le voci di `fvSchemes`. Qualunque sia la scelta, il flusso attraverso la faccia è scritto in forma *linearizzata*,

$$J_f = \mathrm{FluxC}_f\, \phi_C + \mathrm{FluxF}_f\, \phi_F + \mathrm{FluxV}_f, \tag{2.7}$$

con una parte implicita (i coefficienti di ϕ_C e ϕ_F) e una esplicita (FluxV_f, calcolata con i valori dell'iterazione precedente). Sommando sulle facce la (2.6) diventa l'**equazione algebrica** della cella,

$$a_C\phi_C + \sum_{F\sim\mathrm{NB}(C)} a_F\phi_F = b_C, \qquad a_C = \sum_f \mathrm{FluxC}_f, \quad a_F = \mathrm{FluxF}_f, \quad b_C = Q_C V_C - \sum_f \mathrm{FluxV}_f, \tag{2.8}$$

una riga del sistema lineare $\mathbf{A}\boldsymbol{\phi} = \mathbf{b}$ con tanti elementi non nulli quante sono le facce della cella più uno. In OpenFOAM la (2.8) è l'oggetto `fvMatrix`: i coefficienti a_C sono la diagonale `diag()`, i coefficienti a_F sono memorizzati *per faccia* in `upper()` e `lower()` (l'indirizzamento *ldu* del Cap. 3), b_C è `source()`. Le funzioni `UEqn.A()` e

`UEqn.H()` che compaiono nell'algoritmo di pressione (Cap. 8) restituiscono a_C/V_C e $(b_C - \sum_F a_F \phi_F)/V_C$.

2.4 Le proprietà di una buona discretizzazione

Il testo di riferimento (Sez. 5.6) elenca le proprietà che la (2.8) deve avere; sono i criteri con cui nei capitoli seguenti giudicheremo gli schemi di `fvSchemes`.

- **Conservatività.** Il flusso J_f calcolato dalla cella C deve essere l'opposto di quello calcolato dalla cella F per la stessa faccia. Nel FVM è automatico se il flusso è calcolato *una volta per faccia*: OpenFOAM scorre le facce, non le celle, proprio per questo.
- **Consistenza e accuratezza.** L'errore di troncamento deve tendere a zero con h e con Δt, con un ordine noto: secondo ordine nello spazio per la regola del punto medio con interpolazione lineare, primo per upwind. Misurare l'ordine (Cap. 16) è la verifica più severa.
- **Limitatezza** (*boundedness*). Senza sorgenti, ϕ_C non deve uscire dall'intervallo dei valori vicini: lo garantisce una matrice con $a_C \geq \sum_F |a_F|$ e coefficienti a_F di segno opposto a a_C (regola dei segni opposti e regola della somma nulla, Sez. 8.2 del testo). Gli schemi centrati la violano a numeri di Péclet alti; quelli *limitati* (TVD) la ripristinano.
- **Trasportività.** L'informazione viaggia nella direzione del flusso: una perturbazione a valle non deve influenzare il campo a monte quando la convezione domina. Lo schema upwind la possiede, quello centrato no.
- **Stabilità ed economia.** Il processo iterativo deve convergere (dominanza diagonale) e farlo in un tempo ragionevole: entrano qui il sottorilassamento e i solutori lineari (Capp. 6 e 7).

2.5 Come OpenFOAM scrive le equazioni

`fvm` e `fvc`

OpenFOAM mette a disposizione due famiglie di operatori che agiscono su campi definiti nelle celle (`volScalarField`, `volVectorField`) o sulle facce (`surfaceScalarField`):

- `fvm::` (*finite volume method*, **implicito**): restituisce una `fvMatrix`, cioè i coefficienti a_C, a_F e il termine noto della (2.8) per il termine indicato. `fvm::ddt(U)`, `fvm::div(phi, U)`, `fvm::laplacian(nu, U)`, `fvm::Sp(coeff, U)` (sorgente implicita).
- `fvc::` (*finite volume calculus*, **esplicito**): valuta il termine con i valori correnti dei campi e restituisce un campo. `fvc::grad(p)`, `fvc::div(phi)`, `fvc::laplacian(nu, U)`, `fvc::interpolate(U)` (da celle a facce), `fvc::flux(U)` ($\mathbf{U}_f \cdot \mathbf{S}_f$).

Un'equazione è la somma di `fvMatrix` e di campi, con `==` al posto dell'uguale: `solve(fvm::ddt(U) + fvm::div(phi, U) - fvm::laplacian(nu, U) == -fvc::grad(p))` assembla e risolve la quantità di moto con il gradien-

te di pressione esplicito. Ogni operatore cerca in `fvSchemes` *come* discretizzare il proprio termine: `fvm::ddt` in `ddtSchemes`, `fvm::div(phi, U)` nella voce `div(phi,U)` di `divSchemes`, `fvm::laplacian` in `laplacianSchemes`, `fvc::grad` in `gradSchemes`. Se la voce manca il programma si ferma con un messaggio che la nomina: è il modo più rapido per scoprire quali termini un solutore contiene.

Le dimensioni fisiche

Ogni campo e ogni costante di OpenFOAM porta con sé le dimensioni fisiche come vettore di sette esponenti [kg m s K mol A cd]: `dimensions [0 1 -1 0 0 0 0]` è una velocità, `[0 2 -2 0 0 0 0]` la pressione cinematica, `[0 2 -1 0 0 0 0]` la viscosità cinematica. Il codice controlla la coerenza dimensionale di ogni operazione a tempo di esecuzione: sommare una velocità a una pressione produce un errore `FOAM FATAL ERROR: incompatible dimensions`. È una rete di sicurezza preziosa e una fonte di errori dei principianti, che scrivono ν in m^2/s nel caso incomprimibile e μ in Pa s in quello comprimibile senza accorgersi che `p` è cinematica nel primo e assoluta nel secondo.

2.6 Lettura guidata: `icoFoam.C`

Il solutore più semplice della distribuzione, `icoFoam` (incomprimibile, laminare, transitorio, algoritmo PISO), è lungo meno di novanta righe e contiene tutto ciò che questo capitolo ha descritto. Lo si legga una volta ora, per vedere la corrispondenza fra equazioni e codice; lo si rileggerà nel Cap. 8 per l'algoritmo di pressione. Le righe 75–85 sono la (2.5) con ν costante; le righe 88–128 sono il ciclo PISO; la riga 130 scrive i campi su disco agli istanti richiesti da `controlDict`.

letture/icoFoam.C

```
52 int main(int argc, char *argv[])
53 {
54     #include "setRootCase.H"
55     #include "createTime.H"
56     #include "createMesh.H"
57
58     pisoControl piso(mesh);
59
60     #include "createFields.H"
61     #include "initContinuityErrs.H"
62
63     // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
64
65     Info<< "\nStarting time loop\n" << endl;
66
67     while (runTime.loop())
68     {
69         Info<< "Time = " << runTime.userTimeName() << nl << endl;
70
71         #include "CourantNo.H"
72
73         // Momentum predictor
74
75         fvVectorMatrix UEqn
76         (
77             fvm::ddt(U)
```

```
78            + fvm::div(phi, U)
79            - fvm::laplacian(nu, U)
80        );
81
82        if (piso.momentumPredictor())
83        {
84            solve(UEqn == -fvc::grad(p));
85        }
86
87        // --- PISO loop
88        while (piso.correct())
89        {
90            volScalarField rAU(1.0/UEqn.A());
91            volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
92            surfaceScalarField phiHbyA
93            (
94                "phiHbyA",
95                fvc::flux(HbyA)
96              + fvc::interpolate(rAU)*fvc::ddtCorr(U, phi)
97            );
98
99            adjustPhi(phiHbyA, U, p);
100
101           // Update the pressure BCs to ensure flux consistency
102           constrainPressure(p, U, phiHbyA, rAU);
103
104           // Non-orthogonal pressure corrector loop
105           while (piso.correctNonOrthogonal())
106           {
107               // Pressure corrector
108
109               fvScalarMatrix pEqn
110               (
111                   fvm::laplacian(rAU, p) == fvc::div(phiHbyA)
112               );
113
114               pEqn.setReference(pRefCell, pRefValue);
115
116               pEqn.solve();
117
118               if (piso.finalNonOrthogonalIter())
119               {
120                   phi = phiHbyA - pEqn.flux();
121               }
122           }
123
124           #include "continuityErrs.H"
125
126           U = HbyA - rAU*fvc::grad(p);
127           U.correctBoundaryConditions();
128       }
129
130       runTime.write();
131
132       Info<< "ExecutionTime = " << runTime.elapsedCpuTime() << " s"
133           << "  ClockTime = " << runTime.elapsedClockTime() << " s"
134           << nl << endl;
135   }
136
137   Info<< "End\n" << endl;
```

```
138
139     return 0;
140 }
```

Listato 2.1: Il ciclo principale di `icoFoam.C`: equazione della quantità di moto (75–85), ciclo PISO con equazione della pressione (88–128), scrittura dei risultati (130).

Che cosa c'è in queste novanta righe

Le tre `#include` iniziali (54–56) leggono gli argomenti, creano l'oggetto `runTime` (il tempo, con le impostazioni di `controlDict`) e la mesh (da `constant/polyMesh`); `createFields.H` (60) legge p e **U** da `0/` con le loro condizioni al contorno e calcola il flusso `phi` sulle facce. Il ciclo `while (runTime.loop())` (67) avanza il tempo di `deltaT` finché non si raggiunge `endTime`. `UEqn` (75–80) è una `fvVectorMatrix`: tre righe, tre termini, tre voci di `fvSchemes` (`ddt(U)`, `div(phi,U)`, `laplacian(nu,U)`). Il predittore (84) risolve la quantità di moto con il gradiente di pressione *vecchio*; il ciclo PISO (88) costruisce l'equazione della pressione (109–112) a partire dai coefficienti `A` e `H` della matrice della quantità di moto, la risolve (116), corregge il flusso di faccia in modo *conservativo* con `pEqn.flux()` (120) e poi la velocità di cella (126). L'ordine delle operazioni e il significato di `rAU`, `HbyA` e `phiHbyA` sono l'argomento del Cap. 8; per ora basta osservare che l'equazione della pressione è un laplaciano `fvm::` con sorgente `fvc::div`: una equazione di Poisson, come nel Modulo 4.

Nel solutore modulare `incompressibleFluid` di OpenFOAM 12, che useremo per quasi tutti i casi, lo stesso codice è distribuito in più file: `momentumPredictor.C` contiene l'equazione della quantità di moto nella forma generale con il modello di turbolenza (`momentumTransport->divDevSigma(U)`) e le sorgenti opzionali (`fvModels`); `correctPressure.C` contiene il correttore di pressione con le varianti SIMPLE/SIMPLEC/PISO selezionate da `fvSolution`. Sono riportati nell'Appendice A.5 e commentati nel Cap. 8.

2.7 Approfondimenti

2.7.1 Volumi finiti centrati nelle celle e nei vertici

Nella variante *cell-centered* adottata da OpenFOAM le incognite sono i valori medi nelle celle, i volumi di controllo coincidono con le celle della mesh e i centroidi sono i punti in cui le equazioni sono scritte; nella variante *vertex-centered* le incognite stanno nei vertici e i volumi di controllo sono costruiti intorno a essi (mediane). La prima si adatta a qualunque poliedro e ha un volume di controllo per cella, la seconda ha meno incognite su mesh tetraedriche ma richiede la costruzione dei volumi duali. Il testo di riferimento discute le due scelte nella Sez. 5.7; la distinzione conta quando si confrontano codici diversi: su una stessa mesh, un codice vertex-centered ha un numero di incognite diverso e un'accuratezza diversa.

2.7.2 Che cosa significa "secondo ordine"

La regola del punto medio per l'integrale di faccia e la rappresentazione del valore di cella come valore nel centroide sono entrambe approssimazioni del secondo ordine (Sez. 5.4 del testo): $\int_f \phi \, \mathrm{d}S = \phi_f S_f + \mathcal{O}(h^2)$ e $\bar{\phi}_C = \phi(\mathbf{x}_C) + \mathcal{O}(h^2)$. Perché il metodo complessivo sia del

secondo ordine occorre che *tutte* le approssimazioni lo siano: l'interpolazione dei valori di faccia (lineare, non upwind), la valutazione del gradiente (Green–Gauss su mesh regolari), la correzione non ortogonale del laplaciano, lo schema temporale (`backward` o `CrankNicolson`, non `Euler`). Basta un anello del primo ordine perché l'errore globale decresca come h: il Cap. 5 lo mostra misurando l'ordine con schemi diversi.

2.7.3 La forma non conservativa e i suoi rischi

Scrivere la convezione come $(\mathbf{v}\cdot\nabla)\phi$ invece di $\nabla\cdot(\rho\mathbf{v}\phi)$ è lecito nel continuo (per la continuità), ma nel discreto le due forme differiscono di $\phi\,\nabla\cdot(\rho\mathbf{v})$, che è nullo solo a convergenza. Durante le iterazioni la forma non conservativa crea o distrugge ϕ in proporzione all'errore di continuità, con effetti visibili sugli urti e sulle grandezze integrali (portanza, resistenza). OpenFOAM usa sempre la forma conservativa e, nei casi stazionari, offre la variante `bounded Gauss` che sottrae esplicitamente $\phi\,\nabla\cdot(\rho\mathbf{v})$ per migliorare la limitatezza durante le iterazioni (Cap. 5).

Capitolo 3

La mesh ai volumi finiti

3.1 Dalla geometria alle celle

Il secondo passo del processo di discretizzazione (Cap. 2) suddivide il dominio in celle. Nel metodo dei volumi finiti *cell-centered* le celle sono i volumi di controllo, e quindi la mesh non è un dettaglio tecnico: ne dipendono l'accuratezza (la regola del punto medio è del secondo ordine solo se i centroidi sono dove il codice li suppone), la stabilità (celle distorte producono coefficienti di segno sbagliato) e il costo (il numero di celle e la loro forma decidono il tempo di calcolo). Questo capitolo descrive come OpenFOAM rappresenta una mesh poliedrica qualsiasi, come ne calcola le grandezze geometriche, come si giudica la sua qualità, e i due generatori della distribuzione: `blockMesh`, per mesh strutturate a blocchi, e `snappyHexMesh`, per geometrie arbitrarie.

3.2 La rappresentazione `polyMesh`

Indirizzamento per facce

OpenFOAM non memorizza le celle come liste di vertici (come farebbe un codice a elementi finiti) ma usa l'*indirizzamento per facce* (Sez. 7.2 del testo di riferimento): la mesh è un insieme di **punti**, di **facce** (poligoni piani definiti da una lista ordinata di punti) e di **celle** definite implicitamente dalle facce che le delimitano. Ogni faccia ha una cella *owner* (proprietaria) e, se è interna, una cella *neighbour* (vicina); il vettore area $\mathbf{S}_f$ punta dall'owner verso il neighbour, e per convenzione l'owner è la cella con l'indice minore. La cartella `constant/polyMesh` contiene cinque file:

`points`
: la lista delle coordinate (x, y, z) dei vertici, in metri;

`faces`
: per ogni faccia la lista dei suoi punti, `4(0 1 22 21)`: le facce *interne* vengono prima, poi quelle di contorno raggruppate per patch;

`owner`
: per ogni faccia l'indice della cella owner;

`neighbour`
: per ogni faccia *interna* l'indice della cella neighbour (la lista è più corta di `owner`: la differenza è il numero di facce di contorno);

`boundary`
: la lista dei *patch*: per ciascuno il nome, il `type` (`patch`, `wall`, `empty`, `symmetry`, `wedge`, `cyclic`, ...), il numero di facce `nFaces` e la prima faccia `startFace`.

Con questa struttura una cella può avere un numero qualsiasi di facce (esaedri, tetraedri, prismi, poliedri generici tagliati da `snappyHexMesh`), e i cicli del codice scorrono le *facce*, calcolando il flusso una volta per faccia e accumulandolo nell'owner (con segno $+$) e nel neighbour (con segno $-$): è la conservatività automatica del Cap. 2. Il *numero di celle* non è scritto da nessuna parte: è il massimo indice di `owner` più uno.

La cavità: 400 celle, 1640 facce

Per la mesh $20 \times 20 \times 1$ della cavità (Cap. 1) `points` contiene $21 \times 21 \times 2 = 882$ punti, `faces` 1640 facce, di cui 760 interne (`neighbour` ha 760 voci) e 880 di contorno: 20 sul coperchio, 60 sulle altre pareti e $2 \times 400 = 800$ sui due piani `empty`. Il file `boundary` lo riassume:

```
3
(
    movingWall   { type wall;  inGroups List<word> 1(wall); nFaces 20;
    startFace 760; }
    fixedWalls   { type wall;  inGroups List<word> 1(wall); nFaces 60;
    startFace 780; }
    frontAndBack { type empty; inGroups List<word> 1(empty); nFaces 800;
    startFace 840; }
)
```

Lo strumento `foamcase.py` (Appendice A.4) legge questi cinque file e ricostruisce tutto il resto; `python3 foamcase.py casi/c01_cavita` stampa proprio questi numeri.

Le grandezze geometriche

Dalla lista dei punti di una faccia il codice calcola (Sez. 6.5 del testo) il *centroide* $\mathbf{x}_f$ e il *vettore area* $\mathbf{S}_f$ scomponendo il poligono in triangoli con vertice nel centro geometrico dei punti: l'area è la somma dei vettori area dei triangoli (metà del prodotto vettoriale dei lati), il centroide è la media dei centroidi dei triangoli pesata con le aree. Per una faccia piana e convessa il risultato è esatto; per una faccia leggermente non piana (frequente nelle mesh generate automaticamente) la scomposizione definisce *una* faccia equivalente coerente. Il volume e il centroide di una cella si ottengono poi scomponendola in piramidi con base le facce e vertice in un punto interno stimato: $V_C = \frac{1}{3}\sum_f \mathbf{S}_f \cdot (\mathbf{x}_f - \mathbf{x}_{\text{est}})$ e il centroide come media pesata dei centroidi delle piramidi. Questo è esattamente ciò che fa la classe `Mesh` di `foamcase.py`, i cui volumi coincidono con quelli di OpenFOAM alla precisione di macchina. Da queste quantità discendono tutte le altre: il vettore $\mathbf{d}_{CF}$ fra i centroidi di due celle adiacenti, la distanza normale dalla parete $d_n = (\mathbf{x}_f - \mathbf{x}_C) \cdot \mathbf{n}_f$ che compare nelle condizioni al contorno e nello y^+, i pesi di interpolazione lineare $g_f = \overline{fF}/\overline{CF}$.

3.3 Indici di qualità e `checkMesh`

Non ortogonalità, skewness, rapporto di forma

Tre numeri riassumono la qualità di una faccia (Fig. 3.1):

- la **non ortogonalità** è l'angolo θ_f fra $\mathbf{S}_f$ e $\mathbf{d}_{CF}$: è zero su una mesh cartesiana, cresce dove le celle sono distorte. La derivata normale alla faccia, che il termine diffusivo richiede, è approssimata da $(\phi_F - \phi_C)/|\mathbf{d}_{CF}|$ solo se $\theta_f = 0$; altrimenti serve una correzione (Cap. 4) che è esplicita e peggiora con l'angolo. Il codice accetta fino a 70° e segnala sopra; in pratica sotto 40°–50° la correzione funziona bene;
- la **skewness** misura di quanto la retta CF manca il centroide della faccia: la regola del punto medio valuta il flusso nel centroide, ma l'interpolazione lineare fra C ed F fornisce il valore nel punto f' dove la retta attraversa la faccia; la differenza è un errore del primo ordine se non viene corretto. `checkMesh` normalizza la distanza $|\mathbf{x}_f - \mathbf{x}_{f'}|$ con la semi-estensione della faccia nella direzione dello scostamento e segnala sopra 4 (sopra 20 per le facce di contorno); sotto 0,5 l'effetto è trascurabile;
- il **rapporto di forma** (*aspect ratio*) è il rapporto fra la dimensione massima e minima di una cella: valori alti (fino a 10^3–10^4) sono normali e necessari negli strati limite, dove il gradiente è tutto normale alla parete, ma peggiorano il condizionamento del sistema lineare e, se la cella è anche distorta, l'accuratezza.

`checkMesh` calcola questi indici per tutte le facce e riporta massimo e media; con `-allGeometry -allTopology` aggiunge controlli facoltativi (facce concave, celle con facce deformate, *wedge* e *empty* corretti) e scrive in `constant/polyMesh/sets` gli insiemi delle celle che violano le soglie, visualizzabili in ParaView. `foamcase.py` riproduce le stesse definizioni: sulla mesh del Cap. 13 entrambi danno non ortogonalità massima 55,07° e skewness massima 0,150.

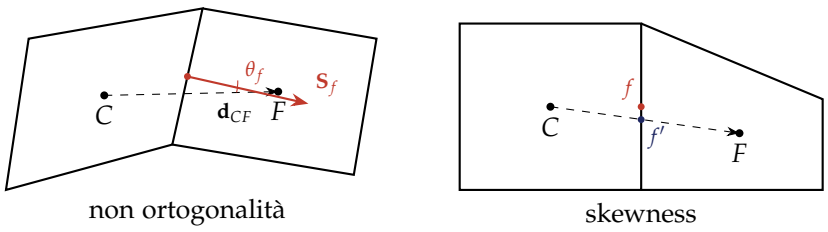


Figura 3.1: Non ortogonalità (angolo fra il vettore area e la congiungente dei centroidi) e skewness (la congiungente attraversa la faccia in f', non nel centroide f).

Che cosa succede con una mesh cattiva

Una mesh con non ortogonalità sopra 70° o con celle a volume negativo fa divergere il calcolo o produce campi a scacchiera; ma il pericolo maggiore è una mesh *mediocre* che converge e dà un risultato sbagliato: celle troppo grandi dove il gradiente è forte (bordo d'attacco, strato limite, urto), transizioni brusche di dimensione (rapporto fra celle adiacenti sopra 1,3–1,5), celle distorte nella scia che diffondono numericamente la vorticità. Nessun indice di `checkMesh` rivela questi difetti: li rivela solo lo studio di convergenza di griglia (Cap. 16).

3.4 `blockMesh`: mesh strutturate a blocchi

`blockMesh` legge `system/blockMeshDict` e costruisce una mesh esaedrica unendo uno o più *blocchi*, ciascuno un esaedro topologico con otto vertici, dodici spigoli (rettilinei per impostazione predefinita, oppure archi, spline, polilinee) e un numero di celle in ciascuna delle tre direzioni con una legge di infittimento. È lo strumento giusto per le geometrie che si lasciano descrivere con pochi blocchi (canali, cavità, cilindri, profili alari con mesh a C o a O, condotti assialsimmetrici) e produce le mesh di qualità migliore: ortogonali o quasi, con l'infittimento esattamente dove serve.

Il dizionario, voce per voce

`convertToMeters`
(o `scale`) fattore che moltiplica tutte le coordinate: si scrive il dizionario in centimetri o in corde e si converte alla fine.

`vertices`
la lista dei vertici, numerati da 0 nell'ordine in cui compaiono; i commenti `// 3` accanto a ciascuno non sono un vezzo: sono l'unico modo di non perdersi.

`blocks`
per ogni blocco `hex (v0 v1 v2 v3 v4 v5 v6 v7) (nx ny nz) simpleGrading (gx gy gz)`. L'ordine dei vertici definisce le direzioni locali: x_1 da `v0` a `v1`, x_2 da `v1` a `v2` (cioè da `v0` a `v3`), x_3 da `v0` a `v4`; la terna deve essere destrorsa ($\mathbf{x}_1 \times \mathbf{x}_2 \cdot \mathbf{x}_3 > 0$), altrimenti il blocco ha volume negativo e `checkMesh` lo segnala. Il *grading* è il rapporto fra l'ultima e la prima cella lungo la direzione: $g > 1$ celle crescenti, $g < 1$ decrescenti; con n celle e rapporto di espansione geometrico r fra celle adiacenti, $g = r^{n-1}$. Il *multi-grading* `((0.5 0.5 0.25) (0.5 0.5 4))` divide la direzione in tratti (frazione di lunghezza, frazione di celle, grading del tratto): infittire verso *entrambe* le pareti di un canale si scrive così.

`edges`
spigoli curvi: `arc v0 v1 (x y z)` arco di cerchio per i due vertici e il punto indicato; `spline v0 v1 ((...) (...) ...)` e `polyLine` per curve per punti (è così che la mesh a C del Cap. 13 segue il profilo). I vertici interni allo spigolo sono distribuiti con il grading del blocco.

`boundary`
i patch: nome, `type`, e le facce come quaterne di vertici (l'ordine dei vertici è libero, ma devono essere quelli di una faccia di blocco). Le facce di blocco non elencate finiscono in un patch `defaultFaces` di tipo `empty`: un avviso nel log lo segnala, ed è quasi sempre un errore.

`mergePatchPairs`
unisce due patch affacciati di blocchi con vertici diversi (mesh non conformi); nei casi di questo corso è vuoto.

Il dizionario può contenere *variabili* (`R 0.5;` e poi `$R`), espressioni `#calc "$R*$c"` (compilate al volo), negazioni `#neg $R`, e inclusioni `#include`: la mesh a O del cilindro ne fa uso per non scrivere trenta volte lo stesso coseno.

Mesh bidimensionali e assialsimmetriche

OpenFOAM è sempre tridimensionale. Un caso *piano* è una mesh con una sola cella nella direzione z e i due piani frontale e posteriore in un patch di tipo `empty`: il codice non risolve le equazioni in quella direzione e non vi richiede condizioni al contorno. Un caso *assialsimmetrico* è un cuneo di piccolo angolo (tipicamente 1°–5°) con una cella in direzione azimutale e le due facce laterali in due patch di tipo `wedge`, che impongono la periodicità rotazionale; l'asse è un patch `empty` con facce degeneri (area nulla). Nel `blockMeshDict` del condotto (caso `c02_tubo`) il blocco è un esaedro con due coppie di vertici coincidenti sull'asse: `hex (0 1 2 3 0 1 4 5)`, e `blockMesh` lo trasforma in un prisma. Lo spessore in z dei casi piani è arbitrario ma conviene renderlo dell'ordine delle celle, perché compare nel calcolo delle forze (che vanno divise per lo spessore) e nel rapporto di forma delle celle riportato da `checkMesh`.

3.5 Tre mesh con `blockMesh`

I tre casi di questo capitolo costruiscono le mesh che useremo in seguito e illustrano le voci del dizionario. Il canale con gradino (Fig. 3.2) usa tre blocchi, il multi-grading verso le pareti e un rapporto di espansione lungo la corrente; la mesh a O intorno al cilindro (Fig. 3.3) quattro blocchi con archi e variabili; il condotto assialsimmetrico un cuneo.

Dati dei casi `c02_canale`, `c02_cilindro`, `c02_tubo`

Canale: altezza 2 cm, lunghezza 10 cm, gradino di 0,5 cm a $x = 4$ cm; blocchi 40×30, 60×30, 60×12 celle; grading in y `((0.5 0.5 0.25) (0.5 0.5 4))`, in x nei blocchi a valle 3. Cilindro: $D = 1$, raggio esterno $R_o = 20D$ (10 raggi), 4 blocchi da 40×30 celle (radiale $\times$ azimutale), espansione radiale 60. Condotto: $R = 1$ cm, $L = 20$ cm, cuneo di 4°, 100×20 celle, grading radiale 0,2 (celle fitte alla parete).

c02_canale/system/blockMeshDict

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

convertToMeters 0.01;          // le coordinate sotto sono in centimetri

// canale alto H = 2 (y da -1 a 1) lungo 10; un gradino abbassa il fondo a y = -1.5
   per x > 4
vertices
(
    (0  -1   0)    // 0
    (4  -1   0)    // 1
    (10 -1   0)    // 2
    (0   1   0)    // 3
    (4   1   0)    // 4
    (10  1   0)    // 5
    (4  -1.5 0)    // 6
    (10 -1.5 0)    // 7

    (0  -1   1)    // 8   stessi punti, spostati di 1 in z
```

```
    (4  -1  1)    // 9
    (10 -1  1)    // 10
    (0   1  1)    // 11
    (4   1  1)    // 12
    (10  1  1)    // 13
    (4  -1.5 1)   // 14
    (10 -1.5 1)   // 15
);

blocks
(
    // blocco a monte: 40 x 30 celle; in y infittimento verso ENTRAMBE le pareti
    // (multi-grading: due meta', ciascuna con il 50% delle celle, rapporti 0.25 e 4)
    hex (0 1 4 3 8 9 12 11) (40 30 1)
        simpleGrading (1 ((0.5 0.5 0.25) (0.5 0.5 4)) 1)
    // blocco a valle, parte alta: 60 x 30; in x celle che si allargano verso l'uscita
    hex (1 2 5 4 9 10 13 12) (60 30 1)
        simpleGrading (3 ((0.5 0.5 0.25) (0.5 0.5 4)) 1)
    // blocco a valle, sotto il gradino: 60 x 12
    hex (6 7 2 1 14 15 10 9) (60 12 1)
        simpleGrading (3 1 1)
);

edges
(
);

boundary
(
    ingresso  { type patch; faces ((0 8 11 3)); }
    uscita    { type patch; faces ((2 5 13 10) (7 2 10 15)); }
    pareti
    {
        type wall;
        faces
        (
            (3 11 12 4) (4 12 13 5)       // parete superiore
            (0 1 9 8)                      // fondo a monte
            (1 6 14 9)                     // faccia verticale del gradino
            (6 7 15 14)                    // fondo a valle
        );
    }
    frontEretro
    {
        type empty;
        faces ((0 3 4 1) (1 4 5 2) (6 1 2 7) (8 9 12 11) (9 10 13 12) (14 15 10 9));
    }
);

mergePatchPairs
```

Listato 3.1: Il `blockMeshDict` del canale con gradino: tre blocchi, multi-grading verso le pareti, patch di ingresso, uscita, pareti e piani `empty`.


c02_cilindro/system/blockMeshDict

```
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

convertToMeters 1;

R   0.5;        // raggio del cilindro
Ro  10;         // raggio del contorno esterno (20 D)
c   0.70710678; // cos(45 gradi)

nR  40;         // celle in direzione radiale (per blocco)
nT  30;         // celle in direzione azimutale (per blocco: 4 blocchi = 120 celle sul cilindro)
gR  60;         // rapporto di espansione radiale (ultima cella / prima cella)

vertices
(
    // sul cilindro, a 45, 135, 225, 315 gradi (z = 0)
    ( #calc "$R*$c"   #calc "$R*$c"  0)   // 0
    ( #calc "-$R*$c"  #calc "$R*$c"  0)   // 1
    ( #calc "-$R*$c"  #calc "-$R*$c" 0)   // 2
    ( #calc "$R*$c"   #calc "-$R*$c" 0)   // 3
    // sul contorno esterno
    ( #calc "$Ro*$c"  #calc "$Ro*$c"  0)  // 4
    ( #calc "-$Ro*$c" #calc "$Ro*$c"  0)  // 5
    ( #calc "-$Ro*$c" #calc "-$Ro*$c" 0)  // 6
    ( #calc "$Ro*$c"  #calc "-$Ro*$c" 0)  // 7
    // stessi punti a z = 1
    ( #calc "$R*$c"   #calc "$R*$c"  1)   // 8
    ( #calc "-$R*$c"  #calc "$R*$c"  1)   // 9
    ( #calc "-$R*$c"  #calc "-$R*$c" 1)   // 10
    ( #calc "$R*$c"   #calc "-$R*$c" 1)   // 11
    ( #calc "$Ro*$c"  #calc "$Ro*$c"  1)  // 12
    ( #calc "-$Ro*$c" #calc "$Ro*$c"  1)  // 13
    ( #calc "-$Ro*$c" #calc "-$Ro*$c" 1)  // 14
    ( #calc "$Ro*$c"  #calc "-$Ro*$c" 1)  // 15
);

blocks
(
    // ogni blocco: prima direzione = radiale (dal cilindro verso l'esterno, con la gradazione), seconda = azimutale
    hex (0 4 5 1 8 12 13 9)   ($nR $nT 1) simpleGrading ($gR 1 1)   // sopra
    hex (1 5 6 2 9 13 14 10)  ($nR $nT 1) simpleGrading ($gR 1 1)   // sinistra
    hex (2 6 7 3 10 14 15 11) ($nR $nT 1) simpleGrading ($gR 1 1)   // sotto
    hex (3 7 4 0 11 15 12 8)  ($nR $nT 1) simpleGrading ($gR 1 1)   // destra
);

edges
(
    // archi sul cilindro: punto intermedio a 90, 180, 270, 0 gradi
    arc 0 1 (0 $R 0)       arc 1 2 (#neg $R 0 0)   arc 2 3 (0 #neg $R 0)   arc 3 0 ($R 0 0)
    arc 8 9 (0 $R 1)       arc 9 10 (#neg $R 0 1)  arc 10 11 (0 #neg $R 1) arc 11 8
```

```
    ($R 0 1)
    // archi sul contorno esterno
    arc 4 5 (0 $Ro 0)      arc 5 6 (#neg $Ro 0 0)  arc 6 7 (0 #neg $Ro 0)  arc 7 4
    ($Ro 0 0)
    arc 12 13 (0 $Ro 1)    arc 13 14 (#neg $Ro 0 1) arc 14 15 (0 #neg $Ro 1) arc 15
    12 ($Ro 0 1)
);

boundary
(
    cilindro
    {
        type wall;
        faces ((0 1 9 8) (1 2 10 9) (2 3 11 10) (3 0 8 11));
    }
    esterno
    {
        type patch;
        faces ((4 5 13 12) (5 6 14 13) (6 7 15 14) (7 4 12 15));
    }
    frontEretro
    {
        type empty;
        faces ((0 4 5 1) (1 5 6 2) (2 6 7 3) (3 7 4 0)
               (8 12 13 9) (9 13 14 10) (10 14 15 11) (11 15 12 8));
    }
);
```

Listato 3.2: La mesh a O intorno al cilindro: variabili, `#calc`, archi sul cilindro e sul contorno esterno, espansione radiale.

c02_tubo/system/blockMeshDict

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

convertToMeters 0.01;

L   20;
R   1;
// semiangolo del cuneo: 2 gradi -> y = R cos(2), z = +-R sin(2)
Ry  0.99939;
Rz  0.034899;

vertices
(
    (0  0 0)                              // 0  asse, ingresso
    ($L 0 0)                              // 1  asse, uscita
    ($L #calc "$R*$Ry" #calc "-$R*$Rz") // 2
    (0  #calc "$R*$Ry" #calc "-$R*$Rz") // 3
    ($L #calc "$R*$Ry" #calc "$R*$Rz")  // 4
    (0  #calc "$R*$Ry" #calc "$R*$Rz")  // 5
);

blocks
(
    // prisma a base triangolare: i vertici sull'asse sono ripetuti (hex degenere)
```

```
    hex (0 1 2 3 0 1 4 5) (100 20 1) simpleGrading (1 0.2 1)   // celle piu' fitte
    verso la parete
);

edges
(
    arc 2 4 ($L $R 0)
    arc 3 5 (0 $R 0)
);

boundary
(
    ingresso { type patch; faces ((0 3 5 0)); }
    uscita   { type patch; faces ((1 2 4 1)); }
    parete   { type wall;  faces ((2 3 5 4)); }
    fronte   { type wedge; faces ((0 1 2 3)); }
    retro    { type wedge; faces ((0 5 4 1)); }
    asse     { type empty; faces ((0 1 1 0)); }
);

// ************************************************************************* //
```

Listato 3.3: Il condotto assialsimmetrico: blocco a cuneo con vertici coincidenti sull'asse, patch `wedge` e asse `empty`.

Risultati di `checkMesh` sulle tre mesh

- `c02_canale`: 3720 celle, 15022 facce; non ortogonalità massima 0,00° (media 0,00°), skewness massima 0,000, rapporto di forma massimo 5,4; `checkMesh` riporta 0,00°, 0,00°, 0,000: Mesh OK.
- `c02_cilindro`: 4800 celle, 19320 facce; non ortogonalità massima 0,00° (media 0,00°), skewness massima 0,038, rapporto di forma massimo 1,9; `checkMesh` riporta 0,00°, 0,00°, 0,038: Mesh OK.
- `c02_tubo`: 2000 celle, 8020 facce; non ortogonalità massima 0,00° (media 0,00°), skewness massima 0,332, rapporto di forma massimo 10,2; `checkMesh` riporta 0,00°, 0,00°, 0,332: Mesh OK.

I valori di `foamcase.py` e di `checkMesh` coincidono alla terza cifra: le definizioni sono le stesse. La mesh a O è ortogonale ovunque (le facce radiali e quelle circonferenziali sono perpendicolari anche alle giunzioni dei blocchi) e la sua skewness residua viene dagli archi; il canale è cartesiano; il cuneo ha le facce degeneri sull'asse che `checkMesh` riconosce come tali (*wedge* e *empty* corretti).

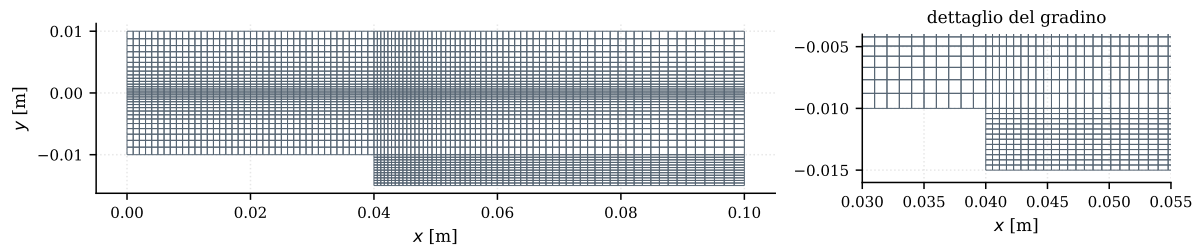


Figura 3.2: La mesh del canale con gradino: infittimento verso entrambe le pareti nel blocco a monte, celle crescenti verso l'uscita; a destra, il dettaglio dello spigolo del gradino.

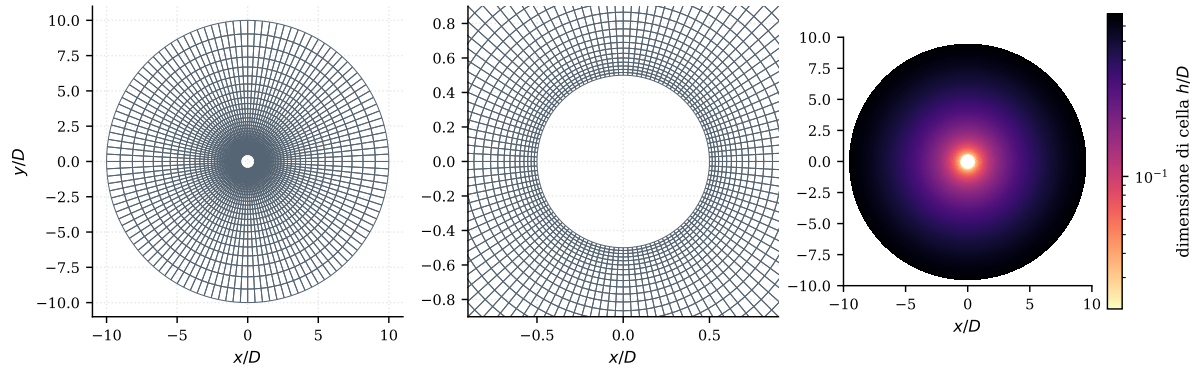


Figura 3.3: La mesh a O del cilindro: vista d'insieme con il contorno esterno circolare e dettaglio della parete con le prime celle, alte $y_1 \approx 0{,}016\,D$; a destra la dimensione delle celle (radice del volume per unità di spessore), che cresce geometricamente dal cilindro al contorno esterno: la mesh polare è ortogonale ovunque, anche dove i blocchi si incontrano a 45°.

3.6 `snappyHexMesh`: geometrie arbitrarie

Quando la geometria è una superficie triangolata (STL, OBJ) esportata da un CAD, `snappyHexMesh` genera una mesh prevalentemente esaedrica in tre fasi, ciascuna attivabile nel dizionario `system/snappyHexMeshDict`:

1. **castellatedMesh**: parte da una mesh di fondo esaedrica (di solito un parallelepipedo di `blockMesh`), raffina le celle che intersecano la superficie (e quelle in regioni indicate) dimezzandole `level` volte, e rimuove le celle dal lato della superficie che non contiene il punto `locationInMesh`. Il risultato è una mesh a gradini (*castellata*) che approssima la geometria;
2. **snap**: sposta i vertici della mesh castellata sulla superficie, iterando con controlli di qualità, fino a farla aderire alla geometria; le celle tagliate diventano poliedri;
3. **addLayers**: inserisce strati di celle prismatiche sottili lungo le superfici indicate, per risolvere lo strato limite; è la fase più delicata, e il numero di strati effettivamente ottenuti va sempre verificato nel log.

La qualità finale è controllata dal sotto-dizionario `meshQualityControls`, con le stesse soglie di `checkMesh`: lo snap e gli strati vengono ridotti dove violerebbero i limiti. Il Cap. 15 applica la procedura a un caso tridimensionale completo, con la cattura degli spigoli vivi (`surfaceFeatures`) e il calcolo parallelo.

3.7 Approfondimenti

3.7.1 L'indirizzamento *ldu* e la matrice

Poiché i coefficienti a_F dell'equazione algebrica sono associati alle facce, OpenFOAM memorizza la matrice del sistema in tre vettori: la diagonale (una voce per cella), i coefficienti *upper* (una voce per faccia interna: il coefficiente con cui il neighbour entra nell'equazione dell'owner) e *lower* (il coefficiente con cui l'owner entra in quella del neighbour), con le liste `owner` e `neighbour` come indirizzi di riga e colonna (Sez. 7.2.4 del testo). La matrice è simmetrica se upper e lower coincidono (il laplaciano della pressione: solutori `PCG`, `GAMG`), asimmetrica altrimenti (convezione: `PBiCGStab`, `smoothSolver`). Questa struttura spiega perché i solutori

lineari del Cap. 7 siano scritti in modo da scorrere le facce e perché il rinumeramento delle celle (`renumberMesh`) acceleri il calcolo: avvicina gli indici di owner e neighbour e migliora l'uso della cache.

3.7.2 Altri generatori e la conversione

Molte mesh arrivano da generatori esterni: Gmsh (libero, `gmshToFoam`), Salome, Pointwise, ANSYS Meshing (`fluentMeshToFoam`), e lo stesso `cfMesh` integrato nella distribuzione ESI. La conversione produce la cartella `polyMesh`; dopo di essa `checkMesh` è obbligatorio, e spesso serve `createPatch` per rinominare e ritipizzare i patch (i convertitori producono `patch` generici dove servono `wall` o `empty`). Per le mesh piane generate da programmi 2D, `extrudeMesh` crea lo spessore unitario in z con i patch `empty`.

3.7.3 Mesh dinamiche e raffinamento adattivo

OpenFOAM supporta mesh in movimento (`dynamicMeshDict`: moto rigido di un corpo, deformazione, zone rotanti) e il raffinamento adattivo esaedrico durante il calcolo, basato su un campo indicatore (il gradiente di una frazione di fase, la vorticità). Sono argomenti oltre lo scopo di questo modulo, ma la rappresentazione per facce è ciò che li rende possibili senza cambiare i solutori: una cella raffinata è semplicemente un insieme di facce in più.

Parte II

Discretizzazione: `fvSchemes` e `fvSolution`

Capitolo 4

Gradienti e termine diffusivo

4.1 Il termine diffusivo su una mesh ortogonale

Il termine diffusivo della (2.6), $\sum_f \Gamma_f (\nabla\phi)_f \cdot \mathbf{S}_f$, richiede la *derivata normale* di ϕ nel centroide di ogni faccia. Su una mesh ortogonale (Fig. 2.1, con $\mathbf{S}_f$ parallelo a $\mathbf{d}_{CF}$) la derivata lungo la normale coincide con quella lungo la congiungente dei centroidi, e la differenza centrata del secondo ordine è immediata:

$$(\nabla\phi)_f \cdot \mathbf{S}_f = |\mathbf{S}_f| \frac{\phi_F - \phi_C}{|\mathbf{d}_{CF}|} = \mathrm{gDiff}_f \, (\phi_F - \phi_C), \qquad \mathrm{gDiff}_f = \frac{|\mathbf{S}_f|}{|\mathbf{d}_{CF}|}. \tag{4.1}$$

Il flusso è lineare in ϕ_C e ϕ_F, quindi *interamente implicito*: $\mathrm{FluxC}_f = -\Gamma_f \mathrm{gDiff}_f$, $\mathrm{FluxF}_f = \Gamma_f \mathrm{gDiff}_f$ nella (2.7) (con il segno della diffusione portato a primo membro). I coefficienti $a_F = -\Gamma_f \mathrm{gDiff}_f$ sono tutti negativi e $a_C = -\sum_F a_F$: la matrice è simmetrica, a dominanza diagonale, e soddisfa la regola dei segni opposti e quella della somma nulla (Sez. 8.2 del testo di riferimento): la soluzione è limitata e il sistema si risolve con il gradiente coniugato. È il caso più favorevole che esista, e corrisponde esattamente al laplaciano a cinque punti del Modulo 2.

Condizioni al contorno nel termine diffusivo

Su una faccia di contorno b la cella F non esiste; la derivata normale si costruisce con il valore *sulla faccia*:

- **Dirichlet** (`fixedValue`): ϕ_b noto, flusso $\Gamma_b |\mathbf{S}_b| (\phi_b - \phi_C)/d_{Cb}$ con d_{Cb} la distanza normale dal centroide alla faccia; contribuisce ad a_C (con $\Gamma_b |\mathbf{S}_b|/d_{Cb}$) e a b_C (con lo stesso coefficiente per ϕ_b);
- **Neumann** (`fixedGradient`, `zeroGradient`): il flusso $\Gamma_b q_b |\mathbf{S}_b|$ è noto e va tutto nel termine noto; con `zeroGradient` è nullo e la faccia semplicemente non contribuisce;
- **mista** (`mixed`, `robin`): $-\Gamma \partial\phi/\partial n = h(\phi_b - \phi_\infty)$, si elimina ϕ_b e si ottiene un coefficiente equivalente $\Gamma_b h |\mathbf{S}_b| / (\Gamma_b + h d_{Cb})$;
- **simmetria** (`symmetry`): flusso diffusivo normale nullo per uno scalare; per un vettore la componente normale è nulla e quella tangenziale ha derivata nulla.

In OpenFOAM queste regole sono dentro le classi delle condizioni al contorno (`fvPatchField`), che forniscono al termine `fvm::laplacian` i due coefficienti `gradientInternalCoeffs` e `gradientBoundaryCoeffs`: è il meccanismo che rende una condizione al contorno indipendente dal solutore (Cap. 11).

4.2 Il calcolo del gradiente

Il gradiente nelle celle serve per tre scopi: valutare termini espliciti (il $-\nabla p$ della quantità di moto), correggere la non ortogonalità del laplaciano, e costruire gli schemi convettivi di ordine superiore (Cap. 5). Il testo di riferimento (Cap. 9) presenta i due metodi che OpenFOAM offre in `gradSchemes`.

Green–Gauss e minimi quadrati

Green–Gauss (`Gauss linear`). Dal teorema del gradiente, $\int_V \nabla\phi \, \mathrm{d}V = \oint \phi \, \mathrm{d}\mathbf{S}$, la media del gradiente sulla cella è

$$(\nabla\phi)_C = \frac{1}{V_C} \sum_f \phi_f \mathbf{S}_f, \tag{4.2}$$

con ϕ_f interpolato linearmente fra C ed F (`linear`). Su mesh regolari è del secondo ordine; sulle mesh con skewness decade al primo e, se la distorsione è irregolare, può non convergere affatto (l'esperimento di questo capitolo lo mostra), perché il valore interpolato non è nel centroide della faccia e l'errore non si cancella fra facce opposte. La (4.2) è esattamente il codice `gaussGrad::gradf` riportato nel Cap. 7 del testo: un ciclo sulle facce interne che accumula $\phi_f \mathbf{S}_f$ nell'owner e $-\phi_f \mathbf{S}_f$ nel neighbour, uno sulle facce di contorno, e la divisione per il volume.

Minimi quadrati (`leastSquares`). Si cerca il gradiente che meglio riproduce, nel senso dei minimi quadrati pesati, le differenze $\phi_F - \phi_C \approx (\nabla\phi)_C \cdot \mathbf{d}_{CF}$ con tutti i vicini F: la soluzione è $(\nabla\phi)_C = \mathbf{G}^{-1} \sum_F w_F (\phi_F - \phi_C) \mathbf{d}_{CF}$, con $\mathbf{G} = \sum_F w_F \mathbf{d}_{CF} \mathbf{d}_{CF}$ una matrice 3×3 che dipende solo dalla geometria e si calcola una volta. Riproduce esattamente i campi lineari su qualunque mesh, e quindi converge anche dove Green–Gauss non converge; è però più sensibile al rumore su mesh con rapporti di forma alti, dove $\mathbf{G}$ è mal condizionata, ed è per questo che nei casi con strati limite si usa di solito `Gauss linear` (sulle celle di strato, quasi ortogonali, i due coincidono).

Limitatori (`cellLimited Gauss linear 1`, `faceLimited`, `cellMDLimited`). Negli schemi convettivi di ordine superiore un gradiente troppo grande produce valori di faccia fuori dall'intervallo dei vicini; il limitatore riduce il gradiente (moltiplicandolo per $\psi \in [0,1]$) quanto basta perché le estrapolazioni $\phi_C + (\nabla\phi)_C \cdot (\mathbf{x}_f - \mathbf{x}_C)$ restino comprese fra il minimo e il massimo dei valori vicini. `cellLimited` applica un unico ψ per cella, `cellMDLimited` lo fa componente per componente; il coefficiente finale (1) attiva il limitatore per intero, 0 lo disattiva.

4.3 Mesh non ortogonali: la correzione

Scomposizione del vettore area

Quando $\mathbf{S}_f$ non è parallelo a $\mathbf{d}_{CF}$ (Fig. 2.1) la derivata lungo $\mathbf{d}_{CF}$ non basta. Si scompone $\mathbf{S}_f = \mathbf{E}_f + \mathbf{T}_f$ con $\mathbf{E}_f$ parallelo a $\mathbf{d}_{CF}$ e $\mathbf{T}_f$ nel piano della faccia; il flusso diffusivo diventa

$$(\nabla\phi)_f \cdot \mathbf{S}_f = \underbrace{|\mathbf{E}_f| \frac{\phi_F - \phi_C}{|\mathbf{d}_{CF}|}}_{\text{implicito, ortogonale}} + \underbrace{\overline{(\nabla\phi)}_f \cdot \mathbf{T}_f}_{\text{esplicito, correzione}}, \tag{4.3}$$

dove il gradiente di faccia della correzione è interpolato dai gradienti di cella (Green–Gauss o minimi quadrati) e valutato con i valori dell'iterazione precedente. La scelta di $\mathbf{E}_f$ non è unica (Sez. 8.6 del testo): con la *correzione minima* $\mathbf{E}_f = (\mathbf{d}_{CF} \cdot \mathbf{S}_f/|\mathbf{d}_{CF}|^2)\mathbf{d}_{CF}$ (proiezione), con quella *ortogonale* $|\mathbf{E}_f| = |\mathbf{S}_f|$, con l'approccio *sovrarilassato* $|\mathbf{E}_f| = |\mathbf{S}_f|^2/(\mathbf{d}_{CF} \cdot \mathbf{S}_f) = |\mathbf{S}_f|/\cos\theta_f$, che è la scelta di OpenFOAM (`corrected`): la parte implicita è la più grande possibile, il che favorisce la convergenza iterativa, e la correzione esplicita si annulla sulle mesh ortogonali. Poiché la correzione dipende dal gradiente, che dipende dalla soluzione, il termine va *iterato*: sono i `nNonOrthogonalCorrectors` di `fvSolution`, che ripetono la soluzione dell'equazione aggiornando la correzione. Nei casi stazionari le iterazioni esterne lo fanno da sé; nei transitori con mesh non ortogonali 1–2 correttori per passo sono necessari.

Le voci di `laplacianSchemes` e `snGradSchemes`

`laplacianSchemes { default Gauss <interpolazione> <snGrad>; }`: `Gauss` è l'unica discretizzazione; la seconda parola dice come interpolare Γ sulle facce (`linear` quasi sempre; `harmonic` per diffusività discontinue, Sez. 8.4 del testo); la terza è lo schema per la derivata normale:

- `orthogonal`: la (4.1) senza correzione, con $|\mathbf{S}_f|/|\mathbf{d}_{CF}|$; corretto solo su mesh ortogonali;
- `uncorrected`: la parte implicita sovrarilassata $|\mathbf{S}_f|/(|\mathbf{d}_{CF}|\cos\theta_f)$ senza correzione esplicita; stabile, ma del primo ordine su mesh non ortogonali;
- `corrected`: la (4.3) completa; secondo ordine, la scelta normale fino a $\theta_f \approx 60°$–$70°$;
- `limited` ψ: correzione esplicita moltiplicata per $\psi \in [0,1]$ (`limited 0.5`, `limited 0.333`) oppure limitata in modo che non superi la parte implicita (`limited corrected 0.33`): un compromesso fra accuratezza e stabilità per le mesh molto distorte, dove la correzione esplicita completa fa divergere.

`snGradSchemes` governa le derivate normali esplicite (`fvc::snGrad`) con le stesse parole chiave, e di norma coincide con la scelta del laplaciano.

4.4 Esperimento: conduzione su mesh ortogonali e distorte

L'esperimento misura quanto ciascuna di queste scelte costa in accuratezza. L'equazione di Laplace $\nabla^2 T = 0$ sul quadrato unitario con $T = \sin\pi x \sinh\pi y/\sinh\pi$ imposto sul contorno ha questa stessa funzione come soluzione esatta; la si risolve con `laplacianFoam` (il solutore stazionario della distribuzione, selezionato con `ddtSchemes steadyState`) su una famiglia di mesh ortogonali $N \times N$ e sulla stessa famiglia *distorta* in parallelogramma (vertici superiori spostati di 0,5 e di 1 a destra, che danno non ortogonalità uniforme di 26,6° e 45° ma skewness interna nulla) e su una terza famiglia in cui i punti interni del quadrato sono spostati a caso di $\pm 0{,}3h$ (non ortogonale *e* skew in modo irregolare, come una mesh reale), con gli schemi `orthogonal`, `uncorrected`, `corrected` e `limited 0.5`, e si misura l'errore massimo rispetto alla soluzione esatta e l'ordine di convergenza. Il valore al contorno non è uniforme: lo scrive uno script Python, `genera.py`, che dopo `blockMesh` legge i centroidi delle facce di contorno con `foamcase.py` e produce il file `0/T` con `nonuniform List<scalar>`. In

un secondo esperimento si assegna il campo esatto in tutte le celle e si calcola il gradiente con `Gauss linear`, `leastSquares` e `Gauss pointLinear` (`foamPostProcess -func "grad(T)"`), confrontandolo con quello analitico.

Dati del caso `c04_diffusione`

Quadrato $[0,1]^2$ (nel parallelogramma, $x \to x + sy$ con $s = 0, 0{,}5, 1$; nella mesh perturbata, spostamenti casuali uniformi in $[-0{,}3h, 0{,}3h]$ con seme fisso); $N = 10, 20, 40, 80$; $D_T = 1$; `laplacianFoam` con `ddtSchemes steadyState`, `nNonOrthogonalCorrectors 20`, tolleranza 10^{-10}; solutore `PCG/DIC`; errore $\max_C |T_C - T(\mathbf{x}_C)|$.

c04_diffusione.zip

```
}

gradSchemes
{
    default         Gauss linear;         // gradiente per la correzione non
    ortogonale e per grad(T)
}

divSchemes
{
    default         none;
}

laplacianSchemes
{
    default         none;
    laplacian(DT,T) Gauss linear corrected;   // <interpolazione di DT> <schema
    della derivata normale>
}

interpolationSchemes
{
    default         linear;
}

snGradSchemes
{
    default         corrected;
}
```

c04_diffusione/system/fvSchemes

Listato 4.1: Gli schemi del caso di conduzione: lo schema del laplaciano è sostituito dallo script per ogni variante.

```
"""
genera.py -- caso c04_diffusione: conduzione stazionaria su mesh ortogonali e
    distorte.

Per ogni mesh (quadrato S = 0, parallelogrammi S = 0,5 e 1, quadrato con i punti
    interni spostati a caso
di +-0,3 h: non ortogonale E skew) e ogni N (10, 20, 40, 80):
  1. scrive system/blockMeshDict dal modello e lancia blockMesh (poi, se richiesto,
    perturba i punti);
  2. legge i centroidi delle facce del patch 'contorno' con foamcase.py e scrive
    0/T con il
     valore ESATTO T = sin(pi x) sinh(pi y)/sinh(pi) su ogni faccia (nonuniform
    List<scalar>);
```

c04_diffusione/genera.py

```
  3. per ogni schema del laplaciano (orthogonal, uncorrected, corrected, limited 0.5) sostituisce la
     voce in fvSchemes, lancia laplacianFoam e misura l'errore massimo nelle celle;
  4. per il gradiente: assegna T esatto nelle celle e calcola grad(T) con foamPostProcess con tre
     gradSchemes, misurando l'errore rispetto al gradiente analitico.
Scrive risultati.json, letto da codice/fig04_diffusione.py.   Uso: python3 genera.py
"""
import os
import re
import sys
import json
import shutil
import subprocess
import numpy as np

QUI = os.path.dirname(os.path.abspath(__file__))
for _d in ("codice", "codici"):                                   # gli strumenti Python: codice/ nel progetto, codici/ nell'archivio pubblicato
    sys.path.insert(0, os.path.join(QUI, "..", "..", _d))
from foamcase import Mesh, Caso                                   # noqa: E402


def esatta(x, y):
    return np.sin(np.pi * x) * np.sinh(np.pi * y) / np.sinh(np.pi)


def grad_esatto(x, y):
    gx = np.pi * np.cos(np.pi * x) * np.sinh(np.pi * y) / np.sinh(np.pi)
    gy = np.pi * np.sin(np.pi * x) * np.cosh(np.pi * y) / np.sinh(np.pi)
    return np.column_stack([gx, gy, np.zeros_like(gx)])


def esegui(cmd, log):
    with open(os.path.join(QUI, log), "w") as fp:
        r = subprocess.run(cmd, cwd=QUI, stdout=fp, stderr=subprocess.STDOUT, shell=True)
    if r.returncode != 0:
        raise RuntimeError(f"comando fallito: {cmd} (vedi {log})")


def scrivi_mesh(S, N):
    t = open(os.path.join(QUI, "system", "blockMeshDict.template")).read()
    t = t.replace("__X2__", f"{1 + S:g}").replace("__X3__", f"{S:g}").replace("__N__", str(N))
    open(os.path.join(QUI, "system", "blockMeshDict"), "w").write(t)
    esegui("blockMesh", "log.blockMesh")


def perturba(ampiezza=0.3, seme=1):
    """Sposta a caso i punti interni della mesh (quadrato unitario) di +-ampiezza*h in x e y, uguale sui due
    piani z: la mesh resta valida ma diventa non ortogonale e skew in modo irregolare, come una mesh reale."""
    m = Mesh(QUI)
    P = m.points.copy()
    h = 1.0 / round(np.sqrt(m.nCells))
    interni = (P[:, 0] > 1e-9) & (P[:, 0] < 1 - 1e-9) & (P[:, 1] > 1e-9) & (P[:, 1] < 1 - 1e-9)
    rng = np.random.default_rng(seme)
```

```
    # stesso spostamento per i punti sovrapposti dei due piani: si indicizza per (x, y)
    chiavi = {}
    for i in np.where(interni)[0]:
        k = (round(P[i, 0], 9), round(P[i, 1], 9))
        if k not in chiavi:
            chiavi[k] = rng.uniform(-ampiezza * h, ampiezza * h, 2)
        P[i, :2] += chiavi[k]
    testo = open(os.path.join(QUI, "constant", "polyMesh", "points")).read()
    i0 = testo.index("(\n(")
    nuovo = testo[:i0] + "(\n" + "\n".join(f"({x:.12g} {y:.12g} {z:.12g})" for x, y, z in P) + "\n)\n"
    open(os.path.join(QUI, "constant", "polyMesh", "points"), "w").write(nuovo)


def scrivi_T(mesh, interno=None):
    """0/T con il valore esatto sulle facce del contorno e, se richiesto, nelle celle."""
    idx = mesh.facce_patch("contorno")
    Tb = esatta(mesh.Cf[idx, 0], mesh.Cf[idx, 1])
    if interno is None:
        campo = "uniform 0"
    else:
        campo = f"nonuniform List<scalar> {len(interno)}\n(\n" + "\n".join(f"{v:.12g}" for v in interno) + "\n)"
    testo = f"""FoamFile
{{
    version     2.0;
    format      ascii;
    class       volScalarField;
    object      T;
}}
dimensions      [0 0 0 1 0 0 0];

internalField   {campo};

boundaryField
{{
    contorno
    {{
        type            fixedValue;
        value           nonuniform List<scalar> {len(Tb)}
(
{chr(10).join(f"{v:.12g}" for v in Tb)}
);
    }}
    frontEretro
    {{
        type            empty;
    }}
}}
"""
    open(os.path.join(QUI, "0", "T"), "w").write(testo)


def imposta_schema(voce, valore, blocco=None):
    """Sostituisce il valore di una voce di fvSchemes (se 'blocco' e' dato, solo dentro quel sotto-dizionario)."""
    p = os.path.join(QUI, "system", "fvSchemes")
    s = open(p).read()
```

```
    if blocco:
        i = s.index(blocco); j = s.index("}", i)
        s = s[:i] + re.sub(rf"({voce}\s+)[^;]*;", rf"\g<1>{valore};", s[i:j]) + s[j:]
    else:
        s = re.sub(rf"({voce}\s+)[^;]*;", rf"\g<1>{valore};", s)
    open(p, "w").write(s)


def pulisci():
    for d in os.listdir(QUI):
        if re.fullmatch(r"[1-9][0-9]*(\.[0-9]+)?", d) or d == "0.orig":
            shutil.rmtree(os.path.join(QUI, d), ignore_errors=True)
    shutil.rmtree(os.path.join(QUI, "postProcessing"), ignore_errors=True)


MESH = {"quadrato": (0.0, False), "S=0.5": (0.5, False), "S=1": (1.0, False), "perturbata": (0.0, True)}


def main():
    risultati = {"diffusione": [], "gradiente": []}
    schemi = {"orthogonal": "Gauss linear orthogonal", "uncorrected": "Gauss linear uncorrected",
              "corrected": "Gauss linear corrected", "limited 0.5": "Gauss linear limited 0.5"}
    for nome_mesh, (S, pert) in MESH.items():
        for N in (10, 20, 40, 80):
            pulisci()
            scrivi_mesh(S, N)
            if pert:
                perturba()
            mesh = Mesh(QUI)
            no = mesh.non_ortogonalita().max(); sk = mesh.skewness().max()
            scrivi_T(mesh)
            for nome, valore in schemi.items():
                imposta_schema("laplacian\\(DT,T\\)", valore, "laplacianSchemes")
                pulisci()
                esegui("laplacianFoam", f"log.laplacianFoam")
                c = Caso(QUI)
                T = c.campo("T")
                err = np.max(np.abs(T - esatta(mesh.C[:, 0], mesh.C[:, 1])))
                risultati["diffusione"].append({"mesh": nome_mesh, "S": S, "N": N, "h": 1.0 / N, "nonorto": float(no), "skew": float(sk),
                                                "schema": nome, "errore": float(err)})
                print(f"{nome_mesh:10s} N = {N:3d}  theta = {no:5.1f}  skew = {sk:5.3f}  {nome:12s}  errore max {err:.3e}")
            # --- gradiente di un campo esatto ---
            if N in (20, 40, 80) and nome_mesh != "S=0.5":
                pulisci()
                scrivi_T(mesh, esatta(mesh.C[:, 0], mesh.C[:, 1]))
                for gs in ("Gauss linear", "leastSquares", "Gauss pointLinear"):
                    imposta_schema("default", gs, "gradSchemes")
                    esegui('foamPostProcess -func "grad(T)" -time 0', "log.foamPostProcess")
                    g = Caso(QUI).campo("grad(T)", tempo="0")
                    ge = grad_esatto(mesh.C[:, 0], mesh.C[:, 1])
                    interni = np.ones(mesh.nCells, bool)
                    # si escludono le celle adiacenti al contorno, dove il
```

```
    gradiente usa il valore di faccia
168                 interni[mesh.owner[mesh.facce_patch("contorno")]] = False
169                 err = np.max(np.linalg.norm(g - ge, axis=1)[interni])
170                 risultati["gradiente"].append({"mesh": nome_mesh, "S": S, "N":
    N, "h": 1.0 / N, "schema": gs, "errore": float(err)})
171                 print(f"      gradiente {gs:18s} errore max (celle interne)
    {err:.3e}")
172             imposta_schema("default", "Gauss linear", "gradSchemes")
173     imposta_schema("laplacian\\(DT,T\\)", "Gauss linear corrected",
    "laplacianSchemes")
174     json.dump(risultati, open(os.path.join(QUI, "risultati.json"), "w"), indent=1)
175     # si lascia nel caso la mesh perturbata con N = 40, risolta con 'corrected',
    per la figura
176     pulisci(); scrivi_mesh(0.0, 40); perturba(); scrivi_T(Mesh(QUI));
    esegui("laplacianFoam", "log.laplacianFoam")
177
178
179 if __name__ == "__main__":
180     main()
```

Listato 4.2: Lo script che genera le mesh (ortogonali e distorte), scrive la condizione al contorno non uniforme dalla soluzione esatta e lancia le varianti.

Risultati del caso `c04_diffusione`

- Mesh cartesiana: con $N = 10, 20, 40, 80$ l'errore massimo è $9{,}10 \times 10^{-3}$, $2{,}68 \times 10^{-3}$, $7{,}20 \times 10^{-4}$, $1{,}86 \times 10^{-4}$: ordine osservato 1,87; i quattro schemi del laplaciano coincidono perché il termine di correzione è nullo.
- Parallelogramma a 45° ($\theta = 45°$ su tutte le facce interne, skewness interna nulla), $N = 40$: `orthogonal` $1{,}37 \times 10^{-1}$, `uncorrected` $1{,}41 \times 10^{-1}$, `corrected` $3{,}21 \times 10^{-2}$, `limited 0.5` $8{,}93 \times 10^{-2}$. Senza correzione l'errore non diminuisce con h (ordine 0,01): lo schema converge alla soluzione di un'equazione diversa; con la correzione l'errore scende con ordine 0,72, limitato dal trattamento del primo ordine delle facce di contorno, tutte oblique.
- Mesh perturbata ($\theta_{\max} = 45°$, skewness massima 0,57, $N = 40$): `uncorrected` $1{,}26 \times 10^{-2}$, `corrected` $5{,}47 \times 10^{-3}$ (ordine 0,91), `limited 0.5` $9{,}73 \times 10^{-3}$: la correzione dimezza l'errore e recupera la convergenza, il limitatore sta a metà.
- Gradiente di un campo esatto (celle interne). Mesh cartesiana: `Gauss linear` e `leastSquares` coincidono (ordine 1,87), `Gauss pointLinear` è anomalo e converge con ordine 3,9 (la sua correzione esplicita cancella il termine d'errore dominante sulle griglie uniformi). Parallelogramma: ancora coincidenti, ordine 1,87, perché la retta CF passa per il centro della faccia. Mesh perturbata: `Gauss linear` *non converge* (errore 1,26 a $N = 80$, ordine $-0{,}06$), `leastSquares` converge (ordine 0,65, errore 0,032 a $N = 80$): è la ragione per cui sulle mesh non strutturate si preferiscono i minimi quadrati.

Come si legge il risultato nei casi reali

Nelle mesh di `snappyHexMesh` la non ortogonalità è concentrata negli strati e nelle zone di transizione, con massimi di 50°–65° e medie di pochi gradi: `corrected` con un correttore è la scelta standard; se il calcolo diverge nelle prime iterazioni si passa a

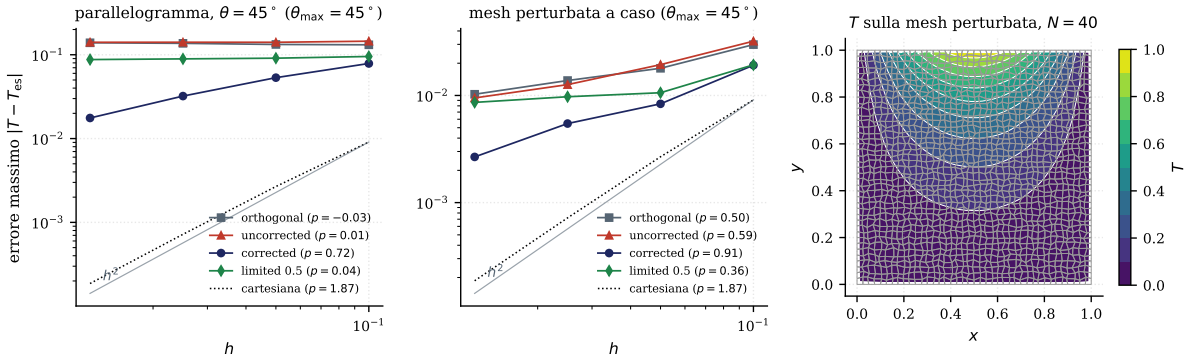


Figura 4.1: Conduzione stazionaria: errore massimo in funzione di h con i quattro schemi del laplaciano sul parallelogramma a 45° (a sinistra) e sulla mesh perturbata a caso (al centro), con la mesh cartesiana come riferimento; a destra la mesh perturbata con il campo T calcolato.

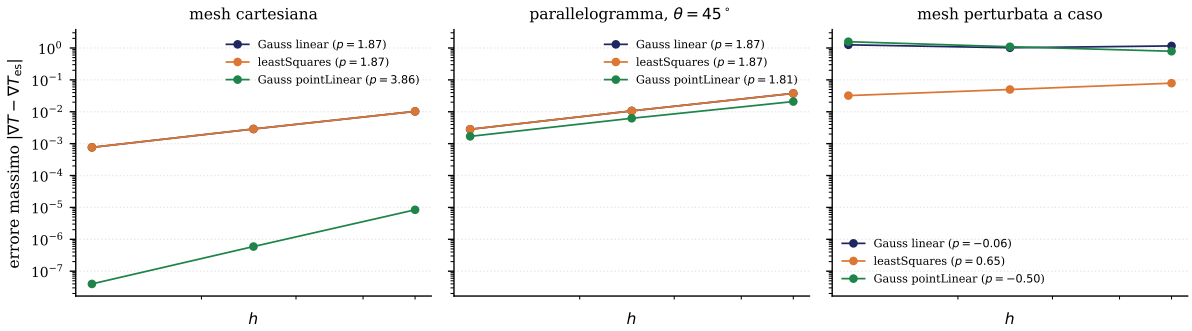


Figura 4.2: Errore del gradiente calcolato con `Gauss linear`, `leastSquares` e `Gauss pointLinear` sulla mesh cartesiana, sul parallelogramma a 45° e sulla mesh perturbata a caso: sul parallelogramma (skewness nulla) i tre schemi sono equivalenti, sulla mesh perturbata solo i minimi quadrati convergono.

`limited 0.5` (o `limited corrected 0.33`, che è quanto i tutorial della Foundation usano per le mesh difficili) accettando una perdita di accuratezza locale. La mesh a C del profilo (Cap. 13) ha 55° al bordo d'attacco e `corrected` funziona senza problemi; la mesh a O del cilindro è ortogonale ovunque tranne alle giunzioni fra blocchi.

4.5 Approfondimenti

4.5.1 Diffusività discontinua e interpolazione armonica

Se Γ salta attraverso una faccia (due materiali, o la viscosità effettiva che varia di ordini di grandezza fra parete e corrente turbolenta) l'interpolazione lineare $\Gamma_f = g_f\Gamma_F + (1-g_f)\Gamma_C$ sovrastima il flusso; la continuità del flusso diffusivo attraverso la faccia impone invece la *media armonica* pesata con le distanze (Sez. 8.4 del testo), $\Gamma_f = \Gamma_C\Gamma_F/(g_f\Gamma_C + (1-g_f)\Gamma_F)$, disponibile come `Gauss harmonic corrected`. Per ν_{eff} nelle equazioni RANS la differenza è piccola perché ν_t varia con continuità; conta nei problemi di conduzione fra solidi diversi.

4.5.2 Diffusione anisotropa

Se la diffusività è un tensore $\mathbf{\Gamma}$ (conduzione in materiali orientati, modelli di turbolenza non lineari) il flusso è $(\mathbf{\Gamma}\nabla\phi)_f \cdot \mathbf{S}_f$ e la scomposizione della (4.3) si applica al vettore $\mathbf{\Gamma}^{\mathrm{T}}\mathbf{S}_f$ al posto di $\mathbf{S}_f$: anche una mesh ortogonale produce allora un termine di correzione esplicito, perché $\mathbf{\Gamma}^{\mathrm{T}}\mathbf{S}_f$ non è parallelo a $\mathbf{d}_{CF}$ (Sez. 8.8 del testo). OpenFOAM lo gestisce con `fvm::laplacian(volTensorField, phi)` e lo stesso `corrected`.

4.5.3 Il gradiente alle pareti e lo sforzo

La derivata normale della velocità alla parete, da cui discendono lo sforzo tangenziale, la resistenza di attrito e lo y^+, è calcolata da OpenFOAM con la stessa formula del contorno di Dirichlet: $(\partial\mathbf{U}/\partial n)_b = (\mathbf{U}_b - \mathbf{U}_C)/d_{Cb}$, del primo ordine, con $\mathbf{U}_C$ nella prima cella. La sua accuratezza dipende quindi solo dalla dimensione della prima cella: è il motivo per cui le mesh per gli strati limite hanno y_1 piccolissimo (Cap. 9) e per cui la resistenza di attrito è la grandezza più sensibile alla mesh (Cap. 13). La funzione `forze` di `foamcase.py` usa esattamente questa formula.

Capitolo 5

Il termine convettivo e gli schemi ad alta risoluzione

5.1 Il problema

Il termine convettivo $\sum_f \dot{m}_f \phi_f$ richiede il valore di ϕ *sulla faccia*, e la scelta di come ricavarlo dai valori di cella è la decisione più importante di tutta la discretizzazione: da essa dipendono la diffusione numerica, le oscillazioni spurie, la stabilità del processo iterativo. La voce `divSchemes` di `fvSchemes` è dove si prende, ed è la voce che più spesso distingue un calcolo credibile da uno che non lo è. Il testo di riferimento vi dedica due capitoli (11 e 12), che questo capitolo segue: prima il problema monodimensionale, dove tutto si calcola a mano, poi la generalizzazione alle mesh non strutturate e gli schemi limitati.

Convezione e diffusione in una dimensione

Per $\mathrm{d}(\rho u \phi)/\mathrm{d}x = \mathrm{d}(\Gamma\, \mathrm{d}\phi/\mathrm{d}x)/\mathrm{d}x$ su $[0, L]$ con $\phi(0) = \phi_0$ e $\phi(L) = \phi_L$ la soluzione esatta è

$$\frac{\phi(x) - \phi_0}{\phi_L - \phi_0} = \frac{e^{Pe\, x/L} - 1}{e^{Pe} - 1}, \qquad Pe = \frac{\rho u L}{\Gamma}, \tag{5.1}$$

dove il **numero di Péclet** misura il rapporto fra trasporto convettivo e diffusivo. Per $Pe \to 0$ il profilo è lineare (diffusione pura); per Pe grande è piatto quasi ovunque e varia bruscamente vicino all'uscita: il valore di ϕ in un punto è determinato da ciò che sta *a monte*. Nella cella C con vicine W (a monte) ed E (a valle), e con flusso $\dot{m}_e > 0$ sulla faccia est, lo schema **centrato** (`linear`) pone $\phi_e = (\phi_C + \phi_E)/2$ e lo schema **upwind** pone $\phi_e = \phi_C$: il primo è del secondo ordine, il secondo del primo. Con il numero di Péclet di cella $Pe_h = \rho u h/\Gamma$ i coefficienti dell'equazione algebrica sono, per lo schema centrato, $a_E = \Gamma/h\,(1 - Pe_h/2)$ e $a_W = \Gamma/h\,(1 + Pe_h/2)$: per $Pe_h > 2$ il coefficiente a_E diventa negativo, la regola dei segni è violata e la soluzione oscilla (Sez. 11.2.3 del testo). Lo schema upwind ha $a_E = \Gamma/h$, $a_W = \Gamma/h + \rho u$, sempre positivi: è *incondizionatamente limitato*, ma il suo errore di troncamento, $\frac{1}{2}\rho u h\, \mathrm{d}^2\phi/\mathrm{d}x^2$, agisce come una **diffusione numerica** $\Gamma_{\mathrm{num}} = \frac{1}{2}\rho u h$ che si somma a quella fisica (Sez. 11.3) e in due dimensioni, per un flusso obliquo rispetto alla griglia, è anche *anisotropa*: è l'effetto che l'esperimento di questo capitolo misura.

5.2 Schemi di ordine superiore

Secondo ordine upwind, QUICK, FROMM

Per avere il secondo ordine senza perdere la trasportività si estrapola dal lato di monte usando due celle: lo schema **second order upwind** (SOU, in OpenFOAM `linearUpwind`) pone $\phi_e = \phi_C + \frac{1}{2}(\phi_C - \phi_W) = \phi_C + (\nabla\phi)_C \cdot (\mathbf{x}_e - \mathbf{x}_C)$; **QUICK** usa una parabola per W, C, E: $\phi_e = \frac{3}{8}\phi_E + \frac{6}{8}\phi_C - \frac{1}{8}\phi_W$, del terzo ordine per il valore di faccia (secondo per il flusso); **FROMM** è la media dei due. Tutti hanno errore di troncamento dispersivo (derivate terze) e, come lo schema centrato, non rispettano la regola dei segni: producono sotto- e sovraelongazioni (*undershoot*, *overshoot*) vicino alle discontinuità e ai gradienti forti, tanto più marcate quanto più lo schema è accurato. Su una mesh non strutturata non esiste una cella W "a monte" lungo la griglia: la generalizzazione (Sez. 11.7) scrive tutto in funzione del gradiente della cella di monte, $\phi_e = \phi_C + (\nabla\phi)_C \cdot \mathbf{r}_{Ce}$ per SOU, che è esattamente ciò che la sintassi `linearUpwind grad(U)` dichiara: il gradiente, calcolato con lo schema indicato (e con il suo eventuale limitatore), è la seconda informazione di monte.

Deferred correction

Gli schemi di ordine superiore coinvolgono più celle di quelle adiacenti e darebbero matrici più larghe e peggio condizionate. Il metodo della *correzione differita* (Sez. 11.8) scrive il flusso come upwind implicito più una correzione esplicita, $\dot{m}_e\phi_e = \dot{m}_e\phi_C^{(\text{impl.})} + \dot{m}_e(\phi_e^{\text{HO}} - \phi_C)^{(\text{espl.})}$: la matrice è quella, ben condizionata, dello schema upwind, e la correzione va nel termine noto con i valori dell'iterazione precedente; a convergenza la soluzione è quella dello schema di ordine superiore. OpenFOAM usa per gli schemi NVD/TVD una variante equivalente: scrive il valore di faccia come combinazione implicita dei due vicini con pesi che dipendono dalla soluzione corrente (il limitatore), e lascia esplicita solo la parte di `linearUpwind` che dipende dal gradiente; in entrambi i casi un calcolo stazionario richiede *iterazioni* perché la correzione si assesti, ed è per questo che un calcolo che converge in 300 iterazioni con `upwind` ne può richiedere il doppio con `linearUpwind`. La scelta di OpenFOAM ha una conseguenza che l'esperimento mette in luce: con i limitatori che arrivano a $\psi = 2$ (`SuperBee`, `MUSCL`) il peso di faccia può diventare interamente *sottovento*, il coefficiente diagonale di una cella si annulla e la matrice stazionaria non è più risolubile; serve il termine temporale (calcolo transitorio) o un rilassamento forte. Per lo stesso motivo *tutti* gli schemi limitati, in un calcolo stazionario, vanno rilassati (`relaxationFactors`, Cap. 6): senza rilassamento anche `vanLeer` diverge sul gradino obliquo.

5.3 Limitatezza: NVF, CBC e TVD

Variabile normalizzata e criterio di limitatezza

Il testo di riferimento (Cap. 12) rende sistematico il confronto fra schemi con la *formulazione a variabile normalizzata* (NVF, Leonard): con $\tilde{\phi} = (\phi - \phi_U)/(\phi_D - \phi_U)$, dove U è la cella a monte della cella di monte C e D quella di valle, ogni schema è una relazione $\tilde{\phi}_f = g(\tilde{\phi}_C)$: upwind $\tilde{\phi}_f = \tilde{\phi}_C$, centrato $\tilde{\phi}_f = \frac{1}{2}(1 + \tilde{\phi}_C)$, SOU $\tilde{\phi}_f = \frac{3}{2}\tilde{\phi}_C$, QUICK $\tilde{\phi}_f = \frac{3}{8} + \frac{3}{4}\tilde{\phi}_C$. Il *criterio di limitatezza della convezione* (CBC, Gaskell e Lau) dice che la

soluzione è limitata se g passa per $(0,0)$ e $(1,1)$ e sta, per $0 \le \tilde{\phi}_C \le 1$, nel triangolo $\tilde{\phi}_C \le \tilde{\phi}_f \le 1$, mentre fuori da $[0,1]$ (estremi locali) coincide con upwind. Nessuno schema lineare di ordine superiore al primo soddisfa il criterio (teorema di Godunov): gli schemi **ad alta risoluzione** (HR) sono *non lineari*, cioè g è una spezzata che segue uno schema di ordine alto dove la soluzione è regolare e torna verso upwind vicino agli estremi. Esempi: MINMOD (il più diffusivo), SMART, STOIC, OSHER, MUSCL, SUPERBEE (il più compressivo).

TVD e il diagramma di Sweby

La formulazione equivalente *Total Variation Diminishing* (Harten, Sweby) scrive il valore di faccia come upwind più una frazione $\psi(r)/2$ della differenza di valle, $\phi_f = \phi_C + \frac{1}{2}\psi(r)(\phi_D - \phi_C)$, con $r = (\phi_C - \phi_U)/(\phi_D - \phi_C)$ il rapporto fra i gradienti di monte e di valle. La variazione totale della soluzione non cresce se ψ sta nella regione di Sweby, $0 \le \psi \le \min(2r, 2)$, e lo schema è del secondo ordine se passa per $\psi(1) = 1$ restando fra le curve dello schema centrato ($\psi = 1$) e di SOU ($\psi = r$). I limitatori classici (Fig. 5.1):

$$\begin{aligned} &\text{MINMOD: } \psi = \max(0, \min(r, 1)), && \text{van Leer: } \psi = \frac{r + |r|}{1 + |r|}, \\ &\text{SUPERBEE: } \psi = \max(0, \min(2r, 1), \min(r, 2)), && \text{MUSCL: } \psi = \max\left(0, \min(2r, \tfrac{r+1}{2}, 2)\right), \end{aligned} \tag{5.2}$$

e la relazione $\tilde{\phi}_f = \tilde{\phi}_C + \frac{1}{2}\psi(r)(1 - \tilde{\phi}_C)$ con $r = \tilde{\phi}_C/(1 - \tilde{\phi}_C)$ li traduce nel linguaggio NVF (Sez. 12.5). Su una mesh non strutturata ϕ_U non esiste e r si valuta con il gradiente della cella di monte: $r = 2(\nabla\phi)_C \cdot \mathbf{d}_{CF}/(\phi_D - \phi_C) - 1$ (Sez. 12.6), che è ciò che OpenFOAM fa in tutti gli schemi `limitedScheme`.

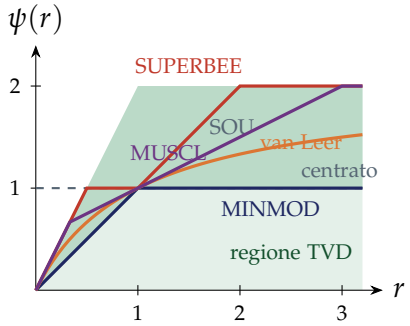


Figura 5.1: Il diagramma di Sweby: la regione TVD (verde chiaro), la regione del secondo ordine (verde scuro, fra le rette dello schema centrato $\psi = 1$ e di SOU $\psi = r$) e i limitatori MINMOD, van Leer, MUSCL e SUPERBEE.

5.4 La sintassi di `divSchemes`

Le parole chiave

Ogni voce ha la forma `div(phi,U) <bounded> Gauss <schema di interpolazione>`: `Gauss` indica l'integrazione sulle facce, e lo schema dice come si ottiene ϕ_f dal flusso `phi` e dai valori di cella. I principali:

`upwind`
primo ordine, limitato, diffusivo; per le prime iterazioni di un calcolo difficile e per le equazioni dei modelli di turbolenza quando si vuole robustezza.

`linear`
centrato, secondo ordine, non limitato: per le LES e per i flussi a basso numero di Péclet (la cavità a $Re = 10$); oscilla altrove.

`linearUpwind grad(U)`
SOU con il gradiente indicato; secondo ordine, poco diffusivo, può dare lievi sovraelongazioni; con `cellLimited Gauss linear 1` come gradiente diventa quasi limitato. È lo schema standard per **U** nei calcoli RANS stazionari.

`QUICK`
terzo ordine sulle mesh uniformi, non limitato.

`limitedLinear` ψ
TVD con limitatore che tende allo schema centrato; il coefficiente $\psi \in [0,1]$ regola quanto rapidamente (`1`: più limitante, più diffusivo e più stabile; `0.2`: più vicino al centrato). Standard per gli scalari (k, ε, ω, T) e, nella variante `limitedLinearV 1`, per **U** nei transitori.

`vanLeer`, `Minmod`, `SuperBee`, `MUSCL`, `vanAlbada`, `OSPRE`, `UMIST`
i limitatori TVD della (5.2) e simili.

`limitedLinear01`, `vanLeer01`
varianti che impongono anche $0 \leq \phi \leq 1$ (frazioni di fase, specie).

suffisso `V`
(`limitedLinearV`, `linearUpwindV`) per i campi vettoriali: un unico limitatore per le tre componenti, calcolato sulla direzione di massima variazione, che evita di limitare in modo diverso le componenti.

prefisso `bounded`
(solo stazionario) sottrae $\phi\,\nabla\cdot(\rho\mathbf{v})$ al termine convettivo, cioè usa la forma non conservativa durante le iterazioni, quando la continuità non è ancora soddisfatta: migliora la limitatezza e la convergenza, e a convergenza non cambia nulla (Approfondimenti del Cap. 2).

Le voci `div((nuEff*dev2(T(grad(U))))) Gauss linear` e simili riguardano termini *espliciti* (`fvc::div`) del tensore degli sforzi: non sono convettivi e `linear` è sempre la scelta giusta.

5.5 Esperimento: il gradino obliquo

Il banco di prova classico (Sez. 11.6 del testo) è il trasporto di un gradino in un campo di velocità uniforme inclinato di 45° rispetto alla griglia, senza diffusione fisica: lo scalare vale 1 all'ingresso di sinistra e 0 a quello inferiore, e la soluzione esatta è un gradino netto lungo la diagonale. Qualunque allargamento del gradino è diffusione numerica, qualunque valore fuori da $[0,1]$ è una violazione della limitatezza. Il caso usa il solutore `functions` di OpenFOAM 12, che integra soltanto la *function object* `scalarTransport` con il campo di velocità fornito, e congelato, dal sotto-solutore `incompressibleFluid`; lo script `genera.py` prova dieci schemi su 40×40 celle e cinque su 80×80, misurando lo spessore del gradino (distanza fra $s = 0{,}1$ e $s = 0{,}9$ lungo la verticale $x = 0{,}5$), il minimo e il massimo del campo, l'errore medio e le iterazioni per convergere.



Dati del caso `c05_convezione`

Quadrato unitario, $\mathbf{U} = (1,1,0)$, $D = 0$ (`diffusivity=none`); $N = 40$ e 80; `ddtSchemes steadyState`, fino a 1500 iterazioni con rilassamento 0,7 dell'equazione di s, `PBiCGStab/DILU` con tolleranza 10^{-10}; profilo lungo $x = 0{,}5$, dove il gradino esatto è in $y = 0{,}5$; si contano le iterazioni fino al residuo iniziale 10^{-6}.




```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

application     foamRun;

solver          functions;
subSolver       incompressibleFluid;
subSolverTime   0;

startFrom       startTime;
startTime       0;
stopAt          endTime;
endTime         1500;           // iterazioni massime dello pseudo-tempo
    (ddtSchemes steadyState)
deltaT          1;
writeControl    timeStep;
writeInterval   1500;
writeFormat     ascii;
writePrecision  8;
timeFormat      general;
timePrecision   6;
runTimeModifiable true;

functions
{
    #includeFunc scalarTransport(s, diffusivity=none)
}
```

Listato 5.1: Il solutore `functions` con il sotto-solutore che fornisce **U** e `phi`, e la function object `scalarTransport`.

```
dimensions      [0 0 0 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    sinistra    { type fixedValue;   value uniform 1; }
    sotto       { type fixedValue;   value uniform 0; }
    destra      { type zeroGradient; }
    sopra       { type zeroGradient; }
    frontEretro { type empty; }
}
```

c05_convezione/0/s

Listato 5.2: Lo scalare trasportato: 1 dall'ingresso di sinistra, 0 da quello inferiore.

Risultati del caso `c05_convezione`

schema (40×40)	$s_{\min}$	$s_{\max}$	spessore	errore medio	iterazioni
`upwind`	0,000	1,000	0,413	0,1076	60
`linear`	-0,056	1,056	0,045	0,0219	1500
`linearUpwind`	-0,049	1,049	0,119	0,0398	62
`QUICK`	-0,046	1,046	0,100	0,0339	171
`vanLeer`	-0,000	1,000	0,135	0,0381	145
`limitedLinear 1`	-0,000	1,000	0,109	0,0316	1500
`Minmod`	0,000	1,000	0,187	0,0516	1500
`SuperBee`			non converge		
`MUSCL`			non converge		
`limitedLinear 0.2`	-0,000	1,000	0,045	0,0157	1500

- Raddoppiando N lo spessore del gradino con `upwind` passa da 0,413 a 0,286 (rapporto 1.44, cioè $\propto \sqrt{h}$ come prevede la diffusione numerica $\Gamma_{\text{num}} \propto h$ su una lunghezza fissa), con `vanLeer` da 0,135 a 0,079 (rapporto 1.70).
- Lo schema `linear` viola la limitatezza: $s \in [-0{,}056,\ 1{,}056]$ contro $[0, 1]$ della soluzione esatta; tutti gli schemi TVD restano in $[0, 1]$ a meno di

$$3{,}7 \times 10^{-4}$$

.
- Errore medio su 40×40: `upwind` 0,1076, `vanLeer` 0,0381, `limitedLinear 1` 0,0316, `limitedLinear 0.2` 0,0157, `Minmod` 0,0516: il limitatore più compressivo dà il gradino più netto, il più diffusivo (`Minmod`) sta fra `upwind` e `vanLeer`.
- `SuperBee` e `MUSCL` non arrivano a una soluzione stazionaria: il limitatore vale 2 su molte facce, il peso di faccia diventa interamente sottovento e il coefficiente diagonale di alcune celle si annulla (il precondizionatore DILU divide per zero). Nei calcoli transitori il termine $\partial/\partial t$ rende la diagonale positiva e questi schemi funzionano; nei calcoli stazionari vanno evitati.
- Con `linear`, `limitedLinear 1`, `Minmod`, `limitedLinear 0.2` il residuo si arresta fra 10^{-4} e 10^{-6} senza scendere oltre nelle 1500 iterazioni: il limitatore cambia valore da un'iterazione all'altra sulle facce vicine al gradino (*limiter cycling*); il campo è però stazionario alla quarta cifra, ed è il comportamento normale degli schemi TVD nei calcoli stazionari, da cui la raccomandazione di giudicare la convergenza anche sulle grandezze integrali.

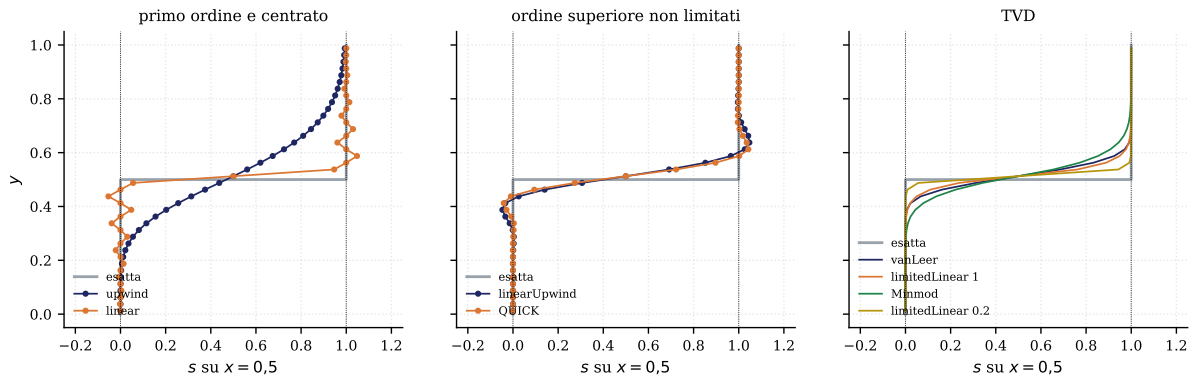


Figura 5.2: Il gradino obliquo su 40×40 celle: profili di s lungo $x = 0{,}5$ per gli schemi del primo ordine e centrato (a sinistra), di ordine superiore non limitati (al centro) e TVD (a destra); la linea grigia è la soluzione esatta.

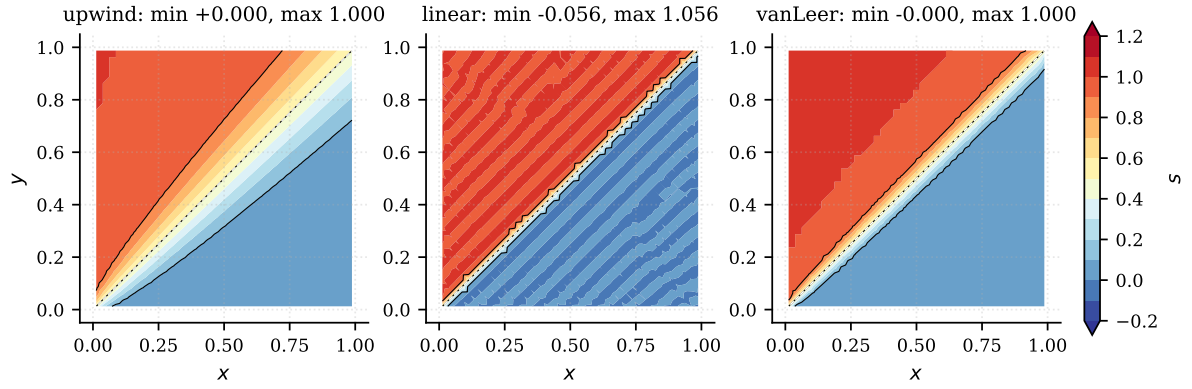


Figura 5.3: Il campo s con `upwind`, `linear` e `vanLeer` (40×40): diffusione numerica del primo, oscillazioni del secondo, gradino netto e limitato del terzo.

Che cosa scegliere, in pratica

L'esperimento riassume la regola che i tutorial della distribuzione applicano. Per la velocità nei calcoli RANS stazionari, `bounded Gauss linearUpwind grad(U);` nei transitori `Gauss linearUpwind grad(U)` o `limitedLinearV 1`; per gli scalari turbolenti `bounded Gauss limitedLinear 1`; per le frazioni di fase `vanLeer01` o `interfaceCompression`; per le LES `linear` o `LUST` (75% centrato, 25% `linearUpwind`); per gli urti (Cap. 10) `vanLeer` o `Minmod` sui flussi ricostruiti. Lo schema `upwind` va usato per *iniziare* un calcolo difficile e poi sostituito, mai per i risultati finali: la sua diffusione numerica in un calcolo aerodinamico è dello stesso ordine della viscosità turbolenta e falsa la resistenza e la separazione. Il `linear` per la velocità produce campi a scacchiera su qualunque mesh a Péclet di cella alto, e il Cap. 8 ne mostra un esempio.

5.6 Approfondimenti

5.6.1 Il limitatore sulle mesh non strutturate e il gradiente

La formula $r = 2(\nabla\phi)_C \cdot \mathbf{d}_{CF}/(\phi_D - \phi_C) - 1$ ricostruisce il valore "virtuale" di monte ϕ_U per estrapolazione lineare dal gradiente di C. Se il gradiente è calcolato con Green–Gauss su una mesh distorta, ϕ_U è impreciso e il limitatore può agire a sproposito, aggiungendo diffusione in zone regolari o lasciando passare oscillazioni: è uno dei motivi per cui la qualità della mesh e lo schema del gradiente influenzano i risultati anche attraverso il termine convettivo, e per cui nelle mesh di `snappyHexMesh` si preferiscono i limitatori più tolleranti (`limitedLinear 1` piuttosto che `SuperBee`).

5.6.2 I limitatori nei calcoli stazionari: la convergenza che si ferma

Un limitatore non lineare può impedire la convergenza di un calcolo stazionario a residui sotto 10^{-4}–10^{-5}: il limitatore "commuta" fra due stati in alcune celle a ogni iterazione, e i residui oscillano senza scendere (*limiter cycling*). Non è un errore del calcolo (le grandezze integrali sono convergenti), ma va riconosciuto guardando la storia dei coefficienti di forza oltre ai residui (Cap. 13); i rimedi sono `limitedLinear` con coefficiente più alto, il limitatore `cellLimited` sul gradiente invece che sullo schema, o il congelamento del limitatore nelle ultime iterazioni, che OpenFOAM non offre direttamente.

5.6.3 Trasporto e dispersione: l'equazione modificata

L'analisi dell'equazione modificata (Modulo 4) spiega il comportamento osservato: l'errore di troncamento di upwind è un termine di derivata seconda (diffusivo: smussa), quello degli schemi del secondo ordine un termine di derivata terza (dispersivo: le componenti di Fourier viaggiano a velocità diverse e il gradino si frange in oscillazioni che lo precedono o lo seguono), quello di QUICK una derivata quarta (iperdiffusivo, con lievi oscillazioni). I limitatori TVD sono, in questa lettura, schemi del secondo ordine con una diffusione aggiuntiva *adattiva*, attivata dai gradienti: è il motivo per cui su una soluzione regolare non costano quasi nulla in accuratezza e vicino a un gradino la salvano.

Capitolo 6

Tempo, sorgenti, sottorilassamento e residui

6.1 Il termine instazionario

Il termine $\int_V \partial(\rho\phi)/\partial t \, \mathrm{d}V \approx V_C \, \partial(\rho\phi)_C/\partial t$ richiede una discretizzazione nel tempo. Il testo di riferimento (Cap. 13) la presenta in due modi equivalenti: come differenza finita della derivata temporale, oppure, in modo più coerente con il metodo, integrando l'equazione su un *volume di controllo temporale* $[t - \Delta t/2,\, t + \Delta t/2]$ e interpolando ϕ nel tempo come si fa nello spazio. In entrambi i casi si ottengono gli schemi che OpenFOAM offre in `ddtSchemes`, tutti *impliciti* nel senso che gli altri termini (convezione, diffusione, sorgenti) sono valutati al nuovo istante $t + \Delta t$.

`Euler`, `backward`, `CrankNicolson`

Con ϕ^n il valore al nuovo istante, ϕ^o al precedente e ϕ^{oo} a quello ancora prima:

$$\texttt{Euler}: \quad \frac{\phi^n - \phi^o}{\Delta t} = L(\phi^n) \text{ (implicito, primo ordine)}, \tag{6.1}$$

$$\texttt{backward}: \quad \frac{3\phi^n - 4\phi^o + \phi^{oo}}{2\Delta t} = L(\phi^n) \text{ (secondo ordine, due livelli)}, \tag{6.2}$$

$$\texttt{CrankNicolson}\ \psi: \quad \frac{\phi^n - \phi^o}{\Delta t} = \tfrac{1}{2}(1+\psi)L(\phi^n) + \tfrac{1}{2}(1-\psi)L(\phi^o) \text{ (secondo ordine)}, \tag{6.3}$$

dove L è l'operatore spaziale discretizzato. `Euler` è incondizionatamente stabile e molto robusto, ma smorza (dissipa) le oscillazioni fisiche e ha errore $\mathcal{O}(\Delta t)$: va bene per raggiungere uno stato stazionario, non per seguire un transitorio. `backward` è del secondo ordine, stabile, leggermente dissipativo; è la scelta normale per i transitori (scia del cilindro, Cap. 14). `CrankNicolson 1` è del secondo ordine e non dissipativo, ma può produrre oscillazioni temporali non smorzate quando il passo è grande rispetto alla scala di diffusione (è solo *marginalmente* stabile: le componenti rapide non decadono); il coefficiente $\psi \in [0,1]$ lo mescola con `Euler` (`CrankNicolson 0.9` è la pratica comune, `0` coincide con `Euler`). `steadyState` elimina il termine e trasforma il ciclo temporale in un ciclo di iterazioni: è ciò che fa di `incompressibleFluid` un solutore stazionario di tipo SIMPLE (Cap. 8). L'implementazione a volumi finiti (Sez. 13.3 del testo) differisce da quella a differenze finite solo nella condizione iniziale e nel modo di trattare passi

non uniformi: OpenFOAM memorizza `oldTime()` e `oldTime().oldTime()` di ogni campo, e `backward` tiene conto del rapporto fra i due passi precedenti.

Il numero di Courant e il passo adattivo

Il numero di Courant di una cella, $\mathrm{Co} = \Delta t \sum_f |\dot{m}_f| / (2\rho V_C)$ (la somma dei flussi uscenti in modulo per il passo, divisa per il volume), misura di quante celle l'informazione avanza in un passo; il log ne stampa media e massimo a ogni passo. Gli schemi impliciti non hanno il limite di stabilità $\mathrm{Co} \leq 1$ degli espliciti (Modulo 2), ma l'*accuratezza* del transitorio e la convergenza degli algoritmi di accoppiamento (PISO in particolare) richiedono $\mathrm{Co} \lesssim 1$ dove la soluzione cambia; con PIMPLE e più iterazioni esterne si sale a $\mathrm{Co} \sim 5$–10 per raggiungere più in fretta uno stato stazionario. Con `adjustTimeStep yes`, `maxCo` e `maxDeltaT` in `controlDict` il passo è ricalcolato a ogni iterazione per mantenere il Courant massimo al valore richiesto (il passo cresce al più del 20% per volta), e `writeControl adjustableRunTime` fa cadere la scrittura sugli istanti voluti. Nei flussi comprimibili il Courant è calcolato con $|\mathbf{U}| + a$ (Cap. 10).

6.2 Esperimento: l'ordine nel tempo

Il caso `c06_tempo` ripete, con `laplacianFoam`, il problema del Modulo 0: la diffusione $\partial T/\partial t = D\,\partial^2 T/\partial x^2$ su $[0,1]$ con $T(x,0) = \sin \pi x$ e $T = 0$ agli estremi, soluzione esatta $\sin \pi x\, e^{-\pi^2 D t}$. Su una mesh fine (400 celle, in modo che l'errore spaziale sia trascurabile) si integra fino a $t = 0{,}1$ con $\Delta t = 0{,}02$, $0{,}01$, $0{,}005$, $0{,}0025$ e i tre schemi, misurando l'errore massimo: i rapporti fra errori successivi devono essere 2 per `Euler` e 4 per `backward` e `CrankNicolson`. Un secondo esperimento trasporta una gaussiana con la function object `scalarTransport` (come nel Cap. 5, ma nel tempo) a $\mathrm{Co} = 0{,}5$, 1, 2 e 4: gli schemi impliciti restano stabili anche a $\mathrm{Co} = 4$, ma la gaussiana si smussa (`Euler`) o oscilla (`CrankNicolson 1`); è il prezzo del passo lungo, che nessuno schema cancella.


c06_tempo.zip

Dati del caso `c06_tempo`

Diffusione: $D = 1$, 400 celle, $t_{\mathrm{fin}} = 0{,}1$, $\Delta t \in \{0{,}02;\ 0{,}01;\ 0{,}005;\ 0{,}0025\}$, schemi `Euler`, `backward`, `CrankNicolson 1`, `CrankNicolson 0.9`. Convezione: dominio $[0, 10]$ con 400 celle, $u = 1$, gaussiana $e^{-(x-2)^2/0{,}18}$ trasportata fino a $t = 5$, schema spaziale `linear`, $\mathrm{Co} \in \{0{,}5;\ 1;\ 2;\ 4\}$.

Risultati del caso `c06_tempo`

- Diffusione, ordine osservato nel tempo: `Euler` 0,97, `backward` 2,08, `CrankNicolson 1` 1,95, `CrankNicolson 0.9` 1,48. Con $\Delta t = 0{,}0025$ s l'errore di Euler è $4{,}49 \times 10^{-3}$, quello di Crank–Nicolson $9{,}53 \times 10^{-5}$: a parità di passo la differenza è di 47 volte.
- Convezione a $t = 5$ s (la gaussiana ha percorso 5 unità, 200 celle): errore massimo

Co	0,5	1	2	4
`Euler`	0,232	0,353	0,485	0,609
`backward`	0,040	0,077	0,183	0,347
`CrankNicolson 1`	0,030	0,040	0,078	0,203

- A Co $= 4$ nessuno schema diverge (sono tutti impliciti), ma `Euler` spiana il picco a 0,393 (diffusione numerica temporale $\propto \Delta t$), `backward` a 0,778, `CrankNicolson 1` a 0,917 con un minimo di -0,069: le oscillazioni dietro il picco sono l'errore di fase dello schema non dissipativo.
- `Euler` a Co $= 0{,}5$ (passo otto volte più piccolo, otto volte il costo) ha ancora un errore di 0,232, maggiore di quello di `backward` a Co $= 2$ (0,183) e 8 volte quello di `CrankNicolson 1` allo stesso passo: nei transitori il primo ordine nel tempo non si recupera riducendo il passo, si cambia schema.

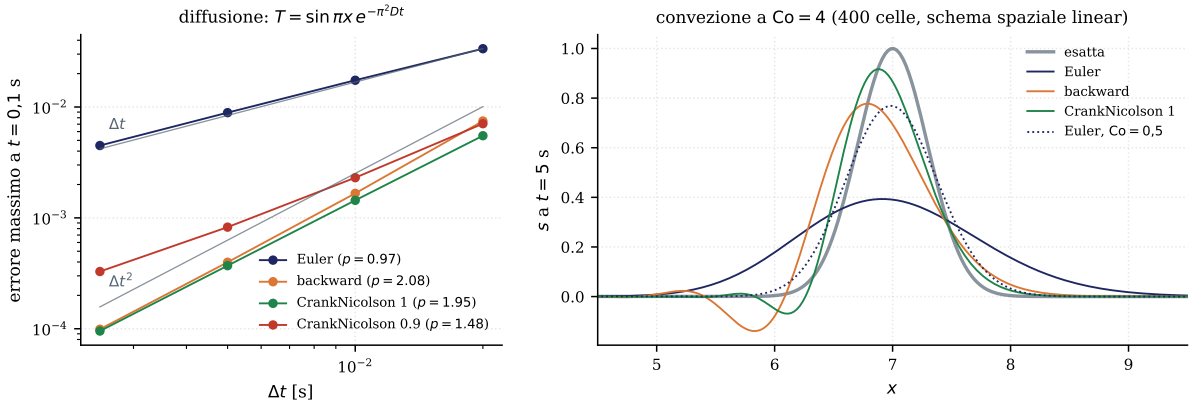


Figura 6.1: Ordine di convergenza nel tempo per la diffusione con i tre schemi (a sinistra); la gaussiana trasportata a Co $= 4$ con `Euler`, `backward` e `CrankNicolson 1` confrontata con la soluzione esatta (a destra).

6.3 Termini sorgente: `fvModels` e `fvConstraints`

Linearizzazione delle sorgenti

Un termine sorgente Q^ϕ che dipende da ϕ va linearizzato come $Q^\phi = S_u + S_p\phi$ (Sez. 14.1 del testo), e per la stabilità del processo iterativo S_p deve essere *negativo*: la parte $S_p\phi_C$ va allora nella diagonale (aumentandola in valore assoluto: dominanza diagonale) e S_u nel termine noto. Se $S_p > 0$ (una sorgente che cresce con ϕ, come la produzione di k in alcuni regimi) si tratta esplicitamente, portando tutto nel termine noto. OpenFOAM fornisce `fvm::Sp(coeff, phi)` (implicito), `fvm::Su(source, phi)` (esplicito) e `fvm::SuSp(coeff, phi)`, che sceglie automaticamente in base al segno del coefficiente cella per cella: è la forma usata nelle equazioni dei modelli di turbolenza. Nel caso, l'utente non scrive codice: le sorgenti si aggiungono con i **fvModels** (`constant/fvModels`): forze di volume (`buoyancyForce`, `accelerationSource`), mezzi porosi (`porosityForce`), sorgenti di calore o di massa in una zona (`semiImplicitSource`, `heatSource`), modelli di attuatore (`actuationDisk`, `rotorDisk`), scambio con particelle, radiazione; e i **fvCon-**

straints (`system/fvConstraints`) impongono vincoli: un valore fissato in una zona (`fixedValueConstraint`), un limite (`limitTemperature`, `limitPressure`), una portata media imposta in un canale periodico (`meanVelocityForce`, che aggiunge un gradiente di pressione calcolato iterativamente). Tutti compaiono nelle equazioni dei moduli come `fvModels().source(U)` e `fvConstraints().constrain(UEqn)`: lo si è visto nel Cap. 2.

6.4 Sottorilassamento

Rilassamento implicito ed esplicito

Nei calcoli stazionari le equazioni sono non lineari (la convezione) e accoppiate (pressione e velocità): risolverle per iterazioni successive, ciascuna con i coefficienti dell'iterazione precedente, può divergere se il cambiamento della soluzione fra due iterazioni è troppo grande. Il *sottorilassamento* (Sez. 14.2 del testo) lo frena. Nella forma *implicita* (Patankar) l'equazione $a_C\phi_C + \sum a_F\phi_F = b_C$ è sostituita da

$$\frac{a_C}{\alpha}\phi_C + \sum_F a_F\phi_F = b_C + \frac{1-\alpha}{\alpha}a_C\phi_C^{\text{prec}}, \qquad 0 < \alpha \le 1, \tag{6.4}$$

che ha la stessa soluzione a convergenza (quando $\phi_C = \phi_C^{\text{prec}}$) ma una diagonale più grande di un fattore $1/\alpha$: più stabile, più lenta. È la voce `relaxationFactors { equations { U 0.7; } }` di `fvSolution`, che OpenFOAM applica con `UEqn.relax()` prima di risolvere. Nella forma *esplicita* si corregge il campo dopo averlo calcolato, $\phi = \phi^{\text{prec}} + \alpha(\phi^{\text{nuovo}} - \phi^{\text{prec}})$: è la voce `fields { p 0.3; }`, usata per la pressione nell'algoritmo SIMPLE (`p.relax()`), dove la correzione di pressione è notoriamente sovrastimata. I valori classici (Sez. 15.8) sono $\alpha_U = 0{,}7$ e $\alpha_p = 0{,}3$ per SIMPLE (con $\alpha_U + \alpha_p \approx 1$), $\alpha_U = 0{,}9$ e $\alpha_p = 1$ per SIMPLEC; per le variabili turbolente 0,5–0,8. Nei transitori con PISO non si rilassa (`"(U|k|epsilon)Final" 1`); con PIMPLE si rilassano le iterazioni esterne intermedie ma non l'ultima (le voci `...Final` a 1), altrimenti il transitorio è falsato. Il Cap. 8 misura l'effetto di α sul numero di iterazioni della cavità.

6.5 I residui: che cosa misura OpenFOAM

La definizione

Il residuo di un'equazione è la misura di quanto il campo corrente non la soddisfa, $\mathbf{r} = \mathbf{b} - \mathbf{A}\boldsymbol{\phi}$ (Sez. 14.4 del testo); va normalizzato per essere confrontabile fra equazioni e casi. OpenFOAM usa

$$R = \frac{\sum_C |b_C - \sum_j a_{Cj}\phi_j|}{\sum_C \left(|\mathbf{A}\boldsymbol{\phi} - \mathbf{A}\bar{\phi}|_C + |\mathbf{b} - \mathbf{A}\bar{\phi}|_C\right)}, \tag{6.5}$$

dove $\bar{\phi}$ è la media del campo: il denominatore misura la "grandezza" dei termini dell'equazione rispetto a un campo uniforme, così che un campo uniforme con sorgenti nulle ha residuo 1 e un campo che soddisfa l'equazione ha residuo 0, indipendentemente dalle unità. Il *residuo iniziale* stampato nel log è calcolato *prima* di risolvere il sistema

lineare, con i coefficienti dell'iterazione corrente e il campo dell'iterazione precedente: è la misura di convergenza del processo esterno (SIMPLE o PIMPLE), ed è quello da guardare. Il *residuo finale* è dopo il solutore lineare, e dice solo quanto il sistema lineare è stato risolto (Cap. 7). Per un campo vettoriale sono stampate le componenti (`Ux`, `Uy`, `Uz`); per la pressione in PISO compare un residuo per correttore.

Residui piccoli non significano soluzione giusta

Il residuo misura la coerenza fra il campo e le equazioni *discrete*, non la distanza dalla soluzione esatta: una mesh grossolana converge a residui 10^{-8} verso una soluzione sbagliata. Inoltre la (6.5) è normalizzata con la soluzione corrente: nei casi dominati da un flusso uniforme (un profilo a bassa incidenza) il denominatore è grande e i residui scendono presto a 10^{-5} anche se le forze non sono ancora assestate; nei casi senza un flusso dominante (la cavità, la convezione naturale) i residui restano alti anche vicino alla convergenza. La convergenza va giudicata su *tre* indicatori insieme: i residui iniziali (in calo di almeno tre ordini di grandezza), la stabilità nel tempo delle grandezze integrali di interesse (coefficienti di forza, portate, temperature medie, con le function object del Cap. 16), e gli errori di continuità. `residualControl` in `fvSolution` (`p 1e-4; U 1e-5;`) ferma automaticamente un calcolo SIMPLE quando tutti i residui iniziali scendono sotto le soglie: è comodo, ma le soglie vanno scelte dopo aver guardato le forze.

6.6 Approfondimenti

6.6.1 Passi temporali locali per lo stazionario

Per raggiungere uno stato stazionario con un solutore transitorio OpenFOAM offre il *local time stepping* (LTS, `ddtSchemes default localEuler`): ogni cella avanza con il proprio passo massimo consentito dal Courant locale, che non è più un tempo fisico ma un'accelerazione della convergenza; i tutorial `pitzDailyLTS` e `angledDuctLTS` lo mostrano. È utile nei flussi comprimibili e multifase dove SIMPLE converge male; nei casi di questo modulo lo stazionario con `steadyState` è sufficiente.

6.6.2 Condizioni iniziali e il primo passo di `backward`

Lo schema `backward` richiede due livelli temporali precedenti: al primo passo ne esiste uno solo, e OpenFOAM usa `Euler` per quel passo (Sez. 13.3.7 del testo). Lo stesso accade quando si riparte da una cartella temporale salvata senza i campi `_0` del livello precedente. L'effetto sull'accuratezza globale è trascurabile per un transitorio lungo, ma spiega perché l'errore al primo passo sia del primo ordine e perché i test di ordine vadano fatti su intervalli di molti passi.

6.6.3 Il transitorio come strumento di convergenza

Molti calcoli "stazionari" difficili (separazioni massicce, scie, getti) non hanno una soluzione stazionaria delle equazioni RANS: SIMPLE oscilla senza convergere perché insegue un'instabilità fisica. In questi casi il transitorio con PIMPLE e un passo moderato, seguito dalla media

nel tempo dei campi (`fieldAverage`), è la via corretta, e i residui "che non scendono" non sono un difetto del calcolo ma un'informazione sulla fisica (il cilindro a $Re = 100$ del Cap. 14 è il caso tipico).

Capitolo 7

La soluzione dei sistemi lineari

7.1 Il problema

Ogni equazione discretizzata, per ogni iterazione esterna o passo temporale, è un sistema lineare $\mathbf{A}\boldsymbol{\phi} = \mathbf{b}$ con tante incognite quante celle: 10^4 nei casi di questo modulo, 10^6–10^8 nei calcoli industriali. La matrice è *sparsa* (al più sette elementi non nulli per riga su una mesh esaedrica), e la sua struttura è quella dell'indirizzamento *ldu* del Cap. 3: diagonale, coefficienti *upper* e *lower* indicizzati per faccia. Il testo di riferimento (Cap. 10) passa in rassegna i metodi; qui ci concentriamo su quelli che OpenFOAM mette a disposizione in `fvSolution`, su come si scelgono, e su che cosa significano davvero `tolerance` e `relTol`, le due voci più fraintese di tutto il caso.

Metodi diretti e iterativi

I metodi *diretti* (eliminazione di Gauss, fattorizzazione LU, l'algoritmo di Thomas per le tridiagonali del Modulo 3) danno la soluzione esatta in un numero finito di operazioni, ma per una matrice sparsa generale la fattorizzazione riempie gli zeri (*fill-in*) e costa $\mathcal{O}(N^{3/2})$–$\mathcal{O}(N^2)$ in tempo e memoria: proibitivo oltre poche decine di migliaia di celle. I metodi *iterativi* partono da una stima e la migliorano; costano $\mathcal{O}(N)$ per iterazione e non riempiono la matrice; sono i soli usati nei codici CFD, e la loro inesattezza non è un difetto: dentro un ciclo SIMPLE i coefficienti stessi sono provvisori, e risolvere esattamente il sistema di un'iterazione intermedia è lavoro sprecato (Sez. 10.3 del testo).

7.2 I metodi iterativi di OpenFOAM

Rilassamento, gradiente coniugato, multigriglia

Jacobi e Gauss–Seidel (`smoothSolver` con `smoother GaussSeidel` o `symGaussSeidel`) aggiornano ogni incognita con l'equazione della sua cella, usando i valori vecchi (Jacobi) o quelli già aggiornati (Gauss–Seidel); convergono se la matrice è a dominanza diagonale, lentamente: eliminano in poche iterazioni le componenti dell'errore a corta lunghezza d'onda (sono ottimi "lisciatori") ma quelle lunghe richiedono $\mathcal{O}(N)$ iterazioni. Vanno bene per le equazioni *con un termine temporale o convettivo dominante* (velocità, k, ε, ω), la cui matrice è molto dominante in diagonale e richiede poche iterazioni.

Gradiente coniugato (`PCG` per matrici simmetriche, `PBiCGStab` per quelle asimmetriche) minimizza l'errore in un sottospazio di Krylov; converge in un numero di iterazioni proporzionale a $\sqrt{\kappa}$, con κ il numero di condizionamento, che per il laplaciano cresce come $N^{2/d}$. Il *precondizionamento* sostituisce $\mathbf{A}$ con $\mathbf{M}^{-1}\mathbf{A}$, dove $\mathbf{M}$ è un'approssimazione di $\mathbf{A}$ facile da invertire: `DIC` (Cholesky incompleta diagonale, simmetrica) e `DILU` (LU incompleta diagonale, asimmetrica) calcolano una fattorizzazione *senza fill-in* (Sez. 10.3.5–10.3.10 del testo); `GAMG` come precondizionatore è spesso la scelta più efficiente per p.

Multigriglia algebrico (`GAMG`, *geometric-algebraic multigrid*) attacca la debolezza dei lisciatori: le componenti lunghe dell'errore, lente su una mesh fine, sono corte su una mesh grossa. Si costruisce una gerarchia di livelli per *agglomerazione* delle celle (`agglomerator faceAreaPair`: si uniscono a coppie le celle con la faccia comune più grande, dimezzando il numero di incognite a ogni livello fino a `nCellsInCoarsestLevel`, tipicamente 10–100), si lisciano poche iterazioni sul livello fine (`nPreSweeps`, `nPostSweeps`), si restringe il residuo al livello grosso, lo si risolve (ricorsivamente) e si prolunga la correzione (Sez. 10.4). Il numero di iterazioni è quasi indipendente da N: è il solutore standard per l'equazione della pressione, la più costosa di tutte.

La sintassi di `fvSolution`: `solvers`

```
solvers
{
    p
    {
        solver          GAMG;              // oppure PCG, PBiCGStab,
    smoothSolver
        smoother        GaussSeidel;       // lisciatore del GAMG
    (GaussSeidel, DICGaussSeidel, DIC)
        tolerance       1e-08;             // residuo assoluto (normalizzato)
    sotto cui fermarsi
        relTol          0.1;               // oppure: residuo ridotto di
    questo fattore rispetto all'iniziale
        nCellsInCoarsestLevel 10;          // dimensione del livello piu'
    grosso (predefinito 10)
        // maxIter 1000; minIter 0;  nPreSweeps 0; nPostSweeps 2;
    cacheAgglomeration true;
    }
    "(U|k|epsilon|omega)"                  // espressione regolare: stesse
    impostazioni per piu' campi
    {
        solver          smoothSolver;
        smoother        symGaussSeidel;
        nSweeps         2;                 // iterazioni per controllo del
    residuo
        tolerance       1e-08;
        relTol          0.1;
    }
    "(U|k|epsilon|omega)Final" { $U; relTol 0; }   // nei transitori:
    l'ultimo correttore senza relTol
}
```

Il solutore si ferma quando il residuo normalizzato (la (6.5), calcolato dentro il solutore lineare) scende sotto `tolerance` *oppure* sotto `relTol` volte il residuo iniziale, quale dei due arriva prima; con `relTol 0` vale solo la tolleranza assoluta. La scelta è legata

all'algoritmo esterno: in SIMPLE (Cap. 8) basta ridurre il residuo di dieci volte per iterazione (`relTol 0.1` per p, anche 0,01; 0,1 per **U**), perché l'iterazione successiva cambierà comunque i coefficienti; in PISO l'ultimo correttore di pressione va risolto a tolleranza assoluta (`pFinal` con `relTol 0`), altrimenti la continuità non è soddisfatta e l'errore si accumula nel tempo. Una `tolerance` troppo stretta (10^{-12}) fa sprecare iterazioni; troppo larga (10^{-5}) ferma SIMPLE prima della convergenza vera, perché il solutore lineare esce al primo controllo e il residuo iniziale esterno non può scendere sotto quel valore.

7.3 Esperimento: i solutori per la pressione

Nel caso `c08_cavita_re100` (cavità a $Re = 100$, 128×128 celle, SIMPLE) lo script esegue 100 iterazioni esterne con otto configurazioni del solutore di p e registra il tempo di calcolo, il numero medio di iterazioni del solutore lineare per iterazione esterna e il residuo esterno raggiunto. Le velocità sono sempre risolte con `smoothSolver`: la loro equazione, dominata dalla diagonale grazie al rilassamento, converge in 2–5 iterazioni qualunque sia il metodo.

Dati dell'esperimento

Cavità $Re = 100$, 128×128 celle (16 384 incognite), SIMPLE con $\alpha_U = 0{,}7$, $\alpha_p = 0{,}3$, 100 iterazioni; solutori per p: `GAMG/GaussSeidel` con `relTol` 0,1 e 0,01, `GAMG/DICGaussSeidel`, `PCG/DIC` con `relTol` 0,1 e 0,01, `PCG` precondizionato `GAMG`, `smoothSolver/GaussSeidel`, `PBiCGStab/DIC`; `tolerance` 10^{-8} ovunque. Tempi su due processori Xeon a 2,8 GHz: i valori assoluti dipendono dalla macchina, i rapporti no.

Risultati dell'esperimento sui solutori

solutore di p	tempo [s]	iter. medie	iter. totali	residuo p (100)
`GAMG GaussSeidel relTol 0.1`	2,3	2,3	234	$1{,}5 \times 10^{-3}$
`GAMG GaussSeidel relTol 0.01`	3,2	5,3	529	$1{,}4 \times 10^{-3}$
`GAMG DICGaussSeidel relTol 0.1`	2,1	2,1	209	$1{,}5 \times 10^{-3}$
`PCG DIC relTol 0.1`	3,6	75,5	7547	$1{,}5 \times 10^{-3}$
`PCG DIC relTol 0.01`	6,6	154,9	15493	$1{,}4 \times 10^{-3}$
`PCG GAMG relTol 0.1`	2,2	1,9	195	$1{,}4 \times 10^{-3}$
`smoothSolver GaussSeidel relTol 0.1`	24,8	1000,0	100000	$8{,}5 \times 10^{-3}$
`PBiCGStab DIC relTol 0.1`	3,7	36,3	3626	$1{,}5 \times 10^{-3}$

- Mesh 128×128 (16 384 celle), 100 iterazioni SIMPLE identiche in tutto tranne il solutore di p (U: `smoothSolver` con `relTol 0.1` ovunque). La configurazione più rapida è `GAMG DICGaussSeidel relTol 0.1` (2,1 s), la più lenta `smoothSolver GaussSeidel relTol 0.1` (24,8 s): un fattore 12,1 a parità di risultato.
- Il residuo iniziale di p dopo 100 iterazioni è lo stesso per tutte le configurazioni che raggiungono `relTol`, a meno di un fattore 1,07: la velocità di convergenza esterna dipende dall'algoritmo SIMPLE e dal rilassamento, non dal solutore lineare, finché `relTol` è abbastanza piccolo da ridurre l'errore di un ordine di grandezza per passo. Il `smoothSolver` con Gauss–Seidel non ci arriva: esaurisce i 1000 `maxIter`

a ogni passo senza raggiungere `relTol 0.1` (il residuo esterno resta a $8{,}5 \times 10^{-3}$): il rilassamento puntuale smorza solo gli errori ad alta frequenza, e l'equazione della pressione è dominata da quelli a bassa frequenza. È il motivo per cui esiste il multigriglia.

- Passare da `relTol 0.1` a `relTol 0.01` con GAMG costa 1,41 volte il tempo (5 contro 2 cicli per passo) e lascia il residuo esterno a $1{,}4 \times 10^{-3}$ contro $1{,}5 \times 10^{-3}$: il lavoro in più è sprecato, perché l'equazione della pressione cambia alla iterazione successiva.
- Il numero di iterazioni di `PCG DIC` (75 per passo) cresce come N (il numero di celle per lato), quello di GAMG (2) resta costante: su mesh grandi il multigriglia vince sempre; qui, su 128^2 celle, il vantaggio è già di 1,6 volte (e di 2,0 con `relTol 0.01`).

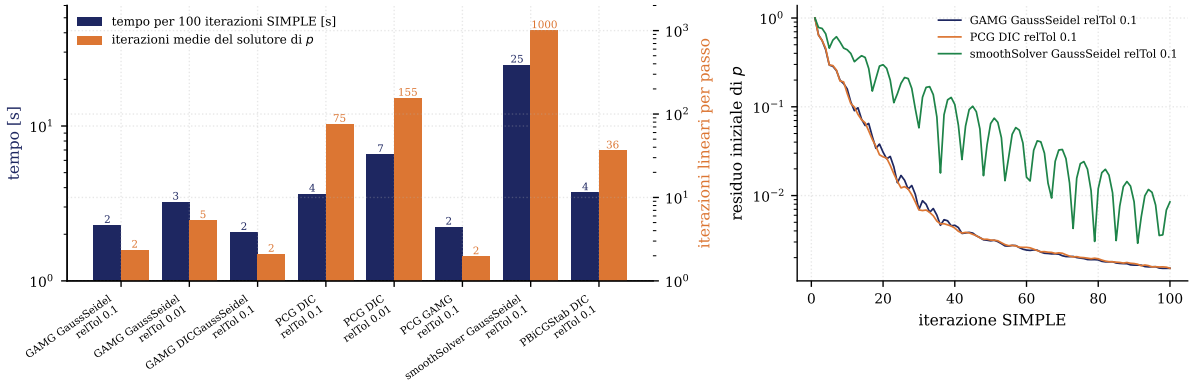


Figura 7.1: Tempo di calcolo per 100 iterazioni SIMPLE e iterazioni medie del solutore lineare della pressione per le otto configurazioni (a sinistra); residuo iniziale di p in funzione dell'iterazione SIMPLE per tre di esse (a destra): la convergenza esterna non dipende dal solutore lineare purché il sistema sia risolto "abbastanza".

Leggere il log del solutore lineare

La riga `GAMG: Solving for p, Initial residual = 0.0123, Final residual = 0.00098, No Iterations 4` dice che il residuo normalizzato della pressione *prima* del solutore era $1{,}23 \times 10^{-2}$ (il residuo esterno), che dopo 4 cicli multigriglia è sceso sotto il 10% (`relTol 0.1`), e che il solutore si è fermato lì. Un numero di iterazioni che cresce di passo in passo, o che raggiunge `maxIter` (1000 per impostazione predefinita, con un avviso nel log), segnala un sistema mal posto: tipicamente una pressione senza riferimento (`pRefCell` mancante in un dominio chiuso) o una mesh con celle degeneri. Un `No Iterations 0` con residuo iniziale sotto `tolerance` dice che l'equazione è già soddisfatta: in un transitorio è il segno dello stato stazionario raggiunto (Cap. 1).

7.4 Approfondimenti

7.4.1 Perché la pressione costa più di tutto

L'equazione della pressione è un'equazione di Poisson: ellittica, con la matrice del solo laplaciano, senza il contributo diagonale del termine temporale o convettivo che rende "facili" le equazioni di trasporto. Il suo numero di condizionamento cresce con il quadrato del numero di celle per lato, e una perturbazione in un punto influenza istantaneamente tutto il dominio: un metodo locale come Gauss–Seidel ha bisogno di $\mathcal{O}(N)$ iterazioni per propagare l'informazione da un capo all'altro. È per questo che nei log il tempo è dominato da `p`, che la multigriglia è stata inventata proprio per questa equazione, e che nel Modulo 2 il sovrarilassamento SOR con ω ottimo era così importante.

7.4.2 Il parallelo e i solutori

In un calcolo parallelo (Cap. 15) ogni processore possiede una porzione di mesh e i solutori lineari scambiano i valori delle celle di confine a ogni iterazione; `GAMG` agglomera anche attraverso i confini (`processorAgglomerator`) e il suo comportamento resta quasi indipendente dal numero di processori, mentre i precondizionatori `DIC/DILU` sono applicati per blocchi e perdono un po' di efficacia. Il numero di iterazioni di `p` che aumenta passando da 1 a 8 processori è normale; se raddoppia, la decomposizione è sfavorevole (troppe facce di confine: `scotch` invece di `simple`).

7.4.3 Solutori accoppiati e GPU

OpenFOAM risolve le equazioni *segregate*: una componente della velocità alla volta, poi la pressione. Esistono formulazioni *accoppiate*, che risolvono simultaneamente **U** e p in un unico sistema a blocchi, più robuste e indipendenti dal rilassamento ma più costose in memoria; la versione ESI ne ha una variante sperimentale per la velocità (`PBiCCCG` per i sistemi vettoriali accoppiati). Le librerie esterne di solutori su GPU (PETSc, AmgX, via il modulo `petsc4Foam`) sostituiscono i solutori interni mantenendo la stessa sintassi di `fvSolution` e sono la frontiera attuale per i calcoli grandi.

Parte III

Algoritmi e modelli

Capitolo 8

L'accoppiamento fra pressione e velocità

8.1 La difficoltà

Nelle equazioni di Navier–Stokes incomprimibili la pressione compare nella quantità di moto ma non ha un'equazione propria: la continuità $\nabla \cdot \mathbf{U} = 0$ è un vincolo sulla velocità che la pressione deve far rispettare, senza che nessuna equazione dica come. Il Modulo 4 ha risolto il problema in due modi: con la formulazione in vorticità e funzione di corrente, che elimina la pressione, e con il metodo a passi frazionari su griglia sfalsata (MAC), dove la pressione è calcolata da un'equazione di Poisson ottenuta imponendo la continuità al campo previsto. Il testo di riferimento (Cap. 15) costruisce la stessa idea nella forma usata da tutti i codici a volumi finiti moderni, su griglia *collocata* (tutte le variabili nei centroidi delle celle): è l'algoritmo SIMPLE e la sua famiglia, con l'interpolazione di Rhie–Chow che rende possibile la collocazione. Il capitolo ne ricava le equazioni, le ritrova riga per riga nel codice di `incompressibleFluid`, e ne misura il comportamento sulla cavità a $Re = 100$.

Il problema della scacchiera

Su una griglia collocata l'equazione della quantità di moto nella cella C contiene il gradiente di pressione $(\nabla p)_C \approx (p_E - p_W)/2h$, che non dipende da p_C: un campo di pressione a scacchiera $(+1, -1, +1, \dots)$ ha gradiente discreto nullo ovunque e non viene "visto" dalla velocità; viceversa, se il flusso di faccia è calcolato interpolando linearmente le velocità di cella, $\dot{m}_e = \rho(u_C + u_E)/2$, un campo di velocità a scacchiera soddisfa la continuità discreta. Le due equazioni sono *disaccoppiate* sui nodi pari e dispari, e la soluzione può contenere oscillazioni spurie che nulla smorza (Sez. 15.2.3 del testo). La griglia sfalsata (Sez. 15.2.4, il MAC del Modulo 4) lo evita memorizzando le velocità sulle facce, dove servono; ma su mesh non strutturate e poliedriche le facce non hanno una direzione privilegiata e lo sfalsamento è impraticabile.

8.2 L'equazione della pressione su griglia collocata

Dalla quantità di moto discretizzata alla pressione

Si scriva l'equazione discretizzata della quantità di moto nella cella C, con il gradiente di pressione tenuto a parte,

$$a_C \mathbf{U}_C + \sum_F a_F \mathbf{U}_F = \mathbf{b}_C - V_C (\nabla p)_C \quad \Longrightarrow \quad \mathbf{U}_C = \underbrace{\frac{\mathbf{b}_C - \sum_F a_F \mathbf{U}_F}{a_C}}_{\mathbf{H}_C / a_C} - \frac{V_C}{a_C} (\nabla p)_C . \tag{8.1}$$

Con la notazione di OpenFOAM, `rAU` $= 1/A$ (con $A = a_C / V_C$) e `HbyA` $= \mathbf{H}/A$ (righe 47–48 di `correctPressure.C`), la (8.1) è `U = HbyA - rAU*grad(p)`: la velocità è la somma di una parte che dipende dai vicini e dalle sorgenti e di una parte proporzionale al gradiente di pressione. Imponendo la continuità $\sum_f \mathbf{U}_f \cdot \mathbf{S}_f = 0$ al campo così espresso, con i valori di faccia ottenuti interpolando *separatamente* i due addendi,

$$\mathbf{U}_f = \overline{\left(\frac{\mathbf{H}}{A}\right)}_f - \overline{\left(\frac{1}{A}\right)}_f (\nabla p)_f , \tag{8.2}$$

si ottiene l'**equazione della pressione**

$$\sum_f \overline{\left(\frac{1}{A}\right)}_f (\nabla p)_f \cdot \mathbf{S}_f = \sum_f \overline{\left(\frac{\mathbf{H}}{A}\right)}_f \cdot \mathbf{S}_f , \tag{8.3}$$

cioè `fvm::laplacian(rAU, p) == fvc::div(phiHbyA)`: un'equazione di Poisson con il laplaciano discretizzato *sulle facce* con la differenza compatta $(p_F - p_C)/|\mathbf{d}_{CF}|$: la pressione nella cella C dipende ora da p_C e dai vicini immediati, la scacchiera non è più invisibile. Questa è l'**interpolazione di Rhie–Chow** (Sez. 15.4 del testo): la differenza fra interpolare la velocità (che dà la scacchiera) e interpolare $\mathbf{H}/A$ e il gradiente separatamente (che la elimina) equivale ad aggiungere al flusso di faccia un termine proporzionale alla differenza fra il gradiente di pressione compatto sulla faccia e quello interpolato dalle celle, $\overline{(1/A)}_f \left[(\nabla p)_f - \overline{(\nabla p)}_f \right] \cdot \mathbf{S}_f$, che è nullo per campi lisci (terzo ordine in h) e grande per la scacchiera: un filtro selettivo. Il flusso di massa `phi` che il codice conserva è quello della (8.2), `phi = phiHbyA - pEqn.flux()` (riga 107), coerente con l'equazione della pressione appena risolta e quindi *conservativo alla precisione del solutore lineare*; la velocità di cella (riga 116) ne è una ricostruzione, e non è esattamente solenoidale: è per questo che gli errori di continuità nel log sono 10^{-9} e non zero, e che il flusso convettivo in `fvm::div(phi, U)` usa `phi` e non $\mathbf{U}$.

8.3 SIMPLE, SIMPLEC, PISO, PIMPLE

La famiglia

Le equazioni (8.1)–(8.3) sono accoppiate e non lineari; gli algoritmi differiscono per come le iterano (Sez. 15.5–15.7 del testo).

SIMPLE
(*Semi-Implicit Method for Pressure-Linked Equations*, Patankar e Spalding 1972; sta-

zionario). A ogni iterazione: (1) si assembla la quantità di moto con il flusso corrente e si rilassa (`equations U 0.7`); (2) si risolve il *predittore* `solve(UEqn == -fvc::grad(p))` con la pressione vecchia (facoltativo: `momentumPredictor`); (3) si calcolano `rAU` e `HbyA`; (4) si risolve l'equazione della pressione; (5) si corregge il flusso e si ricostruisce la velocità; (6) si rilassa la pressione in modo esplicito (`fields p 0.3`) prima di usarla nella successiva quantità di moto. Il rilassamento della pressione è necessario perché nel ricavare la (8.3) si trascura il contributo dei vicini alla correzione di velocità ($\sum a_F \mathbf{U}'_F$): la correzione di pressione è sovrastimata e va smorzata.

SIMPLEC

(*Consistent*, Van Doormaal e Raithby 1984). Approssima il termine trascurato con $\mathbf{U}'_F \approx \mathbf{U}'_C$, cioè sostituisce $1/A$ con $1/(A - H_1)$ dove $H_1 = \sum_F a_F / V_C$ (righe 67–73: `rAtU`); la correzione di pressione è più coerente e non richiede rilassamento (`fields p 1`, $\alpha_U = 0{,}9$): converge in meno iterazioni, ma è meno robusto nelle prime. Si attiva con `consistent yes`.

PISO

(*Pressure-Implicit with Splitting of Operators*, Issa 1986; transitorio). Dentro un passo temporale la quantità di moto è assemblata *una volta* e la pressione è corretta più volte (`nCorrectors 2`–3) riusando `HbyA` con le velocità aggiornate: ogni correttore in più riduce l'errore di separazione fra pressione e velocità di un ordine in Δt. Richiede Co $\lesssim 1$ e nessun rilassamento.

PIMPLE

(SIMPLE + PISO). Ripete per `nOuterCorrectors` volte, dentro il passo temporale, l'intera sequenza (quantità di moto, correttori di pressione, turbolenza), con rilassamento nelle iterazioni intermedie e senza nell'ultima (le voci `Final`): con un solo correttore esterno è PISO; con molti e `turbOnFinalIterOnly`, `outerCorrectorResidualControl` è un SIMPLE a ogni passo che consente Co di 5–20. In OpenFOAM 12 lo stesso modulo `incompressibleFluid` fa tutte e quattro le cose: `steadyState` in `fvSchemes` attiva la modalità SIMPLE (il dizionario `SIMPLE` in `fvSolution`), `Euler`/`backward` quella PIMPLE (il dizionario `PIMPLE`).

Le voci di `SIMPLE`/`PIMPLE` in `fvSolution`

`nNonOrthogonalCorrectors` ripete l'equazione della pressione aggiornando la correzione non ortogonale (Cap. 4): 0 su mesh esaedriche regolari, 1–2 su mesh distorte. `pRefCell`/`pRefValue` (o `pRefPoint`) fissano il livello della pressione quando nessun patch lo fa (domini chiusi: la cavità); con un'uscita a `fixedValue` non servono, e il codice lo verifica (`p.needReference()`, riga 58). `momentumPredictor yes` risolve il predittore: conviene nei flussi ad alta velocità, si può omettere nei multifase. `consistent yes` attiva SIMPLEC. `residualControl` ferma SIMPLE. Nei transitori: `nCorrectors`, `nOuterCorrectors`, e le voci `outerCorrectorResidualControl { U { tolerance 1e-5; relTol 0; } }` per uscire dalle iterazioni esterne a convergenza raggiunta.

8.4 Lettura guidata: `correctPressure.C`


letture/correctPressure.C

```
void Foam::solvers::incompressibleFluid::correctPressure()
{
    volScalarField& p(p_);
    volVectorField& U(U_);
    surfaceScalarField& phi(phi_);

    fvVectorMatrix& UEqn = tUEqn.ref();

    volScalarField rAU(1.0/UEqn.A());
    volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
    surfaceScalarField phiHbyA
    (
        "phiHbyA",
        fvc::flux(HbyA)
      + fvc::interpolate(rAU)*fvc::ddtCorr(U, phi, Uf)
    );

    MRF.makeRelative(phiHbyA);

    if (p.needReference())
    {
        fvc::makeRelative(phiHbyA, U);
        adjustPhi(phiHbyA, U, p);
        fvc::makeAbsolute(phiHbyA, U);
    }

    tmp<volScalarField> rAtU(rAU);

    if (pimple.consistent())
    {
        rAtU = 1.0/max(1.0/rAU - UEqn.H1(), 0.1/rAU);
        phiHbyA +=
            fvc::interpolate(rAtU() - rAU)*fvc::snGrad(p)*mesh.magSf();
        HbyA -= (rAU - rAtU())*fvc::grad(p);
    }

    if (pimple.nCorrPiso() <= 1)
    {
        tUEqn.clear();
    }

    // Update the pressure BCs to ensure flux consistency
    constrainPressure(p, U, phiHbyA, rAtU(), MRF);

    // Evaluate any volume sources
    fvScalarMatrix p_rghEqnSource(fvModels().sourceProxy(p));

    // Non-orthogonal pressure corrector loop
    while (pimple.correctNonOrthogonal())
    {
        fvScalarMatrix pEqn
        (
            fvm::laplacian(rAtU(), p)
         ==
            fvc::div(phiHbyA)
          - p_rghEqnSource
        );

```

```
97          pEqn.setReference
98          (
99              pressureReference.refCell(),
100             pressureReference.refValue()
101         );
102
103         pEqn.solve();
104
105         if (pimple.finalNonOrthogonalIter())
106         {
107             phi = phiHbyA - pEqn.flux();
108         }
109     }
110
111     continuityErrors();
112
113     // Explicitly relax pressure for momentum corrector
114     p.relax();
115
116     U = HbyA - rAtU*fvc::grad(p);
117     U.correctBoundaryConditions();
118     fvConstraints().constrain(U);
```

Listato 8.1: Il correttore di pressione del modulo `incompressibleFluid` (OpenFOAM 12): `rAU` e `HbyA` (47–48), il flusso previsto (49–54), la variante SIMPLEC (67–73), l'equazione della pressione nel ciclo non ortogonale (87–109), il flusso conservativo (107), il rilassamento e la ricostruzione della velocità (114–117).

Riga per riga

`UEqn.A()` è la diagonale per unità di volume della matrice della quantità di moto (già rilassata), `UEqn.H()` il vettore $\mathbf{H}$ della (8.1); `constrainHbyA` impone sui patch con velocità assegnata che `HbyA` valga la velocità di parete, così che il flusso previsto attraverso le pareti sia nullo. `phiHbyA` (49–54) è il flusso $\overline{(\mathbf{H}/A)}_f \cdot \mathbf{S}_f$ più il termine `ddtCorr`, una correzione del flusso temporale che mantiene la coerenza fra il flusso di faccia dell'istante precedente e la velocità di cella (senza di essa il transitorio mostrerebbe un errore di tipo scacchiera smorzato male). `adjustPhi` (61) corregge il flusso sui patch di uscita perché la portata entrante e uscente si bilancino esattamente, condizione necessaria perché l'equazione di Poisson di Neumann abbia soluzione. `constrainPressure` (81) aggiorna le condizioni `fixedFluxPressure`, che impongono alla parete il gradiente di pressione coerente con il flusso nullo (nei casi con gravità o forze di volume `zeroGradient` non basterebbe). Il ciclo delle righe 87–109 è l'equazione (8.3) con la correzione non ortogonale; `setReference` fissa il livello; `pEqn.flux()` è il flusso *calcolato dalla matrice* $\overline{(1/A)}_f (\nabla p)_f \cdot \mathbf{S}_f$, che sottratto a `phiHbyA` dà il flusso conservativo `phi`. `p.relax()` (114) è il rilassamento esplicito α_p (attivo solo in SIMPLE); la riga 116 è la (8.1).

8.5 Esperimenti sulla cavità a $Re = 100$

Il caso `c08_cavita_re100` è la cavità del Cap. 1 con $L = 1$, $U = 1$, $\nu = 0{,}01$ su 128×128 celle: il riferimento sono i profili di velocità di Ghia, Ghia e Shin (1982), già usati nel Modulo 4. Lo script `genera.py` esegue, modificando i dizionari con `foamDictionary`: il calcolo SIMPLE di base; una serie di calcoli con coppie diverse di fattori di rilassamento e con SIMPLEC;

tre schemi convettivi su una mesh grossolana 32×32; il transitorio PISO fino a $t = 20$ con Co $= 0{,}5$; e la prova dei solutori lineari del Cap. 7.

Dati del caso `c08_cavita_re100`

$Re = 100$; 128×128 celle; schemi `steadyState`, `bounded Gauss linearUpwind grad(U)`, laplaciano `corrected`; `GAMG` per p (`relTol 0.1`), `smoothSolver` per **U**; `residualControl` 10^{-6}; rilassamento $\alpha_U = 0{,}7$, $\alpha_p = 0{,}3$ (caso base). PISO: `Euler`, $\Delta t = 0{,}004$ (Co $= 0{,}5$), `nCorrectors 2`, nessun rilassamento, fino a $t = 20$.

c08_cavita_re100.zip

c08_cavita_re100/system/fvSolution

```
16
17 solvers
18 {
19     p
20     {
21         solver          GAMG;
22         smoother        GaussSeidel;
23         tolerance       1e-08;
24         relTol          0.1;
25     }
26     U
27     {
28         solver          smoothSolver;
29         smoother        symGaussSeidel;
30         tolerance       1e-08;
31         relTol          0.1;
32     }
33 }
34
35 SIMPLE
36 {
37     nNonOrthogonalCorrectors 0;
38     consistent      no;
39     pRefCell        0;
40     pRefValue       0;
41     residualControl
42     {
43         p               1e-12;
44         U               1e-06;
45     }
46 }
47
48 relaxationFactors
49 {
50     fields
51     {
52         p               0.3;
53     }
54     equations
55     {
56         U               0.7;
57     }
58 }
```

Listato 8.2: Solutori, dizionario `SIMPLE` con `residualControl` e fattori di rilassamento del caso base.

Risultati del caso `c08_cavita_re100`

- Caso base ($\alpha_U = 0{,}7$, $\alpha_p = 0{,}3$, `linearUpwind`, 128×128): `residualControl` raggiunto dopo 3810 iterazioni in 71,5 s; scarto massimo dai dati di Ghia et al. 0,0047 su u e 0,0088 su v (i dati di riferimento sono su 129×129 nodi).
- Rilassamento (iterazioni fino a 10^{-6}): $(0{,}7,\ 0{,}3) \to 3810$; $(0{,}5,\ 0{,}2) \to$ non converge in 6000; $(0{,}9,\ 0{,}1) \to 1345$; $(0{,}7,\ 0{,}5) \to 3810$; $(0{,}8,\ 0{,}4) \to 2435$; SIMPLEC $\alpha_U = 0{,}90 \to 1221$; SIMPLEC $\alpha_U = 0{,}95 \to 639$. Il numero di iterazioni è governato da α_U: con $\alpha_U = 0{,}7$ le coppie $(0{,}7,\ 0{,}3)$ e $(0{,}7,\ 0{,}5)$ convergono nello stesso numero di iterazioni (3810 e 3810), perché la velocità asintotica di convergenza è quella del modo più lento, smorzato dal rilassamento della quantità di moto; α_p decide invece la stabilità (con $\alpha_p = 0{,}9$ il calcolo diverge nelle prime iterazioni). Portare α_U a 0,9 riduce le iterazioni a 1345; SIMPLEC con $\alpha_U = 0{,}95$ e $\alpha_p = 1$ ne richiede 639, 6,0 volte meno del caso base: su questo problema, laminare e su mesh ortogonale, è la scelta migliore.
- PISO (Euler, $\Delta t = 0{,}004$ s, $\mathrm{Co}_{\max} = 0{,}50$): 5000 passi in 174 s, cioè 2,4 volte il tempo di SIMPLE; a $t = 20$ s lo scarto dal profilo stazionario di SIMPLE è $3{,}6 \times 10^{-4}$ (a $t = 5$ s era $1{,}8 \times 10^{-2}$): lo stato stazionario è lo stesso, raggiunto per via transitoria.
- Schemi su 32×32 (errore massimo su u contro Ghia): `upwind` 0,0232, `linear` 0,0042, `linearUpwind` 0,0042; la mesh grossa con `upwind` sottostima la velocità di ritorno al fondo del 12%.

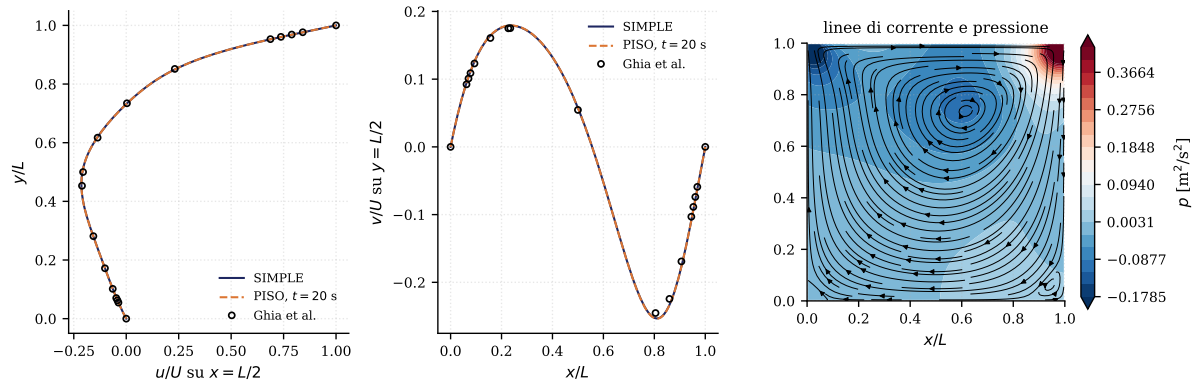


Figura 8.1: Cavità a $Re = 100$, 128×128 celle: profili di u lungo la verticale e di v lungo l'orizzontale di mezzeria da SIMPLE e da PISO a $t = 20$, confrontati con i dati di Ghia et al. (simboli); a destra le linee di corrente e il campo di pressione.

Che cosa insegnano i numeri

Tre cose, che valgono per ogni calcolo stazionario. (1) I fattori di rilassamento decidono il numero di iterazioni ma non il risultato: tutte le combinazioni convergenti danno gli stessi profili alla quinta cifra. La coppia $(0{,}7;\ 0{,}3)$ è un compromesso sicuro; α_p troppo alto fa oscillare o divergere, troppo basso rallenta; SIMPLEC con $\alpha_p = 1$ è il più rapido quando converge. (2) PISO transitorio e SIMPLE stazionario convergono alla stessa soluzione (differenze $\sim 10^{-4}$, dovute al residuo finito e al diverso schema per il termine convettivo), ma il transitorio costa venti volte di più: lo stazionario va usato quando esiste, il transitorio quando lo stazionario non converge perché il flusso è fisicamente instazionario. (3) Lo schema convettivo pesa più dell'algoritmo: su 32×32 `upwind`

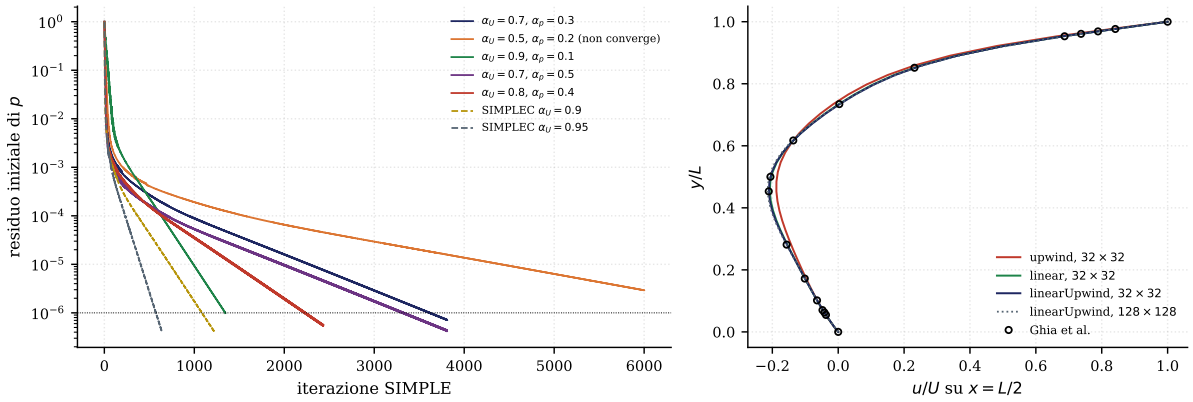


Figura 8.2: Residui iniziali di p e U_x per i diversi fattori di rilassamento e per SIMPLEC (a sinistra); profilo di u sulla mesh 32×32 con `upwind`, `linear` e `linearUpwind` contro Ghia (a destra): la diffusione numerica di `upwind` appiattisce il profilo.

sbaglia il minimo di u del 20%, `linearUpwind` del 3%.

8.6 Approfondimenti

8.6.1 Le condizioni al contorno della pressione

Dove la velocità è imposta (pareti, ingressi) la pressione ha `zeroGradient` o, meglio, `fixedFluxPressure`, che impone il gradiente normale tale che il flusso della (8.2) sia esattamente quello prescritto: in assenza di forze di volume le due coincidono. Dove la pressione è imposta (uscite: `fixedValue`, `totalPressure`) la velocità ha `zeroGradient` o `inletOutlet` o `pressureInletOutletVelocity` (Cap. 11). In un dominio senza nessun patch a pressione imposta (la cavità, un canale periodico) l'equazione di Poisson è di Neumann puro: la matrice è singolare, la soluzione definita a meno di una costante, e il termine noto deve avere integrale nullo (`adjustPhi`): `pRefCell` rimuove la singolarità fissando una cella. Dimenticarlo produce un solutore lineare che non converge (`maxIter` raggiunto) o una pressione che deriva nel tempo.

8.6.2 Il termine `ddtCorr` e la coerenza temporale

Nel transitorio la velocità di cella ricostruita e il flusso di faccia dell'istante precedente non sono coerenti fra loro (il primo è $\mathbf{U}^o$ interpolato, il secondo è $\texttt{phi}^o$ di Rhie–Chow); se il flusso previsto usa solo `HbyA`, costruito dalle velocità di cella, perde l'informazione del flusso precedente e introduce una dipendenza spuria dal passo temporale e dal rilassamento (Sez. 15.9 del testo, trattamento del termine instazionario con Rhie–Chow). `fvc::ddtCorr(U, phi)` aggiunge $\overline{(1/A)}_f \, [\,\texttt{phi}^o - (\mathbf{U}^o)_f \cdot \mathbf{S}_f\,]/\Delta t$, smorzato da un coefficiente che lo annulla dove $\mathbf{U}^o$ e $\texttt{phi}^o$ sono già coerenti: è un dettaglio di implementazione che fa la differenza fra una soluzione transitoria indipendente da Δt e una che non lo è.

8.6.3 Comprimibile e multifase: la stessa idea

Nei flussi comprimibili (Cap. 10) la densità dipende dalla pressione e l'equazione della pressione acquista un termine temporale $\partial\rho/\partial t = \psi\,\partial p/\partial t$ e un termine convettivo in p: diventa un'equazione di convezione–diffusione (Sez. 16.5 del testo), iperbolica nel limite supersonico; i moduli `fluid` e `isothermalFluid` usano lo stesso schema PIMPLE con questa equazione. Nei flussi multifase la densità variabile entra nel flusso di massa e nel termine di gravità con la pressione modificata $p_{\mathrm{rgh}} = p - \rho\mathbf{g}\cdot\mathbf{x}$: l'algoritmo non cambia.

Capitolo 9

La turbolenza: modelli RANS e trattamento di parete

9.1 Dalle equazioni istantanee a quelle mediate

Quasi tutti i flussi aeronautici sono turbolenti: fluttuazioni tridimensionali, instazionarie, su scale che vanno dalla dimensione del corpo alla scala di Kolmogorov $\eta \sim L\,Re^{-3/4}$. Risolverle tutte (DNS) costa $Re^{9/4}$ celle: 10^{14} per un'ala. Le equazioni *mediate alla Reynolds* (RANS) rinunciano alle fluttuazioni e descrivono il campo medio; il prezzo è un termine incognito, il tensore degli sforzi di Reynolds, che va *modellato*. Il testo di riferimento (Cap. 17) presenta la derivazione e i modelli a due equazioni; questo capitolo li riassume e si concentra su ciò che decide la qualità di un calcolo RANS in OpenFOAM: la scelta del modello, il trattamento della parete, la mesh vicino a essa e le condizioni all'ingresso.

Media di Reynolds e ipotesi di Boussinesq

Scomponendo $\mathbf{u} = \bar{\mathbf{U}} + \mathbf{u}'$ e mediando le equazioni di Navier–Stokes (Sez. 17.2 del testo) la continuità resta $\nabla \cdot \bar{\mathbf{U}} = 0$ e la quantità di moto acquista il termine $-\nabla \cdot (\overline{\mathbf{u}'\mathbf{u}'})$, il tensore degli *sforzi di Reynolds* $\boldsymbol{\tau}^R = -\rho\overline{\mathbf{u}'\mathbf{u}'}$ (sei incognite). L'ipotesi di Boussinesq (Sez. 17.3) lo lega al gradiente della velocità media come uno sforzo viscoso con una *viscosità turbolenta* ν_t:

$$-\overline{u_i' u_j'} = \nu_t\Big(\frac{\partial \bar{U}_i}{\partial x_j} + \frac{\partial \bar{U}_j}{\partial x_i}\Big) - \tfrac{2}{3}k\delta_{ij}, \qquad k = \tfrac{1}{2}\overline{u_i' u_i'}, \tag{9.1}$$

così che le equazioni medie sono le (2.5) con $\nu_{\text{eff}} = \nu + \nu_t$ e il termine $-\frac{2}{3}k$ assorbito nella pressione. Tutto il problema si riduce a calcolare $\nu_t(\mathbf{x})$, che non è una proprietà del fluido ma del *flusso*: i modelli a due equazioni la ricavano da due grandezze turbolente trasportate, con equazioni della forma generale (2.3) e termini di produzione e distruzione modellati. Il termine `momentumTransport->divDevSigma(U)` nel codice (Cap. 2) è $-\nabla \cdot [\nu_{\text{eff}}(\nabla\mathbf{U} + (\nabla\mathbf{U})^{\mathrm{T}})]$ con la parte deviatorica, e il modello selezionato in `momentumTransport` decide come ν_t è calcolata.

I modelli: k–ε, k–ω SST, Spalart–Allmaras

k–ε **standard** (Launder e Spalding 1974): trasporta l'energia cinetica turbolenta k e la sua dissipazione ε, con $\nu_t = C_\mu k^2/\varepsilon$, $C_\mu = 0{,}09$, $C_{\varepsilon1} = 1{,}44$, $C_{\varepsilon2} = 1{,}92$, $\sigma_k = 1$, $\sigma_\varepsilon = 1{,}3$ (Sez. 17.5.1). Robusto, ben calibrato per flussi liberi (getti, scie) e per gli strati limite attaccati con funzioni di parete; non valido fino alla parete (richiede funzioni di parete), sovrastima la produzione nei punti di ristagno e nei gradienti di pressione avversi, con separazioni ritardate o assenti. Varianti: `realizableKE`, `RNGkEpsilon`, `LaunderSharmaKE` (basso Reynolds).
k–ω **SST** (Menter 1994, Sez. 17.5.4): trasporta k e la frequenza specifica $\omega = \varepsilon/(C_\mu k)$, con $\nu_t = a_1 k/\max(a_1\omega, SF_2)$; usa il modello k–ω di Wilcox vicino alla parete (integrabile fino a essa, senza funzioni di parete se $y^+ \lesssim 1$) e passa al k–ε nel flusso libero tramite una funzione di raccordo F_1, evitando la sensibilità del k–ω al valore di ω all'esterno; il limitatore dello sforzo (F_2) migliora i gradienti avversi e la separazione. È il modello di riferimento per l'aerodinamica esterna e per i flussi con separazione moderata; in OpenFOAM `kOmegaSST`.
Spalart–Allmaras (1992): una sola equazione per una viscosità modificata $\tilde{\nu}$, calibrata sugli strati limite aerodinamici; economico, robusto, eccellente per profili e ali a incidenza moderata, meno per i flussi liberi e le separazioni massicce; in OpenFOAM `SpalartAllmaras`, con il campo `nuTilda`.
Tutti richiedono la *distanza dalla parete* y (per le funzioni di smorzamento e F_1, F_2), che OpenFOAM calcola con `wallDist { method meshWave; }` in `fvSchemes`.

9.2 La parete: legge universale e funzioni di parete

La legge di parete

Vicino a una parete liscia, con la velocità d'attrito $u_\tau = \sqrt{\tau_w/\rho}$ e le variabili adimensionali $y^+ = y u_\tau/\nu$, $u^+ = \bar{U}/u_\tau$, il profilo di velocità è universale (Sez. 17.8.1 del testo):

$$u^+ = y^+ \quad (y^+ \lesssim 5), \qquad u^+ = \frac{1}{\kappa}\ln(Ey^+) \quad (30 \lesssim y^+ \lesssim 300), \tag{9.2}$$

la prima nel sottostrato viscoso, la seconda nella regione logaritmica, con $\kappa = 0{,}41$ ed $E = 9{,}8$ (si incrociano a $y^+ = 11{,}53$). Le **funzioni di parete** sfruttano questa universalità: se la prima cella ha il centroide nella regione logaritmica, lo sforzo di parete si ricava dalla velocità della cella *senza* risolvere il sottostrato, imponendo una viscosità turbolenta equivalente sulla faccia di parete (Sez. 17.8.2),

$$\nu_{t,w} = \nu\left(\frac{y^+\kappa}{\ln(Ey^+)} - 1\right) \quad \text{per } y^+ > 11{,}53, \qquad \nu_{t,w} = 0 \text{ altrimenti}, \tag{9.3}$$

così che $\tau_w = (\nu + \nu_{t,w})\,\bar{U}_P/y_P$ riproduce la legge logaritmica; y^+ è calcolato con $u_\tau = C_\mu^{1/4}\sqrt{k_P}$ (la variante "k" delle funzioni di parete, insensibile ai punti di separazione dove $\tau_w \to 0$). In OpenFOAM è la condizione `nutkWallFunction` sul campo `nut`, accompagnata da `kqRWallFunction` (gradiente nullo) per k, `epsilonWallFunction` o `omegaWallFunction` per ε e ω, che fissano nella prima cella i valori di equilibrio della regione logaritmica. `nutUSpaldingWallFunction` usa invece la legge continua di Spalding, valida a qualunque y^+ (Sez. 17.8.3, funzioni di parete "migliorate"): è la

scelta robusta quando lo y^+ varia lungo la parete, com'è inevitabile su un profilo alare. `nutLowReWallFunction` impone $\nu_{t,w} = 0$: per le mesh risolte ($y^+ \lesssim 1$) con i modelli che lo consentono (SST, SA, k–ε a basso Reynolds).

Regola pratica per la prima cella

Due strategie, da non mescolare: (a) **funzioni di parete**: $30 \lesssim y^+ \lesssim 300$ nella prima cella ($y^+ \approx 50$ è l'obiettivo), con `nutkWallFunction`; mesh economica, accuratezza buona negli strati limite attaccati, incerta nelle separazioni e nei gradienti avversi forti; (b) **parete risolta**: $y^+ \lesssim 1$ e almeno 10–15 celle entro $y^+ = 30$ con rapporto di espansione $\leq 1{,}2$, con `nutLowReWallFunction` e un modello integrabile (SST, SA); mesh costosa (lo strato di celle sottili domina il conteggio), accuratezza migliore per attrito, transizione e separazione. Lo y^+ non si conosce prima del calcolo: si stima con una correlazione per la lastra piana, $C_f \approx 0{,}058\, Re_x^{-0,2}$, $u_\tau = U\sqrt{C_f/2}$, $y_1 = y^+ \nu/u_\tau$, si calcola, si misura con la function object `yPlus` e si corregge la mesh. La regione $5 < y^+ < 30$ (*buffer*) è la peggiore per entrambe le strategie, e con `nutkWallFunction` va evitata; `nutUSpaldingWallFunction` la tollera.

Condizioni all'ingresso per le grandezze turbolente

All'ingresso servono k, ε o ω (o $\tilde{\nu}$). Si assegnano da un'*intensità di turbolenza* $I = u'/U$ (1% in galleria del vento, 5% in un condotto, 10% dietro una griglia) e da una *scala di lunghezza* ℓ (il 7% del diametro idraulico in un condotto; lo spessore dello strato limite o una frazione della corda in aerodinamica):

$$k = \tfrac{3}{2}(UI)^2, \qquad \varepsilon = C_\mu^{3/4}\frac{k^{3/2}}{\ell}, \qquad \omega = \frac{k^{1/2}}{C_\mu^{1/4}\ell}, \qquad \nu_t = C_\mu \frac{k^2}{\varepsilon}, \tag{9.4}$$

oppure con le condizioni che fanno il conto da sé: `turbulentIntensityKineticEnergyInlet` (`intensity 0.05`), `turbulentMixingLengthDissipationRateInlet` e `turbulentMixingLengthFrequencyInlet` (`mixingLength 0.01`). Per Spalart–Allmaras, $\tilde{\nu}_\infty = 3\nu$–$5\nu$ in corrente libera. Un errore classico è un ν_t/ν all'ingresso di 10^3–10^4 (da ℓ troppo grande), che falsa il flusso ben oltre l'ingresso; il rapporto ν_t/ν (`turbulenceFields`) va controllato: 1–10 in galleria del vento, 10–100 in un condotto.

I file del caso turbolento

`constant/momentumTransport`: `simulationType RAS; RAS { model kOmegaSST; turbulence on; printCoeffs on; }` (le costanti del modello si possono sovrascrivere in un sotto-dizionario `kOmegaSSTCoeffs`). In `0/`: `k`, `omega` (o `epsilon`, `nuTilda`) con le condizioni all'ingresso di sopra, `zeroGradient` o `inletOutlet` all'uscita, e le funzioni di parete sulle pareti; `nut` con `calculated` all'ingresso e all'uscita e la funzione di parete scelta sulle pareti. In `fvSchemes`: `div(phi,k) bounded Gauss limitedLinear 1;` (e per `omega`, `epsilon`, `nuTilda`); `wallDist { method meshWave; }`. In `fvSolution`: i solutori per `"(k|omega|epsilon|nuTilda)"` e i rilassamenti 0,7.

9.3 Esperimento 1: la lastra piana

Il caso `c09_lastra` calcola lo strato limite su una lastra piana a gradiente di pressione nullo, laminare a $Re_L = 10^5$ (confronto con Blasius, $C_f = 0{,}664/\sqrt{Re_x}$, dal Modulo 3) e turbolento a $Re_L = 10^7$ con tre trattamenti: `kOmegaSST` su mesh risolta ($y^+ \approx 1$), `kOmegaSST` e `kEpsilon` su mesh con funzioni di parete ($y^+ \approx 50$), confrontando il coefficiente d'attrito locale con le correlazioni di Schultz–Grunow ($C_f = 0{,}370(\log_{10} Re_x)^{-2{,}584}$) e di White ($C_f = 0{,}455/\ln^2(0{,}06 Re_x)$) e il profilo di velocità in variabili di parete con la legge logaritmica. Il coefficiente d'attrito è calcolato in due modi: con la function object `wallShearStress` di OpenFOAM e con la funzione `forze` di `foamcase.py`, che devono coincidere.

Dati del caso `c09_lastra`

Lastra di lunghezza $L = 1$ m preceduta da un tratto libero di 0,2 m (patch `symmetry`); altezza del dominio 0,5 m; $U = 1$ m/s; laminare: $\nu = 10^{-5}$ ($Re_L = 10^5$), mesh 200×80 con $y_1 = 2 \times 10^{-4}$ m; turbolento: $\nu = 10^{-7}$ ($Re_L = 10^7$), $I = 1\%$, $\nu_t/\nu = 5$ all'ingresso; mesh risolta $y_1 = 2 \times 10^{-6}$ m ($y^+ \approx 1$), mesh per funzioni di parete $y_1 = 10^{-4}$ m ($y^+ \approx 50$); SIMPLE, `bounded Gauss linearUpwind`; residui 10^{-6}.

c09_lastra.zip

c09_lastra/0/nut

```
16
17 dimensions      [ 0 2 -1 0 0 0 0 ];
18
19 internalField   uniform 0;
20
21 boundaryField
22 {
23     ingresso
24     {
25         type            calculated;
26         value           uniform 0;
27     }
28     uscita
29     {
30         type            calculated;
31         value           uniform 0;
32     }
33     sopra
34     {
35         type            calculated;
36         value           uniform 0;
37     }
38     simmetria
39     {
40         type            symmetryPlane;
41     }
42     lastra
43     {
44         type            nutkWallFunction;
45         value           uniform 0;
```

Listato 9.1: La viscosità turbolenta: `nutkWallFunction` sulla lastra (variante con funzioni di parete; `nutLowReWallFunction` nella variante risolta), `calculated` altrove.

Risultati del caso `c09_lastra`

variante	iter.	y^+ medio	$C_f(0{,}5)$	riferimento	scarto medio
laminare	188	0,4	0,00307	0,00298	2,8%
k–ω SST, $y^+ \approx 1$	3000	0,4	0,00261	0,00272	5,1%
k–ω SST, funzioni di parete	3000	18,5	0,00256	0,00272	6,7%
k–ε, funzioni di parete	3000	19,0	0,00271	0,00272	1,6%

- Il caso laminare ($\mathrm{Re}_L = 10^5$) segue Blasius con errore medio del 2,8%: è la verifica che la mesh e lo schema risolvono lo strato limite ($\delta/x \approx 5/\sqrt{\mathrm{Re}_x}$, circa 79 celle nello strato a $x = 1$ m).
- Nel caso turbolento risolto ($y^+ \approx 0{,}4$) il profilo ricostruisce il sottostrato viscoso e la legge logaritmica senza alcun modello di parete; nelle varianti con funzioni di parete ($y^+ \approx 18$) il primo centro cella cade al limite inferiore della regione logaritmica (`nutkWallFunction` applica la legge logaritmica sopra $y^+ \approx 11$ e quella lineare sotto) e il profilo è noto solo da lì in su.
- C_f a $x = 0{,}5$ m: SST risolta 0,00261, SST con funzioni di parete 0,00256, k–ε 0,00271 contro 0,00272 (Schultz–Grunow) e 0,00286 (White): tutti entro il 6,7%, che è anche la dispersione fra le correlazioni (e fra gli esperimenti) a questo numero di Reynolds; il modello SST risolto a parete è il più basso, come nei confronti della NASA TMR sulla lastra piana.
- Lo sforzo di parete calcolato da `foamcase.py` e quello della function object `wallShearStress` differiscono al più di 0,20% (stessa formula: $\nu_{\mathrm{eff}}\,\partial u/\partial n$ con il gradiente della prima cella).

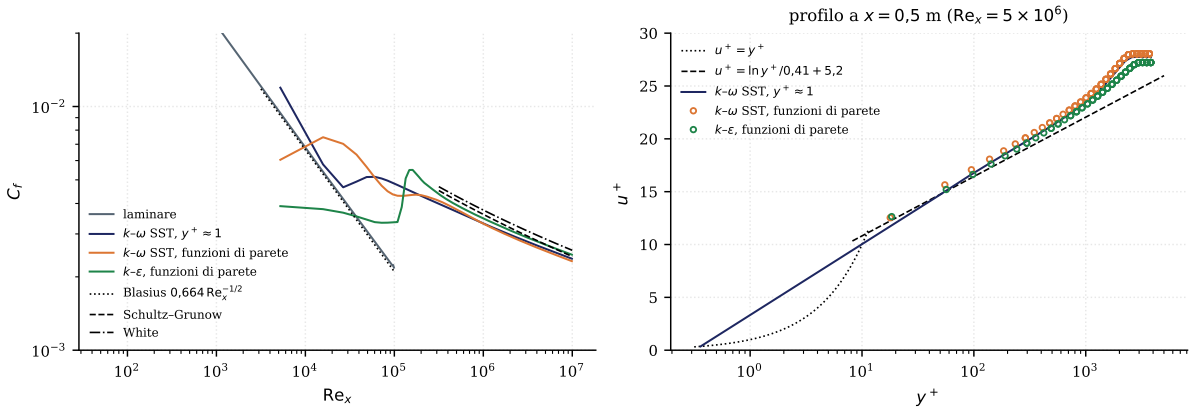


Figura 9.1: Lastra piana: coefficiente d'attrito locale in funzione di Re_x per il caso laminare (Blasius) e per i tre calcoli turbolenti con le correlazioni di Schultz–Grunow e di White (a sinistra); profili di velocità in variabili di parete a $Re_x = 5 \times 10^6$ con la legge lineare e quella logaritmica (a destra).

9.4 Esperimento 2: il gradino a valle

Il gradino a valle (*backward-facing step*) è il banco di prova classico dei modelli di turbolenza per i flussi separati: la lunghezza di riattacco x_r dipende dal modello in modo noto. Il caso `c10_gradino` riproduce la configurazione di Driver e Seegmiller (1985): rapporto di

espansione 1,125, $Re_h = 37\,500$, riattacco misurato a $x_r/h = 6{,}26 \pm 0{,}10$; i modelli k–ε (noto per sottostimare, $\approx$ 5,2–5,5), k–ω SST ($\approx$ 6,3–6,5) e Spalart–Allmaras ($\approx$ 6,0–6,2) sono confrontati sulla stessa mesh con funzioni di parete, e il riattacco è individuato dal cambio di segno dello sforzo di parete sul fondo.


c10_gradino.zip

Dati del caso `c10_gradino`

$h = 0{,}0127$ m, canale a monte alto $8h$, a valle $9h$; tratto a monte lungo $30h$ (lo strato limite si sviluppa fino a $\delta \approx 1{,}5h$ al gradino, come nell'esperimento), a valle $40h$; $U_\infty = 44{,}2$ m/s, $\nu = 1{,}5 \times 10^{-5}$ ($Re_h = 37\,400$); $I = 1\%$; mesh di tre blocchi con y_1 tale che $y^+ \approx 40$ sulle pareti; SIMPLE, `bounded Gauss linearUpwind grad(U)`; modelli `kEpsilon`, `kOmegaSST`, `SpalartAllmaras`.

Risultati del caso `c10_gradino`

modello	x_r/h	scarto	iterazioni	y^+ fondo	$C_{f,\min} \times 10^3$
k–ε	5,33	-15%	4000	11	-1,48
k–ω SST	6,07	-3%	4000	9	-1,40
Spalart–Allmaras	5,55	-11%	4000	9	-2,09
esperimento	6,26	–	–	–	–

- Il k–ε riattacca troppo presto ($x_r/h = 5{,}33$, scarto del 15%): la sua viscosità turbolenta, troppo alta dove il gradiente di pressione è avverso, fa diffondere lo strato di taglio e chiude la bolla prima. È il difetto documentato del modello che ha motivato le correzioni di Menter.
- SST e Spalart–Allmaras danno $x_r/h = 6{,}07$ e $x_r/h = 5{,}55$: entro il 11% dell'esperimento, che è quanto ci si può aspettare da un calcolo RANS 2D con funzioni di parete ($y^+ \approx 9$).
- Il minimo di C_f nel ricircolo è di $-1{,}40 \times 10^{-3}$ per SST: lo sforzo è negativo perché il fluido a parete scorre verso il gradino; il riattacco è il punto dove C_f torna positivo, la definizione usata da `genera.py`.
- Convergenza: i residui si assestano intorno a 10^{-4} (U) e 2×10^{-3} (p) dopo circa 1000 iterazioni senza scendere oltre, ma il punto di riattacco è fermo: per SST vale 6,075, 6,075, 6,075, 6,075 alle iterazioni 1000, 2000, 3000, 4000. È un caso tipico di *stallo del residuo* con soluzione integrale convergente (Cap. 6): si giudica sulle grandezze d'interesse, non sulla sola riga del registro.
- Lo sforzo di parete sul fondo calcolato da `foamcase.py` e dalla function object `wallShearStress` differiscono al più di 0,02%.

9.5 Approfondimenti

9.5.1 Transizione e modelli a basso Reynolds

I modelli RANS descritti assumono il flusso turbolento ovunque; su un profilo a Re di qualche milione la transizione avviene nel primo 5–20% della corda e l'ipotesi è accettabile, ma a $Re < 10^6$ (modelli, UAV, pale) la bolla laminare e la posizione della transizione dominano

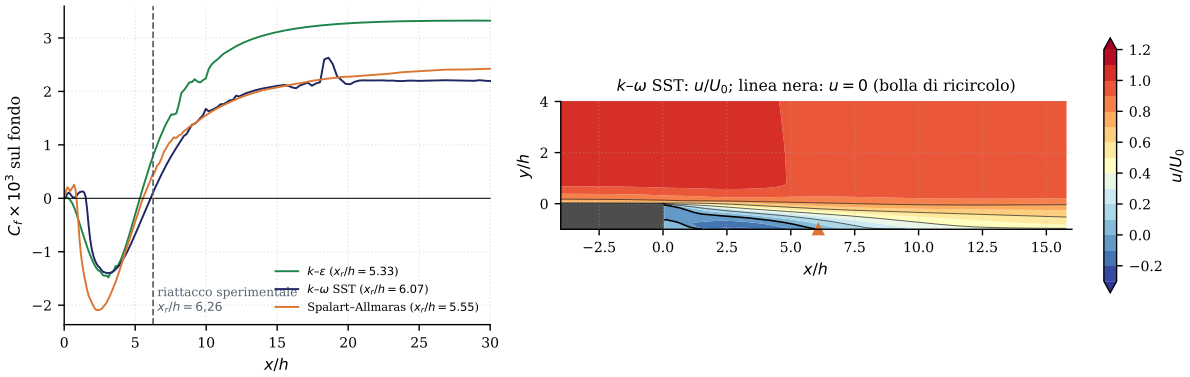


Figura 9.2: Gradino a valle: coefficiente d'attrito sul fondo a valle del gradino per i tre modelli, con il riattacco sperimentale (a sinistra); linee di corrente e campo di velocità con k–ω SST (a destra).

portanza e resistenza. I modelli di transizione (`kOmegaSSTLM` di Langtry–Menter, con due equazioni aggiuntive per l'intermittenza γ e il numero di Reynolds di transizione; `kkLOmega`) sono disponibili e richiedono mesh risolte ($y^+ < 1$) e intensità di turbolenza all'ingresso realistiche, perché la transizione dipende da essa.

9.5.2 LES e DES

La simulazione dei grandi vortici (LES) risolve le strutture turbolente più grandi della mesh e modella le altre (`simulationType LES`, modelli `Smagorinsky`, `kEqn`, `WALE`); richiede mesh fini in tutte e tre le direzioni ($\Delta^+ \sim 20$–50 a parete), passi temporali piccoli (Co < 1), schemi poco dissipativi (`linear`, `LUST`, `backward`), medie nel tempo lunghe, ed è tridimensionale per natura: costa da 10^2 a 10^4 volte un calcolo RANS. I metodi ibridi DES/DDES/IDDES (`SpalartAllmarasDDES`, `kOmegaSSTDES`) usano RANS negli strati limite e LES nelle scie e nelle separazioni massicce, e sono il compromesso attuale per le configurazioni in stallo o con corpi tozzi. Il tutorial `pitzDailyLES` è il punto di partenza.

9.5.3 Sulla scelta del modello

Non esiste il modello migliore: esiste quello calibrato per la classe di flussi in esame, e la letteratura (le pagine del *Turbulence Modeling Resource* della NASA raccolgono casi di validazione con risultati di riferimento per SA e SST) dice quanto aspettarsi. Per i flussi attaccati e i profili a incidenza moderata SA e SST sono equivalenti ed eccellenti (C_L entro il 2%, C_D entro il 5–10%); per le separazioni indotte da gradiente di pressione SST è preferibile; per i getti e le scie libere k–ε realizable; per le separazioni massicce e lo stallo profondo nessun RANS è affidabile e si passa a DES. In ogni caso il risultato va sempre accompagnato dallo studio di sensibilità alla mesh e al modello (Cap. 16).

Capitolo 10

Flussi comprimibili

10.1 Che cosa cambia

Quando la densità varia, la continuità non è più un vincolo sulla sola velocità ma un'equazione di evoluzione per ρ, la pressione è legata alla densità e alla temperatura dall'equazione di stato, e serve l'equazione dell'energia. Il testo di riferimento (Cap. 16) estende l'algoritmo SIMPLE a questo caso mostrando che l'equazione della pressione diventa di convezione–diffusione, con un carattere che passa da ellittico (subsonico) a iperbolico (supersonico): è l'approccio *pressure-based*, adottato dai moduli `fluid` e `isothermalFluid` di OpenFOAM, valido da Mach zero a Mach qualche unità. L'alternativa *density-based* (`shockFluid`, l'erede di `rhoCentralFoam`) risolve le equazioni di Eulero/Navier–Stokes in forma conservativa con flussi numerici costruiti per catturare le discontinuità: è quella giusta per urti ed espansioni, e la useremo per i due casi del capitolo.

Le equazioni e la chiusura termodinamica

In forma conservativa, per un gas perfetto non viscoso (Eulero):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{U}) = 0, \qquad \frac{\partial (\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U}\mathbf{U}) + \nabla p = 0, \qquad \frac{\partial (\rho E)}{\partial t} + \nabla \cdot \left[(\rho E + p)\mathbf{U} \right] = 0, \tag{10.1}$$

con $E = e + \frac{1}{2}|\mathbf{U}|^2$, $p = \rho R T$, $e = c_v T$; la velocità del suono è $a = \sqrt{\gamma R T}$ e il numero di Mach $Ma = |\mathbf{U}|/a$. In OpenFOAM la pressione è *assoluta* (Pa, `dimensions [1 -1 -2 0 0 0 0]`), e la termodinamica è nel file `constant/physicalProperties` con il dizionario `thermoType`: `type hePsiThermo` (termodinamica basata sulla compressibilità $\psi = \partial\rho/\partial p = 1/RT$, per i gas), `equationOfState perfectGas`, `thermo hConst` (calore specifico costante: `Cp 1004.5`), `transport const` (`mu` e `Pr`; `mu 0` per il flusso non viscoso), `energy sensibleInternalEnergy` (l'equazione risolta è quella di e; `sensibleEnthalpy` per h), `specie { molWeight 28.96; }` da cui $R = 8314/M = 287\ \mathrm{J/(kg\,K)}$ e $\gamma = c_p/(c_p - R) = 1{,}4$. I campi in `0/` sono `p`, `T` e `U`; ρ e e sono calcolati.

Il metodo centrale-upwind di Kurganov e Tadmor

`shockFluid` discretizza le (10.1) con lo schema *centrale-upwind* di Kurganov, Noelle e Petrova (2001), una variante di Kurganov–Tadmor: su ogni faccia i campi primitivi ρ, $\mathbf{U}$, T sono *ricostruiti* dai due lati con uno schema limita-

to (`reconstruct(rho) vanLeer`, `reconstruct(U) vanLeerV`, `reconstruct(T) vanLeer` in `interpolationSchemes`), si calcolano le velocità massime di propagazione dei due lati, $a^{\pm} = \max(|\mathbf{U}_f^{\pm} \cdot \mathbf{n}| + a_f^{\pm}, \dots)$, e il flusso numerico è una media dei flussi dei due lati pesata con $a^{\pm}$ più un termine di dissipazione proporzionale al salto: $\mathbf{F}_f = \dfrac{a^+\mathbf{F}^- + a^-\mathbf{F}^+}{a^+ + a^-} - \dfrac{a^+a^-}{a^+ + a^-}(\mathbf{q}^+ - \mathbf{q}^-)$ (con `fluxScheme Kurganov`; `Tadmor` usa $a^+ = a^-$). Non richiede la soluzione del problema di Riemann (è un metodo *senza* Riemann), è del secondo ordine dove la soluzione è regolare e cattura gli urti in 2–3 celle senza oscillazioni grazie ai limitatori. L'avanzamento è esplicito (`Euler`, con `maxCo` $\leq 0{,}5$ calcolato con $|\mathbf{U}| + a$); i termini viscosi e di conduzione, se presenti, sono trattati implicitamente in un passo correttore. La stessa famiglia di schemi era usata nel Modulo 2 per il tubo d'urto e l'equazione di Burgers.

Condizioni al contorno nel comprimibile

Il numero di condizioni da imporre su un contorno è quello delle caratteristiche entranti: in un **ingresso supersonico** tutte le grandezze sono imposte (p, T, $\mathbf{U}$ `fixedValue`); in un'**uscita supersonica** nessuna (`zeroGradient` per tutte); in un **ingresso subsonico** si impongono T (o la temperatura totale, `totalTemperature`) e $\mathbf{U}$ (o la pressione totale, `totalPressure`) e si lascia p libera (`zeroGradient`); in un'**uscita subsonica** si impone solo p (`fixedValue` o, per lasciar uscire le onde senza riflessioni, `waveTransmissive` con `gamma 1.4` e `lInf`, `fieldInf`) e il resto è `zeroGradient`. Sulle **pareti** non viscose `slip` per $\mathbf{U}$ e `zeroGradient` per p e T (adiabatica); viscose `noSlip` e, per T, `fixedValue` o `zeroGradient`. Imporre una condizione di troppo (la pressione a un'uscita supersonica) produce un urto numerico fittizio al contorno.

10.2 Esperimento 1: il tubo d'urto

Il tubo d'urto di Sod è il banco di prova per eccellenza: due stati di gas in quiete separati da un diaframma, che al tempo zero si rompe; la soluzione esatta (un'onda d'urto, una discontinuità di contatto e un ventaglio di espansione) si calcola risolvendo il problema di Riemann, come nel Modulo 2. Il caso `c11_tubo_urto` è il tutorial di OpenFOAM con rapporto di pressione 10: il confronto riguarda la posizione delle tre onde e i valori dei plateau di pressione, densità e velocità, e lo script `fig11_sod.py` calcola la soluzione esatta e misura l'errore con tre ricostruzioni (`vanLeer`, `vanAlbada`, `Minmod`) e due risoluzioni.

Dati del caso `c11_tubo_urto`

Tubo $[-5, 5]$ m, 1000 celle (e 4000); stato sinistro $p = 10^5$ Pa, $T = 348{,}4$ K; stato destro $p = 10^4$ Pa, $T = 278{,}7$ K (densità 1,0 e 0,125 kg/m^3: il problema di Sod); aria, $\gamma = 1{,}4$, non viscosa; $\Delta t = 5 \times 10^{-6}$ s, $t_{\text{fin}} = 7$ ms; `fluxScheme Kurganov`, ricostruzione `vanAlbada`.


c11_tubo_urto.zip

```
16
17 thermoType
18 {
19     type            hePsiThermo;
20     mixture         pureMixture;
21     transport       const;
22     thermo          hConst;
23     equationOfState perfectGas;
24     specie          specie;
25     energy          sensibleInternalEnergy;
26 }
27
28 mixture
29 {
30     specie
31     {
32         molWeight       28.96;
33     }
34     thermodynamics
35     {
36         Cp              1004.5;
37         hf              2.544e+06;
38     }
39     transport
40     {
41         mu              0;
42         Pr              1;
43     }
44 }
```



c11_tubo_urto/constant/physicalProperties

Listato 10.1: La termodinamica del gas perfetto non viscoso.

```
16
17 fluxScheme      Kurganov;
18
19 ddtSchemes
20 {
21     default         Euler;
22 }
23
24 gradSchemes
25 {
26     default         Gauss linear;
27 }
28
29 divSchemes
30 {
31     default         none;
32 }
33
34 laplacianSchemes
35 {
36     default         Gauss linear orthogonal;
37 }
38
39 interpolationSchemes
40 {
41     default         none;
42
43     reconstruct(rho) vanAlbada;
```



c11_tubo_urto/system/fvSchemes

```
    reconstruct(U)   vanAlbadaV;
    reconstruct(T)   vanAlbada;

    flux(rhoU)       linear;
}

snGradSchemes
{
    default          orthogonal;
}
```

Listato 10.2: Gli schemi del solutore density-based: flusso di Kurganov e ricostruzione limitata delle variabili primitive.

Risultati del caso `c11_tubo_urto`

- Soluzione esatta a $t = 7$ ms: $p^* = 0{,}3031\, p_L$, $u^* = 293{,}3$ m/s, densità 0,4262 (dietro il ventaglio) e 0,2655 kg/m^3 (dietro l'urto); urto in $x = 3{,}879$ m (velocità 554,2 m/s), contatto in $x = 2{,}053$ m, ventaglio fra $-2{,}620$ e $-0{,}156$ m.
- Con 1000 celle e `vanAlbada` l'urto è in $x = 3{,}886$ m ed è risolto in 5,1 celle; il plateau dietro l'urto ha uno scostamento di -0,01% dal valore esatto: le posizioni e i salti sono quelli giusti perché lo schema è conservativo.
- Errore L_1 sulla densità (normalizzato con $\rho_L - \rho_R$):

celle	`vanLeer`	`vanAlbada`	`Minmod`
1000	$1{,}87 \times 10^{-3}$	$1{,}95 \times 10^{-3}$	$2{,}21 \times 10^{-3}$
4000	$7{,}13 \times 10^{-4}$	$7{,}33 \times 10^{-4}$	$8{,}14 \times 10^{-4}$

  Passando da 1000 a 4000 celle l'errore si riduce di 2,66 volte: ordine osservato 0,71, cioè fra 1/2 e 1 come ci si aspetta quando la soluzione ha discontinuità (l'ordine formale dello schema vale solo nelle regioni lisce; l'urto e il contatto occupano un numero fisso di celle, quindi il loro contributo all'errore scala come h, e il contatto, che non si autocomprime come l'urto, si allarga nel tempo).
- `Minmod` è il più diffusivo (urto risolto in 6 celle contro 5), `vanLeer` il più netto; la differenza fra i tre è del 18% sull'errore L_1.

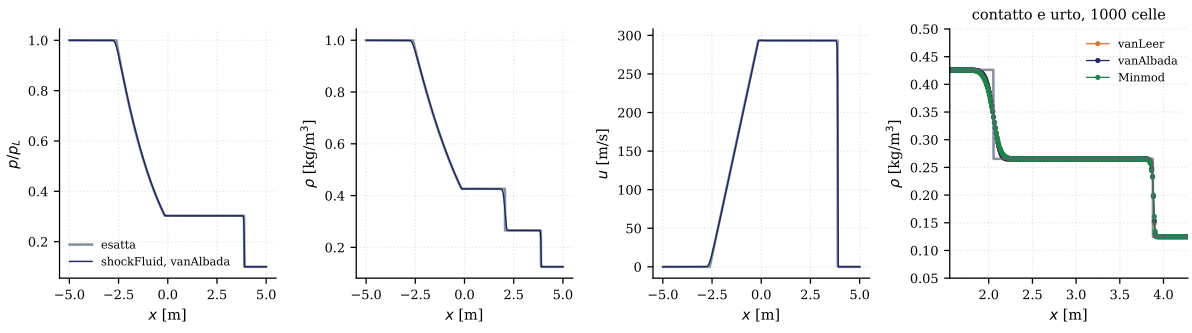


Figura 10.1: Tubo d'urto a $t = 7$ ms: pressione, densità e velocità calcolate (1000 celle, `vanAlbada`) contro la soluzione esatta; a destra il dettaglio dell'urto e del contatto con le tre ricostruzioni.

10.3 Esperimento 2: l'urto obliquo

Una corrente supersonica che incontra un cuneo di semiangolo θ devia attraverso un urto obliquo inclinato di β rispetto alla corrente; la relazione θ–β–Ma e i rapporti attraverso l'urto sono esatti (teoria dell'urto obliquo, dal corso di aerodinamica):

$$\tan\theta = 2\cot\beta\,\frac{Ma_1^2\sin^2\beta - 1}{Ma_1^2(\gamma + \cos 2\beta) + 2},$$
$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma+1}\left(Ma_1^2\sin^2\beta - 1\right), \qquad Ma_{n2}^2 = \frac{1 + \frac{\gamma-1}{2}Ma_{n1}^2}{\gamma Ma_{n1}^2 - \frac{\gamma-1}{2}}. \tag{10.2}$$

Il caso `c12_cuneo` è il tutorial del cuneo a 15° portato in aria reale a $Ma = 2{,}5$: l'angolo dell'urto misurato dal campo di pressione e il rapporto p_2/p_1 nella regione a valle sono confrontati con la (10.2), e il numero di Mach con la function object `MachNo`.


c12_cuneo.zip

Dati del caso `c12_cuneo`

Aria, $p_1 = 101\,325$ Pa, $T_1 = 300$ K, $U_1 = 868$ m/s ($Ma_1 = 2{,}5$); cuneo di 15° con vertice in $x = 0$, dominio da $-0{,}152$ a 0,305 m, altezza 0,152 m; mesh $120 \times 100 + 240 \times 100$ celle; ingresso supersonico con tutto imposto, uscita `zeroGradient`, cuneo `slip`, contorno superiore `symmetryPlane`; `maxCo 0.5`, fino a $t = 2$ ms (≈ 4 attraversamenti); ricostruzione `vanLeer`.

Risultati del caso `c12_cuneo`

- Teoria (urto debole, $Ma_1 = 2{,}5$, $\theta = 15°$): $\beta = 36{,}94°$, $Ma_{n1} = 1{,}503$, $p_2/p_1 = 2{,}468$, $T_2/T_1 = 1{,}322$, $Ma_2 = 1{,}874$ (ancora supersonico, come deve essere per un urto debole).
- Calcolo: l'urto, individuato dall'isobara $(p_1 + p_2)/2$ su 123 colonne di celle, è una retta (R^2 della regressione: 0,99998) con $\beta = 36{,}94°$: scarto 0,00° (0,0%), meno della risoluzione angolare della mesh ($\Delta y/\Delta x$ per cella).
- Nella regione a valle (media su 1846 celle): $p_2/p_1 = 2{,}467$ (-0,01%), $T_2/T_1 = 1{,}323$ (0,06%), $Ma_2 = 1{,}871$ (-0,11%); a monte $Ma = 2{,}499$ e $p/p_1 = 1{,}0000$: la corrente libera è trasportata senza alterazioni.
- Su $y = 0{,}1$ m l'urto è in $x = 0{,}1328$ m contro 0,1330 m teorici; la transizione occupa 3–4 celle ($\Delta x = 1{,}27$ mm), come nel tubo d'urto: la ricostruzione limitata non introduce oscillazioni a monte né a valle.

Pressure-based o density-based?

Per i flussi aeronautici subsonici e transonici (profili a $Ma \leq 0{,}8$, prese d'aria, gondole) il modulo `fluid` con PIMPLE e lo schema `linearUpwind` per $\mathbf{U}$ e e è la scelta normale: condivide con l'incomprimibile tutto l'apparato dei Capp. 8 e 9, funziona a qualunque Mach basso senza il problema di rigidità dei metodi density-based a $Ma \to 0$, e cattura gli urti transonici deboli in modo accettabile (il tutorial `aerofoilNACA0012` lo mostra a $Ma = 0{,}72$). Per i flussi supersonici con urti forti, le espansioni, gli ugelli, `shockFluid` è più accurato e più robusto. Entrambi richiedono mesh fini dove ci sono gli urti, e la loro posizione non è nota a priori: il raffinamento adattivo o un calcolo preliminare

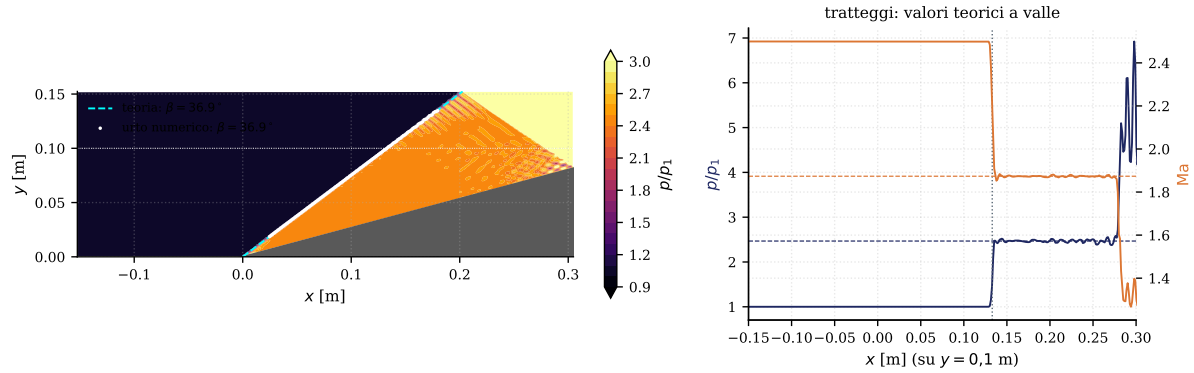


Figura 10.2: Urto obliquo a $Ma = 2{,}5$ su cuneo di 15°: campo di pressione con l'angolo teorico dell'urto sovrapposto (a sinistra); pressione e numero di Mach lungo una retta orizzontale a $y = 0{,}1$ m con i valori teorici a valle dell'urto (a destra).

grossolano aiutano.

10.4 Approfondimenti

10.4.1 L'equazione della pressione comprimibile

Nel pressure-based (Sez. 16.5 del testo) la continuità $\partial\rho/\partial t + \nabla\cdot(\rho\mathbf{U}) = 0$ con $\rho = \psi p$ e $\mathbf{U} = \mathbf{H}/A - (1/A)\nabla p$ diventa $\partial(\psi p)/\partial t + \nabla\cdot(\psi p\,\mathbf{H}/A) - \nabla\cdot(\rho/A\,\nabla p) = 0$: un termine temporale e uno convettivo *in* p si aggiungono al laplaciano. A bassi Mach il laplaciano domina ed è l'equazione incomprimibile; ad alti Mach domina la convezione e l'equazione diventa iperbolica, con la pressione che si propaga lungo le caratteristiche; il termine convettivo va discretizzato con uno schema upwind (`div(phid,p) Gauss limitedLinear 1` in `fvSchemes` del modulo `fluid`: `phid` è il flusso $\psi\mathbf{U}_f\cdot\mathbf{S}_f$). La densità nel flusso di massa è trattata con la *linearizzazione di Newton* $\rho' = \psi p'$, che accoppia correzione di pressione e di densità.

10.4.2 Il problema di Riemann e i solutori approssimati

Lo schema di Godunov risolve esattamente, su ogni faccia, il problema di Riemann fra gli stati ricostruiti; è costoso e i solutori approssimati (Roe, HLL, HLLC, AUSM) ne approssimano il flusso con poche operazioni. Kurganov–Tadmor appartiene alla famiglia *centrale*, che evita del tutto la struttura delle onde usando solo le velocità massime: è leggermente più dissipativo di Roe o HLLC sulle discontinuità di contatto (visibile nel caso di Sod), ma non ha i difetti di entropia di Roe ed è semplice da estendere a equazioni di stato arbitrarie. La versione ESI di OpenFOAM offre anche un solutore HLLC (`rhoCentralFoam` con `fluxScheme HLLC`); le discussioni in letteratura (LeVeque, Toro) sono nei riferimenti del Modulo 2.

10.4.3 Mach basso e precondizionamento

I metodi density-based soffrono a $Ma \to 0$: le velocità delle onde acustiche (a) e convettive ($|\mathbf{U}|$) sono molto diverse, il passo esplicito è limitato dalla prima mentre la soluzione evolve con la seconda, e il flusso numerico diventa troppo dissipativo; il *precondizionamento a basso Mach* (Turkel, Weiss–Smith) riscala le equazioni per riavvicinare le velocità. OpenFOAM non lo

implementa nei suoi solutori density-based perché la soluzione è usare il pressure-based, che a basso Mach è nativo: è la ragione pratica per cui, in un'ala subsonica, `fluid` è la scelta e `shockFluid` no.

Capitolo 11

Le condizioni al contorno

11.1 Come funziona una condizione al contorno

Nel metodo dei volumi finiti il contorno entra nelle equazioni attraverso i flussi delle facce di contorno: la faccia b di una cella C contribuisce al termine convettivo con $\dot{m}_b \phi_b$ e a quello diffusivo con $\Gamma_b (\nabla\phi)_b \cdot \mathbf{S}_b$, e ogni condizione al contorno è, in ultima analisi, una regola per esprimere ϕ_b e $(\nabla\phi)_b \cdot \mathbf{n}$ in funzione di ϕ_C e di dati noti (Sez. 5.3 del testo di riferimento e Cap. 18). OpenFOAM organizza questa idea in una gerarchia di classi, le `fvPatchField`, che il Cap. 18 del testo descrive e che qui riassumiamo prima del catalogo, perché spiega che cosa accade davvero quando si scrive `type inletOutlet`.

I quattro coefficienti

Ogni condizione al contorno deve fornire al discretizzatore quattro cose. Per il valore sulla faccia, usato dai termini convettivi e dalle interpolazioni: $\phi_b = \text{vIC} \cdot \phi_C + \text{vBC}$ (`valueInternalCoeffs`, `valueBoundaryCoeffs`); per il gradiente normale, usato dal laplaciano: $(\partial\phi/\partial n)_b = \text{gIC} \cdot \phi_C + \text{gBC}$ (`gradientInternalCoeffs`, `gradientBoundaryCoeffs`). Le due condizioni fondamentali:

- `fixedValue` ($\phi_b = \phi_{\text{imp}}$): vIC $= 0$, vBC $= \phi_{\text{imp}}$; gIC $= -1/d_{Cb}$, gBC $= \phi_{\text{imp}}/d_{Cb}$ (la derivata normale è $(\phi_{\text{imp}} - \phi_C)/d_{Cb}$: contribuisce alla diagonale e al termine noto);
- `zeroGradient` ($\partial\phi/\partial n = 0$): vIC $= 1$, vBC $= 0$ (il valore di faccia è quello della cella); gIC $=$ gBC $= 0$.

Tutte le altre sono combinazioni, varianti o specializzazioni di queste due (`mixed`: una frazione f di Dirichlet e $1 - f$ di Neumann, cella per cella), con l'aggiunta di una funzione `updateCoeffs()` che ricalcola il valore imposto a ogni iterazione in base ad altri campi: è così che `inletOutlet` guarda il flusso `phi` per decidere se una faccia è entrante, che `totalPressure` ricava la pressione statica dalla velocità, che le funzioni di parete calcolano ν_t da k. Un `fvPatchField` non sa nulla del solutore in cui è usato: la stessa `fixedValue` serve per $\mathbf{U}$, T, k e per qualunque campo che un utente inventerà; è la ragione della straordinaria riusabilità del codice.

11.2 Il catalogo ragionato

Condizioni di base e di vincolo

`fixedValue`, `zeroGradient`, `fixedGradient`, `mixed`
le fondamentali; `fixedGradient` con `gradient` assegnato (flusso termico imposto); `mixed` con `refValue`, `refGradient`, `valueFraction`.

`calculated`
il valore è calcolato da un altro oggetto (la viscosità turbolenta dal modello): non è una condizione, è un segnaposto; un errore `calculated ... cannot be called` dice che si è messo `calculated` dove serve una condizione vera.

`empty`
le facce dei piani di un caso 2D: nessuna equazione nella direzione normale. `wedge` le facce laterali di un caso assialsimmetrico. `symmetry`, `symmetryPlane` (quest'ultimo richiede facce esattamente planari e va preferito): componente normale della velocità nulla, derivata normale nulla per gli scalari e per la componente tangenziale. `cyclic` (periodicità fra due patch con `neighbourPatch`: canali e turbomacchine), `cyclicAMI` per patch non conformi, `processor` fra i sottodomini di un calcolo parallelo (generati automaticamente).

Velocità e pressione

`noSlip`
$\mathbf{U} = 0$ (equivale a `fixedValue (0 0 0)`); `movingWallVelocity` per pareti in moto con mesh mobile; `slip` componente normale nulla, tangenziale libera (pareti non viscose, contorni lontani); `rotatingWallVelocity`.

`inletOutlet`
`zeroGradient` dove il flusso esce, `fixedValue inletValue` dove rientra: è la condizione corretta per $\mathbf{U}$, k, ε, T alle uscite, dove un ricircolo può far rientrare fluido; senza di essa un'uscita con flusso entrante fa divergere. `outletInlet` è la speculare.

`pressureInletOutletVelocity`
per un'uscita (o un ingresso) a pressione imposta: `zeroGradient` in uscita, velocità ricavata dal flusso in ingresso; si accoppia con `totalPressure` per p (ingresso a pressione totale imposta, `p0`: serbatoi, prese d'aria, camini) o con `fixedValue` per p.

`freestreamVelocity`, `freestreamPressure`
(con `freestreamValue`) per i contorni lontani dell'aerodinamica esterna: si comportano da ingresso o da uscita a seconda del segno locale del flusso, e permettono di usare *un solo* patch esterno circolare o rettangolare qualunque sia l'incidenza (Cap. 13).

`flowRateInletVelocity`
portata volumetrica o massica imposta (`volumetricFlowRate`, `massFlowRate`); `uniformFixedValue` con una `uniformValue` funzione del tempo (vedi sotto); `fixedProfile` profilo lungo una direzione.

`fixedFluxPressure`
per p sulle pareti e sugli ingressi a velocità imposta quando ci sono forze di volume (gravità, `fvModels`): impone il gradiente coerente con il flusso; in assenza di forze equivale a `zeroGradient`. `fixedMean`, `waveTransmissive` (uscite comprimibili non riflettenti), `totalPressure`.

Funzioni del tempo e dello spazio: `Function1` e `coded`

Molte condizioni accettano un valore che è una **funzione del tempo** (`Function1`): `uniformValue constant 1;` `table ((0 0) (1 2) (5 2))` (interpolazione lineare per punti); `tableFile` da un file esterno; `sine` con `amplitude`, `frequency`, `level`; `square`; `polynomial`; le rampe `linearRamp`, `halfCosineRamp` (per avviare dolcemente un calcolo); `scale` per comporle; `coded` per scriverne una in C++. Per le dipendenze dallo *spazio* ci sono `fixedProfile` e, in generale, le condizioni **codificate**: `codedFixedValue` e `codedMixed` accettano un blocco di codice C++ nel dizionario (fra `#{` e `#}`) che viene compilato alla prima esecuzione in una libreria dentro `dynamicCode/` e caricato; nel codice sono disponibili `patch().Cf()` (i centri delle facce), `patch().nf()`, `db().time().value()` (il tempo), `db().lookupObject<volScalarField>("p")` (altri campi), e `operator==(valori)` assegna il valore. È il modo più rapido di scrivere una condizione che non esiste; per un uso ripetuto si scrive una classe vera (Cap. 12).

Gli errori più comuni

Imporre la velocità *e* la pressione sullo stesso patch (sovradeterminazione: la continuità non può essere soddisfatta); `zeroGradient` per **U** a un'uscita dove il fluido rientra (divergenza: usare `inletOutlet`); un'uscita troppo vicina a un corpo, dove la condizione di gradiente nullo è falsa e deforma la scia (regola: almeno 10–20 lunghezze caratteristiche a valle, Cap. 13); un contorno esterno troppo vicino con `fixedValue` (blocca la deflessione della corrente e aumenta la portanza: il *bloccaggio*); `calculated` invece di una funzione di parete su `nut`; dimenticare `value` in una condizione che lo richiede per l'avvio (`codedFixedValue`, `totalPressure`, le funzioni di parete) o scrivere `value` con il tipo sbagliato (`uniform 0` per un vettore).

11.3 Esperimento: il condotto di Poiseuille con profilo codificato

Il caso `c13_poiseuille` usa la mesh assialsimmetrica del Cap. 3 e impone all'ingresso, con `codedFixedValue`, il profilo parabolico $u = 2U_m(1 - r^2/R^2)$: il flusso è allora sviluppato dall'ingresso all'uscita e la soluzione esatta di Hagen–Poiseuille dà il gradiente di pressione $\mathrm{d}p/\mathrm{d}x = -8\mu U_m/R^2$ (in pressione cinematica $-8\nu U_m/R^2$) e lo sforzo di parete $\tau_w = 4\mu U_m/R$. Si verificano: il profilo all'uscita (che deve coincidere con quello d'ingresso), la caduta di pressione sul condotto, il fattore d'attrito $f = 64/Re$, e lo sforzo di parete calcolato con `foamcase.py`. Un secondo calcolo con ingresso *uniforme*, su un condotto lungo il doppio ($L/D = 20$, perché a $Re = 200$ lo sviluppo richiede circa 12 diametri), mostra la lunghezza di sviluppo $L_e/D \approx 0{,}06\,Re$ e la caduta di pressione aggiuntiva dell'imbocco.

Dati del caso `c13_poiseuille`

$R = 1$ cm, $L = 20$ cm ($L/D = 10$), cuneo di 4°, 100×20 celle con infittimento alla parete; $U_m = 0{,}1$ m/s, $\nu = 10^{-5}$ ($Re = UD/\nu = 200$); SIMPLEC; soluzione esatta: $\Delta p/\rho = 8\nu U_m L/R^2 = 1{,}6 \times 10^{-2}$ m^2/s^2, $u_{\max} = 0{,}2$ m/s, $\tau_w/\rho = 4\nu U_m/R = 4 \times 10^{-4}$ m^2/s^2.

c13_poiseuille.zip

c13_poiseuille/0/U

```
// Condotto: all'ingresso il profilo parabolico di Poiseuille u = 2 U_m (1 - r^2/R^2), scritto in C++
// direttamente nel dizionario (codedFixedValue): e' compilato al primo avvio e caricato come libreria.
dimensions      [0 1 -1 0 0 0 0];

internalField   uniform (0.1 0 0);

boundaryField
{
    ingresso
    {
        type            codedFixedValue;
        name            profiloParabolico;          // nome della libreria generata (dynamicCode/profiloParabolico)
        code
        #{
            const scalar Um = 0.1, R = 0.01;        // velocita' media e raggio
            const vectorField& Cf = patch().Cf();   // centri delle facce del patch
            vectorField Uf(Cf.size(), vector::zero);
            forAll(Cf, i)
            {
                const scalar r = sqrt(sqr(Cf[i].y()) + sqr(Cf[i].z()));
                Uf[i] = vector(2*Um*(1 - sqr(r/R)), 0, 0);
            }
            operator==(Uf);                         // assegna il valore al patch
        #};
        value           uniform (0.1 0 0);          // valore iniziale, prima della compilazione
    }
    uscita      { type zeroGradient; }
    parete      { type noSlip; }
    fronte      { type wedge; }
    retro       { type wedge; }
    asse        { type empty; }
}
```

Listato 11.1: Il profilo parabolico scritto in C++ dentro il dizionario con `codedFixedValue`.

Risultati del caso `c13_poiseuille`

- Il profilo codificato è esatto sulle facce d'ingresso (scarto massimo dalla parabola $2{,}10 \times 10^{-8}$ relativo); all'uscita il profilo nelle celle differisce dalla parabola al più del 0,11% di $u_{\max}$: il flusso resta sviluppato lungo tutto il condotto.
- Caduta di pressione fra $x = 0{,}10$ e $x = 19{,}90$ cm: $1{,}596 \times 10^{-2}$ contro $1{,}584 \times 10^{-2}$ m^2/s^2 (0,76%); fattore d'attrito $f = 0{,}3224$ contro $64/Re = 0{,}3200$.
- Sforzo di parete (`foamcase.py`, media nel tratto $5 < x < 19$ cm): τ_w/ρ =$4{,}005 \times 10^{-4}$ contro $4\nu U_m/R$ =$4{,}000 \times 10^{-4}$ m^2/s^2 (0,12%); la forza viscosa totale sulla

parete del cuneo divisa per la sua area dà $4{,}006 \times 10^{-4}$: stesso valore, stessa formula (gradiente della prima cella, $y_1 = 0{,}105$ mm).

- Ingresso uniforme ($L/D = 20$): la velocità sull'asse raggiunge il 99% di $2U_m$ in $L_e/D = 14{,}7$ (le correlazioni classiche danno 0,05–0,065 Re, cioè 10–13 diametri: la definizione al 99% è severa perché l'avvicinamento al profilo finale è esponenziale); la caduta di pressione in eccesso rispetto a Poiseuille vale $K = 1{,}05$ in unità di $\frac{1}{2}U_m^2$ (il valore classico della perdita d'imbocco laminare è $K \approx 1{,}2$–$1{,}3$).

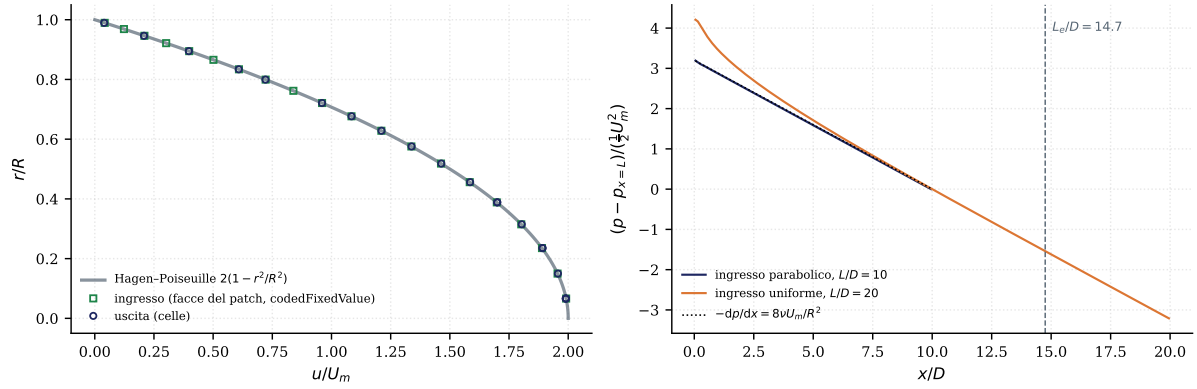


Figura 11.1: Condotto di Poiseuille: profilo di velocità all'ingresso (codificato) e all'uscita con la parabola esatta (a sinistra); pressione lungo l'asse per l'ingresso parabolico e per quello uniforme, con la retta di Hagen–Poiseuille (a destra).

11.4 Approfondimenti

11.4.1 Condizioni al contorno e ordine di accuratezza

La derivata normale alla parete $(\phi_b - \phi_C)/d_{Cb}$ è del primo ordine, ma la soluzione nelle celle resta del secondo perché la faccia di contorno *è* nel punto in cui il valore è imposto (lo schema è un'approssimazione unilatera consistente); la quantità che è del primo ordine è lo sforzo o il flusso termico *derivato* dal campo, come osservato nel Cap. 4. Le condizioni che estrapolano (`zeroGradient` a un'uscita) sono invece una vera approssimazione fisica, e il loro errore non diminuisce con la mesh: diminuisce allontanando il contorno.

11.4.2 Patch, gruppi e espressioni regolari

Nel `boundaryField` si possono usare espressioni regolari fra virgolette, `"(parete|cuneo).*" { type noSlip; }`, e i *gruppi* definiti in `boundary` (`inGroups (wall)`): `wall { type noSlip; }` si applica a tutti i patch del gruppo `wall`, e le voci specifiche hanno la precedenza. Con `#includeEtc "caseDicts/setConstraintTypes"` si assegnano in un colpo solo le condizioni di vincolo (`empty`, `wedge`, `cyclic`, `symmetry`) in base al tipo del patch: è la prima riga del `boundaryField` di quasi tutti i tutorial, e conviene adottarla.

11.4.3 Le condizioni al contorno nel calcolo parallelo e nelle mesh mobili

Nel calcolo parallelo i confini fra sottodomini sono patch di tipo `processor` creati da `decomposePar`: per il codice sono condizioni al contorno accoppiate che scambiano i valori con il processore vicino a ogni valutazione, e l'utente non li vede nei file di `0/` (sono aggiunti ai campi decomposti). Nelle mesh mobili le condizioni `movingWallVelocity` e `fixedFluxPressure` tengono conto del flusso della mesh (`meshPhi`), e la velocità relativa è ciò che conta: è il caso di una pala che ruota (`MRF`) o di un corpo che oscilla.

Capitolo 12

Dentro il codice: leggere e scrivere OpenFOAM

12.1 Perché aprire i sorgenti

Tutto ciò che OpenFOAM fa è scritto in C++ in una forma che ricalca da vicino le equazioni dei capitoli precedenti: `fvm::laplacian(nu, U)` *è* il termine diffusivo della (2.5), con la discretizzazione scelta in `fvSchemes`. Questa è la differenza fra OpenFOAM e un codice commerciale: la documentazione definitiva di ogni parola chiave, di ogni condizione al contorno e di ogni modello è il sorgente, e chi sa leggerlo non dipende da manuali incompleti o da forum. Il capitolo insegna tre cose, in ordine di difficoltà crescente: orientarsi nell'albero dei sorgenti e trovare la risposta a una domanda ("che cosa fa esattamente `nutUSpaldingWallFunction`?"); leggere un solutore; scrivere ed esercitare con `wmake` una condizione al contorno e un solutore propri. Il lettore che non intende programmare può fermarsi alla prima parte: saper *trovare* il codice è già metà della consapevolezza.

L'albero dei sorgenti e le variabili d'ambiente

Dopo `source .../etc/bashrc` le variabili d'ambiente puntano alle cartelle dell'installazione:

`$WM_PROJECT_DIR`
: la radice (`OpenFOAM-12/`): `src/` le librerie, `applications/` i programmi, `etc/` la configurazione, `tutorials/` i casi d'esempio, `platforms/` i binari compilati (`bin/`, `lib/`), `wmake/` il sistema di compilazione, `bin/` gli script di shell (`foamNewBC`, `foamInfo`, `foamSearch`, `foamCloneCase`, `foamLog`, `paraFoam`).

`$FOAM_SRC`
: `src/`: `OpenFOAM/` (contenitori, campi, dizionari, matrici `ldu`, tempo, I/O, `Function1`), `finiteVolume/` (mesh `fvMesh`, campi `volScalarField`, operatori `fvm`/`fvc`, schemi, condizioni al contorno in `fields/fvPatchFields/`, solutori lineari in `fvMatrices/`), `meshTools/`, `MomentumTransportModels/` (i modelli di turbolenza: `momentumTransportModels/RAS/kOmegaSST/kOmegaSST.C`), `ThermophysicalTransportModels/`, `thermophysicalModels/`, `functionObjects/`, `fvModels/` e `fvConstraints/`, `sampling/`, `Pstream/` (lo scambio di messaggi in parallelo).

$FOAM_APP
applications/: solvers/foamRun/ (l'eseguibile unico), modules/ (i solutori come librerie: incompressibleFluid/, fluid/, shockFluid/, functions/, multiphaseEuler/...), legacy/ (i vecchi icoFoam, laplacianFoam), utilities/ (mesh/, preProcessing/, postProcessing/, parallelProcessing/, miscellaneous/).

$FOAM_ETC
etc/: controlDict (interruttori di debug, InfoSwitches, allowSystemOperations), caseDicts/ (i dizionari che #includeEtc e #includeFunc trovano: functions/, annotated/, setConstraintTypes), codeTemplates/ (gli scheletri di foamNewBC e del codice coded), thermoData/.

$FOAM_TUTORIALS
i casi d'esempio, organizzati per modulo (incompressibleFluid/, fluid/, shockFluid/...): foamSearch $FOAM_TUTORIALS fvSchemes divSchemes/div(phi,U) elenca tutti i valori usati nei tutorial per uno schema, ed è il modo più rapido di sapere "che cosa usano gli autori".

$WM_PROJECT_USER_DIR
(~/OpenFOAM/utente-12/) la cartella personale: $FOAM_RUN per i casi, $FOAM_USER_APPBIN e $FOAM_USER_LIBBIN per i programmi e le librerie compilati dall'utente, cercati *prima* di quelli della distribuzione.

La ricerca nei sorgenti si fa con grep: grep -rl nutUSpaldingWallFunction $FOAM_SRC -include=*.H trova la classe, foamInfo nutUSpaldingWallFunction stampa la descrizione nel suo intestazione e i tutorial che la usano. Ogni file .H comincia con un blocco Description e, per le condizioni al contorno e i modelli, con un blocco Usage che è la documentazione ufficiale delle parole chiave.

12.2 Le classi che si incontrano sempre

Dieci classi per leggere qualunque solutore

Time runTime
il tempo (o il contatore di iterazioni), costruito da controlDict: runTime.run(), runTime++, runTime.deltaT(), runTime.write() (scrive tutti i campi AUTO_WRITE se è un istante di scrittura), runTime.name() (il nome della cartella corrente). Deriva da objectRegistry: è il "database" in cui ogni campo è registrato con il suo nome, e da cui lookupObject<volScalarField>("p") lo recupera.

fvMesh mesh
la mesh dei volumi finiti: mesh.C() i centroidi, mesh.V() i volumi, mesh.Sf() e mesh.magSf() i vettori d'area, mesh.Cf() i centri di faccia, mesh.owner(), mesh.neighbour(), mesh.boundary() i patch, mesh.deltaCoeffs() ($1/|\mathbf{d}_{CF}|$), mesh.weights() (g_f), mesh.nonOrthDeltaCoeffs(): sono

esattamente le quantità dei Cap. 2–4.

`volScalarField`, `volVectorField`, `surfaceScalarField`
i campi: valori nelle celle (`internalField()`, un `Field<scalar>`), valori sui patch (`boundaryField()[patchi]`), dimensioni fisiche (`dimensions()`, controllate a ogni operazione: sommare una velocità a una pressione è un errore a tempo di esecuzione), nome e regole di lettura/scrittura (`IOobject`). Un `surfaceScalarField` vive sulle facce: `phi` è il flusso $\mathbf{U}_f \cdot \mathbf{S}_f$.

`IOobject`
(nome, cartella, registro, `MUST_READ/READ_IF_PRESENT/NO_READ`, `AUTO_WRITE/NO_WRITE`): dice da dove viene un campo e se va scritto.

`fvScalarMatrix`, `fvVectorMatrix`
il sistema lineare: la matrice `ldu` (`diag()`, `upper()`, `lower()`), il termine noto `source()`, i coefficienti di contorno `internalCoeffs()` e `boundaryCoeffs()` (i quattro coefficienti del Cap. 11), e i metodi `relax()`, `solve()`, `A()` (la diagonale per unità di volume), `H()` (il termine $\mathbf{H}$ dell'algoritmo SIMPLE, Cap. 8), `flux()`.

`fvm::`, `fvc::`
gli operatori *impliciti* (restituiscono una matrice: `fvm::ddt`, `fvm::div`, `fvm::laplacian`, `fvm::Sp`, `fvm::SuSp`) ed *espliciti* (restituiscono un campo: `fvc::grad`, `fvc::div`, `fvc::curl`, `fvc::interpolate`, `fvc::snGrad`, `fvc::flux`). Ciascuno legge il proprio schema da `fvSchemes` usando il nome dell'operando come chiave: `fvm::laplacian(DT, T)` cerca `laplacian(DT,T)`.

`dictionary`
il lettore dei file di input: `dict.lookup<scalar>("Um")` (obbligatorio), `dict.lookupOrDefault<vector>("centro", Zero)` (facoltativo), `dict.found()`, `dict.subDict()`, `dict.lookupOrAddDefault()`.

`Function1<Type>`
una funzione di una variabile (di solito il tempo) scelta a run time dal dizionario: `constant`, `table`, `sine`, `halfCosineRamp`, `coded...`; `f->value(t)` la valuta, `f->integral(t1, t2)` la integra.

`autoPtr<T>`, `tmp<T>`
i puntatori intelligenti di OpenFOAM: `autoPtr` possiede un oggetto creato con `new` (e lo distrugge), `tmp` trasporta un campo temporaneo fra le espressioni senza copiarlo (`tmp<volScalarField> t = fvc::grad(p) & U`).

la selezione a run time
ogni classe scelta per nome da un dizionario (schemi, condizioni al contorno, modelli, `Function1`, solutori) è registrata in una tabella con le macro `TypeName("nome")` nell'intestazione e `addToRunTimeSelectionTable` o `makePatchTypeField` nel sorgente; `Classe::New(dict)` consulta la tabella e costruisce l'oggetto giusto. Per aggiungere una condizione al contorno basta quindi compilare una libreria che si registra e caricarla con `libs`: il solutore non va toccato.

12.3 Leggere un solutore: `trasportoFoam`

Il modo migliore di imparare a leggere un solutore è leggerne uno corto. `trasportoFoam` risolve l'equazione di trasporto di uno scalare passivo T in un campo di velocità $\mathbf{U}$ letto da file e tenuto fisso,

$$\frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{U}T) - \nabla \cdot (D_T \nabla T) = 0, \tag{12.1}$$

cioè il problema del Cap. 5 e del Cap. 6 scritto come programma invece che come *function object*; è il solutore `scalarTransportFoam` delle versioni precedenti, riscritto con le intestazioni di OpenFOAM 12 e commentato riga per riga (Listato 12.1). La struttura è quella di tutti i solutori della distribuzione: inclusioni, `main`, creazione di `runTime` e `mesh`, lettura dei campi (`createFields.H`, Listato 12.2), il ciclo temporale, dentro il quale si costruisce l'equazione come somma di operatori `fvm` e la si risolve.


c14_codice/trasportoFoam/trasportoFoam.C

```
17 #include "argList.H"           // gli argomenti di riga di comando (-case,
       -parallel, ...)
18 #include "simpleControl.H"     // il ciclo esterno con i correttori non ortogonali
       (porta con se' Time e fvMesh)
19 #include "fvcFlux.H"           // fvc::flux(U), usato da createPhi.H
20 #include "fvmDdt.H"            // gli operatori impliciti: derivata temporale,
21 #include "fvmDiv.H"            //   divergenza (convezione),
22 #include "fvmLaplacian.H"      //   laplaciano (diffusione)
23
24 using namespace Foam;         // nelle applicazioni si scrive Info, fvm::,
       volScalarField senza il prefisso Foam::
25
26 int main(int argc, char *argv[])
27 {
28     #include "setRootCase.H"   // legge gli argomenti (-case, -parallel, ...)
29     #include "createTime.H"    // crea l'oggetto runTime da system/controlDict
30     #include "createMesh.H"    // legge constant/polyMesh e crea la fvMesh
31
32     simpleControl simple(mesh);
33
34     #include "createFields.H"  // T, U, phi, DT (file accanto a questo)
35
36     Info<< "\nTrasporto dello scalare T nel campo di velocita' assegnato\n" << endl;
37
38     while (simple.loop(runTime))              // avanza il tempo (o l'iterazione)
       e controlla l'arresto
39     {
40         Info<< "Time = " << runTime.userTimeName() << nl << endl;
41
42         while (simple.correctNonOrthogonal()) // ripete per
       nNonOrthogonalCorrectors + 1 volte
43         {
44             fvScalarMatrix TEqn               // l'equazione, termine per termine:
       ogni fvm:: legge fvSchemes
45             (
46                 fvm::ddt(T)
47               + fvm::div(phi, T)
48               - fvm::laplacian(DT, T)
49             );
50
51             TEqn.relax();                     // rilassamento implicito (fattore
       in fvSolution, se presente)
52             TEqn.solve();                     // il solutore lineare scelto in
       fvSolution per T
```

```
        }

        runTime.write();                        // scrive T agli istanti di
    controlDict

        Info<< "ExecutionTime = " << runTime.elapsedCpuTime() << " s"
            << "  ClockTime = " << runTime.elapsedClockTime() << " s"
            << nl << endl;
    }

    Info<< "End\n" << endl;

    return 0;
}
```

Listato 12.1: Il solutore completo: inclusioni, ciclo temporale con `simpleControl`, costruzione dell'equazione con gli operatori impliciti, rilassamento e soluzione.

```
// --- i campi del solutore: letti da 0/ (MUST_READ) o calcolati

Info<< "Lettura del campo T\n" << endl;
volScalarField T
(
    IOobject
    (
        "T",
        runTime.name(),           // la cartella temporale corrente
        mesh,
        IOobject::MUST_READ,      // deve esistere in 0/
        IOobject::AUTO_WRITE      // viene scritto a ogni istante di scrittura
    ),
    mesh
);

Info<< "Lettura del campo U\n" << endl;
volVectorField U
(
    IOobject
    (
        "U",
        runTime.name(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

Info<< "Lettura della diffusivita' DT da physicalProperties\n" << endl;
IOdictionary physicalProperties
(
    IOobject
    (
        "physicalProperties",
        runTime.constant(),
        mesh,
        IOobject::MUST_READ_IF_MODIFIED,
        IOobject::NO_WRITE
    )
);
```

c14_codice/trasportoFoam/createFields.H

```
43 dimensionedScalar DT("DT", dimKinematicViscosity, physicalProperties);   //
       [m^2/s]: il controllo dimensionale e' automatico
44
45 // il flusso di faccia phi = U_f . S_f: letto se esiste (es. da un calcolo
       precedente), altrimenti calcolato
46 #include "createPhi.H"
```

Listato 12.2: La lettura dei campi: `T` e `U` con `MUST_READ`, la diffusività dal dizionario `physicalProperties` con le dimensioni dichiarate, il flusso `phi` da `createPhi.H`.

Riga per riga

- `#include "setRootCase.H"` costruisce `argList args` dagli argomenti della riga di comando (`-case`, `-parallel`, `-noFunctionObjects`); `createTime.H` e `createMesh.H` sono file di una riga che costruiscono `runTime` e `mesh`: in OpenFOAM i blocchi di codice ripetuti sono inclusi, non copiati.
- `simpleControl simple(mesh)` legge il sotto-dizionario `SIMPLE` di `fvSolution`; `simple.loop(runTime)` avanza il tempo, controlla `endTime` e `residualControl`, ed esegue le *function object*; `simple.correctNonOrthogonal()` ripete il blocco `nNonOrthogonalCorrectors`+1 volte (Cap. 4).
- `fvScalarMatrix TEqn(fvm::ddt(T) + fvm::div(phi, T) - fvm::laplacian(DT, T))`: tre matrici sommate. `fvm::ddt` legge `ddtSchemes` (con `steadyState` restituisce una matrice nulla: il programma è stazionario senza cambiare una riga), `fvm::div(phi, T)` legge `div(phi,T)` in `divSchemes` e costruisce i coefficienti $\dot{m}_f w_f$ del Cap. 5, `fvm::laplacian(DT, T)` legge `laplacian(DT,T)` e aggiunge la parte implicita della diffusione e, nel termine noto, la correzione non ortogonale.
- `TEqn.relax()` applica il fattore `equations/T` di `fvSolution` se esiste (rilassamento implicito, Cap. 6); `TEqn.solve()` sceglie il solutore lineare dalla voce `solvers/T` e stampa la riga `Solving for T` del registro.
- `runTime.write()` scrive T (dichiarato `AUTO_WRITE`) agli istanti di `writeControl`; **U** è `AUTO_WRITE` per comodità (compare nelle cartelle di uscita), `phi` viene scritto da `createPhi.H` con `READ_IF_PRESENT`.
- Le dimensioni: `dimensionedScalar DT("DT", dimKinematicViscosity, physicalProperties)` legge `DT 0 [m^2/s];` e rifiuta un valore con unità diverse. Nei file di OpenFOAM 12 le unità si scrivono fra parentesi quadre dopo il numero; la forma storica `[0 2 -1 0 0 0 0]` (gli esponenti di kg, m, s, K, mol, A, cd) è ancora accettata.

Compilare: `wmake`

Ogni programma o libreria ha una cartella `Make/` con due file. `files` elenca i sorgenti da compilare e il nome del risultato: `EXE = $(FOAM_USER_APPBIN)/trasportoFoam` per un eseguibile, `LIB = $(FOAM_USER_LIBBIN)/libprofiloParabolico` per una libreria (senza estensione). `options` elenca le cartelle delle intestazioni (`EXE_INC`, con `-I$(LIB_SRC)/finiteVolume/lnInclude`) e le librerie da collegare (`EXE_LIBS` o `LIB_LIBS`, con `-lfiniteVolume -lmeshTools`). Le cartelle `lnInclude/` so-

no raccolte di collegamenti simbolici a tutte le intestazioni di una libreria, create da `wmakeLnInclude`: è per questo che si include `"fvmLaplacian.H"` senza percorso. I comandi: `wmake` compila nella cartella corrente (eseguibile), `wmake libso` compila una libreria condivisa, `wclean` rimuove gli oggetti, `wmake -j` usa tutti i processori. Gli oggetti finiscono in `Make/linux64GccDPInt32Opt/`: il nome codifica architettura, compilatore, precisione (`DP`), dimensione degli indici (`Int32`) e ottimizzazione (`Opt`; `Debug` per usare `gdb`). Un errore frequente: compilare con un ambiente diverso da quello con cui si esegue (due versioni installate): il binario cerca librerie di un'altra versione e fallisce con `undefined symbol`.

12.4 Scrivere una condizione al contorno

Una condizione al contorno è una classe derivata da `fvPatchField<Type>` (o, quasi sempre, da una delle sue specializzazioni `fixedValueFvPatchField`, `fixedGradientFvPatchField`, `mixedFvPatchField`) che fornisce i quattro coefficienti del Cap. 11. Derivando da `fixedValue` si eredita tutto tranne il *valore* da imporre: basta scrivere `updateCoeffs()`, che lo calcola e lo assegna con `operator==`, e i costruttori di rito. La classe `profiloParabolico` impone il profilo di Poiseuille

$$\mathbf{U}_b = 2U_m\left(1 - \frac{r^2}{R^2}\right) f(t)\,\hat{\mathbf{e}}, \qquad r = \left|(\mathbf{x}_b - \mathbf{x}_0) - ((\mathbf{x}_b - \mathbf{x}_0)\cdot\hat{\mathbf{e}})\hat{\mathbf{e}}\right|, \tag{12.2}$$

con $\mathbf{x}_0$ un punto dell'asse, $\hat{\mathbf{e}}$ il versore e $f(t)$ una `Function1` facoltativa (una rampa, un seno: l'ingresso pulsante di un condotto): è la versione "vera" del blocco `codedFixedValue` del Cap. 11, con il controllo degli argomenti e la scrittura ordinata nel file del campo.

```
32 #ifndef profiloParabolicoFvPatchVectorField_H
33 #define profiloParabolicoFvPatchVectorField_H
34
35 #include "fixedValueFvPatchFields.H"
36 #include "Function1.H"
37
38 namespace Foam
39 {
40
41 class profiloParabolicoFvPatchVectorField
42 :
43     public fixedValueFvPatchVectorField
44 {
45     // Private Data
46
47         //- Velocita' media
48         scalar Um_;
49
50         //- Raggio del condotto (o semialtezza del canale)
51         scalar R_;
52
53         //- Un punto dell'asse
54         vector centro_;
55
56         //- Versore dell'asse (verso della velocita')
57         vector direzione_;
58
59         //- Fattore moltiplicativo funzione del tempo
```

c14_codice/profiloParabolico/profiloParabolicoFvPatchVectorField.H

```
60          autoPtr<Function1<scalar>> fattore_;
61
62
63  public:
64
65      //- Runtime type information: il nome con cui la condizione e' scelta nel
        dizionario
66      TypeName("profiloParabolico");
```

Listato 12.3: L'intestazione: i dati membri e il nome con cui la condizione è scelta nel dizionario (`TypeName`); seguono le dichiarazioni dei quattro costruttori, di `clone`, `updateCoeffs` e `write`.




```
9   Foam::profiloParabolicoFvPatchVectorField::profiloParabolicoFvPatchVectorField
10  (
11      const fvPatch& p,
12      const DimensionedField<vector, volMesh>& iF,
13      const dictionary& dict
14  )
15  :
16      fixedValueFvPatchVectorField(p, iF),
17      Um_(dict.lookup<scalar>("Um")),
18      R_(dict.lookup<scalar>("R")),
19      centro_(dict.lookupOrDefault<vector>("centro", vector::zero)),
20      direzione_(normalised(dict.lookupOrDefault<vector>("direzione", vector(1, 0,
        0)))),
21      fattore_
22      (
23          dict.found("fattore")
24        ? Function1<scalar>::New("fattore", db().time().userUnits(), dimless, dict)
25        : autoPtr<Function1<scalar>>(nullptr)
26      )
27  {
28      // valore iniziale: quello del profilo (se il campo e' gia' disponibile) o
        'value' se presente
29      if (dict.found("value"))
30      {
31          fvPatchVectorField::operator=(vectorField("value", iF.dimensions(), dict,
        p.size()));
32      }
33      else
34      {
35          updateCoeffs();
36      }
37  }
38
39
40  Foam::profiloParabolicoFvPatchVectorField::profiloParabolicoFvPatchVectorField
```

Listato 12.4: Il costruttore dal dizionario: lettura dei parametri obbligatori e facoltativi, la `Function1` opzionale, il valore iniziale.




```
74 void Foam::profiloParabolicoFvPatchVectorField::updateCoeffs()
75 {
76     if (updated())
77     {
78         return;
79     }
80
81     // distanza di ogni centro di faccia dall'asse: componente di (Cf - centro)
       ortogonale alla direzione
82     const vectorField d(patch().Cf() - centro_);
83     const scalarField r(mag(d - (d & direzione_)*direzione_));
84
85     scalar f = 1;
86     if (fattore_.valid())
87     {
88         f = fattore_->value(db().time().value());
89     }
90
91     // il profilo di Poiseuille: 2 Um (1 - r^2/R^2), mai negativo
92     fixedValueFvPatchVectorField::operator==
93     (
94         direzione_*(2*Um_*f*max(1 - sqr(r/R_), scalar(0)))
95     );
96
97     fixedValueFvPatchVectorField::updateCoeffs();
98 }
99
100
101 void Foam::profiloParabolicoFvPatchVectorField::write(Ostream& os) const
102 {
103     fvPatchVectorField::write(os);
104     writeEntry(os, "Um", Um_);
105     writeEntry(os, "R", R_);
106     writeEntry(os, "centro", centro_);
107     writeEntry(os, "direzione", direzione_);
108     if (fattore_.valid())
109     {
110         writeEntry(os, db().time().userUnits(), dimless, fattore_());
111     }
112     writeEntry(os, "value", *this);
113 }
114
115
116 // * * * * * * * * * * * * * * * * Build Macro Function  * * * * * * * * * * * * * //
117
118 namespace Foam
119 {
120     // registra la classe: da qui in poi 'type profiloParabolico' e' riconosciuto
       per i campi vettoriali
121     makePatchTypeField
122     (
123         fvPatchVectorField,
124         profiloParabolicoFvPatchVectorField
125     );
126 }
```

Listato 12.5: `updateCoeffs`: la distanza dall'asse per ogni centro di faccia, il profilo, l'assegnazione con `operator==`; `write`; la macro che registra la classe.

Che cosa chiede l'infrastruttura

- **Quattro costruttori**: dal dizionario (quello che si usa leggendo `0/U`); per mappatura su un'altra mesh (`mapFields`, raffinamento, decomposizione parallela: copia i parametri e lascia alla classe base la mappatura dei valori); di copia con nuovo campo interno; il `clone`. Il costruttore di copia semplice è vietato (`= delete`) perché una condizione senza campo interno non ha senso.
- **`updateCoeffs()`**: chiamata dal discretizzatore prima di usare i coefficienti; comincia con `if (updated()) return;` per non ricalcolare due volte nello stesso passo, finisce con la chiamata alla classe base, che segna la condizione come aggiornata. Dentro si hanno `patch().Cf()`, `patch().Sf()`, `patch().nf()`, `patch().magSf()`, `patchInternalField()` (i valori delle celle adiacenti), `db().time().value()`, e con `db().lookupObject<volScalarField>("p")` qualunque altro campo registrato.
- **`write()`**: scrive le voci del dizionario così come lette (`writeEntry`), più `value`, perché il file scritto possa essere riletto come condizione iniziale di un calcolo successivo.
- **La registrazione**: `TypeName("profiloParabolico")` nell'intestazione e `makePatchTypeField(fvPatchVectorField, profiloParabolicoFvPatchVectorField)` nel sorgente aggiungono la classe alla tabella delle condizioni per campi vettoriali; da quel momento `type profiloParabolico;` è riconosciuto da *qualunque* solutore che carichi la libreria con `libs ("libprofiloParabolico.so");` in `controlDict`.
- **Il template `foamNewBC`**: `foamNewBC -f -v profiloParabolico` genera lo scheletro (intestazione, sorgente, `Make/`) di una condizione `fixedValue` (`-f`) vettoriale (`-v`): si riempiono `updateCoeffs` e i dati membri. Le opzioni `-m` (mixed) e `-s` (scalare) coprono gli altri casi. Il codice del caso `c14_codice` è stato scritto partendo da lì.

Codice che gira in parallelo

Una condizione al contorno è costruita su ogni processore con la *sua* parte del patch: su alcuni processori `patch().size()` può essere zero, e un calcolo che divide per la somma dei valori del patch (la portata, l'area) deve sommare su tutti i processori con `reduce(somma, sumOp<scalar>())` o `gSum(campo)`, altrimenti ogni processore normalizza con la propria porzione e il risultato dipende dalla decomposizione. Nel nostro esempio non ci sono somme sul patch: il profilo è locale a ogni faccia e la classe funziona in parallelo senza modifiche. Lo stesso vale per i solutori: `gMax`, `gMin`, `gAverage`, `returnReduce` sono le versioni "globali" delle riduzioni, e `Pstream::master()` dice se si è sul processore che deve scrivere.

12.5 Esperimento: i due programmi alla prova

Il caso `c14_codice` contiene i due sorgenti, i loro `Make/`, e due casi di prova: `casoTrasporto` è il gradino obliquo del Cap. 5 risolto con `trasportoFoam` (schema `vanLeer`, 40×40, rilassamento 0,7) da confrontare con la *function object* `scalarTransport`;

`casoProfilo` è il condotto del Cap. 11 con `type profiloParabolico` al posto del blocco `codedFixedValue`. Lo script `Allrun` compila ed esegue tutto.


c14_codice/Allrun

```
#!/bin/bash
# Caso c14_codice: compila la condizione al contorno profiloParabolico e il solutore trasportoFoam con wmake,
# poi esegue i due casi di prova. Richiede l'ambiente OpenFOAM caricato (source .../etc/bashrc).
cd "${0%/*}" || exit 1

echo "1) compilazione della libreria libprofiloParabolico.so (in \$FOAM_USER_LIBBIN)"
(cd profiloParabolico && wmake libso) > log.wmake_profiloParabolico 2>&1 || { echo "   errore: vedi log.wmake_profiloParabolico"; exit 1; }

echo "2) compilazione del solutore trasportoFoam (in \$FOAM_USER_APPBIN)"
(cd trasportoFoam && wmake) > log.wmake_trasportoFoam 2>&1 || { echo "   errore: vedi log.wmake_trasportoFoam"; exit 1; }

echo "3) caso di prova del solutore: gradino obliquo (confronto con scalarTransport, caso c05)"
(cd casoTrasporto && blockMesh > log.blockMesh 2>&1 && trasportoFoam > log.trasportoFoam 2>&1 && tail -n 2 log.trasportoFoam)

echo "4) caso di prova della condizione al contorno: condotto di Poiseuille (confronto con codedFixedValue, caso c13)"
(cd casoProfilo && blockMesh > log.blockMesh 2>&1 && foamRun > log.foamRun 2>&1 && (grep "SIMPLE solution converged" log.foamRun || tail -n 2 log.foamRun))
```

Listato 12.6: Compilazione con `wmake` ed esecuzione dei due casi di prova.


c14_codice/casoProfilo/0/U

```
boundaryField
{
    ingresso
    {
        type            profiloParabolico;      // la classe compilata in libprofiloParabolico.so
        Um              0.1;                    // velocita' media [m/s]
        R               0.01;                   // raggio [m]
        centro          (0 0 0);                // un punto dell'asse (facoltativo, default origine)
        direzione       (1 0 0);                // direzione dell'asse e del moto (facoltativo)
        fattore         halfCosineRamp;         // Function1 facoltativa: rampa dolce da 0 a 1 in 'duration'
        fattoreCoeffs   { start 0; duration 50; }
    }
    uscita      { type zeroGradient; }
    parete      { type noSlip; }
    fronte      { type wedge; }
    retro       { type wedge; }
    asse        { type empty; }
}
```

Listato 12.7: La condizione compilata in uso: i parametri e la rampa temporale (una `Function1`).

Risultati del caso `c14_codice`

- `trasportoFoam` e `scalarTransport` sullo stesso gradino obliquo (40×40, `vanLeer`, rilassamento 0,7): i due campi coincidono cifra per cifra (scarto massimo nullo alle 8 cifre scritte); stesse 1500 iterazioni con residui sovrapponibili (Fig. 12.1): le due implementazioni eseguono le stesse operazioni, perché `fvm::ddt`, `fvm::div` e `fvm::laplacian` sono le stesse funzioni e leggono gli stessi dizionari.
- La classe `profiloParabolico` e il blocco `codedFixedValue` del Cap. 11 producono lo stesso profilo d'ingresso (scarto nullo alle cifre scritte) e lo stesso campo nel condotto (scarto massimo $5{,}0 \times 10^{-8}$); la classe aggiunge il controllo degli argomenti, la rampa temporale (`fattore`) e la scrittura ordinata nel file del campo, ed è riutilizzabile in qualunque caso con `libs ("libprofiloParabolico.so")`.
- Compilazione: `wmake libso` produce `libprofiloParabolico.so` in `$FOAM_USER_LIBBIN`, `wmake` produce `trasportoFoam` in `$FOAM_USER_APPBIN`; i due script `Allrun` e `Allclean` del caso compilano, eseguono e ripuliscono (`wclean`).

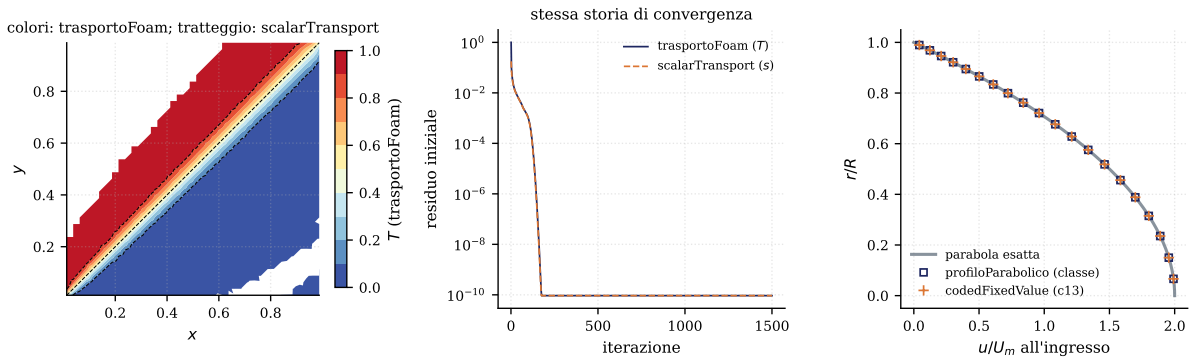


Figura 12.1: Il campo T di `trasportoFoam` con le isolinee di `scalarTransport` sovrapposte (a sinistra); le due storie dei residui (al centro); il profilo d'ingresso della classe `profiloParabolico` e del blocco `codedFixedValue` (a destra).

Quando scrivere codice e quando no

Prima di scrivere un solutore conviene chiedersi se l'equazione in più non sia già disponibile come *function object* (`scalarTransport`, `age`, `comfort`, le `coded`), come `fvModel` (una sorgente, una forza di volume, un disco attuatore: `coded` anche qui) o come `fvConstraint`; e prima di scrivere una condizione al contorno se non basti `codedFixedValue`, `uniformFixedValue` con una `Function1`, o `fixedProfile`. Il codice compilato si giustifica quando lo stesso oggetto serve in molti casi, quando ha parametri da documentare, o quando le prestazioni contano (il codice `coded` è compilato allo stesso modo, ma è legato al caso). Un solutore nuovo, nella versione modulare di OpenFOAM 12, si scrive come *modulo*: una classe derivata da `solver` (o da `fluidSolver`) con i metodi `preSolve`, `momentumPredictor`, `thermophysicalPredictor`, `pressureCorrector`, `postCorrector`, che `foamRun` chiama in sequenza; i moduli `incompressibleFluid/` e `isothermalFluid/` sono il modello da copiare, e il Cap. 8 ne ha già letto i file principali.

12.6 Approfondimenti

12.6.1 Eseguire con il debugger e gli interruttori di debug

Compilato con `WM_COMPILE_OPTION=Debug` (in `etc/bashrc`, poi `./Allwmake` delle librerie interessate), OpenFOAM controlla gli indici fuori dai vettori e si lascia seguire passo passo con `gdb -args foamRun`; lo script `mpirunDebug` fa lo stesso in parallelo. Senza ricompilare, il file `etc/controlDict` (o la sua copia in `$WM_PROJECT_USER_DIR`) ha gli interruttori `DebugSwitches`: `lduMatrix 1` stampa i residui a ogni iterazione del solutore lineare, `SolverPerformance 1` i dettagli di convergenza, il nome di una classe attiva le sue stampe di debug. `FOAM_SIGFPE` (attivo per default) ferma il programma alla prima operazione non valida (divisione per zero, radice di un negativo) e stampa la pila delle chiamate: è la traccia che si legge nei log dei calcoli divergenti, e la funzione più in alto della pila dice *dove* è nato il NaN. `FOAM_SETNAN` inizializza la memoria con NaN per scoprire campi non assegnati.

12.6.2 Le differenze fra le versioni

Il codice di questo capitolo è per OpenFOAM 12 (Foundation). Nelle versioni ESI (`v2312` e seguenti) e nelle Foundation fino alla 10 le differenze principali sono: l'intestazione unica `"fvCFD.H"` al posto delle inclusioni esplicite; `runTime.timeName()` per `runTime.name()`; `Function1<scalar>::New("fattore", dict)` senza le unità; `fvPatchFieldMapper` per `fieldMapper`; `dict.get<scalar>("Um")` (ESI) per `dict.lookup<scalar>("Um")`; i solutori come eseguibili separati (`simpleFoam`, `pimpleFoam`, `rhoCentralFoam`) con `transportProperties` e `turbulenceProperties` al posto di `physicalProperties` e `momentumTransport`. La struttura (costruttori, `updateCoeffs`, `write`, `makePatchTypeField`, `wmake`) è identica, e la traduzione è meccanica; la sezione A.2 dell'appendice raccoglie la tabella delle corrispondenze.

12.6.3 Una *function object* propria

Anche le *function object* si scrivono con lo stesso schema: una classe derivata da `fvMeshFunctionObject` con i metodi `read(dict)`, `execute()` (a ogni passo) e `write()` (agli istanti di scrittura), registrata con `addToRunTimeSelectionTable`; `foamNewFunctionObject` genera lo scheletro. Per un calcolo una tantum basta la `coded` *function object* nel `controlDict`, che accetta i blocchi `codeExecute` e `codeWrite` con lo stesso C++ e lo stesso accesso al database (`mesh().lookupObject<volVectorField>("U")`): la sezione 16.2 del Cap. 16 ne dà un esempio che calcola e scrive una grandezza integrale.

Parte IV

Applicazioni aeronautiche

Capitolo 13

Il profilo NACA 0012: polare e convergenza di griglia

13.1 Il caso di riferimento dell'aerodinamica numerica

Il NACA 0012 a $Re_c = 6 \times 10^6$ e $Ma = 0{,}15$ è il caso su cui i codici RANS vengono confrontati da trent'anni: le misure di Ladson in galleria (con transizione forzata) coprono la polare fino allo stallo, le distribuzioni di pressione di Gregory e O'Reilly completano il quadro, e la NASA Turbulence Modeling Resource pubblica i risultati di CFL3D e FUN3D con Spalart–Allmaras su griglie fino a 1793×513 celle, che per questo modello sono la "soluzione esatta" con cui confrontarsi. L'esercizio ha due facce: la *validazione* (quanto il modello SA riproduce l'esperimento: bene a bassa incidenza, meno vicino allo stallo) e la *verifica* (quanto il nostro calcolo, su una mesh di poche decine di migliaia di celle, riproduce i codici di riferimento sullo stesso modello). Il capitolo costruisce la mesh a C con lo script `nacamesh.py`, calcola la polare da 0° a 15°, confronta C_L, C_D e C_p, esegue lo studio di convergenza di griglia e, soprattutto, racconta ciò che un calcolo aerodinamico "vero" richiede per convergere: è il capitolo in cui tutte le scelte dei precedenti vengono messe alla prova insieme.

La mesh a C

Per un profilo con scia la topologia classica è la C: linee di mesh che avvolgono il profilo, si chiudono a monte in un arco e proseguono a valle lungo la scia fino all'uscita. Con `blockMesh` bastano sei blocchi (Fig. 13.1): due sul dorso e due sul ventre (anteriori e posteriori, divisi a $x_m = 0{,}3c$ dove lo spessore è massimo), due nella scia (sopra e sotto la linea $y = 0$ che prolunga la corda). La superficie è una `spline` per i punti del profilo (la formula NACA a quattro cifre, 200 punti per lato con spaziatura a coseno), il contorno esterno un arco di raggio $R = 20c$ e il piano d'uscita a $L = 20c$. La direzione normale ha la stessa distribuzione in tutti i blocchi (è l'unico modo perché le facce dei blocchi coincidano): N celle con la prima alta y_1 e rapporto geometrico calcolato dallo script (Esercizio 2); lungo il profilo le celle si infittiscono al bordo d'attacco e a quello d'uscita, nella scia crescono verso l'uscita. Il prezzo della semplicità: le celle sono allineate alla direzione del profilo e non alla normale locale, e al bordo d'attacco, dove la superficie curva di 90° in pochi centesimi di corda, la non ortogonalità arriva a 55°; e le celle della scia restano alte y_1 fino all'uscita, con rapporti di forma di 10^4–10^5. Entrambe le cose sono accettabili per il calcolo, ma non gratuite: la Sez. 13.4 ne dà conto.

13.2 Impostazione del caso

Dati del caso `c15_naca0012`

Corda $c = 1$, $U_\infty = 1$, $\nu = 1{,}6667 \times 10^{-7}$ ($Re = 6 \times 10^6$); mesh a C media: 80 celle normali con $y_1 = 2 \times 10^{-5} c$ ($y^+ \approx 2$), $40 + 60$ per lato, 60 nella scia, 25 600 celle; grossa e fine con $1/\sqrt{2}$ e $\sqrt{2}$ celle per direzione (y_1 scalata di conseguenza); contorno esterno `freestreamVelocity`/`freestreamPressure` a $R = L = 20c$ (variante a $40c$ per la sensibilità al dominio); Spalart–Allmaras con $\tilde{\nu}_\infty = 3\nu$ e `nutUSpaldingWallFunction`; incidenza data ruotando $\mathbf{U}_\infty$ e le direzioni di `forceCoeffs`; $\alpha = 0, 4, 8, 10, 12, 15°$; SIMPLE con `potentialFoam` (file `0/Phi` di Neumann), 300 iterazioni `upwind` e poi `linearUpwind` fino a 4000, rilassamento 0,2/0,5.

c15_naca0012.zip

c15_naca0012/0.orig/U

```
// corrente asintotica unitaria; l'incidenza e' data ruotando il vettore (lo script genera.py)
dimensions      [0 1 -1 0 0 0 0];
internalField   uniform (1 0 0);
boundaryField
{
    esterno     { type freestreamVelocity; freestreamValue $internalField; }   // ingresso o uscita secondo il flusso locale
    profilo     { type noSlip; }
    frontEretro { type empty; }
}
```

Listato 13.1: Un solo patch esterno con la condizione `freestreamVelocity`: ingresso o uscita faccia per faccia secondo il segno del flusso, qualunque sia l'incidenza.

c15_naca0012/system/forzeDict

```
coefficienti
{
    type            forceCoeffs;
    libs            ( "libforces.so" );
    writeControl    timeStep;
    writeInterval   1;
    log             no;
    patches         ( profilo );
    rho             rhoInf;
    rhoInf          1;
    liftDir         ( -0.173648 0.984808 0 );
    dragDir         ( 0.984808 0.173648 0 );
    CofR            ( 0.25 0 0 );
    pitchAxis       ( 0 0 1 );
    magUInf         1;
    lRef            1;
    Aref            1;
}
```

Listato 13.2: La *function object* `forceCoeffs` con le direzioni di portanza e resistenza ruotate dell'incidenza (qui 10°) dallo script.


c15_naca0012/genera.py

```
"""
genera.py -- caso c15_naca0012: profilo NACA 0012 a Re = 6e6 con Spalart-Allmaras.

  python3 genera.py mesh [livello]   genera la mesh a C con nacamesh.py (livelli: grossa, media, fine)
  python3 genera.py polare           incidenze 0, 2, 4, 6, 8, 10, 12, 15 gradi sulla mesh media: C_L, C_D, C_m, Cp
  python3 genera.py gci              alfa = 10 gradi su tre mesh: indice di convergenza di griglia (Roache)
  python3 genera.py dominio          alfa = 10 gradi con il contorno esterno a 20 e 40 corde: sensibilita' al dominio
  python3 genera.py tutto
L'incidenza e' imposta ruotando la velocita' asintotica (la mesh non cambia); liftDir e dragDir della
function object forceCoeffs ruotano con essa. Risultati in risultati.json; Cp e storia in dati/.
"""
import os
import sys
import json
import shutil
import subprocess
import numpy as np

QUI = os.path.dirname(os.path.abspath(__file__))
CODICE = next(d for d in (os.path.join(QUI, "..", "..", n) for n in ("codice", "codici")) if os.path.isdir(d))   # codici/ nell'archivio pubblicato
sys.path.insert(0, CODICE)
from foamcase import Caso                                                    # noqa: E402
from foamlog import leggi_log                                                # noqa: E402
from nacamesh import scrivi_blockmesh                                        # noqa: E402

LIVELLI = {  # nNorm, nRear, nFront, nWake, y1 (rapporto sqrt(2) fra livelli); gWake: celle della scia crescenti verso l'uscita
    "grossa": dict(nNorm=57, nRear=42, nFront=28, nWake=42, y1=2.83e-5, gWake=50),
    "media": dict(nNorm=80, nRear=60, nFront=40, nWake=60, y1=2e-5, gWake=50),
    "fine": dict(nNorm=113, nRear=85, nFront=57, nWake=85, y1=1.41e-5, gWake=50),
}
ITER_UPWIND = 300      # iterazioni della prima fase (upwind, rilassamento forte) dopo l'inizializzazione potenziale
ITER_MAX = 4000        # iterazioni della seconda fase (linearUpwind): le forze si assestano entro 2500-3500 (vedi storia)
RAGGIO = 20.0          # raggio dell'arco a monte e lunghezza della scia, in corde
PHI = """FoamFile { format ascii; class volScalarField; object Phi; }
// Potenziale di velocita' per potentialFoam: Neumann su tutto il contorno, cosi' il flusso al contorno e' quello di U.
// Senza questo file potentialFoam impone Phi = 0 sul patch 'esterno' (freestreamPressure e' una condizione mista,
// che 'fissa il valore') e restituisce un campo quasi in quiete: vedi testo.
dimensions      [0 2 -1 0 0 0 0];
internalField   uniform 0;
boundaryField
{
    esterno     { type zeroGradient; }
    profilo     { type zeroGradient; }
    frontEretro { type empty; }
}
"""
ALFA = [0, 4, 8, 10, 12, 15]

```

```
49
50  def sh(cmd, log=None):
51      if log:
52          with open(os.path.join(QUI, log), "w") as fp:
53              return subprocess.run(cmd, cwd=QUI, stdout=fp,
        stderr=subprocess.STDOUT, shell=True).returncode
54      r = subprocess.run(cmd, cwd=QUI, shell=True, capture_output=True, text=True)
55      if r.returncode != 0:
56          raise RuntimeError(r.stderr)
57      return r.stdout
58
59
60  def imposta(file, voce, valore):
61      sh(f'foamDictionary -entry "{voce}" -set "{valore}" {file}')
62
63
64  def pulisci():
65      """Rimuove i risultati e ripristina 0/ da 0.orig (potentialFoam e incidenza()
        lo modificano)."""
66      for d in os.listdir(QUI):
67          if d[0].isdigit() and os.path.isdir(os.path.join(QUI, d)) and d != "0.orig":
68              shutil.rmtree(os.path.join(QUI, d))
69      shutil.copytree(os.path.join(QUI, "0.orig"), os.path.join(QUI, "0"))
70      shutil.rmtree(os.path.join(QUI, "postProcessing"), ignore_errors=True)
71
72
73  def mesh(livello="media", R=RAGGIO, etichetta=None):
74      scrivi_blockmesh(QUI, "0012", R=R, L=R, **LIVELLI[livello])
75      etichetta = etichetta or livello
76      sh("blockMesh", f"log.blockMesh_{etichetta}")
77      sh("checkMesh", f"log.checkMesh_{etichetta}")
78      return Caso(QUI).mesh.nCells
79
80
81  def incidenza(alfa):
82      a = np.radians(alfa)
83      U = f"uniform ({np.cos(a):.8f} {np.sin(a):.8f} 0)"
84      imposta("0/U", "internalField", U)
85      imposta("0/U", "boundaryField/esterno/value", U)      # valore iniziale del
        patch: e' il flusso al contorno di potentialFoam
86      with open(os.path.join(QUI, "0", "Phi"), "w") as fp:
87          fp.write(PHI)
88      imposta("system/forzeDict", "coefficienti/liftDir", f"({-np.sin(a):.8f}
        {np.cos(a):.8f} 0)")
89      imposta("system/forzeDict", "coefficienti/dragDir", f"({np.cos(a):.8f}
        {np.sin(a):.8f} 0)")
90
91
92  def coefficienti():
93      """Legge postProcessing/coefficienti/*/coefficient.dat (una cartella per ogni
        avvio) e media le ultime 50 righe."""
94      cart = os.path.join(QUI, "postProcessing", "coefficienti")
95      tempi = sorted(os.listdir(cart), key=float)
96      blocchi = []
97      for t in tempi:
98          f = [x for x in os.listdir(os.path.join(cart, t)) if x.endswith(".dat")][0]
99          blocchi.append(np.loadtxt(os.path.join(cart, t, f), comments="#"))
100         with open(os.path.join(cart, t, f)) as fp:
101             intest = [l for l in fp if l.startswith("#")][-1].strip("# \n").split()
102     d = np.vstack(blocchi)
```

```
    col = {n: i for i, n in enumerate(intest)}
    ult = d[-50:]
    return {k: float(ult[:, col[k]].mean()) for k in ("Cd", "Cl", "Cm") if k in col}, d, col


def esegui_alfa(alfa, etichetta):
    """Esecuzione in tre passi (vedi testo): potentialFoam per un campo iniziale solenoidale (con il file 0/Phi di
    Neumann); ITER_UPWIND iterazioni con upwind; poi linearUpwind fino a ITER_MAX. Il rilassamento resta 0,2/0,5 in
    entrambe le fasi: 0,3/0,7 e' stabile a R = 20 ma diverge a R = 40. I due registri di foamRun sono concatenati in
    log.foamRun_<etichetta> (foamlog li legge come un'unica storia)."""
    pulisci()
    incidenza(alfa)
    log = f"log.foamRun_{etichetta}"
    sh("potentialFoam", f"log.potentialFoam_{etichetta}")
    imposta("system/controlDict", "startFrom", "startTime")
    imposta("system/controlDict", "endTime", ITER_UPWIND)
    imposta("system/controlDict", "writeInterval", ITER_UPWIND)
    imposta("system/fvSchemes", "divSchemes/div(phi,U)", "bounded Gauss upwind")
    imposta("system/fvSchemes", "divSchemes/div(phi,nuTilda)", "bounded Gauss upwind")
    imposta("system/fvSolution", "relaxationFactors/fields/p", 0.2)
    imposta("system/fvSolution", "relaxationFactors/equations/U", 0.5)
    ok = sh("foamRun", log)
    imposta("system/controlDict", "startFrom", "latestTime")
    imposta("system/controlDict", "endTime", ITER_MAX)
    imposta("system/controlDict", "writeInterval", ITER_MAX)
    imposta("system/fvSchemes", "divSchemes/div(phi,U)", "bounded Gauss linearUpwind grad(U)")
    imposta("system/fvSchemes", "divSchemes/div(phi,nuTilda)", "bounded Gauss linearUpwind grad(nuTilda)")
    imposta("system/fvSolution", "relaxationFactors/fields/p", 0.2)
    imposta("system/fvSolution", "relaxationFactors/equations/U", 0.5)
    with open(os.path.join(QUI, log), "a") as fp:
        ok = ok or subprocess.run("foamRun", cwd=QUI, stdout=fp, stderr=subprocess.STDOUT, shell=True).returncode
    imposta("system/controlDict", "startFrom", "startTime")
    log = leggi_log(os.path.join(QUI, f"log.foamRun_{etichetta}"))
    c = Caso(QUI)
    co, storia, col = coefficienti()
    # Cp sul profilo: p cinematica / (0,5 U^2) con U = 1
    m = c.mesh
    idx = m.facce_patch("profilo")
    pf = c.valore_patch("p", "profilo")
    xy = m.Cf[idx, :2]
    yp = c.yplus("profilo", 1.6667e-7)
    nu = 1.6667e-7
    Fp, Fv = c.forze("profilo", nu)
    a = np.radians(alfa)
    L_py = -Fp[0] * np.sin(a) + Fp[1] * np.cos(a) - Fv[0] * np.sin(a) + Fv[1] * np.cos(a)
    D_py = (Fp[0] + Fv[0]) * np.cos(a) + (Fp[1] + Fv[1]) * np.sin(a)
    spess = m.points[:, 2].max() - m.points[:, 2].min()
    out = dict(alfa=alfa, Cl=co["Cl"], Cd=co["Cd"], Cm=co.get("Cm", float("nan")),
               Cl_python=float(2 * L_py / spess), Cd_python=float(2 * D_py / spess),
               Cd_pressione=float(2 * (Fp[0] * np.cos(a) + Fp[1] * np.sin(a)) / spess),
```

```
153                 Cd_attrito=float(2 * (Fv[0] * np.cos(a) + Fv[1] * np.sin(a)) /
    spess),
154                 iterazioni=len(log["campi"]["p"]["t"]), tempo=float(log["exec"][-1,
    1]), ok=ok == 0,
155                 yplus_medio=float(yp.mean()), yplus_max=float(yp.max()),
156                 convergenza="SIMPLE solution converged" in open(os.path.join(QUI,
    f"log.foamRun_{etichetta}")).read())
157     os.makedirs(os.path.join(QUI, "dati"), exist_ok=True)
158     np.savez(os.path.join(QUI, "dati", f"{etichetta}.npz"), x=xy[:, 0], y=xy[:, 1],
    Cp=2 * pf,
159              storia=storia, colonne=np.array(list(col)),
    res_t=log["campi"]["p"]["t"], res_p=log["campi"]["p"]["res0"],
160              res_Ux=log["campi"]["Ux"]["res0"])
161     print(f"{etichetta:12s} alfa {alfa:4.1f}: Cl {co['Cl']:.4f} Cd {co['Cd']:.5f}
    (python {out['Cl_python']:.4f} {out['Cd_python']:.5f};"
162           f" pressione {out['Cd_pressione']:.5f} attrito {out['Cd_attrito']:.5f})
    it {out['iterazioni']}  y+ medio {out['yplus_medio']:.2f}")
163     return out
164
165
166 def carica():
167     p = os.path.join(QUI, "risultati.json")
168     return json.load(open(p)) if os.path.exists(p) else {}
169
170
171 def main(cosa):
172     ris = carica()
173     if cosa in ("mesh",):
174         n = mesh(sys.argv[2] if len(sys.argv) > 2 else "media"); print("celle:", n)
175     if cosa in ("polare", "tutto"):
176         n = mesh("media"); ris["celle_media"] = n
177         ris["polare"] = [esegui_alfa(a, f"a{a:02d}") for a in ALFA]
178         json.dump(ris, open(os.path.join(QUI, "risultati.json"), "w"), indent=1)
179     if cosa in ("dominio", "tutto"):
180         ris["dominio"] = {}
181         for R in (40.0,):                                # il confronto con R = 20 e'
    il punto a 10 gradi della polare
182             n = mesh("media", R=R, etichetta=f"R{R:.0f}")
183             r = esegui_alfa(10, f"R{R:.0f}"); r["celle"] = n; r["R"] = R
184             ris["dominio"][f"R{R:.0f}"] = r
185             json.dump(ris, open(os.path.join(QUI, "risultati.json"), "w"), indent=1)
186     if cosa in ("gci", "tutto"):
187         ris["gci"] = {}
188         for liv in ("grossa", "fine", "media"):      # la media per ultima: la mesh
    e i campi a 10 gradi restano nel caso
189             n = mesh(liv)
190             r = esegui_alfa(10, f"gci_{liv}"); r["celle"] = n
191             ris["gci"][liv] = r
192             json.dump(ris, open(os.path.join(QUI, "risultati.json"), "w"), indent=1)
193     json.dump(ris, open(os.path.join(QUI, "risultati.json"), "w"), indent=1)
194
195
196 if __name__ == "__main__":
197     main(sys.argv[1] if len(sys.argv) > 1 else "tutto")
```

Listato 13.3: Lo script della polare e dello studio di griglia: mesh da `nacamesh.py`, incidenza, le tre fasi del calcolo, lettura dei coefficienti e verifica indipendente delle forze con `foamcase.py`.

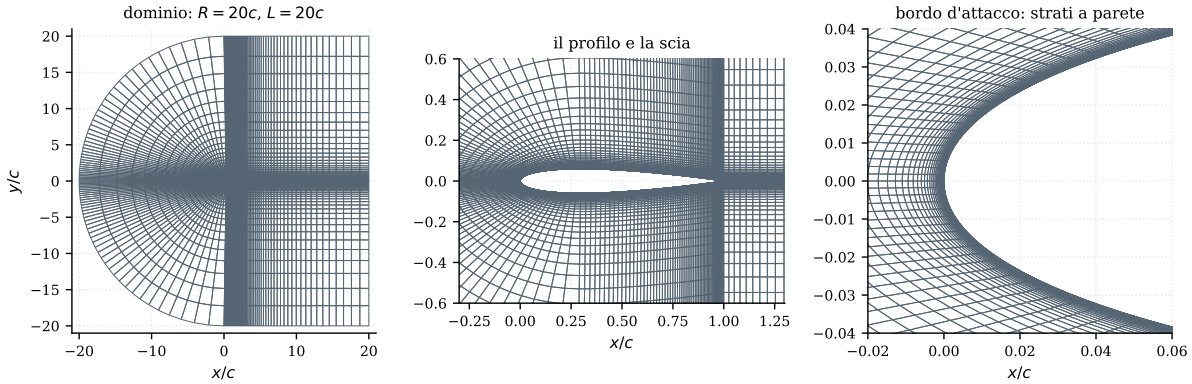


Figura 13.1: La mesh a C della mesh media: il dominio con l'arco a monte e il piano d'uscita (a sinistra), il profilo e l'inizio della scia (al centro), gli strati a parete al bordo d'attacco (a destra).

13.3 Risultati

Risultati del caso `c15_naca0012`

α	C_L	C_L Ladson	C_D	C_D Ladson	$C_{D,p}$	$C_{D,f}$	iter.
0°	-0,0000	0,0004	0,00955	0,00809	0,00191	0,00769	4000
4°	0,4308	0,4272	0,01096	0,00823	0,00338	0,00761	4000
8°	0,8465	0,8557	0,01557	0,01028	0,00824	0,00737	4000
10°	1,0425	1,0586	0,01933	0,01191	0,01221	0,00715	4000
12°	1,2248	1,2493	0,02430	0,01321	0,01748	0,00684	4000
15°	1,4495	1,4938	0,03545	0,01831	0,02924	0,00622	4000

- Mesh media: 25600 celle, y^+ medio 2,35 (massimo 7,85) a 10°: lo strato limite è risolto fino alla parete, come richiede `nutUSpaldingWallFunction` usata in modalità "low-Re".
- Pendenza della retta di portanza (fit 0–8°): $C_{L\alpha} = 6{,}063$ per radiante, cioè $0{,}965 \cdot 2\pi$ (la teoria del profilo sottile dà 2π, l'esperimento circa $0{,}97 \cdot 2\pi$ per lo spessore e lo strato limite).
- A $\alpha = 10°$: $C_L = 1{,}0425$ contro 1,0909 (CFL3D), 1,0983 (FUN3D), 1,0707 a 10,12° (Ladson); $C_D = 0{,}01933$ contro 0,01231 (CFL3D) e 0,01201 (Ladson): scarto 4,4% su C_L e 57,0% su C_D rispetto a CFL3D.
- Ripartizione della resistenza contro i valori di riferimento sulla griglia 897×257 (CFL3D a 10° [24], Flow360 a 0° e 15° [27]): a 0° attrito 0,00769 contro 0,00681 e pressione 0,00191 contro 0,00127; a 10° attrito 0,00715 contro 0,00620 e pressione 0,01221 contro 0,00620; a 15° attrito 0,00622 contro 0,00511 e pressione 0,02924 contro 0,01647. L'attrito è sovrastimato di un 17% circa a tutte le incidenze; la resistenza di pressione è sovrastimata di poco a portanza nulla e sempre di più al crescere di C_L.
- A 15° $C_L = 1{,}450$ contro 1,546 (CFL3D) e 1,494 (Ladson): il dominio a $20c$ abbassa la portanza anche qui, e CFL3D sta sopra l'esperimento perché il modello SA, senza transizione, ritarda lo stallo.

- Le forze integrate da `foamcase.py` coincidono con `forceCoeffs`: scarto massimo 0,01% su C_L e 0,45% su C_D (lo stesso gradiente di parete, la stessa somma $p\,\mathbf{S}_f$).
- GCI a 10° su 12768, 25600 e 51302 celle ($r \approx \sqrt{2}$): C_L = 1,0451, 1,0425, 1,0392, ordine apparente 0,72, valore estrapolato 1,0277, $\mathrm{GCI}_{\mathrm{fine}} = 1{,}39\%$; C_D = 0,02051, 0,01933, 0,01890, ordine 2,91, estrapolato 0,01866, $\mathrm{GCI}_{\mathrm{fine}} = 1{,}62\%$ (convergenza monotona).
- Dominio: con il contorno a $R = L = 40c$ (stessa mesh media, stessa prima cella) $C_L = 1{,}0703$ e $C_D = 0{,}01819$ ($C_{D,p} = 0{,}01108$) contro 1,0425 e 0,01933 a $20c$: 2,7% su C_L e -11 conti su C_D, cioè 16% dello scarto da CFL3D. L'errore di dominio decresce come c/R: l'estrapolazione lineare in c/R dà $C_{L,\infty} \approx 1{,}098$ (CFL3D 1,091, FUN3D 1,098) e $C_{D,\infty} \approx 0{,}0171$. A $60c$ e $100c$ la stessa famiglia di mesh (80 celle normali, scia alta y_1 fino all'uscita) diverge: per allontanare il contorno serve ridisegnare la mesh.

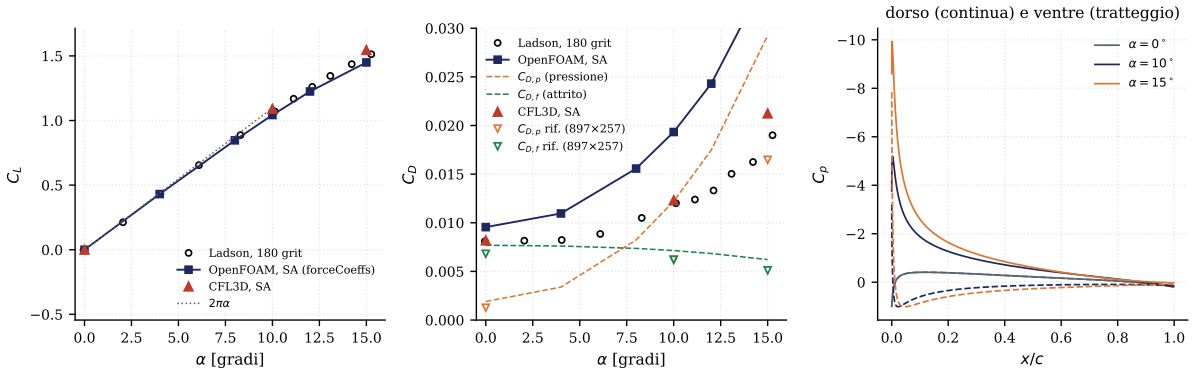


Figura 13.2: Polare del NACA 0012 a $Re = 6 \times 10^6$: $C_L(\alpha)$ e $C_D(\alpha)$ calcolati sulla mesh media, confrontati con i dati di Ladson e con CFL3D (SA, griglia fine), con la ripartizione della resistenza in pressione e attrito; a destra il coefficiente di pressione a 0°, 10° e 15°.

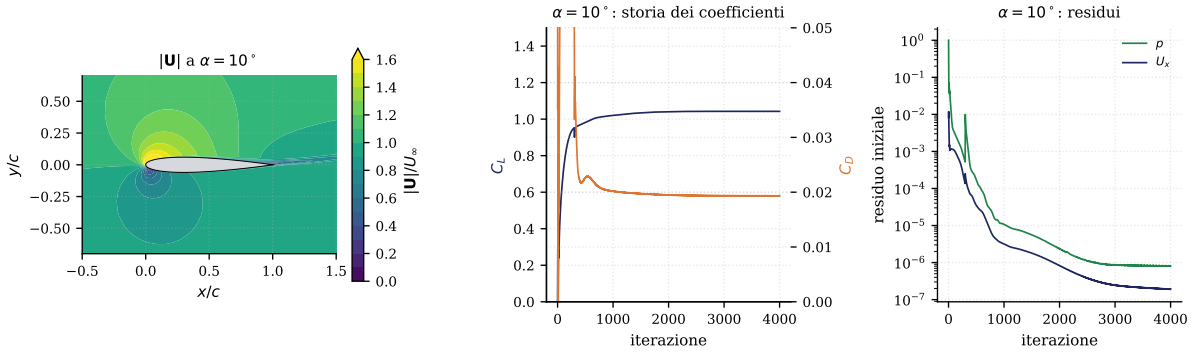


Figura 13.3: Il campo di velocità a 10° (a sinistra); la storia dei coefficienti (al centro) e dei residui (a destra) nelle due fasi di `foamRun` a 10° (300 iterazioni `upwind`, poi `linearUpwind`): le forze si assestano molto prima dei residui.

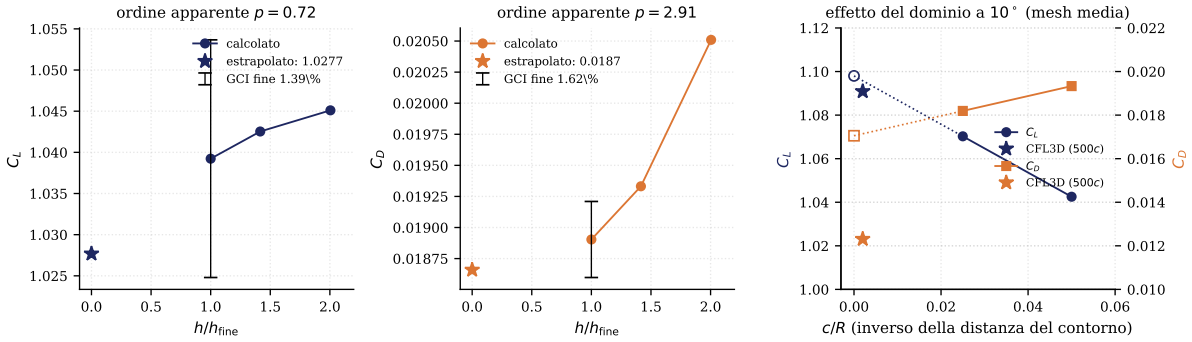


Figura 13.4: Convergenza di griglia a 10° per C_L e C_D sulle tre mesh (grossa, media, fine), con il valore estrapolato di Richardson e la banda del GCI sulla mesh fine (a sinistra e al centro); effetto della distanza del contorno esterno, $20c$ e $40c$, sulla mesh media, con i valori di CFL3D sulla griglia a $500c$ (a destra): l'errore di dominio è più grande dell'errore di griglia.

13.4 Che cosa insegna questo caso

Il caso è stato messo a punto per questa monografia con il metodo che si consiglia al lettore: provare, osservare, capire; e le prove fallite sono istruttive quanto quella riuscita.

Sette modi di sbagliare un calcolo aerodinamico

1. **Il campo iniziale uniforme con celle sottili a parete.** Con $\mathbf{U} = \mathbf{U}_\infty$ in tutte le celle, comprese quelle alte $2 \times 10^{-5}c$ addossate alla parete, la prima iterazione di SIMPLE deve annullare la velocità in quelle celle e la pressione necessaria a deviare il flusso su una cella così sottile è enorme ($p \sim U^2 \Delta x/\Delta y \sim 10^3$): dopo due iterazioni la velocità al bordo d'attacco vale 70 volte U_∞ e il calcolo non si riprende. Sulla mesh del tutorial `airFoil2D` ($y^+ \approx 50$) lo stesso avvio è innocuo. Rimedio: `potentialFoam` (un campo solenoidale che aggira il profilo) e la prima fase con `upwind` e rilassamento forte ($\alpha_p = 0{,}2$, $\alpha_U = 0{,}5$).

2. **`potentialFoam` con i contorni di corrente libera.** `potentialFoam` impone Φ di Dirichlet su ogni patch in cui p "fissa il valore", e `freestreamPressure`, che è una condizione mista, lo fa: $\Phi = 0$ su tutto il contorno esterno e il campo risultante è quasi in quiete ($|\mathbf{U}| \approx 0{,}05\,U_\infty$), senza alcun messaggio d'errore. Il calcolo che parte da lì converge, ma deve prima *riempire* il campo lontano cella dopo cella: dopo seimila iterazioni la corona esterna era ancora a $0{,}85\,U_\infty$, le forze derivavano lentamente e lo studio di griglia misurava l'errore iterativo invece di quello di discretizzazione. Rimedio: un file `0/Phi` con `zeroGradient` su tutti i patch (il flusso al contorno viene allora da $\mathbf{U}$), `PhiRefCell` in `potentialFlow` e la voce `value` sul patch esterno di `0/U`; il campo potenziale è allora quello giusto e le forze si assestano in 2500 iterazioni. La lezione vale in generale: un campo iniziale va *guardato* prima di spendere ore di calcolo.

3. **Il modello di turbolenza nelle prime iterazioni.** Dalla velocità iniziale sbagliata nasce uno sforzo enorme, il modello SA produce $\tilde{\nu}$ fino a $10^5\nu$ e il flusso diventa uno sciroppo che separa dal bordo d'attacco: il calcolo converge a una soluzione stallata e asimmetrica anche a $\alpha = 0$. È una conseguenza del punto precedente, e lo stesso rimedio lo cura.

4. **Il gradiente limitato usato ovunque.** `grad(U) cellLimited Gauss linear`

`1` in `gradSchemes` sembra prudente, ma quel gradiente entra anche nel termine esplicito degli sforzi `div((nuEff*dev2(T(grad(U)))))` e nella correzione non ortogonale del laplaciano: limitarlo lì rompe la coerenza fra parte implicita ed esplicita del termine viscoso e il calcolo diverge al bordo d'attacco entro cento iterazioni. Il limitatore va dato solo allo schema convettivo (`linearUpwind limited` con `limited` definito a parte) o non usato.

5. **`bounded` durante un transitorio violento.** Il prefisso `bounded` sottrae $\nabla \cdot (\phi)\,\mathbf{U}$ con `fvm::Sp`: a convergenza è nullo, ma quando gli errori di continuità sono grandi (le prime iterazioni di un avvio sbagliato) può rendere negativa la diagonale. Ridurre il rilassamento non aiuta, anzi: con $\alpha_p = 0{,}1$ gli errori di continuità per iterazione crescono.

6. **SIMPLEC con rilassamento spinto.** $\alpha_U = 0{,}9$, $\alpha_p = 1$ (ottimi nella cavità del Cap. 8) qui divergono anche da un campo iniziale sano; e $\alpha_U = 0{,}7$, $\alpha_p = 0{,}3$ divergono al passaggio da `upwind` a `linearUpwind` appena il contorno è portato a $40c$. Con 0,2/0,5 il calcolo è più lento (2500–3500 iterazioni) ma converge a tutte le incidenze e su tutti i domini: su una mesh con celle di rapporto di forma 10^4 la stabilità si paga.

7. **Il raffinamento della scia e la mesh con blocchi del naso.** Due tentativi di migliorare la mesh (celle della scia che si allargano verso l'uscita con `edgeGrading`; due blocchi in più al bordo d'attacco per portare la non ortogonalità da 55° a 31°) hanno prodotto mesh *migliori secondo* `checkMesh` e calcoli *peggiori*: con celle sottili che cambiano altezza lungo la scia i centroidi si spostano e le facce sulla linea di scia diventano quasi parallele a $\mathbf{d}_{CF}$; i blocchi del naso cambiano la distribuzione delle celle lungo la superficie. Non tutto ciò che migliora un indice migliora il calcolo: la mesh va giudicata dal risultato.

Leggere i numeri: un bilancio degli errori

La portanza è la grandezza robusta. La pendenza della retta di portanza è entro l'1% di quella sperimentale, e lo scarto a 10° dai codici di riferimento (-4%) è quasi tutto errore di *dominio*: l'estrapolazione in c/R dei due calcoli a $20c$ e $40c$ dà $C_L \approx 1{,}098$, fra CFL3D e FUN3D (Fig. 13.4, a destra), mentre la griglia vale l'1–1,5%.
La resistenza è un'altra storia, e il riquadro permette di separarne le cause. A 10° il calcolo sulla mesh media dà settanta conti (70×10^{-4}) più di CFL3D. Il dominio ne spiega una ventina: la portanza sottostimata dal contorno vicino riduce la spinta del bordo d'attacco, la forza in avanti dovuta alla depressione sul naso, che entra in C_D con il segno meno e fa di C_D una differenza fra contributi grandi e opposti (sommando le facce del profilo a 10°, circa $-0{,}13$ e $+0{,}14$); per questo l'errore di dominio sulla resistenza cresce con C_L ed è quasi nullo a portanza nulla. La griglia ne spiega altri sette (GCI). L'attrito ne vale una decina: è sovrastimato di circa il 17% a tutte le incidenze, e tre prove a $\alpha = 0$ hanno escluso le cause più ovvie: raddoppiare le celle normali dimezzando la prima cella lo abbassa dell'1,6%, raddoppiare quelle lungo il profilo lo alza del 3%, sostituire la funzione di parete di Spalding con `nutLowReWallFunction` lo lascia invariato alla quarta cifra. Restano una trentina di conti di pressione che né il dominio né questa famiglia di griglie spiegano; e l'ordine apparente di C_D nello studio di griglia, 2,9, superiore all'ordine formale degli schemi, avverte che la famiglia non è ancora nell'intervallo asintotico e che il GCI dell'1,6% è indicativo, non una banda di

confidenza.
La lezione è quella del Cap. 16: con 25 000 celle e il contorno a venti corde si ottiene la portanza al percento e la resistenza al 50%; per scendere al 5% servono il dominio a centinaia di corde (o la correzione di vortice), griglie di 10^5–10^6 celle con la prima cella a $y^+ < 1$, e il confronto della propria catena (modello, distanza dalla parete, schemi) con i casi di verifica della NASA TMR, a cominciare dalla lastra piana con lo stesso modello. Infine le forze calcolate da `foamcase.py` coincidono con la *function object*: due programmi indipendenti che leggono gli stessi campi danno lo stesso numero, e questo è il controllo più semplice contro gli errori di post-processing (Esercizio 16).

13.5 Approfondimenti

13.5.1 La correzione di vortice per il contorno esterno

Con il contorno a $20c$ il campo di un profilo portante è ancora quello di un vortice di circolazione $\Gamma = \frac{1}{2} C_L c U_\infty$, con velocità indotta $\Gamma/(2\pi R) \approx C_L/(4\pi \cdot 20) \approx 0{,}4\%$ di U_∞ a $C_L = 1$. Sembra poco, ma imporre la corrente uniforme a quella distanza equivale a un'incidenza efficace diversa di qualche decimo di grado (l'errore sulla velocità indotta cambia segno fra sopra e sotto il profilo e si somma lungo tutto il contorno), e il risultato è un C_L più basso di qualche per cento e un C_D più alto di decine di conti, come mostra il confronto fra $20c$ e $40c$ del riquadro dei risultati. La *vortex correction* (la condizione di corrente libera più il campo del vortice, con Γ aggiornata a ogni iterazione) permette contorni a $10c$ con lo stesso errore; OpenFOAM non la fornisce di serie, ed è un buon esercizio per una condizione al contorno propria (Cap. 12) che legga C_L dalla *function object*.

13.5.2 Transizione e stallo

I dati di Ladson con rugosità (transizione forzata al bordo d'attacco) sono i giusti termini di confronto per un calcolo completamente turbolento; quelli con transizione libera, allo stesso Re, hanno resistenza minore del 10–15% a bassa incidenza e stallano prima. Il modello `kOmegaSSTLM` (Langtry–Menter, quattro equazioni) prevede la transizione dalla turbolenza della corrente libera e riproduce quel comportamento, al prezzo di una mesh con $y^+ < 1$ e di una sensibilità marcata ai valori d'ingresso. Oltre lo stallo (qui 16°–17°) nessun calcolo RANS stazionario è affidabile: la separazione è instazionaria e tridimensionale, e il risultato dipende da come SIMPLE si ferma; URANS o DES (Cap. 14) sono necessari, e l'esperimento stesso ha una dispersione di decimi di C_L.

Capitolo 14

Un transitorio: la scia di von Kármán

14.1 Perché un calcolo transitorio

Tutti i casi dei capitoli precedenti cercavano uno stato stazionario. Molti flussi aeronautici non ne hanno uno: la scia di un cilindro, di un carrello, di un'antenna o di una fusoliera tozza oscilla periodicamente, e lo stesso fanno un profilo in stallo profondo, un getto, un flusso in cavità aperta. Per questi casi la soluzione RANS stazionaria non converge (i residui si fermano, Cap. 6) o converge a una media che non esiste in natura, e bisogna integrare nel tempo con PISO o PIMPLE (Cap. 8) e analizzare *segnali*: storie temporali di forze, pressioni, velocità in punti scelti, da cui estrarre frequenze, ampiezze e medie. Il cilindro a $Re = 100$ è il banco di prova classico: la scia è laminare e bidimensionale, il distacco alternato dei vortici è periodico con una frequenza nota con grande precisione dagli esperimenti (il numero di Strouhal $St = fD/U$ di Williamson e di Roshko), e il calcolo dura pochi minuti.

Che cosa si deve decidere in un transitorio

Lo schema temporale
`Euler` è del primo ordine e smorza (Cap. 6): in un calcolo di distacco di vortici fa calare l'ampiezza e può spostare la frequenza; `backward` (secondo ordine, tre livelli temporali) è la scelta standard per i transitori incomprimibili, `CrankNicolson 0.9` l'alternativa meno dissipativa ma meno robusta.

Il passo temporale
fissato dalla fisica, non dalla stabilità: il periodo di distacco è $T = D/(St\,U) \approx 6D/U$ e deve essere coperto da almeno 50–100 passi; il numero di Courante risultante ($\mathrm{Co}_{\max} \lesssim 1$ con PISO, qualche unità con PIMPLE e più correttori esterni) è una conseguenza. Con `adjustTimeStep yes` e `maxCo` il passo segue la velocità massima, ma in un distacco periodico è meglio un passo costante: le analisi in frequenza vogliono campioni equispaziati.

L'algoritmo
PISO (`nOuterCorrectors 1`, `nCorrectors 2`) a $\mathrm{Co} < 1$; PIMPLE con 2–3 correttori esterni e rilassamento se si vuole un passo più grande. Il solutore di `pFinal` (l'ultimo correttore) va a tolleranza piena (`relTol 0`): è quello che fissa la conservazione della massa nel passo.

La durata
un transitorio iniziale (la scia si forma, il distacco si innesca da un'asimmetria

numerica) seguito dal ciclo limite; l'analisi va fatta solo su quest'ultimo, scartando il transitorio, e deve coprire molti periodi (10–20) perché la risoluzione in frequenza della trasformata di Fourier è l'inverso della durata della finestra.

Che cosa registrare
le *function object* a ogni passo (`writeControl timeStep`): `forceCoeffs` per le forze, `probes` per i segnali puntuali, e i campi solo ogni tanto (`writeInterval` grande e `purgeWrite` per non riempire il disco). I campi derivati (`vorticity`, `Q`, `Lambda2`) si scrivono con gli stessi istanti.

14.2 Esperimento: cilindro a $Re = 100$

Il caso `c16_cilindro` usa la mesh a O del Cap. 3 infittita (prima cella $\approx 0{,}008D$, 160 celle sulla circonferenza) con il contorno esterno a 20 diametri e le condizioni `freestream`; il fluido ha $\nu = 0{,}01$ ($Re = UD/\nu = 100$); si integra con PISO e `backward` fino a $tU/D = 150$, registrando C_L, C_D e la velocità in due punti della scia a ogni passo. Lo script `fig16_cilindro.py` scarta il transitorio, calcola lo spettro di C_L e del segnale della sonda con la trasformata di Fourier (finestra di Hann, interpolazione parabolica del picco), e confronta il numero di Strouhal con le correlazioni sperimentali

$$St = 0{,}2175 - \frac{5{,}1064}{Re} \text{ (Williamson, 1988)}, \qquad St = 0{,}212\left(1 - \frac{21{,}2}{Re}\right) \text{ (Roshko, 1954)}, \tag{14.1}$$

valide nel regime laminare bidimensionale ($49 < Re < 180$).

Dati del caso `c16_cilindro`

$D = 1$, $U = 1$, $\nu = 0{,}01$; mesh a O, 4 blocchi da 60×40 celle, espansione radiale 100, $R_o = 20D$; `backward`, PISO con 2 correttori, $\Delta t = 0{,}005$ ($\mathrm{Co}_{\max} \approx 0{,}4$), $t_{\mathrm{fin}} = 150$; `forceCoeffs` con $l_{\mathrm{ref}} = A_{\mathrm{ref}} = D$ (spessore 1), sonde in $(3D, 0{,}5D)$ e $(6D, 0{,}5D)$; finestra di analisi $90 \le tU/D \le 150$.

c16_cilindro.zip

c16_cilindro/system/controlDict

```
application     foamRun;
solver          incompressibleFluid;
startFrom       startTime;
startTime       0;
stopAt          endTime;
endTime         150;              // in unita' D/U: circa 20 periodi di distacco dopo il transitorio
deltaT          0.005;            // Co massimo ~ 1 vicino al cilindro
writeControl    runTime;
writeInterval   10;
purgeWrite      3;                // si conservano solo gli ultimi tre istanti
writeFormat     ascii;
writePrecision  8;
timeFormat      general;
timePrecision   6;
runTimeModifiable true;

functions
{
    coefficienti
```

```
    {
        type            forceCoeffs;
        libs            ("libforces.so");
        writeControl    timeStep;
        writeInterval   1;
        log             no;
        patches         (cilindro);
        rho             rhoInf;
        rhoInf          1;
        liftDir         (0 1 0);
        dragDir         (1 0 0);
        CofR            (0 0 0);
        pitchAxis       (0 0 1);
        magUInf         1;
        lRef            1;          // diametro
        Aref            1;          // diametro x spessore (la mesh e' spessa 1)
    }
    sonda
    {
        type            probes;
        libs            ("libsampling.so");
        writeControl    timeStep;
        writeInterval   1;
        fields          (U p);
        probeLocations  ((3 0.5 0.5) (6 0.5 0.5));     // nella scia, fuori asse
    (la componente v oscilla)
    }
    #includeFunc vorticity          // il campo della vorticita' agli istanti di
    scrittura
}
```

Listato 14.1: Passo costante, scrittura dei campi rada con `purgeWrite`, le *function object* `forceCoeffs` e `probes` a ogni passo e la vorticità agli istanti di scrittura.

```
solvers
{
    p
    {
        solver          GAMG;
        smoother        GaussSeidel;
        tolerance       1e-07;
        relTol          0.01;
    }
    pFinal  { $p; relTol 0; }
    "U.*"   { solver smoothSolver; smoother symGaussSeidel; tolerance 1e-08; relTol
    0; }
}

PIMPLE
{
    nOuterCcorrectors 1;            // PISO
    nCorrectors     2;
    nNonOrthogonalCorrectors 0;
}
```

c16_cilindro/system/fvSolution

Listato 14.2: PISO: un correttore esterno, due di pressione, `pFinal` a tolleranza piena, nessun rilassamento.

Risultati del caso `c16_cilindro`

- Mesh a O di 9600 celle; 30000 passi di $\Delta t = 0{,}005\,D/U$ in 8 min, $\mathrm{Co}_{\max} = 0{,}42$ (PISO con `backward`, due correttori).
- Il distacco si instaura spontaneamente (nessuna perturbazione imposta) dopo circa $tU/D \approx 61$ (ampiezza 0,05) e satura a un ciclo limite: C_L oscilla con ampiezza $\pm 0{,}339$ (valore efficace 0,238), $C_D = 1{,}377 \pm 0{,}015$; la resistenza oscilla alla frequenza doppia (2,00 volte quella di C_L), perché ogni vortice, da sopra o da sotto, la fa crescere allo stesso modo.
- Numero di Strouhal dalla FFT di C_L: $\mathrm{St} = 0{,}1679$; dalla componente v alla sonda in $(6D,\ 0{,}5D)$: 0,1679. Correlazioni sperimentali a $Re = 100$: Williamson $0{,}2175 - 5{,}1064/Re = 0{,}1664$, Roshko $0{,}212(1 - 21{,}2/Re) = 0{,}1671$: scarto 0,9%. Valori numerici di riferimento in letteratura: $C_D = 1{,}33$–$1{,}37$, $\mathrm{St} = 0{,}164$–$0{,}167$.
- La risoluzione in frequenza della FFT è $\Delta f = 1/T = 0{,}0167$ (finestra di 60 D/U, circa 10 periodi): l'interpolazione parabolica del picco porta l'incertezza su St sotto il $\pm 1\%$; con una finestra più corta il risultato sarebbe illusoriamente preciso solo sulla terza cifra.

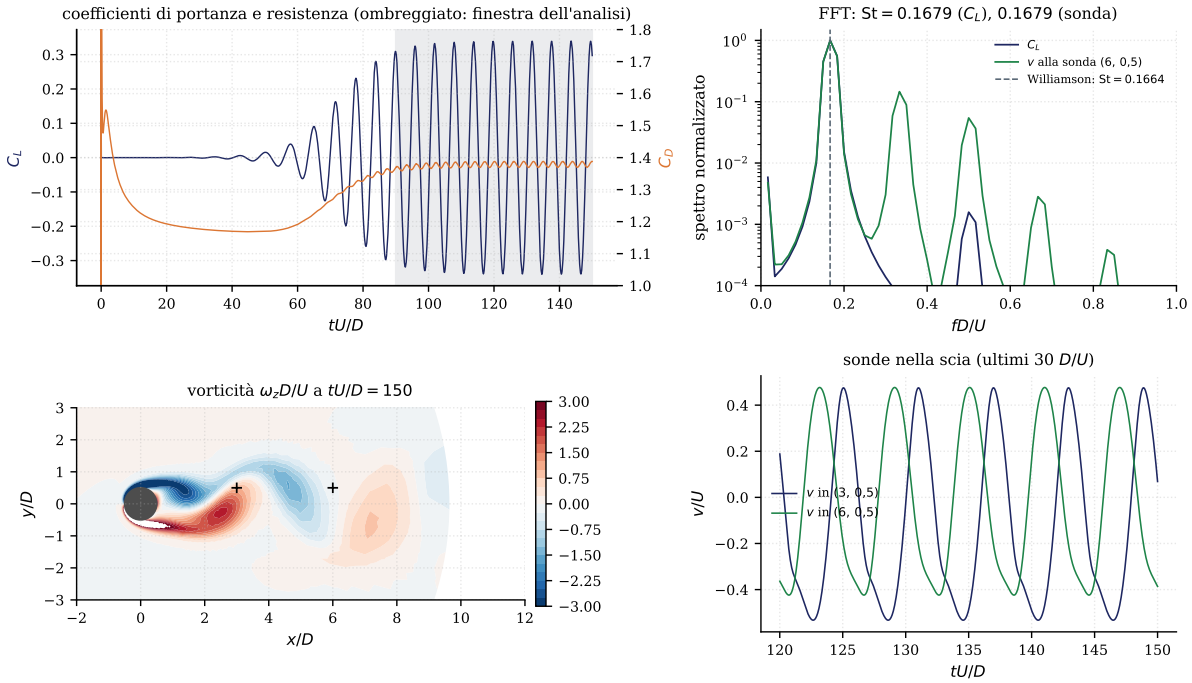


Figura 14.1: Cilindro a $Re = 100$: storie di C_L e C_D con la finestra d'analisi (in alto a sinistra); spettri normalizzati di C_L e del segnale di velocità nella scia con il numero di Strouhal sperimentale (in alto a destra); campo di vorticità all'istante finale con le due sonde (in basso a sinistra); i segnali delle sonde negli ultimi 30 D/U (in basso a destra).

Leggere i segnali

Tre regole che l'esperimento illustra. *Primo*: il transitorio iniziale è lungo (qui 60–80 D/U) e non ha significato fisico, perché dipende da come il distacco si innesca (dal rumore numerico, in un caso perfettamente simmetrico); in un calcolo industriale conviene innescarlo con una perturbazione iniziale o partire da un campo asimmetrico. *Secondo*:

la resistenza oscilla alla frequenza doppia della portanza e intorno a una media più alta di quella del flusso stazionario simmetrico (che si otterrebbe forzando la simmetria): la media temporale di un flusso instazionario non coincide con la soluzione stazionaria delle stesse equazioni, ed è per questo che un calcolo RANS stazionario di un corpo tozzo sottostima la resistenza. *Terzo*: lo spettro va calcolato su una finestra di molti periodi (qui dieci) e la frequenza del picco va interpolata; il numero di Strouhal così ottenuto coincide con le correlazioni entro l'1%, mentre contare i periodi "a occhio" su pochi cicli dà il 5%.

14.3 Approfondimenti

14.3.1 Da Re 100 a Re 3900: quando il 2D non basta

Oltre $Re \approx 190$ la scia del cilindro diventa tridimensionale (modi A e B di Williamson) e il calcolo 2D laminare sovrastima la forza oscillante; a $Re = 3900$, il caso di riferimento per le LES, la scia è turbolenta e il distacco di vortici coesiste con la turbolenza a piccola scala: serve un calcolo 3D con LES (`simulationType LES`, modello `WALE` o `kEqn`, schemi centrati o `LUST`, `backward`, Co $< 0{,}5$) su una mesh di milioni di celle e un'estensione in apertura di almeno πD con condizioni `cyclic`. Il metodo di analisi è lo stesso (segnali, spettri, medie di fase), con l'aggiunta delle medie temporali dei campi (`fieldAverage`) e delle statistiche turbolente (`UPrime2Mean`). Fra i due estremi, URANS (RANS instazionario con k–ω SST, Cap. 9) cattura il distacco di vortici di grande scala con un costo contenuto ed è quanto si usa per corpi tozzi aeronautici (carrelli, cupole, carichi esterni) quando interessa la frequenza e la forza oscillante; la LES o la DES (ibrido, `SpalartAllmarasDDES`) quando interessano gli spettri di pressione (rumore, buffeting).

14.3.2 Flussi instazionari su profili

Lo stesso schema (PIMPLE, `backward`, segnali) serve per il profilo oscillante (`dynamicMeshDict` con `solidBody` e `oscillatingRotatingMotion`, o `dynamicMotionSolverFvMesh` con la mesh che si deforma), per lo stallo dinamico e per le raffiche (`uniformFixedValue` con una `Function1 table` all'ingresso). Le forze vanno lette con `forceCoeffs` rispetto a un sistema di riferimento che ruota con il profilo (`CofR`, `liftDir` aggiornati) o, più semplicemente, calcolate a posteriori con `foamcase.py` dai campi scritti, come si fa nel Cap. 13 per verificare la *function object*.

Capitolo 15

Un caso tridimensionale: la semiala con `snappyHexMesh` in parallelo

15.1 Dal 2D al 3D: che cosa cambia davvero

Le equazioni, gli schemi e gli algoritmi non cambiano passando a tre dimensioni; cambiano la generazione della mesh, il costo del calcolo e l'organizzazione del lavoro. Per le geometrie reali `blockMesh` non basta, e si usa `snappyHexMesh`, che parte da una mesh esaedrica di fondo e la adatta a una superficie triangolata (STL, OBJ) in tre fasi; il numero di celle passa da decine di migliaia a milioni, e il calcolo si distribuisce su più processori con `decomposePar` e `mpirun`; le mesh risultanti sono *non strutturate* nel senso pieno (poliedri agli intersecamenti dei livelli, non ortogonalità di 60°–70° negli strati), e gli schemi vanno scelti di conseguenza (`limited corrected 0.33`, gradienti `cellLimited` per i soli termini convettivi, Cap. 4–5). Questo capitolo percorre l'intera catena su un caso aeronautico semplice ma completo, la semiala rettangolare NACA 0012 di allungamento $AR = 4$ a 5° d'incidenza, la cui portanza è prevista dalla teoria della linea portante del Modulo 2: con $a_0 = 2\pi$ e distribuzione ellittica,

$$C_L = \frac{a_0\,\alpha}{1 + a_0/(\pi\,AR)} = \frac{2\pi\alpha}{1 + 2/AR}, \qquad C_{D,i} = \frac{C_L^2}{\pi\,AR\,e}, \tag{15.1}$$

che per $AR = 4$, $\alpha = 5°$ danno $C_L = 0{,}366$ e $C_{D,i} = 0{,}0107$ ($e = 1$); per un'ala rettangolare $e \approx 0{,}9$–$0{,}95$ e la pendenza è leggermente inferiore, e lo spessore del profilo riduce ulteriormente a_0 rispetto a 2π (Cap. 13).

`snappyHexMesh` in tre fasi

Geometria
una o più superfici chiuse (`triSurfaceMesh` da file STL/OBJ in `constant/geometry/`) e forme analitiche (`searchableBox`, `searchableSphere`, `searchableCylinder`) usate come regioni di raffinamento. La superficie deve essere *chiusa e orientata* (`surfaceCheck` lo verifica) e i suoi spigoli vivi vanno estratti con `surfaceFeatures` (`surfaceFeaturesDict`: `includedAngle 150` considera spigolo ogni coppia di triangoli con angolo diedro minore di 150°) in un file `.eMesh`.

Castellamento
(`castellatedMesh`) parte dalla mesh di fondo di `blockMesh` (celle cubiche: il *livello 0*) e raffina per dimezzamento: le celle attraversate dalla superficie fino

al livello indicato in `refinementSurfaces` (`level (5 6)`: 5 ovunque, 6 dove la superficie forma angoli superiori a `resolveFeatureAngle`), le celle attraversate dagli spigoli (`features`), le celle dentro una regione o entro una distanza (`refinementRegions`, modo `inside` o `distance`); poi rimuove le celle dalla parte della superficie opposta a `insidePoint`. `nCellsBetweenLevels` impone un numero minimo di celle fra due livelli consecutivi (transizioni graduali); `maxGlobalCells` è il tetto di celle.

Snap
sposta i vertici della superficie a gradini sulla superficie vera e, con `explicitFeatureSnap`, sugli spigoli del file `.eMesh` (bordo d'uscita, estremità alare), poi rilassa la mesh per rispettare i criteri di `meshQualityDict` (`maxNonOrtho 65`, `maxBoundarySkewness 20`, `maxInternalSkewness 4`, `minVol`, `minTetQuality`): se una modifica violerebbe un criterio, la cella non viene spostata e la superficie resta localmente a gradini.

Strati
(`addLayers`) estrude dalla superficie `nSurfaceLayers` strati prismatici con `expansionRatio`, di spessore totale fissato (in modo relativo alla cella di superficie, `relativeSizes true`, o assoluto); dove la geometria non lo permette (angoli concavi, celle troppo distorte) gli strati vengono ridotti o omessi, e il rapporto fra lo spessore ottenuto e quello richiesto è stampato nel registro (`thicknessFraction`): un valore sotto il 70–80% dice che la mesh a parete non è quella che si credeva, e lo y^+ va sempre verificato a posteriori.

Il processo è deterministico ma sensibile ai parametri: si lavora per iterazioni (`castellatedMesh` solo, poi con `snap`, poi con gli strati), guardando la mesh in ParaView a ogni passo; `snappyHexMesh -overwrite` scrive la mesh finale in `constant/polyMesh/` invece che in cartelle temporali successive.

Il calcolo parallelo

OpenFOAM parallelizza per *decomposizione del dominio*: `decomposePar` divide la mesh in N sottodomini (`processor0/`, ...) secondo `decomposeParDict` (`scotch` minimizza le facce di interfaccia bilanciando le celle, `simple` e `hierarchical` tagliano con piani, `manual` da una lista), e ogni processo risolve le sue celle scambiando i valori delle celle adiacenti all'interfaccia con MPI attraverso i patch `processor`. Tutte le applicazioni si lanciano con `mpirun -np N applicazione -parallel`, compresi `snappyHexMesh` (la mesh di fondo viene decomposta e il raffinamento ribilanciato: `maxLoadUnbalance`) e `checkMesh`; `reconstructPar` ricompone i campi (e la mesh, se è cambiata) per il post-processing seriale, `redistributePar` cambia il numero di processori senza ricomporre. Nei file di campo la voce `#includeEtc "caseDicts/setConstraintTypes"` fornisce automaticamente le condizioni per i patch vincolati (`processor`, `symmetryPlane`, `cyclic`, `empty`), che altrimenti andrebbero elencati a mano. L'efficienza è buona finché ogni processo ha almeno 30–50 mila celle: sotto, lo scambio di messaggi domina; GAMG scala peggio di PCG ma resta più veloce in assoluto. I residui e i risultati di un calcolo parallelo differiscono da quelli seriali solo per l'aritmetica dei solutori (l'ordine delle somme, il precondizionatore DIC applicato per sottodominio): è una buona verifica da fare una volta su un caso piccolo.

15.2 Esperimento: la semiala NACA 0012

Il caso `c17_ala` costruisce la superficie dell'ala con lo script `wingstl.py` (una sezione NACA 0012 estrusa per $2c$ di semiapertura, prolungata di $0{,}05c$ oltre il piano di simmetria perché il piano tagli il solido invece di coincidere con la sua faccia, estremità piana chiusa), la mesh con `snappyHexMesh` su due processori e risolve con `incompressibleFluid` e Spalart–Allmaras a $Re_c = 10^6$ con funzioni di parete; la portanza e la resistenza vengono dalla *function object* `forceCoeffs` con la direzione di portanza ruotata dell'incidenza e A_{ref} pari alla superficie della semiala.


c17_ala.zip

Dati del caso `c17_ala`

Corda $c = 1$, semiapertura $2c$ ($AR = 4$), NACA 0012, $\alpha = 5°$, $Re_c = 10^6$; dominio $-6 < x < 12$, $-6 < y < 6$, $0 < z < 6$ con piano di simmetria in $z = 0$; mesh di fondo $36 \times 24 \times 12$ (celle di $0{,}5c$), livelli 5–6 sulla superficie ($0{,}016$–$0{,}008c$), 4 strati con rapporto 1,3, scatola di raffinamento di livello 3 intorno all'ala e nella scia; SIMPLEC con rilassamento 0,9, 1000 iterazioni; `nutUSpaldingWallFunction`.


c17_ala/Allrun

```
#!/bin/bash
# Caso c17_ala: semiala NACA 0012 (AR = 4) a 5 gradi con snappyHexMesh, in parallelo su 2 processori
cd "${0%/*}" || exit 1
STRUMENTI=../../codice; [ -d $STRUMENTI ] || STRUMENTI=../../codici        # codici/ nell'archivio pubblicato
python3 $STRUMENTI/wingstl.py constant/geometry/ala.stl 0012 2.0 120 > log.wingstl 2>&1
surfaceFeatures  > log.surfaceFeatures 2>&1     # spigoli vivi (TE, tip) -> constant/geometry/ala.eMesh
blockMesh        > log.blockMesh 2>&1           # mesh di fondo
decomposePar -copyZero > log.decomposePar 2>&1  # partizione della mesh di fondo in 2
mpirun -np 2 snappyHexMesh -overwrite -parallel > log.snappyHexMesh 2>&1   # castellamento, snap, strati
mpirun -np 2 checkMesh -parallel > log.checkMesh 2>&1
mpirun -np 2 potentialFoam -parallel > log.potentialFoam 2>&1              # campo iniziale solenoidale
mpirun -np 2 foamRun -parallel > log.foamRun 2>&1
reconstructPar -latestTime > log.reconstructPar 2>&1                       # ricompone mesh e campi per il post-processing
tail -n 3 log.foamRun
```

Listato 15.1: La catena completa: superficie, spigoli, mesh di fondo, decomposizione, `snappyHexMesh` e calcolo in parallelo, ricomposizione.


c17_ala/system/snappyHexMeshDict

```
castellatedMesh true;
snap            true;
addLayers       true;

geometry
{
    ala                                     // la superficie STL scritta da codice/wingstl.py
    {
        type triSurfaceMesh;
        file "ala.stl";
    }
```

```
    scatolaScia                             // regione di raffinamento intorno all'ala e nella scia
    {
        type searchableBox;
        min (-0.5 -0.6 0);
        max ( 3.0  0.6 2.4);
    }
}

castellatedMeshControls
{
    maxLocalCells       500000;
    maxGlobalCells      1500000;
    minRefinementCells  10;
    maxLoadUnbalance    0.10;
    nCellsBetweenLevels 3;                  // celle di transizione fra due livelli (2:1 piu' graduale)

    features                                // spigoli vivi (bordo d'uscita, tip) dal file eMesh di surfaceFeatures
    (
        { file "ala.eMesh"; level 6; }
    );

    refinementSurfaces
    {
        ala
        {
            level (5 6);                    // 0,5/2^5 = 0,016 c sulla superficie, 0,008 c dove la curvatura e' alta
            patchInfo { type wall; }
        }
    }
    resolveFeatureAngle 30;

    refinementRegions
    {
        scatolaScia { mode inside; level 3; }                       // 0,06 c nella scatola
        ala         { mode distance; levels ((0.05 5) (0.2 4)); }   // 0,016 c entro 0,05 c dall'ala, 0,03 c entro 0,2 c
    }

    insidePoint (-3 0 3);                   // un punto nel fluido: si tiene la regione che lo contiene
    allowFreeStandingZoneFaces true;
}
```

Listato 15.2: Geometria, raffinamento e castellamento: la superficie, la scatola della scia, i livelli sulla superficie e per distanza, lo spigolo del bordo d'uscita.

```
snapControls
{
    nSmoothPatch        3;
    tolerance           2.0;
    nSolveIter          30;
    nRelaxIter          5;
    nFeatureSnapIter    10;
    implicitFeatureSnap false;
    explicitFeatureSnap true;
    multiRegionFeatureSnap false;
}

addLayersControls
{
    relativeSizes       true;               // spessori relativi alla cella di
    superficie
    layers
    {
        ala { nSurfaceLayers 4; }
    }
    expansionRatio      1.3;
    finalLayerThickness 0.4;                // l'ultimo strato e' il 40% della
    cella adiacente
    minThickness        0.05;
    nGrow               0;
    featureAngle        110;
    slipFeatureAngle    30;
    nRelaxIter          5;
    nSmoothSurfaceNormals 1;
    nSmoothNormals      3;
    nSmoothThickness    10;
    maxFaceThicknessRatio 0.5;
    maxThicknessToMedialRatio 0.3;
    minMedianAxisAngle  90;
    nBufferCellsNoExtrude 0;
    nLayerIter          50;
}

meshQualityControls
{
    #include "meshQualityDict"
}

writeFlags ( scalarLevels layerSets layerFields );
mergeTolerance 1e-6;
```

c17_ala/system/snappyHexMeshDict

Listato 15.3: Snap e strati a parete; i criteri di qualità sono in `meshQualityDict`.

```
"""
wingstl.py -- scrive la superficie STL (triangoli, ASCII) di un'ala rettangolare
    NACA a quattro cifre per
snappyHexMesh: corda unitaria, semiapertura b2, radice prolungata di 'eccesso'
    oltre il piano di simmetria z = 0
(cosi' il piano di simmetria taglia l'ala senza che la superficie coincida con il
    contorno) e tip piano chiuso.
Uso: python3 wingstl.py <file.stl> [sigla] [b2] [n]      (es. python3 wingstl.py
    ala.stl 0012 2.0 120)
"""
import os
import sys
```

wingstl.py

```python
import numpy as np
from nacamesh import naca4


def ala(sigla="0012", b2=2.0, n=120, eccesso=0.05, nz=40):
    dorso, ventre = naca4(sigla, n)
    # contorno chiuso della sezione, da TE (dorso) a LE a TE (ventre), senza ripetere LE e TE
    sez = np.vstack([dorso[::-1], ventre[1:-1]])          # (2n-2, 2), orientata in senso antiorario visto da +z
    z = np.linspace(-eccesso, b2, nz)
    tri = []
    # superficie laterale: strisce fra due sezioni consecutive
    m = len(sez)
    for k in range(nz - 1):
        for i in range(m):
            j = (i + 1) % m
            a = (*sez[i], z[k]); b = (*sez[j], z[k]); c = (*sez[j], z[k + 1]); d = (*sez[i], z[k + 1])
            tri.append((a, b, c)); tri.append((a, c, d))
    # tappi (radice e tip): ventaglio di triangoli dal centro della sezione
    for zz, verso in ((z[0], -1), (z[-1], 1)):
        c0 = (0.5, 0.0, zz)
        for i in range(m):
            j = (i + 1) % m
            a = (*sez[i], zz); b = (*sez[j], zz)
            tri.append((c0, a, b) if verso > 0 else (c0, b, a))
    return np.array(tri)


def scrivi_stl(percorso, tri, nome="ala"):
    os.makedirs(os.path.dirname(os.path.abspath(percorso)), exist_ok=True)
    with open(percorso, "w") as fp:
        fp.write(f"solid {nome}\n")
        for a, b, c in tri:
            nrm = np.cross(np.subtract(b, a), np.subtract(c, a)); nn = np.linalg.norm(nrm)
            nrm = nrm / nn if nn > 0 else nrm
            fp.write(f"  facet normal {nrm[0]:.6e} {nrm[1]:.6e} {nrm[2]:.6e}\n  outer loop\n")
            for v in (a, b, c):
                fp.write(f"      vertex {v[0]:.7e} {v[1]:.7e} {v[2]:.7e}\n")
            fp.write("    endloop\n  endfacet\n")
        fp.write(f"endsolid {nome}\n")


if __name__ == "__main__":
    out = sys.argv[1] if len(sys.argv) > 1 else "ala.stl"
    sigla = sys.argv[2] if len(sys.argv) > 2 else "0012"
    b2 = float(sys.argv[3]) if len(sys.argv) > 3 else 2.0
    n = int(sys.argv[4]) if len(sys.argv) > 4 else 120
    t = ala(sigla, b2, n)
    scrivi_stl(out, t)
    print(f"scritto {out}: {len(t)} triangoli, semiapertura {b2}, {n} punti per lato")
```

Listato 15.4: Lo script che scrive la superficie STL dell'ala: sezioni NACA estruse, strisce di triangoli, tappi di radice ed estremità.

Risultati del caso `c17_ala`

- Mesh: 339948 celle (fondo $36 \times 24 \times 12$, livelli 5–6 sulla superficie, 4 strati richiesti: 3,88 strati medi e 97% dello spessore nominale raggiunti), `checkMesh` senza errori; 1000 iterazioni SIMPLEC in parallelo su 2 processori, 16 minuti.
- y^+ sull'ala: minimo 7,1, massimo 302, medio 45: funzione di parete di Spalding su tutto l'intervallo.
- $C_L = 0{,}3134$, $C_D = 0{,}0230$ (function object `forceCoeffs`; dalle sole pressioni, con `foamcase.py`: $C_L = 0{,}3138$, $C_D = 0{,}0137$); variazione relativa di C_L nelle ultime 200 iterazioni $1{,}2 \times 10^{-6}$, residui finali 10^{-7}.
- Linea portante di Prandtl con $a_0 = 2\pi$: $C_L = 0{,}366$; con la pendenza del profilo calcolata nel Cap. 13 ($a_0 = 6{,}06$ rad^{-1}): $C_L = 0{,}357$; correzione di Helmbold per allungamento basso: $C_L = 0{,}332$. Il calcolo è del 5,6% sotto Helmbold.
- Attrito: $C_{D,f} = C_D - C_{D,p} = 0{,}0093$, contro la stima di lastra piana turbolenta a $Re_c = 10^6$ (Schultz–Grunow, due facce, con la correzione di sovravelocità di Hoerner) $2C_f(1 + 2t/c) = 0{,}00895$: la funzione di parete di Spalding a y^+ medio 45 dà l'attrito giusto.
- Resistenza totale: $C_D - C_L^2/(\pi AR) = 0{,}0152$ (tolta la resistenza indotta ellittica, 0,0078), contro la resistenza di profilo $2C_f(1 + 2t/c + 60(t/c)^4) = 0{,}00904$ della formula di Hoerner: attribuendo tutto l'eccesso alla resistenza indotta si avrebbe un fattore di Oswald $e = 0{,}56$, mentre la linea portante dà per un'ala rettangolare di $AR = 4$ valori superiori a 0,9. Una parte della resistenza di pressione, dell'ordine di 0,0053, è dunque *spuria*: dissipazione numerica nella mesh di fondo (celle di $0{,}06c$ nella scatola della scia, livello 3), che l'integrazione delle pressioni sulla parete non sa distinguere dalla resistenza fisica. È il limite noto del calcolo della resistenza in "campo vicino"; i codici industriali la decompongono in "campo lontano" (piano di Trefftz) proprio per separare la parte indotta da quella spuria.

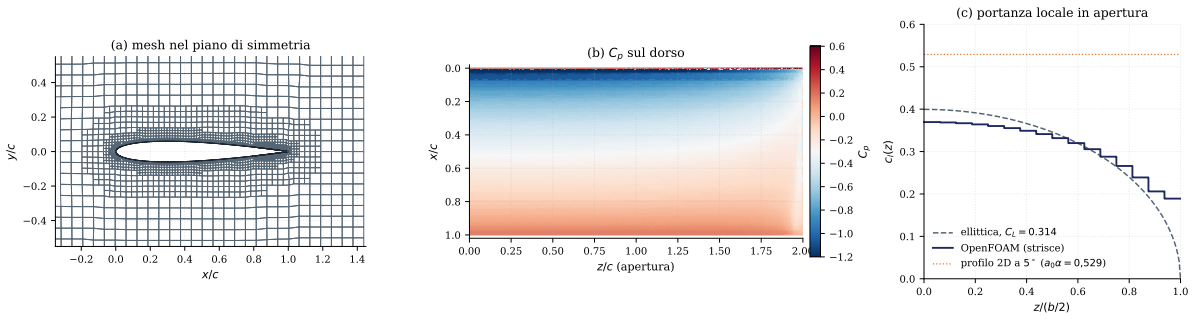


Figura 15.1: Semiala NACA 0012, $AR = 4$, $\alpha = 5°$: la mesh di `snappyHexMesh` nel piano di simmetria e sulla superficie (a sinistra); coefficiente di pressione sul dorso dell'ala in pianta, dal bordo d'attacco in alto (al centro); distribuzione in apertura del coefficiente di portanza locale, calcolata per strisce dalle pressioni con `foamcase.py`, confrontata con la distribuzione ellittica di pari portanza totale e con la portanza del profilo 2D alla stessa incidenza (a destra): l'ala finita perde portanza su tutta l'apertura, non solo all'estremità, perché l'incidenza indotta riduce l'incidenza effettiva di ogni sezione.

Che cosa guardare in un calcolo 3D

Nell'ordine: il registro di `snappyHexMesh` (numero di celle per fase, copertura degli strati), `checkMesh` (non ortogonalità massima e numero di facce oltre 65°, skewness, celle con determinante piccolo), lo y^+ a calcolo finito (`yPlus` scrive minimo, massimo e media per ogni parete nel registro: la funzione di parete di Spalding tollera tutto l'intervallo ma la risoluzione della transizione e della separazione no), la storia dei coefficienti (convergenza alle grandezze integrali: in 3D con mesh non strutturate e geometrie complesse i residui spesso si fermano a 10^{-4}–10^{-5} e il criterio sono le forze, Cap. 6; qui, con la geometria semplice e la mesh regolare, scendono a 10^{-7}), e infine i campi in ParaView: la pressione sulla superficie, le linee di corrente a parete (`wallShearStress`) che rivelano separazioni, la vorticità nella scia con il vortice d'estremità. I numeri da confrontare con la teoria sono pochi e robusti (portanza totale, distribuzione in apertura) e i dettagli dipendono dalla mesh molto più che in 2D: la convergenza di griglia costa un fattore 8 di celle per livello, e nelle applicazioni si fa quasi sempre sul caso 2D della sezione.

15.3 Approfondimenti

15.3.1 Mesh ibride e `snappyHexMesh` su geometrie complesse

Su una fusoliera completa, un'ala con ipersostentatori o un elicottero la stessa procedura funziona con più superfici (una per componente, ciascuna con il suo `level` e il suo patch), più regioni di raffinamento (`searchableCylinder` per i vortici d'estremità, `distance` con più soglie), e `nSurfaceLayers` differenziati; i punti critici sono gli spigoli concavi fra componenti (gli strati collassano), le superfici sottili (bordi d'uscita: servono almeno due celle di livello massimo nello spessore, o il bordo viene arrotondato) e i piccoli dettagli geometrici (vanno rimossi dalla CAD, non risolti). I casi `motorBike` e `drivaerFastback` dei tutorial della distribuzione sono i modelli da studiare; `foamyHexMesh` e i generatori esterni (`cfMesh`, `gmsh`, Pointwise) con `gmshToFoam`, `fluentMeshToFoam`, `ideasUnvToFoam` sono le alternative.

15.3.2 Verso i calcoli industriali

Quattro abitudini distinguono un calcolo riproducibile: lo script `Allrun` che rifà tutto da zero (mesh compresa), i parametri in un unico file incluso (`#include "parametri"` con incidenza, velocità, numero di processori), le *function object* per tutte le grandezze d'interesse (nessun post-processing a mano sui campi), e i casi piccoli di verifica accanto al caso grande (la sezione 2D, Cap. 13; la lastra, Cap. 9). Il Cap. 16 le raccoglie in una lista di controllo. Per il calcolo su cluster, `foamJob` e gli script di coda (`sbatch`) lanciano `mpirun` con `-hostfile`; `decomposePar -cellDist` scrive la mappa dei sottodomini per controllare il bilanciamento; `writeFormat binary` e `purgeWrite` contengono lo spazio su disco; il formato collated (`fileHandler collated`) riunisce i file dei processori.

Capitolo 16

Verifica, validazione e buone pratiche: dal problema al caso

16.1 Tre domande distinte

Un risultato CFD è credibile quando si è risposto a tre domande, nell'ordine. *Verifica del codice*: il programma risolve correttamente le equazioni che dice di risolvere? Lo si controlla su problemi con soluzione esatta e misurando l'ordine di convergenza (i Cap. 4, 6 e 10 lo hanno fatto: ordine 2 nello spazio e nel tempo dove la soluzione è liscia, ordine 1 con le discontinuità). *Verifica della soluzione*: su *questa* mesh, con *questo* passo e *questo* livello di convergenza iterativa, quanto è lontano il risultato dalla soluzione esatta delle equazioni discretizzate? È l'incertezza numerica, e si stima con lo studio di convergenza di griglia (il GCI del Cap. 13) e con il controllo della convergenza iterativa sulle grandezze integrali. *Validazione*: le equazioni e i modelli (turbolenza, condizioni al contorno, ipotesi 2D) descrivono la fisica? Si risponde solo confrontando con esperimenti di qualità nota (Ghia, Driver–Seegmiller, Ladson, Williamson in questa monografia), e la risposta vale per quella classe di flussi. Confondere le tre domande è l'errore più frequente: un calcolo che "torna" con l'esperimento su una mesh grossa può farlo per compensazione di errori (diffusione numerica che imita la turbolenza), e un calcolo verificato con cura può essere lontano dall'esperimento perché il modello RANS non descrive il fenomeno (il k–ε sul gradino).

L'indice di convergenza di griglia (GCI)

Con tre mesh di passo rappresentativo $h_1 < h_2 < h_3$ (rapporto di raffinamento $r = h_2/h_1$, consigliato $r \geq 1{,}3$) e i valori f_1, f_2, f_3 di una grandezza integrale, l'ordine osservato p si ricava da

$$p = \frac{1}{\ln r_{21}} \left| \ln \left| \frac{\varepsilon_{32}}{\varepsilon_{21}} \right| + q(p) \right|, \qquad q(p) = \ln \frac{r_{21}^{p} - s}{r_{32}^{p} - s}, \quad s = \operatorname{sgn} \frac{\varepsilon_{32}}{\varepsilon_{21}}, \tag{16.1}$$

con $\varepsilon_{21} = f_2 - f_1$, $\varepsilon_{32} = f_3 - f_2$ (iterazione a punto fisso, $q = 0$ se $r_{21} = r_{32}$); il valore estrapolato alla mesh infinitamente fine e l'indice di convergenza della mesh fine sono

$$f_{\text{ext}} = \frac{r_{21}^{p} f_1 - f_2}{r_{21}^{p} - 1}, \qquad \text{GCI}_{\text{fine}} = \frac{1{,}25\,|e_a|}{r_{21}^{p} - 1}, \quad e_a = \left| \frac{f_1 - f_2}{f_1} \right|, \tag{16.2}$$

dove 1,25 è il fattore di sicurezza di Roache per tre mesh. Il GCI è una stima dell'incertezza numerica (una banda, non un errore noto), valida se le tre soluzioni sono nell'*intervallo*

asintotico: lo si controlla con $\mathrm{GCI}_{23}/(r^p\,\mathrm{GCI}_{12}) \approx 1$ e con un ordine osservato vicino a quello formale (fra 1 e 2 per i nostri schemi; valori molto più alti o oscillanti dicono che le mesh sono troppo grosse o che qualcos'altro è cambiato fra una mesh e l'altra: y^+, funzioni di parete, convergenza iterativa). Lo script `foamgci.py` implementa la procedura; nel Cap. 13 è applicata a C_L e C_D del profilo. Il passo rappresentativo di una mesh non strutturata è $h = (\sum V_i/N)^{1/d}$; con mesh ottenute per raffinamento uniforme basta il rapporto dei numeri di celle elevato a $1/d$.

Convergenza iterativa: quando fermarsi

I residui normalizzati di OpenFOAM (Cap. 6) misurano quanto le equazioni discretizzate sono soddisfatte *relativamente* al campo corrente: sono il primo indicatore, non l'unico. Il criterio completo per un calcolo stazionario è: residui iniziali sotto 10^{-5}–10^{-6} *oppure* fermi da molte iterazioni; grandezze integrali (C_L, C_D, portate, temperature medie) stabili alla cifra richiesta sulle ultime centinaia di iterazioni (`forceCoeffs`, `patchAverage`, `flowRatePatch` in `functions`); bilanci globali chiusi (la portata entrante uguale a quella uscente: la riga `time step continuity errors` e la *function object* `flowRatePatch` sui patch d'ingresso e d'uscita); nessuna deriva lenta dei campi fra due scritture successive. I casi del Cap. 9 (gradino) e 13 (profilo) mostrano residui fermi a 10^{-4} con forze stabili alla quarta cifra: calcoli convergenti; un residuo che scende a 10^{-8} con una forza che deriva è il contrario. Nei transitori il criterio è il numero di Courante, la convergenza dei correttori PISO/PIMPLE dentro il passo (il residuo finale di `pFinal`) e l'indipendenza dal passo temporale (ripetere con $\Delta t/2$).

16.2 Il catalogo delle *function object*

Le *function object* sono il modo giusto di estrarre risultati: calcolano durante il calcolo (anche in parallelo, anche a ogni passo) e scrivono file di testo pronti per Python. Si dichiarano in `functions` di `controlDict`, per esteso o con `#includeFunc nome(argomenti)` che carica il modello da `etc/caseDicts/functions/`; `foamPostProcess -func nome` le applica a posteriori ai campi scritti; `foamPostProcess -list` elenca quelle disponibili.

Le *function object* più utili per l'aerodinamica

Forze

`forces` (forze e momenti in N e N m, con la ripartizione pressione/viscosa) e `forceCoeffs` (coefficienti: servono `liftDir`, `dragDir`, `pitchAxis`, `CofR`, `magUInf`, `lRef`, `Aref`, `rhoInf` per i casi incomprimibili con `rho rhoInf`); `binField` per la distribuzione lungo una direzione (la portanza in apertura).

Parete

`wallShearStress`, `yPlus` (minimo, massimo, media nel registro e campo), `wallHeatFlux`, `wallHeatTransferCoeff`.

Campi derivati

`vorticity`, `Q`, `Lambda2`, `enstrophy`, `MachNo`, `CourantNo`, `PecletNo`, `grad(p)`, `div(phi)`, `mag(U)`, `components(U)`, `turbulenceFields` (il tensore degli sforzi di Reynolds, k, ε dal modello), `streamFunction` (2D), `age`.

Statistiche e sonde

`fieldAverage` (medie temporali e varianze: `UMean`, `UPrime2Mean`), `probes` (segnali puntuali: posizioni in `probeLocations`), `fieldMinMax`, `volFieldValue` e `surfaceFieldValue` (integrali e medie su volumi, patch, piani: portata con `flowRatePatch`, pressione media con `patchAverage`), `residuals` (i residui iniziali in un file, pronti per il grafico).

Campionamento

`sets` o `graphUniform` (profili lungo rette, con i tipi `uniform`, `face`, `midPoint`, `cloud`), `surfaces` e `cutPlaneSurface` (sezioni e isosuperfici in formato VTK), `streamlines`, `writeCellCentres`, `writeVTK`.

Controllo

`writeObjects` (scrive campi aggiuntivi), `timeActivatedFileUpdate` (cambia un dizionario a un istante dato: per passare da `upwind` a `linearUpwind` dopo n iterazioni senza fermare il calcolo), `systemCall`, `abort` (ferma il calcolo se compare un file), `stopAt`.

Trasporto

`scalarTransport` (uno scalare passivo con i campi correnti: tracciante, età dell'aria, Cap. 5), `comfort`, `particles`.

Una *function object* `coded`

Quando manca quella giusta, si scrive nel `controlDict` un blocco C++ compilato alla prima esecuzione, con lo stesso accesso ai campi di un solutore (Cap. 12). L'esempio calcola, a ogni istante di scrittura, l'energia cinetica totale del dominio e la scrive nel registro:

```
energiaCinetica
{
    type            coded;
    libs            ("libutilityFunctionObjects.so");
    name            energiaCinetica;
    writeControl    writeTime;
    codeWrite
    #{
        const volVectorField& U = mesh().lookupObject<volVectorField>("U");
        const scalar Ek = 0.5*gSum(mesh().V()*magSqr(U.primitiveField()));
    // gSum: somma su tutti i processori
        Info<< "energia cinetica = " << Ek << " m^5/s^2" << endl;
    #};
}
```

`mesh()` dà accesso alla mesh (e, per `lookupObject`, a tutti i campi registrati), `time()` al tempo; `codeExecute` è eseguito a ogni passo, `codeWrite` agli istanti di scrittura; i risultati si scrivono con un `OFstream` nella cartella `postProcessing/` come fanno le *function object* della distribuzione.

16.3 Dal problema al caso: una lista di controllo

Impostare un calcolo nuovo

1. **Il problema.** Quali grandezze servono e con quale precisione (un C_L al 2% o una separazione sì/no?); quali numeri adimensionali (Re, Ma, Strouhal attesi); stazionario o no; 2D, assialsimmetrico o 3D; quali dati di confronto esistono.
2. **Il modello.** Incomprimibile sotto $Ma \approx 0{,}3$ (`incompressibleFluid`), altrimenti `fluid` o `shockFluid`; laminare solo se Re lo giustifica; RANS con k–ω SST come scelta predefinita, Spalart–Allmaras per l'aerodinamica esterna attaccata, k–ε per flussi interni senza separazioni; transizione (`kOmegaSSTLM`) se $Re_c < 10^6$ e la posizione della transizione conta.
3. **Il dominio.** Contorni lontani: almeno 50 corde per i profili portanti se interessa la resistenza (a $20c$ l'errore di dominio supera quello di griglia: Cap. 13), o `freestream` con correzione di vortice; 10–20 lunghezze a valle dei corpi tozzi, 5–10 diametri attorno; piani di simmetria solo se la soluzione è simmetrica (non per le scie instazionarie); uscite dove il flusso è parallelo.
4. **La mesh.** Prima cella da y^+ obiettivo ($y^+ < 1$ risolta, 30–100 con funzioni di parete; mai 5–30); espansione $\leq 1{,}2$ negli strati; celle allineate con il flusso dove possibile; raffinamento nelle zone di gradiente (bordo d'attacco, bordo d'uscita, scia, urti); non ortogonalità $< 65°$, skewness < 4, transizioni di dimensione graduali; `checkMesh` sempre. Tre mesh (rapporto $\geq 1{,}3$) per il GCI delle grandezze principali.
5. **Le condizioni al contorno.** Una tabella con, per ogni patch e ogni campo, tipo e valore; i valori turbolenti d'ingresso da I e ℓ (Cap. 9); `inletOutlet` alle uscite; `value` dove richiesto; la pressione fissata in un solo punto o patch.
6. **Gli schemi.** Stazionario: `steadyState`, `bounded Gauss linearUpwind grad(U)` per **U**, `bounded Gauss limitedLinear 1` per gli scalari turbolenti, `Gauss linear corrected` (o `limited corrected 0.33` su mesh difficili). Transitorio: `backward`, `Gauss linearUpwind` o `LUST`, Co < 1. Prima di ogni risultato finale, almeno un confronto `upwind`/ordine superiore per misurare la diffusione numerica.
7. **L'algoritmo.** SIMPLE con $\alpha_U = 0{,}7$, $\alpha_p = 0{,}3$ (o SIMPLEC 0,9/1) e `residualControl`; PISO/PIMPLE per i transitori; GAMG per p, `smoothSolver` per il resto, `relTol 0.1/0.01`; inizializzazione con `potentialFoam` (controllando che il campo ottenuto sia sensato: con i contorni di corrente libera serve il file `Phi`) e avvio con `upwind` e rilassamento forte per i casi difficili (Cap. 13).
8. **Le *function object*.** Forze, portate, sonde, y^+ e residui dichiarati *prima* di lanciare: si decide la convergenza su di essi.
9. **L'esecuzione.** Script `Allrun` riproducibile; un caso piccolo di prova prima del grande; in parallelo con 30–50 mila celle per processore; registro salvato e letto con `foamlog.py` o `foamLog`.
10. **La verifica e la validazione.** Convergenza iterativa sulle grandezze integrali; GCI; confronto con i dati; una frase onesta sull'incertezza accanto a ogni numero riportato.

Gli errori che costano di più

In ordine di frequenza: lo y^+ mai controllato (funzioni di parete nella zona di raccordo, o mesh "risolta" con $y^+ = 5$); la convergenza giudicata solo dai residui, in un verso o nell'altro; l'uscita troppo vicina o un ricircolo all'uscita con `zeroGradient`; lo schema `upwind` lasciato nel risultato finale; il confronto con l'esperimento senza sapere l'incertezza di quest'ultimo (i dati di Ladson al 2% su C_L e al 5–10% su C_D); il GCI fatto cambiando anche altro fra le mesh; la mesh di `snappyHexMesh` con gli strati al 50% di copertura; il calcolo 2D di un fenomeno 3D (la scia oltre $Re \approx 190$); un risultato riportato con sei cifre quando l'incertezza è al secondo decimale.

16.4 Approfondimenti

16.4.1 Incertezza numerica e incertezza del modello

Il GCI stima l'incertezza di discretizzazione; quella iterativa si stima dalla variazione della grandezza nelle ultime n iterazioni; quella del modello non si stima dal calcolo, ma dalla letteratura di validazione del modello sulla classe di flussi in esame (i rapporti della NASA Turbulence Modeling Resource, i workshop AIAA sulla resistenza e sull'ipersostentazione) o ripetendo il calcolo con due modelli diversi e prendendo la differenza come banda minima. Le tre si combinano in quadratura. La norma ASME V&V 20 e le linee guida ERCOFTAC danno la procedura completa; per il corso basta la disciplina di riportare sempre il GCI delle grandezze principali e il modello usato.

16.4.2 Automazione e analisi parametrica

Gli script `genera.py` dei casi di questa monografia sono il modello per le analisi parametriche: un ciclo Python che modifica i dizionari con `foamDictionary` (o con sostituzioni in file modello), lancia le applicazioni con `subprocess`, legge i risultati con `foamcase.py` e `foamlog.py` e li raccoglie in un file JSON; le figure sono prodotte da uno script separato che legge solo il JSON. Lo stesso schema, con `#include` di un file di parametri nei dizionari e `#calc` per le quantità derivate, serve per le polari, gli studi di sensibilità e l'ottimizzazione; PyFoam e le interfacce come `foam-extend/pyvista` sono alternative più strutturate, ma le poche funzioni di `foamcase.py` bastano a tutto ciò che il corso richiede e mostrano come i file sono fatti.

Appendice A

Tabelle di riferimento, strumenti e istruzioni

A.1 I dizionari in una pagina

Tabella A.1: Le voci di `system/controlDict`.

voce	significato e valori tipici
`application`, `solver`	`foamRun` e il modulo: `incompressibleFluid`, `fluid`, `shockFluid`, `functions`, `multiphaseEuler`... (OpenFOAM 12); nelle versioni precedenti il nome del solutore (`simpleFoam`).
`startFrom`, `startTime`	`startTime` (da `startTime`), `latestTime` (riavvio dall'ultima cartella), `firstTime`.
`stopAt`, `endTime`	`endTime`, `writeNow`, `noWriteNow`, `nextWrite`; il tempo finale o il numero di iterazioni.
`deltaT`	passo temporale (1 nei calcoli stazionari).
`adjustTimeStep`, `maxCo`, `maxDeltaT`	passo adattato al numero di Courante massimo.
`writeControl`, `writeInterval`	`timeStep` (ogni n passi), `runTime` (ogni t secondi), `adjustableRunTime` (idem con passo adattato), `cpuTime`, `clockTime`.
`purgeWrite`	numero di cartelle temporali conservate (0 = tutte).
`writeFormat`, `writePrecision`, `writeCompression`	`ascii`/`binary`, cifre significative, `on`/`off`.
`timeFormat`, `timePrecision`	nomi delle cartelle temporali (`general`, `fixed`, `scientific`).
`runTimeModifiable`	`true`: i dizionari sono riletti se modificati durante il calcolo.
`libs`	librerie da caricare: `("libprofiloParabolico.so")`.
`functions`	le *function object* (Cap. 16).

A.2 Le due linee di sviluppo: corrispondenze

OpenFOAM esiste in due distribuzioni con lo stesso nucleo e sintassi leggermente diverse: la Foundation (`https://openfoam.org`, versioni numerate: 10, 11, 12) e ESI (`https://`

Tabella A.2: Le voci di `system/fvSchemes` (Cap. 4–6).

blocco	valori
`ddtSchemes`	`steadyState`, `Euler`, `backward`, `CrankNicolson` ψ, `localEuler` (pseudo-transitorio locale), `bounded Euler`.
`gradSchemes`	`Gauss linear`, `leastSquares`, `Gauss pointLinear`, `fourth`; limitati: `cellLimited <schema> 1`, `cellMDLimited`, `faceLimited`, `faceMDLimited`.
`divSchemes`	`Gauss upwind`, `linear`, `linearUpwind grad(U)`, `LUST grad(U)`, `QUICK`, `limitedLinear` ψ, `vanLeer`, `Minmod`, `SuperBee`, `MUSCL`, `vanAlbada`, `limitedLinearV`, `limitedLinear01`, `vanLeer01`; prefisso `bounded` nei calcoli stazionari.
`laplacianSchemes`	`Gauss <interp> <snGrad>`: `linear/harmonic` e `orthogonal`, `uncorrected`, `corrected`, `limited corrected` ψ (sinonimo `limited` ψ).
`interpolation-Schemes`	`linear` (quasi sempre); `reconstruct(...)` per `shockFluid`: `vanLeer`, `vanAlbada`, `Minmod`, con suffisso `V` per i vettori.
`snGradSchemes`	come il secondo argomento del laplaciano.
`wallDist`	`method meshWave` (necessario ai modelli di turbolenza).
`fluxScheme`	`Kurganov` o `Tadmor` (solo `shockFluid`).

`openfoam.com`, versioni v2306, v2312, v2406, ...). I casi di questa monografia sono per OpenFOAM 12; la Tabella A.5 dà le corrispondenze per usarli altrove.

A.3 Glossario delle utility

Mesh

`blockMesh`, `snappyHexMesh`, `extrudeMesh` (da un patch 2D a 3D, o per ottenere un caso 2D/assialsimmetrico da una superficie), `checkMesh`, `renumberMesh` (riordina le celle per ridurre la larghezza di banda: più veloce), `refineMesh`, `mirrorMesh`, `transformPoints` (scala, trasla, ruota: `"scale=(0.001 0.001 0.001)"` dai millimetri ai metri), `topoSet` e `setSet` (insiemi di celle, facce, punti), `createPatch`, `createBaffles`, `mergeMeshes`, `stitchMesh`, `subsetMesh`, `splitMeshRegions`; conversione: `gmshToFoam`, `fluentMeshToFoam`, `ideasUnvToFoam`, `star4ToFoam`, `foamMeshToFluent`.

Superfici

`surfaceCheck`, `surfaceFeatures`, `surfaceConvert`, `surfaceTransformPoints`, `surfaceRefineRedGreen`, `surfaceOrient`, `surfaceClean`.

Pre-processing

`setFields` (campi iniziali per regioni), `mapFields` e `mapFieldsPar` (da una mesh a un'altra), `changeDictionary`, `foamDictionary` (legge e modifica voci: `-entry`, `-set`, `-add`, `-remove`, `-expand`), `potentialFoam`, `foamCloneCase`, `foamCleanCase`, `foamListTimes`.

Parallelo

`decomposePar`, `reconstructPar`, `redistributePar`, `reconstructParMesh`, `mpirun -np N ... -parallel`, `foamJob`.

Tabella A.3: Le voci di `system/fvSolution` (Cap. 7–8).

voce	valori
`solvers/<campo>`	`solver`: `PCG` (simmetrico), `PBiCGStab`, `PBiCG` (asimmetrici), `GAMG`, `smoothSolver`, `diagonal`; `preconditioner`: `DIC`, `DILU`, `GAMG`, `FDIC`, `diagonal`, `none`; `smoother`: `GaussSeidel`, `symGaussSeidel`, `DIC`, `DICGaussSeidel`, `DILU`; `tolerance`, `relTol`, `maxIter`, `minIter`, `nSweeps`; per GAMG `nCellsInCoarsestLevel`, `agglomerator` `faceAreaPair`, `nPreSweeps`, `nPostSweeps`, `cacheAgglomeration`.
`<campo>Final`	il solutore dell'ultimo correttore PISO/PIMPLE (di solito `relTol 0`).
`SIMPLE`	`nNonOrthogonalCorrectors`, `consistent` (SIMPLEC), `residualControl`, `pRefCell/pRefValue` (o `pRefPoint`).
`PIMPLE/PISO`	`nOuterCorrectors`, `nCorrectors`, `nNonOrthogonalCorrectors`, `momentumPredictor`, `outerCorrectorResidualControl`, `turbOnFinalIterOnly`.
`relaxationFactors`	`fields { p 0.3; }` (esplicito), `equations { U 0.7; }` (implicito); `".*"` per tutti; `UFinal 1`.
`potentialFlow`	`nNonOrthogonalCorrectors`, `PhiRefCell/PhiRefValue` per `potentialFoam` (necessari se Φ è di Neumann ovunque: Cap. 13).

Post-processing

`foamPostProcess` (`-func`, `-funcs`, `-list`, `-latestTime`, `-time`), `foamToVTK`, `foamToEnsight`, `paraFoam` (apre il caso in ParaView; equivalente a creare un file `caso.foam` vuoto e aprirlo), `foamLog` (estrae i residui dal registro in file), `foamMonitor`.

Informazioni

`foamInfo <parola>` (documentazione di una classe e i tutorial che la usano), `foamSearch <cartella> <file> <voce>` (i valori di una voce in tutti i casi: `foamSearch $FOAM_TUTORIALS fvSchemes divSchemes/div(phi,U)`), `foamGet` (copia un file modello da `etc/caseDicts`), `foamNew`, `foamNewBC`, `foamNewApp`, `foamNewFunctionObject`, `foamUnits`.

A.4 Gli strumenti Python

Gli strumenti stanno in `codice/` e si usano con `import foamcase` dalla stessa cartella (o aggiungendo la cartella a `sys.path`, come fanno gli script `genera.py` dei casi). Dipendono solo da NumPy, SciPy e Matplotlib. Qui sono riportati per intero i tre principali; gli script delle figure (`fig01_cavita.py`, ..., `fig17_ala.py`) e `figutil.py` sono in rete.

foamcase.py

```python
"""
foamcase.py -- lettura di un caso OpenFOAM in Python puro (NumPy), nello spirito
di uFVM: la mesh poliedrica (points, faces, owner, neighbour, boundary), i campi
(volScalarField, volVectorField, in formato ASCII, anche compressi .gz) e le
grandezze geometriche che il metodo dei volumi finiti richiede (centri e aree
delle facce, centri e volumi delle celle, vettore d_CF, non ortogonalita').

Uso tipico:
    from foamcase import Caso
    c = Caso("casi/c01_cavita")
    m = c.mesh                        # oggetto Mesh: m.C (centri celle), m.V, m.Cf, m.Sf, m.owner, ...
    p = c.campo("p")                  # ultimo istante; c.campo("U", tempo=0.5)
    y, u = c.linea("U", (0.5, 0, 0.005), (0.5, 1, 0.005), 100)     # campionamento lungo un segmento

Il modulo e' volutamente semplice (nessuna dipendenza oltre NumPy e, per il
campionamento, SciPy): serve a CAPIRE che cosa contengono i file di un caso,
non a sostituire ParaView o le function object di OpenFOAM.
"""
import os
import re
import gzip
import numpy as np


# ============================================================================
# lettura dei file di OpenFOAM (ASCII)
# ============================================================================
def _apri(percorso):
    """Apre un file di OpenFOAM, anche nella versione compressa .gz."""
    if os.path.exists(percorso):
        return open(percorso, encoding="utf-8", errors="replace").read()
    if os.path.exists(percorso + ".gz"):
        return gzip.open(percorso + ".gz", "rt", encoding="utf-8", errors="replace").read()
    raise FileNotFoundError(percorso)


def _senza_commenti(testo):
    testo = re.sub(r"/\*.*?\*/", "", testo, flags=re.S)
    return re.sub(r"//[^\n]*", "", testo)


def _intestazione(testo):
    """Il dizionario FoamFile { ... } in testa a ogni file."""
    m = re.search(r"FoamFile\s*\{(.*?)\}", testo, flags=re.S)
    voci = {}
    if m:
        for riga in m.group(1).split(";"):
            parti = riga.split()
            if len(parti) >= 2:
                voci[parti[0]] = parti[1]
    return voci


def _lista_numeri(testo, inizio):
    """Legge 'N ( ... )' a partire da testo[inizio:]; restituisce (array, posizione dopo la parentesi)."""
    m = re.compile(r"\s*(\d+)\s*\(").match(testo, inizio)
```

```
    n = int(m.group(1))
    i = m.end()
    j = testo.index(")", i) if n == 0 else _chiudi(testo, i - 1)
    corpo = testo[i:j].replace("(", " ").replace(")", " ")
    return np.array(corpo.split(), dtype=float), j + 1, n


def _chiudi(testo, i):
    """Indice della parentesi che chiude quella aperta in testo[i]."""
    livello = 0
    for k in range(i, len(testo)):
        if testo[k] == "(":
            livello += 1
        elif testo[k] == ")":
            livello -= 1
            if livello == 0:
                return k
    raise ValueError("parentesi non chiusa")


def leggi_lista_etichette(percorso):
    """owner, neighbour: lista di interi."""
    t = _senza_commenti(_apri(percorso))
    t = t[t.index("}", t.index("FoamFile")) + 1:]           # dopo l'intestazione
    v, _, _ = _lista_numeri(t, 0)
    return v.astype(int)


def leggi_punti(percorso):
    t = _senza_commenti(_apri(percorso))
    t = t[t.index("}", t.index("FoamFile")) + 1:]
    v, _, n = _lista_numeri(t, 0)
    return v.reshape(n, 3)


def leggi_facce(percorso):
    """faces: lista di liste di etichette di punti (facce con numero di vertici
    variabile)."""
    t = _senza_commenti(_apri(percorso))
    t = t[t.index("}", t.index("FoamFile")) + 1:]
    m = re.compile(r"\s*(\d+)\s*\(").match(t, 0)
    n = int(m.group(1))
    corpo = t[m.end():_chiudi(t, m.end() - 1)]
    facce = [np.array(f.split(), dtype=int) for f in
    re.findall(r"\d+\s*\(([^)]*)\)", corpo)]
    assert len(facce) == n, "numero di facce incoerente"
    return facce


def leggi_boundary(percorso):
    """boundary: lista di dizionari (nome, type, nFaces, startFace, ...)
    nell'ordine del file."""
    t = _senza_commenti(_apri(percorso))
    t = t[t.index("}", t.index("FoamFile")) + 1:]
    patches = []
    for m in re.finditer(r"(\w+)\s*\{([^{}]*)\}", t):
        voci = {"nome": m.group(1)}
        for riga in m.group(2).split(";"):
            parti = riga.split()
            if len(parti) >= 2:
```

```
                voci[parti[0]] = parti[1] if len(parti) == 2 else "
    ".join(parti[1:])
        voci["nFaces"] = int(voci["nFaces"]); voci["startFace"] =
    int(voci["startFace"])
        patches.append(voci)
    return patches


def leggi_campo(percorso, n_celle=None):
    """Campo vol*Field ASCII: restituisce (interno, contorno, dimensioni).
    interno: array (n,) o (n,3); contorno: dizionario patch -> {"type": ...,
    "value": array o None}."""
    t = _senza_commenti(_apri(percorso))
    intest = _intestazione(t)
    classe = intest.get("class", "volScalarField")
    ncomp = 3 if "Vector" in classe else (6 if "SymmTensor" in classe else (9 if
    "Tensor" in classe else 1))
    dim = re.search(r"dimensions\s*\[([^\]]*)\]", t)
    dimensioni = [float(x) for x in dim.group(1).split()] if dim else None
    interno = _valore(t, t.index("internalField"), ncomp, n_celle)
    contorno = {}
    ib = t.find("boundaryField")
    if ib >= 0:
        i = t.index("{", ib) + 1
        corpo = t[i:_chiudi_graffa(t, i - 1)]
        for m in re.finditer(r"([\w\"|().*]+)\s*\{", corpo):
            nome = m.group(1).strip('"')
            j = m.end()
            fine = _chiudi_graffa(corpo, j - 1)
            blocco = corpo[j:fine]
            tipo = re.search(r"type\s+(\w+)", blocco)
            voce = {"type": tipo.group(1) if tipo else None, "value": None}
            mv = re.search(r"(?<![\w])value\s+(uniform|nonuniform)", blocco)   # la
    voce 'value' (non inletValue, freestreamValue...)
            if mv:
                voce["value"] = _valore(blocco, mv.start(), ncomp, None)
            contorno[nome] = voce
    return interno, contorno, dimensioni


def _chiudi_graffa(testo, i):
    livello = 0
    for k in range(i, len(testo)):
        if testo[k] == "{":
            livello += 1
        elif testo[k] == "}":
            livello -= 1
            if livello == 0:
                return k
    raise ValueError("graffa non chiusa")


def _valore(testo, i, ncomp, n):
    """Legge 'uniform x' / 'uniform (x y z)' / 'nonuniform List<...> N (...)' a
    partire da testo[i:]."""
    m = re.compile(r"\w+\s+(uniform|nonuniform)").match(testo, i)
    if m.group(1) == "uniform":
        j = m.end()
        if ncomp == 1:
            v = float(re.compile(r"\s*([-+0-9.eE]+)").match(testo, j).group(1))
```

```
            return np.full(n, v) if n else v
        mm = re.compile(r"\s*\(([^)]*)\)").match(testo, j)
        v = np.array(mm.group(1).split(), dtype=float)
        return np.tile(v, (n, 1)) if n else v
    j = testo.index(">", m.end()) + 1
    v, _, nn = _lista_numeri(testo, j)
    return v.reshape(nn, ncomp) if ncomp > 1 else v


# ============================================================================
# la mesh e le sue grandezze geometriche
# ============================================================================
class Mesh:
    """Mesh poliedrica di OpenFOAM con le grandezze geometriche calcolate come fa il codice:
    centro e area di ogni faccia per decomposizione in triangoli intorno al centro geometrico,
    centro e volume di ogni cella per decomposizione in piramidi con vertice nel centro stimato."""

    def __init__(self, cartella):
        pm = os.path.join(cartella, "constant", "polyMesh")
        self.points = leggi_punti(os.path.join(pm, "points"))
        self.faces = leggi_facce(os.path.join(pm, "faces"))
        self.owner = leggi_lista_etichette(os.path.join(pm, "owner"))
        self.neighbour = leggi_lista_etichette(os.path.join(pm, "neighbour"))
        self.boundary = leggi_boundary(os.path.join(pm, "boundary"))
        self.nFaces = len(self.faces)
        self.nInternalFaces = len(self.neighbour)
        self.nCells = int(self.owner.max()) + 1
        self.nPoints = len(self.points)
        self._geometria()

    # --- facce ---------------------------------------------------------------
    def _geometria(self):
        P = self.points
        Cf = np.zeros((self.nFaces, 3)); Sf = np.zeros((self.nFaces, 3))
        for i, f in enumerate(self.faces):
            pts = P[f]
            if len(f) == 3:
                Cf[i] = pts.mean(axis=0)
                Sf[i] = 0.5 * np.cross(pts[1] - pts[0], pts[2] - pts[0])
                continue
            cg = pts.mean(axis=0)                    # stima del centro
            sumN = np.zeros(3); sumA = 0.0; sumAc = np.zeros(3)
            for k in range(len(f)):
                a, b = pts[k], pts[(k + 1) % len(f)]
                n = 0.5 * np.cross(b - a, cg - a)  # normale del triangolo (a, b, cg)
                area = np.linalg.norm(n)
                sumN += n; sumA += area; sumAc += area * (a + b + cg) / 3
            Cf[i] = sumAc / sumA if sumA > 1e-300 else cg
            Sf[i] = sumN
        self.Cf, self.Sf = Cf, Sf
        self.magSf = np.linalg.norm(Sf, axis=1)
        # --- celle: piramidi faccia-centro ---
        nC = self.nCells
        cEst = np.zeros((nC, 3)); nF = np.zeros(nC)
        for i in range(self.nFaces):
            cEst[self.owner[i]] += Cf[i]; nF[self.owner[i]] += 1
```

```
            if i < self.nInternalFaces:
                cEst[self.neighbour[i]] += Cf[i]; nF[self.neighbour[i]] += 1
        cEst /= nF[:, None]
        V = np.zeros(nC); C = np.zeros((nC, 3))
        for i in range(self.nFaces):
            o = self.owner[i]
            pyr = np.dot(Sf[i], Cf[i] - cEst[o])   # 3 x volume della piramide (con segno)
            V[o] += pyr; C[o] += pyr * (0.75 * Cf[i] + 0.25 * cEst[o])
            if i < self.nInternalFaces:
                nb = self.neighbour[i]
                pyr = -np.dot(Sf[i], Cf[i] - cEst[nb])
                V[nb] += pyr; C[nb] += pyr * (0.75 * Cf[i] + 0.25 * cEst[nb])
        self.C = C / V[:, None]
        self.V = V / 3.0

    # --- grandezze derivate ------------------------------------------------
    def dCF(self):
        """Vettore dal centro della cella owner a quello della neighbour (facce interne)."""
        n = self.nInternalFaces
        return self.C[self.neighbour] - self.C[self.owner[:n]]

    def non_ortogonalita(self):
        """Angolo (gradi) fra S_f e d_CF per ogni faccia interna. checkMesh ne riporta il massimo e la
        'media' calcolata come arccos della media dei coseni (metodo media_non_ortogonalita)."""
        d = self.dCF(); S = self.Sf[:self.nInternalFaces]
        cosang = np.einsum("ij,ij->i", d, S) / (np.linalg.norm(d, axis=1) * np.linalg.norm(S, axis=1))
        self._cos_no = np.clip(cosang, -1, 1)
        return np.degrees(np.arccos(self._cos_no))

    def media_non_ortogonalita(self):
        self.non_ortogonalita()
        return np.degrees(np.arccos(self._cos_no.mean()))

    def skewness(self, contorno=True):
        """Skewness con la definizione di checkMesh. Facce interne: distanza fra il centro della faccia e
        il punto in cui la retta C-F la attraversa, normalizzata con la semi-estensione della faccia nella
        direzione dello scostamento (non meno di 0,2 |d_CF|). Facce di contorno: scostamento fra il centro
        della faccia e il piede della normale dal centro della cella (normalizzazione non meno di 0,4 |d|)."""
        n = self.nInternalFaces
        C_o = self.C[self.owner[:n]]; d = self.dCF(); S = self.Sf[:n]; Cf = self.Cf[:n]
        Cpf = Cf - C_o
        lam = np.einsum("ij,ij->i", S, Cpf) / (np.einsum("ij,ij->i", S, d) + 1e-300)
        sv = Cpf - lam[:, None] * d                        # vettore di scostamento (giace sulla faccia)
        msv = np.linalg.norm(sv, axis=1)
        svHat = sv / (msv[:, None] + 1e-300)
        fd = 0.2 * np.linalg.norm(d, axis=1)
        for i in range(n):
            if msv[i] > 0:
                fd[i] = max(fd[i], np.abs((self.points[self.faces[i]] - Cf[i]) @
```

```
    svHat[i]).max())
        sk = msv / fd
        if not contorno:
            return sk
        idx = np.arange(n, self.nFaces)
        Cpf = self.Cf[idx] - self.C[self.owner[idx]]
        nrm = self.Sf[idx] / (self.magSf[idx][:, None] + 1e-300)
        dd = np.einsum("ij,ij->i", nrm, Cpf)[:, None] * nrm
        sv = Cpf - dd; msv = np.linalg.norm(sv, axis=1); svHat = sv / (msv[:, None]
    + 1e-300)
        fd = 0.4 * np.linalg.norm(dd, axis=1)
        for k, i in enumerate(idx):
            if msv[k] > 0:
                fd[k] = max(fd[k], np.abs((self.points[self.faces[i]] - self.Cf[i])
    @ svHat[k]).max())
        return np.concatenate([sk, msv / fd])

    def patch(self, nome):
        for b in self.boundary:
            if b["nome"] == nome:
                return b
        raise KeyError(nome)

    def facce_patch(self, nome):
        b = self.patch(nome)
        return np.arange(b["startFace"], b["startFace"] + b["nFaces"])

    def piano_xy(self, z=None):
        """Per una mesh 2D (una cella di spessore): indici delle facce 'empty' su
    un piano, per disegnarla."""
        import collections
        facce = []
        for b in self.boundary:
            if b.get("type") == "empty":
                idx = self.facce_patch(b["nome"])
                zf = self.Cf[idx, 2]
                facce.append(idx[np.isclose(zf, zf.min())] if z is None else
    idx[np.isclose(zf, z)])
        return np.concatenate(facce) if facce else np.array([], dtype=int)

    def disegna(self, ax, colore="#566573", lw=0.3):
        """Disegna il reticolo di una mesh 2D (facce empty del piano inferiore) con
    Matplotlib."""
        from matplotlib.collections import LineCollection
        segs = []
        for i in self.piano_xy():
            f = self.faces[i]; pts = self.points[f][:, :2]
            segs.extend([(pts[k], pts[(k + 1) % len(f)]) for k in range(len(f))])
        ax.add_collection(LineCollection(segs, colors=colore, linewidths=lw))
        ax.autoscale()


# ============================================================================
# il caso
# ============================================================================
class Caso:
    def __init__(self, cartella):
        self.cartella = cartella
        self._mesh = None

```

```
    @property
    def mesh(self):
        if self._mesh is None:
            self._mesh = Mesh(self.cartella)
        return self._mesh

    def tempi(self):
        """Le cartelle temporali presenti, ordinate."""
        t = []
        for d in os.listdir(self.cartella):
            try:
                v = float(d)
            except ValueError:
                continue
            if os.path.isdir(os.path.join(self.cartella, d)):
                t.append((v, d))
        return [d for v, d in sorted(t)]

    def campo(self, nome, tempo=None, contorno=False):
        """Campo di cella all'istante indicato (predefinito: l'ultimo)."""
        if tempo is None:
            tempo = self.tempi()[-1]
        elif not isinstance(tempo, str):
            tempo = self._cartella_tempo(tempo)
        interno, bordo, dim = leggi_campo(os.path.join(self.cartella, tempo, nome),
    self.mesh.nCells)
        return (interno, bordo) if contorno else interno

    def _cartella_tempo(self, t):
        for d in self.tempi():
            if np.isclose(float(d), t):
                return d
        raise FileNotFoundError(f"istante {t} assente in {self.cartella}")

    def valore_patch(self, nome_campo, patch, tempo=None):
        """Valori del campo sulle facce di un patch: il 'value' scritto nel file se
    c'e',
        altrimenti (zeroGradient e simili) il valore della cella adiacente."""
        interno, bordo = self.campo(nome_campo, tempo, contorno=True)
        voce = bordo.get(patch)
        idx = self.mesh.facce_patch(patch)
        if voce is not None and voce["value"] is not None and
    np.ndim(voce["value"]) > 0 \
                and len(np.atleast_1d(voce["value"])) == len(idx):
            return np.asarray(voce["value"])
        if voce is not None and voce["value"] is not None:
            v = np.asarray(voce["value"])
            return np.tile(v, (len(idx), 1)) if v.ndim == 1 and interno.ndim == 2
    else np.full((len(idx),) + interno.shape[1:], v)
        if voce is not None and voce["type"] in ("noSlip", "empty", "wedge",
    "symmetryPlane", "symmetry"):
            # noSlip non scrive 'value': la velocita' di parete e' nulla; per i
    vincoli geometrici il valore e' quello della cella
            if voce["type"] == "noSlip":
                return np.zeros((len(idx),) + interno.shape[1:])
        return interno[self.mesh.owner[idx]]

    def linea(self, nome_campo, p0, p1, n=100, tempo=None, metodo="cella"):
        """Campiona un campo lungo il segmento p0-p1: 'cella' = valore della cella
    piu' vicina
```

```
        (come il sample 'cell' di OpenFOAM), 'idw' = media pesata con l'inverso della distanza delle 4 celle piu' vicine."""
        from scipy.spatial import cKDTree
        v = self.campo(nome_campo, tempo)
        t = np.linspace(0, 1, n)[:, None]
        pts = np.asarray(p0, float) + t * (np.asarray(p1, float) - np.asarray(p0, float))
        albero = cKDTree(self.mesh.C)
        if metodo == "cella":
            _, idx = albero.query(pts)
            # ascissa: proiezione del centro della cella scelta sulla retta (cosi' i valori sono quelli esatti delle celle)
            e = (np.asarray(p1, float) - np.asarray(p0, float)); e /= np.linalg.norm(e)
            return (self.mesh.C[idx] - np.asarray(p0, float)) @ e, v[idx]
        d, idx = albero.query(pts, k=4)
        w = 1.0 / np.maximum(d, 1e-12)
        val = (v[idx] * w[..., None]).sum(axis=1) / w.sum(axis=1)[:, None] if v.ndim == 2 else (v[idx] * w).sum(axis=1) / w.sum(axis=1)
        return np.linalg.norm(pts - pts[0], axis=1), val

    # --- forze su un patch (come la function object 'forces', scritta per esteso) ----------
    def forze(self, patch, nu, rho=1.0, tempo=None, nut="nut", p_cinematica=True):
        """Forza di pressione e viscosa [N] esercitate dal fluido su un patch, per un caso incomprimibile
        (come la function object 'forces'):  F_p = sum rho p_f S_f ;  F_v = sum rho (nu + nu_t,f) (dU/dn)_f |S_f|,
        con (dU/dn)_f = (U_P - U_f)/d_n la derivata della velocita' tangenziale nella direzione che va dalla
        parete verso il fluido. Restituisce (F_pressione, F_viscosa) come vettori."""
        m = self.mesh
        idx = m.facce_patch(patch)
        pf = self.valore_patch("p", patch, tempo)
        Uf = self.valore_patch("U", patch, tempo)
        UP = self.campo("U", tempo)[m.owner[idx]]
        try:
            nutf = self.valore_patch(nut, patch, tempo)
        except (FileNotFoundError, KeyError):
            nutf = np.zeros(len(idx))
        Sf = m.Sf[idx]; nf = Sf / m.magSf[idx][:, None]
        dn = np.einsum("ij,ij->i", m.Cf[idx] - m.C[m.owner[idx]], nf)      # distanza normale centro cella-faccia
        gradUn = (UP - Uf) / dn[:, None]                                    # derivata normale di U (verso l'esterno)
        gradUn -= np.einsum("ij,ij->i", gradUn, nf)[:, None] * nf           # componente tangenziale
        fatt = rho if p_cinematica else 1.0
        Fp = (fatt * pf[:, None] * Sf).sum(axis=0)
        Fv = (rho * (nu + nutf)[:, None] * gradUn * m.magSf[idx][:, None]).sum(axis=0)    # forza DEL fluido SULLA parete
        return Fp, Fv

    def sforzo_parete(self, patch, nu, rho=1.0, tempo=None, nut="nut"):
        """Sforzo tangenziale [Pa] su ogni faccia di un patch di parete: tau = rho (nu + nu_t,f) (dU/dn)_tang,
        con la derivata normale del primo ordine (U_P - U_f)/d_n, come in OpenFOAM (wallShearStress).
        Restituisce (centri delle facce, vettore sforzo per faccia, orientato come
```

```
    la velocita' della cella)."""
424         m = self.mesh
425         idx = m.facce_patch(patch)
426         Uf = self.valore_patch("U", patch, tempo); UP = self.campo("U",
    tempo)[m.owner[idx]]
427         try:
428             nutf = self.valore_patch(nut, patch, tempo)
429         except (FileNotFoundError, KeyError):
430             nutf = np.zeros(len(idx))
431         nf = m.Sf[idx] / m.magSf[idx][:, None]
432         dn = np.abs(np.einsum("ij,ij->i", m.Cf[idx] - m.C[m.owner[idx]], nf))
433         dU = UP - Uf; dU -= np.einsum("ij,ij->i", dU, nf)[:, None] * nf
434         tau = rho * (nu + nutf)[:, None] * dU / dn[:, None]
435         return m.Cf[idx], tau
436
437     def yplus(self, patch, nu, tempo=None, nut="nut"):
438         """y+ delle celle adiacenti a una parete: y+ = y_P u_tau / nu, u_tau =
    sqrt((nu + nu_t) |dU/dn|)."""
439         m = self.mesh
440         idx = m.facce_patch(patch)
441         Uf = self.valore_patch("U", patch, tempo); UP = self.campo("U",
    tempo)[m.owner[idx]]
442         try:
443             nutf = self.valore_patch(nut, patch, tempo)
444         except (FileNotFoundError, KeyError):
445             nutf = np.zeros(len(idx))
446         nf = m.Sf[idx] / m.magSf[idx][:, None]
447         dn = np.abs(np.einsum("ij,ij->i", m.Cf[idx] - m.C[m.owner[idx]], nf))
448         dU = UP - Uf; dU -= np.einsum("ij,ij->i", dU, nf)[:, None] * nf
449         utau = np.sqrt((nu + nutf) * np.linalg.norm(dU, axis=1) / dn)
450         return dn * utau / nu
451
452
453 if __name__ == "__main__":
454     import sys
455     c = Caso(sys.argv[1] if len(sys.argv) > 1 else ".")
456     m = c.mesh
457     print(f"caso {c.cartella}: {m.nPoints} punti, {m.nFaces} facce
    ({m.nInternalFaces} interne), {m.nCells} celle")
458     print(f"   volume totale {m.V.sum():.6e}; patch: " + ", ".join(f"{b['nome']}
    ({b['nFaces']}, {b.get('type')})" for b in m.boundary))
459     no = m.non_ortogonalita()
460     print(f"   non ortogonalita' massima {no.max():.2f} gradi, media
    {m.media_non_ortogonalita():.2f}; skewness massima {m.skewness().max():.3f}")
461     print(f"   istanti: {c.tempi()}")
```

Listato A.1: Il lettore di mesh e campi: formati ASCII di `polyMesh/` e dei campi, grandezze geometriche come in OpenFOAM, indici di qualità come `checkMesh`, campionamento, forze e sforzi di parete.

```
1 """
2 foamlog.py -- lettura del log di un solutore OpenFOAM: residui iniziali e finali di
3 ogni equazione, numero di iterazioni del solutore lineare, errori di continuita',
4 numero di Courant, tempo di calcolo; grafico dei residui in scala semilogaritmica.
5
6 Uso:   python3 foamlog.py casi/c01_cavita/log.foamRun [nome_figura]
7        from foamlog import leggi_log;  d = leggi_log("log.foamRun");  d["p"]["t"],
    d["p"]["res0"]
8
9 Il log e' l'unico "strumento" sempre disponibile per giudicare una simulazione:
```

foamlog.py

```
questo modulo fa in Python cio' che fa foamLog (lo script di OpenFOAM) e la
function object 'residuals', e si presta a essere esteso.
"""
import re
import sys
import numpy as np

RE_TEMPO = re.compile(r"^Time = ([-+0-9.eE]+)")
RE_RES = re.compile(r"(\w+):\s+Solving for (\w+), Initial residual = ([-+0-9.eE]+),
    Final residual = ([-+0-9.eE]+), No Iterations (\d+)")
RE_CONT = re.compile(r"time step continuity errors : sum local = ([-+0-9.eE]+),
    global = ([-+0-9.eE]+), cumulative = ([-+0-9.eE]+)")
RE_CO = re.compile(r"Courant Number mean: ([-+0-9.eE]+) max: ([-+0-9.eE]+)")
RE_EXEC = re.compile(r"ExecutionTime = ([-+0-9.eE]+) s")
RE_DT = re.compile(r"^deltaT = ([-+0-9.eE]+)")


def leggi_log(percorso):
    """Restituisce un dizionario: per ogni campo risolto
    {'t','res0','resF','iter','solver'} (array);
    inoltre 'continuita' (t, locale, globale, cumulativo), 'Co' (t, medio, max),
    'exec' (t, s), 'deltaT'."""
    campi = {}
    cont, co, ex, dts = [], [], [], []
    t = None
    visti = set()
    with open(percorso, encoding="utf-8", errors="replace") as fp:
        for riga in fp:
            m = RE_TEMPO.match(riga)
            if m:
                t = float(m.group(1)); visti = set(); continue
            if t is None:
                continue
            m = RE_RES.search(riga)
            if m:
                solv, nome, r0, rf, it = m.groups()
                # in un passo con piu' correttori si tiene il PRIMO residuo
    iniziale (il piu' significativo)
                if nome in visti:
                    continue
                visti.add(nome)
                d = campi.setdefault(nome, {"t": [], "res0": [], "resF": [],
    "iter": [], "solver": solv})
                d["t"].append(t); d["res0"].append(float(r0));
    d["resF"].append(float(rf)); d["iter"].append(int(it))
                continue
            m = RE_CONT.search(riga)
            if m:
                cont.append((t,) + tuple(float(x) for x in m.groups())); continue
            m = RE_CO.search(riga)
            if m:
                co.append((t, float(m.group(1)), float(m.group(2)))); continue
            m = RE_EXEC.search(riga)
            if m:
                ex.append((t, float(m.group(1)))); continue
            m = RE_DT.match(riga)
            if m:
                dts.append((t, float(m.group(1))))
    for d in campi.values():
        for k in ("t", "res0", "resF", "iter"):
```

```
            d[k] = np.array(d[k])
    out = {"campi": campi}
    out["continuita"] = np.array(cont) if cont else None
    out["Co"] = np.array(co) if co else None
    out["exec"] = np.array(ex) if ex else None
    out["deltaT"] = np.array(dts) if dts else None
    return out


def riassunto(d):
    """Stringa con l'ultimo residuo iniziale di ogni campo, il tempo di calcolo e il numero di passi."""
    righe = []
    for nome, c in d["campi"].items():
        righe.append(f"  {nome:10s} {c['solver']:14s} passi {len(c['t']):6d}  residuo iniziale finale {c['res0'][-1]:.2e}"
                     f"  (max {c['res0'].max():.2e})  iterazioni medie {c['iter'].mean():.1f}")
    if d["exec"] is not None:
        righe.append(f"  tempo di calcolo {d['exec'][-1, 1]:.1f} s per {len(d['exec'])} passi")
    if d["continuita"] is not None:
        righe.append(f"  errore di continuita' cumulativo {d['continuita'][-1, 3]:.2e}")
    if d["Co"] is not None:
        righe.append(f"  numero di Courant massimo {d['Co'][:, 2].max():.3f}")
    return "\n".join(righe)


def grafico_residui(d, ax, campi=None, iniziali=True, **kw):
    """Residui iniziali (o finali) in funzione del tempo/iterazione, in scala semilogaritmica."""
    for nome, c in d["campi"].items():
        if campi and nome not in campi:
            continue
        ax.semilogy(c["t"], c["res0"] if iniziali else c["resF"], label=nome, **kw)
    ax.set_xlabel("tempo / iterazione"); ax.set_ylabel("residuo iniziale" if iniziali else "residuo finale")
    ax.legend(fontsize=7)


if __name__ == "__main__":
    import matplotlib.pyplot as plt
    from stile_grafici import salva
    d = leggi_log(sys.argv[1])
    print(riassunto(d))
    fig, ax = plt.subplots(figsize=(5.2, 3.0))
    grafico_residui(d, ax, lw=1.0)
    fig.tight_layout()
    salva(fig, sys.argv[2] if len(sys.argv) > 2 else "residui")
```

Listato A.2: Il lettore del registro: residui iniziali e finali, iterazioni dei solutori, errori di continuità, numero di Courante, tempi di calcolo.


nacamesh.py

```python
"""
nacamesh.py -- genera il blockMeshDict di una mesh a C intorno a un profilo NACA a quattro cifre (sei blocchi, bordi 'spline' sul profilo e 'arc' sul contorno anteriore),
con l'altezza della prima cella a parete e i rapporti di espansione calcolati dai parametri richiesti.

Uso:  python3 nacamesh.py [--sigla 0012] [--caso casi/c15_naca0012] [--y1 2e-5]
                          [--nNorm 80] [--nRear 60] [--nFront 40] [--nWake 60] [--R 20] [--L 20]
Scrive <caso>/system/blockMeshDict. Topologia (meta' superiore; la inferiore e' speculare):

           o2 (0,R) ------------------------ t (1,R) ------- o1 (L,R)
          .    |   blocco A-up               |   blocco W-up      |
        .      |                             |                    |
      o3(-R,0) |  F-up                       |                    |
        .      LE ----- UM ---- (profilo) -- TE ---------------- w1 (L,0)
"""
import argparse
import os
import numpy as np
from scipy.optimize import brentq


# ---------------------------------------------------------------------------- profilo NACA
def naca4(sigla, n=200):
    """Coordinate (x, y) del dorso e del ventre di un NACA a quattro cifre, bordo d'uscita chiuso.
    Spaziatura a coseno (fitta ai bordi). Restituisce due array (n, 2) ordinati dal bordo d'attacco al bordo d'uscita."""
    m, p, t = int(sigla[0]) / 100, int(sigla[1]) / 10, int(sigla[2:]) / 100
    beta = np.linspace(0, np.pi, n)
    x = 0.5 * (1 - np.cos(beta))
    yt = 5 * t * (0.2969 * np.sqrt(x) - 0.1260 * x - 0.3516 * x**2 + 0.2843 * x**3 - 0.1036 * x**4)
    if m == 0:
        yc = np.zeros_like(x); dyc = np.zeros_like(x)
    else:
        yc = np.where(x < p, m / p**2 * (2 * p * x - x**2), m / (1 - p)**2 * (1 - 2 * p + 2 * p * x - x**2))
        dyc = np.where(x < p, 2 * m / p**2 * (p - x), 2 * m / (1 - p)**2 * (p - x))
    th = np.arctan(dyc)
    dorso = np.column_stack([x - yt * np.sin(th), yc + yt * np.cos(th)])
    ventre = np.column_stack([x + yt * np.sin(th), yc - yt * np.cos(th)])
    return dorso, ventre


# ---------------------------------------------------------------------------- gradazioni
def rapporto_da_prima_cella(H, n, y1):
    """Rapporto di espansione geometrico r tale che n celle con la prima alta y1 coprano H;
    restituisce il 'grading' di blockMesh (ultima/prima) = r^(n-1)."""
    if n * y1 >= H:
        return 1.0
    f = lambda r: y1 * (r**n - 1) / (r - 1) - H
    r = brentq(f, 1.0 + 1e-9, 10.0)
    return r ** (n - 1)
```

```
51
52
53  def punto(xy, z):
54      return f"({xy[0]:.7g} {xy[1]:.7g} {z})"
55
56
57  # ------------------------------------------------------------------------
        generazione
58  def scrivi_blockmesh(caso, sigla="0012", R=20.0, L=20.0, xm=0.3, nNorm=80,
        nRear=60, nFront=40, nWake=60,
59                       y1=2e-5, gTE=0.15, gLE=0.08, gWake=None, gUscita=None,
        n_spline=200):
60      dorso, ventre = naca4(sigla, n_spline)
61      # punti notevoli
62      LE = (0.0, 0.0); TE = (1.0, 0.0)
63      iUM = np.argmin(np.abs(dorso[:, 0] - xm)); UM = tuple(dorso[iUM])
64      iLM = np.argmin(np.abs(ventre[:, 0] - xm)); LM = tuple(ventre[iLM])
65      c45 = np.sqrt(0.5)
66      V = {  # nome -> (x, y)
67          "TE": TE, "UM": UM, "LE": LE, "LM": LM,
68          "w1": (L, 0.0), "o1": (L, R), "t": (1.0, R), "o2": (0.0, R), "o3": (-R,
        0.0),
69          "o4": (0.0, -R), "b": (1.0, -R), "o5": (L, -R),
70      }
71      nomi = list(V)                          # indice dei vertici: z = 0 -> 0..11, z =
        1 -> 12..23
72      idx = {n: i for i, n in enumerate(nomi)}
73      def v(n, z=0):
74          return idx[n] + 12 * z
75      # gradazione normale al profilo (dalla parete verso l'esterno): H = R per i
        blocchi anteriori, R per i posteriori
76      gN = rapporto_da_prima_cella(R, nNorm, y1)
77      if gWake is None:
78          gWake = 8.0                         # celle che si allargano verso l'uscita
        nella scia
79      righe = []
80      a = righe.append
81      a("/*--------------------------------*- C++
        -*----------------------------------*\\")
82      a(f"  blockMeshDict generato da nacamesh.py: NACA {sigla}, mesh a C, R = {R} c,
        L = {L} c,")
83      a(f"  {nNorm} celle normali (prima cella {y1:g} c, rapporto {gN:.1f}),
        {nFront}+{nRear} celle per lato, {nWake} nella scia.")
84      a("\\*" + "-"*75 + "*/")
85      a("FoamFile\n{\n    format      ascii;\n    class       dictionary;\n    object
            blockMeshDict;\n}")
86      a("\nconvertToMeters 1;\n")
87      a("vertices\n(")
88      for z in (0, 1):
89          for n in nomi:
90              a(f"    {punto(V[n], z)}    // {v(n, z)} {n}")
91      a(");\n")
92      a("blocks\n(")
93      # (b0 b1 b2 b3 b4 b5 b6 b7): b0->b1 e' la direzione 1 (lungo il profilo),
        b1->b2 la direzione 2 (normale)
94      def hex_(p0, p1, p2, p3, n1, n2, g1, g2, nome, g2b=None):
95          """Blocco con gradazione semplice; se g2b e' dato, la gradazione normale e'
        g2 sullo spigolo p1-p2 e g2b
96          sullo spigolo opposto p0-p3 (edgeGrading: le celle si allargano lungo la
        scia)."""
```

```
        if g2b is None:
            grad = f"simpleGrading ({g1} {g2} 1)"
        else:
            grad = f"edgeGrading ({g1} {g1} {g1} {g1}  {g2b} {g2} {g2} {g2b}  1 1 1 1)"
        return (f"    hex ({v(p0)} {v(p1)} {v(p2)} {v(p3)} {v(p0,1)} {v(p1,1)} {v(p2,1)} {v(p3,1)})"
                f" ({n1} {n2} 1) {grad}    // {nome}")
    # gradazioni lungo il profilo: verso TE (gTE) nel blocco posteriore, verso LE (gLE) in quello anteriore
    # orientamento: direzione 1 x direzione 2 deve puntare verso +z (volumi positivi):
    # sul dorso si percorre il profilo da LE a TE, sul ventre da TE a LE
    a(hex_("UM", "TE", "t", "o2", nRear, nNorm, f"{gTE:g}", f"{gN:.4g}", "A-up: dorso posteriore (celle fitte verso TE)"))
    a(hex_("LE", "UM", "o2", "o3", nFront, nNorm, f"{1/gLE:g}", f"{gN:.4g}", "F-up: dorso anteriore (fitte verso LE)"))
    a(hex_("LM", "LE", "o3", "o4", nFront, nNorm, f"{gLE:g}", f"{gN:.4g}", "F-low: ventre anteriore"))
    a(hex_("TE", "LM", "o4", "b", nRear, nNorm, f"{1/gTE:g}", f"{gN:.4g}", "A-low: ventre posteriore"))
    # scia: sullo spigolo al TE la gradazione normale e' quella del profilo (gN), sullo spigolo d'uscita e' piu' dolce (gUscita):
    # le celle sottili dello strato limite si allargano lungo la scia invece di restare alte y1 fino all'uscita
    if gUscita is None:          # celle della scia alte y1 fino all'uscita (ortogonali ma con rapporto di forma altissimo)
        a(hex_("TE", "w1", "o1", "t", nWake, nNorm, f"{gWake:g}", f"{gN:.4g}", "W-up: scia superiore (celle crescenti verso l'uscita)"))
        a(hex_("w1", "TE", "b", "o5", nWake, nNorm, f"{1/gWake:g}", f"{gN:.4g}", "W-low: scia inferiore"))
    else:
        a(hex_("TE", "w1", "o1", "t", nWake, nNorm, f"{gWake:g}", f"{gUscita:.4g}", "W-up: scia superiore (celle crescenti verso l'uscita)", g2b=f"{gN:.4g}"))
        a(hex_("w1", "TE", "b", "o5", nWake, nNorm, f"{1/gWake:g}", f"{gN:.4g}", "W-low: scia inferiore", g2b=f"{gUscita:.4g}"))
    a(");\n")
    a("edges\n(")
    def spline(p0, p1, pts, z):
        a(f"    spline {v(p0, z)} {v(p1, z)}\n    (")
        for xy in pts:
            a(f"        {punto(xy, z)}")
        a("    )")
    for z in (0, 1):
        spline("TE", "UM", dorso[iUM + 1:-1][::-1], z)          # dal TE verso UM (punti interni)
        spline("UM", "LE", dorso[1:iUM][::-1], z)
        spline("LE", "LM", ventre[1:iLM], z)
        spline("LM", "TE", ventre[iLM + 1:-1], z)
        a(f"    arc {v('o2', z)} {v('o3', z)} ({-R*c45:.7g} {R*c45:.7g} {z})")
        a(f"    arc {v('o3', z)} {v('o4', z)} ({-R*c45:.7g} {-R*c45:.7g} {z})")
    a(");\n")
    a("boundary\n(")
    a("    profilo\n    {\n        type wall;\n        faces\n        (")
    for p0, p1 in (("TE", "UM"), ("UM", "LE"), ("LE", "LM"), ("LM", "TE")):
        a(f"            ({v(p0)} {v(p1)} {v(p1,1)} {v(p0,1)})")
    a("        );\n    }")
    a("    esterno\n    {\n        type patch;\n        faces\n        (")
    for p0, p1 in (("t", "o2"), ("o2", "o3"), ("o3", "o4"), ("o4", "b"), ("o1", "t"), ("b", "o5"), ("w1", "o1"), ("o5", "w1")):
```

```
        a(f"            ({v(p0)} {v(p1)} {v(p1,1)} {v(p0,1)})")
    a("        );\n    }")
    a("    frontEretro\n    {\n        type empty;\n        faces\n        (")
    for blk in (("UM", "TE", "t", "o2"), ("LE", "UM", "o2", "o3"), ("LM", "LE", "o3", "o4"), ("TE", "LM", "o4", "b"),
                ("TE", "w1", "o1", "t"), ("w1", "TE", "b", "o5")):
        a(f"            ({v(blk[0])} {v(blk[3])} {v(blk[2])} {v(blk[1])})")
        a(f"            ({v(blk[0],1)} {v(blk[1],1)} {v(blk[2],1)} {v(blk[3],1)})")
    a("        );\n    }")
    a(");\n\nmergePatchPairs\n(\n);\n\n// ************************************************************************* //")
    os.makedirs(os.path.join(caso, "system"), exist_ok=True)
    percorso = os.path.join(caso, "system", "blockMeshDict")
    with open(percorso, "w", encoding="utf-8") as fp:
        fp.write("\n".join(righe) + "\n")
    n_celle = (2 * nRear + 2 * nFront + 2 * nWake) * nNorm
    print(f"scritto {percorso}: {n_celle} celle, rapporto normale {gN:.1f}, prima cella {y1:g} c")
    return percorso


if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--sigla", default="0012"); ap.add_argument("--caso", default=".")
    ap.add_argument("--R", type=float, default=20.0); ap.add_argument("--L", type=float, default=20.0)
    ap.add_argument("--y1", type=float, default=2e-5); ap.add_argument("--nNorm", type=int, default=80)
    ap.add_argument("--nRear", type=int, default=60); ap.add_argument("--nFront", type=int, default=40)
    ap.add_argument("--nWake", type=int, default=60)
    a = ap.parse_args()
    scrivi_blockmesh(a.caso, a.sigla, a.R, a.L, nNorm=a.nNorm, nRear=a.nRear, nFront=a.nFront, nWake=a.nWake, y1=a.y1)
```

Listato A.3: Il generatore della mesh a C per i profili NACA a quattro cifre: `blockMeshDict` con sei blocchi, spline sul profilo, gradazioni calcolate dalla prima cella.



foamgci.py

```
"""
foamgci.py -- indice di convergenza di griglia (GCI) di Roache secondo la procedura ASME V&V 20 / Celik et al. (2008).

Date tre mesh di dimensione rappresentativa h1 < h2 < h3 (fine, media, grossa) e i tre valori f1, f2, f3 di una
grandezza integrale (C_L, C_D, una caduta di pressione...), si calcola:
  - l'ordine apparente p (iterazione a punto fisso, perche' i rapporti di raffinamento possono differire),
  - il valore estrapolato alla mesh infinitamente fine f_ext (Richardson),
  - l'errore relativo approssimato e_a e quello estrapolato e_ext,
  - il GCI della mesh fine, GCI = Fs |e_a| / (r^p - 1) con fattore di sicurezza Fs = 1,25.
Uso da riga di comando:  python3 foamgci.py h1 h2 h3 f1 f2 f3
"""
import sys
import numpy as np


def gci(h, f, Fs=1.25, iterazioni=50):
```

```
    """h = (h1,h2,h3) con h1<h2<h3, f = (f1,f2,f3) valori corrispondenti.
    Restituisce un dizionario."""
    h1, h2, h3 = h; f1, f2, f3 = f
    r21, r32 = h2 / h1, h3 / h2
    e21, e32 = f2 - f1, f3 - f2
    if e21 == 0 or e32 == 0:
        raise ValueError("due soluzioni coincidono: il GCI non e' definito")
    s = np.sign(e32 / e21)
    if s < 0:
        convergenza = "oscillatoria"
    else:
        convergenza = "monotona"
    # ordine apparente: p = |ln|e32/e21| + q(p)| / ln r21,  q(p) = ln((r21^p -
    s)/(r32^p - s))
    p = abs(np.log(abs(e32 / e21))) / np.log(r21)
    for _ in range(iterazioni):
        q = np.log((r21**p - s) / (r32**p - s))
        p_nuovo = abs(np.log(abs(e32 / e21)) + q) / np.log(r21)
        if abs(p_nuovo - p) < 1e-10:
            break
        p = p_nuovo
    f_ext = (r21**p * f1 - f2) / (r21**p - 1)
    e_a = abs((f1 - f2) / f1)
    e_ext = abs((f_ext - f1) / f_ext)
    GCI_fine = Fs * e_a / (r21**p - 1)
    GCI_media = Fs * abs((f2 - f3) / f2) / (r32**p - 1)
    # il rapporto GCI_media / (r^p GCI_fine) vale 1 nell'intervallo asintotico
    asintotico = GCI_media / (r21**p * GCI_fine)
    return dict(p=float(p), f_ext=float(f_ext), e_a=float(e_a), e_ext=float(e_ext),
    GCI_fine=float(GCI_fine), GCI_media=float(GCI_media),
                r21=float(r21), r32=float(r32), convergenza=convergenza,
    asintotico=float(asintotico))


def h_da_celle(N, dimensioni=2):
    """Dimensione rappresentativa h = (1/N)^(1/d) per una mesh di N celle in d
    dimensioni (dominio di misura unitaria)."""
    return (1.0 / np.asarray(N, float)) ** (1.0 / dimensioni)


if __name__ == "__main__":
    if len(sys.argv) != 7:
        print(__doc__); sys.exit(1)
    v = [float(a) for a in sys.argv[1:]]
    r = gci(v[:3], v[3:])
    for k, val in r.items():
        print(f"{k:12s} {val}")
```

Listato A.4: L'indice di convergenza di griglia secondo Roache (procedura ASME).

A.5 Le letture dai sorgenti

I file di OpenFOAM 12 letti nel testo sono raccolti in `casi/letture/`: `icoFoam.C` e `createFields.H` (Cap. 2), `momentumPredictor.C` e `correctPressure.C` del modulo `incompressibleFluid` (Cap. 8). Altri file che vale la pena leggere, nell'ordine: `src/finiteVolume/finiteVolume/gradSchemes/gaussGrad/gaussGrad.C` (il gradiente di Green–Gauss, dieci righe); `.../laplacianSchemes/`

`gaussLaplacianScheme/gaussLaplacianScheme.C` (la correzione non ortogonale); `.../convectionSchemes/gaussConvectionScheme/gaussConvectionScheme.C` con `.../limitedSchemes/LimitedScheme/LimitedScheme.C` (i pesi degli schemi limitati); `src/finiteVolume/fvMatrices/fvMatrix/fvMatrix.C` (`relax`, `solve`, `H`, `A`); `src/finiteVolume/fields/fvPatchFields/basic/fixedValue/fixedValueFvPatchField.C` e `.../mixed/mixedFvPatchField.C` (i quattro coefficienti); `src/MomentumTransportModels/momentumTransportModels/RAS/kOmegaSST/kOmegaSST.C` e `.../derivedFvPatchFields/wallFunctions/nutWallFunctions/` (la cartella `nutUSpaldingWallFunction/`, con il sorgente della funzione di parete di Spalding); `applications/modules/shockFluid/` (il flusso di Kurganov–Tadmor in `fluxPredictor.C`); `applications/solvers/foamRun/foamRun.C` (il ciclo che chiama i metodi del modulo).

A.6 Come riprodurre i calcoli

Installazione e prova

1. **Linux (Ubuntu 22.04/24.04, anche in WSL2 su Windows).** `sudo sh -c "wget -O - https://dl.openfoam.org/gpg.key > /etc/apt/trusted.gpg.d/openfoam.asc"; sudo add-apt-repository http://dl.openfoam.org/ubuntu; sudo apt update && sudo apt install openfoam12`; poi `source /opt/openfoam12/etc/bashrc` (da aggiungere a `~/.bashrc`). ParaView: `sudo apt install paraview`. Python: `sudo apt install python3-numpy python3-scipy python3-matplotlib`.
2. **macOS.** Il contenitore Docker ufficiale (`https://openfoam.org/download/12-macos/`): lo script `openfoam12-macos` avvia una shell con l'ambiente e la cartella corrente montata; ParaView si installa nativamente e apre i casi con un file `.foam`.
3. **Dai sorgenti** (ciò che è stato fatto per questa monografia): `git clone https://github.com/OpenFOAM/OpenFOAM-12`, `cd OpenFOAM-12 && source etc/bashrc && ./Allwmake -j`; servono `build-essential`, `flex`, `libfl-dev`, `cmake`, `zlib1g-dev`, `libopenmpi-dev`, `libscotch-dev`, `libptscotch-dev`, `paraview-dev` facoltativo; la compilazione completa richiede alcune ore.
4. **Prova.** `mkdir -p $FOAM_RUN && cd $FOAM_RUN`, `cp -r $FOAM_TUTORIALS/incompressibleFluid/cavity/cavity .`, `cd cavity && blockMesh && foamRun`: in pochi secondi compaiono le cartelle temporali. `paraFoam` per vedere il risultato.
5. **I casi della monografia.** Si scarica l'archivio del caso dalla pagina `https://braucci.github.io/CFD/monografia5/casi/` (i QR nel margine dei listati puntano ai singoli file), si entra nella cartella del caso e si esegue `./Allrun`; `./Allclean` riporta la cartella allo stato iniziale. I casi con `genera.py` richiedono Python con NumPy e SciPy e la cartella degli strumenti Python accanto a `casi/` (`codici/` nell'archivio pubblicato, `https://braucci.github.io/CFD/monografia5/codici/`; `codice/` nel progetto LaTeX); i tempi indicativi su

due processori sono: Cap. 1–7 pochi minuti ciascuno, cavità a $Re = 100$ 15 minuti, lastra e gradino 10 minuti, tubo d'urto e cuneo 5 minuti, profilo NACA un'ora (polare e GCI), cilindro 10 minuti, semiala 3D 40 minuti.

6. **Le figure.** `cd codice && python3 figNN_nome.py` dopo aver eseguito il caso corrispondente: ogni script scrive la figura in `figure/` e il riquadro dei risultati in `capitoli/gen_cNN_risultati.tex`, così che la monografia si ricompila con i numeri dei calcoli del lettore.

Se qualcosa non funziona

`command not found`: l'ambiente non è caricato (`source .../etc/bashrc`). `Cannot find file "points"`: manca la mesh (`blockMesh`). `keyword ... is undefined`: una voce mancante in un dizionario, e il messaggio dice quale e in quale file. `Cannot find patchField entry for ...`: un patch della mesh non ha la condizione al contorno nel file del campo. `Floating point exception`: divergenza (Cap. 6: rilassamento, schemi, mesh, condizioni iniziali). `This code should not be executed by someone with administrator rights`: il codice `coded` e `#calc` rifiutano di compilare da `root` (nei contenitori Docker si usa un utente normale). `wmake`: `command not found` o `undefined symbol`: ambiente di compilazione diverso da quello di esecuzione.

Tabella A.4: Le condizioni al contorno più usate (Cap. 11); ϕ sta per un campo qualunque.

tipo	uso e voci richieste
`fixedValue`, `zeroGradient`, `fixedGradient`, `mixed`	le fondamentali: `value`; `gradient`; `refValue`, `refGradient`, `valueFraction`.
`noSlip`, `slip`, `movingWallVelocity`	pareti (**U**).
`inletOutlet`, `outletInlet`	`inletValue`, `value`: uscite con possibile riflusso.
`freestreamVelocity`, `freestreamPressure`, `freestream`	`freestreamValue`: contorni lontani (aerodinamica esterna).
`totalPressure`, `pressureInletOutletVelocity`	`p0`; ingressi e uscite a pressione totale/statica.
`flowRateInletVelocity`	`volumetricFlowRate` o `massFlowRate`.
`uniformFixedValue`	`uniformValue` come `Function1` (`constant`, `table`, `sine`, `halfCosineRamp`, `coded`).
`codedFixedValue`, `codedMixed`	`name`, `code #{ ... #}`, `value`.
`empty`, `wedge`, `symmetryPlane`, `symmetry`, `cyclic`, `processor`	vincoli geometrici (`#includeEtc "caseDicts/setConstraintTypes"`).
`kqRWallFunction`, `epsilonWallFunction`, `omegaWallFunction`, `nutkWallFunction`, `nutUSpaldingWallFunction`, `nutLowReWallFunction`	funzioni di parete per k, ε, ω, ν_t (`value` richiesto).
`turbulentIntensityKineticEnergyInlet`, `turbulentMixingLengthDissipationRateInlet`, `turbulentMixingLengthFrequencyInlet`	valori turbolenti d'ingresso da `intensity` e `mixingLength`.
`calculated`	segnaposto per i campi calcolati dal modello (`nut`, `alphat`).

Tabella A.5: Corrispondenze fra OpenFOAM 12 (Foundation), Foundation ≤ 10 ed ESI.

OpenFOAM 12	Foundation 7–10	ESI v2xxx
`foamRun + solver incompressibleFluid`	`simpleFoam, pimpleFoam, pisoFoam`	idem
`solver fluid`	`rhoSimpleFoam, rhoPimpleFoam`	idem
`solver shockFluid`	`rhoCentralFoam`	idem
`solver functions + scalarTransport`	`scalarTransportFoam`	idem
`foamPostProcess`	`postProcess`	`postProcess`
`physicalProperties` (`viscosityModel constant; nu 1e-5 [m^2/s];`)	`transportProperties` (`transportModel Newtonian; nu [0 2 -1 0 0 0 0] 1e-5;`)	come Foundation ≤ 10
`momentumTransport` (`RAS { model kOmegaSST; }`)	`turbulenceProperties` (`RAS { RASModel kOmegaSST; }`)	`turbulenceProperties`
`physicalProperties` con `thermoType`	`thermophysical-Properties`	`thermophysical-Properties`
`fvModels, fvConstraints`	`fvOptions`	`fvOptions`
`#includeFunc`	idem	idem
`surfaceFeatures`	`surfaceFeatureExtract` (Foundation ≤ 7)	`surfaceFeatureExtract`
`blockMesh` con `#calc`	idem	idem (`#eval` in più)
`runTime.name(), "argList.H"` nei solutori	`runTime.timeName(), "fvCFD.H"`	`runTime.timeName(), "fvCFD.H"`
`dict.lookup<scalar>("x")`	idem	`dict.get<scalar>("x")`
`Function1::New(nome, unità, dict)`	`Function1::New(nome, dict)`	`Function1::New(nome, dict)`
`fieldMapper`	`fvPatchFieldMapper`	`fvPatchFieldMapper`

Appendice B

Esercizi svolti

I venti esercizi seguono l'ordine dei capitoli: i primi tre riguardano la struttura di un caso e la mesh, poi la discretizzazione (gradienti, convezione, tempo, rilassamento, residui, solutori), l'accoppiamento pressione–velocità, la turbolenza, i flussi comprimibili, le condizioni al contorno e il codice, e infine le applicazioni (forze, convergenza di griglia, segnali, parallelo, impostazione di un calcolo). Ciascuno ha testo, impostazione, svolgimento, risultato e commento; quelli che richiedono di eseguire OpenFOAM indicano il caso da modificare, e i numeri riportati sono quelli ottenuti con OpenFOAM 12 sui casi della monografia.

B.1 Esercizio 1 – La cavità a numero di Reynolds diverso

Testo

Modificare il caso `c01_cavita` ($L = 0{,}1$ m, $U = 1$ m/s, 20×20 celle) per portare il numero di Reynolds da 10 a 100 senza cambiare la mesh, scegliendo la viscosità e il passo temporale in modo che il numero di Courante massimo resti sotto 0,5; prevedere il tempo fisico necessario a raggiungere lo stato stazionario e confrontare con il caso `c08_cavita_re100`.

Impostazione. $Re = UL/\nu$ fissa ν; il numero di Courante $\mathrm{Co} = U\Delta t/\Delta x$ fissa Δt con $\Delta x = L/20$; il tempo di stazionarietà è dell'ordine di qualche tempo diffusivo L^2/ν o, meglio, di alcune decine di tempi convettivi L/U, perché a $Re = 100$ il trasporto è convettivo.

Svolgimento. $\nu = UL/Re = 1 \cdot 0{,}1/100 = 10^{-3}$ m^2/s: `foamDictionary -entry nu -set "1e-3 [m^2/s]" constant/physicalProperties`. $\Delta x = 0{,}005$ m, $\Delta t = 0{,}5\,\Delta x/U = 0{,}0025$ s (il tutorial a $Re = 10$ usa 0,005 con $\mathrm{Co}_{\max} \approx 0{,}85$ perché la velocità massima nelle celle è minore di U). Si pone `endTime 2` (venti tempi convettivi) e si osservano i residui: a $t = 1$ s il residuo di U_x è sotto 10^{-5}; a $t = 2$ s sotto 10^{-6}.

Risultato

$\nu = 10^{-3}$ m^2/s, $\Delta t = 0{,}0025$ s, 800 passi per $t = 2$ s. La velocità minima sulla mezzeria è $u_{\min}/U \approx -0{,}17$ sulla mesh 20×20, contro $-0{,}21$ dei dati di Ghia (Cap. 8): la mesh grossa sottostima il ricircolo del 20%, il doppio di quanto faceva a $Re = 10$.

Commento. Al crescere di Re gli strati di taglio si assottigliano e la mesh che bastava a $Re = 10$ non basta più: il numero di celle necessario cresce almeno come $\sqrt{Re}$ in ogni direzione nei flussi laminari.

B.2 Esercizio 2 – La gradazione da una prima cella assegnata

Testo

Un blocco di `blockMesh` deve coprire $H = 20$ corde in direzione normale con $N = 80$ celle e una prima cella alta $y_1 = 2 \times 10^{-5}$ corde. Calcolare il rapporto di espansione fra celle consecutive r, il `simpleGrading` (rapporto ultima/prima) e l'altezza dell'ultima cella; dire che cosa succede a r se si raddoppia N a parità di y_1.

Impostazione. Le altezze formano una progressione geometrica: $y_1(1 + r + \cdots + r^{N-1}) = y_1(r^N - 1)/(r - 1) = H$; l'equazione in r è trascendente e si risolve per bisezione (`rapporto_da_prima_cella` in `nacamesh.py`); il grading è r^{N-1}.

Svolgimento. Con $y_1(r^{80} - 1)/(r - 1) = 20$ si trova $r = 1{,}1617$; grading $= r^{79} = 1{,}39 \times 10^5$; ultima cella $y_1 r^{79} = 2{,}78$ corde. Con $N = 160$: $r = 1{,}072$, grading $6{,}6 \times 10^4$, ultima cella 1,3 corde.

Risultato

$r = 1{,}162$, `simpleGrading (1 1.39e5 1)`, ultima cella $2{,}8c$; raddoppiando N il rapporto scende a 1,07 e l'ultima cella si dimezza.

Commento. Un rapporto di 1,16 è al limite di quanto si accetta negli strati limite (1,1–1,2); l'ultima cella di 2,8 corde al contorno esterno è grossolana ma lì il flusso è uniforme. È il compromesso della mesh *media* del Cap. 13; la mesh *fine* ha $r = 1{,}11$.

B.3 Esercizio 3 – Non ortogonalità e skewness di una faccia

Testo

Due celle adiacenti hanno centroidi $C = (0,0)$ e $F = (0{,}3,\ 1)$; la faccia comune è orizzontale, con vettore d'area $\mathbf{S}_f = (0,\ 0{,}5)$ e centro $\mathbf{x}_f = (0{,}1,\ 0{,}5)$. Calcolare l'angolo di non ortogonalità, il punto f' in cui la retta CF attraversa la faccia e la skewness con la definizione di `checkMesh`; dire se la faccia supera le soglie.

Impostazione. $\theta = \arccos\big(\mathbf{d}_{CF} \cdot \mathbf{S}_f / (|\mathbf{d}_{CF}||\mathbf{S}_f|)\big)$; $f' = C + t\,\mathbf{d}_{CF}$ con t tale che $(f' - \mathbf{x}_f) \cdot \mathbf{S}_f = 0$; skewness $= |\mathbf{x}_f - f'|/(0{,}2|\mathbf{d}_{CF}|)$ per una faccia interna (Cap. 3).

Svolgimento. $\mathbf{d}_{CF} = (0{,}3,\ 1)$, $|\mathbf{d}_{CF}| = 1{,}044$; $\cos\theta = 0{,}5/(1{,}044 \cdot 0{,}5) = 0{,}958$, $\theta = 16{,}7°$. $t = (\mathbf{x}_f \cdot \mathbf{S}_f)/(\mathbf{d}_{CF} \cdot \mathbf{S}_f) = 0{,}25/0{,}5 = 0{,}5$, $f' = (0{,}15,\ 0{,}5)$; $|\mathbf{x}_f - f'| = 0{,}05$; skewness $= 0{,}05/(0{,}2 \cdot 1{,}044) = 0{,}24$.

Risultato

$\theta = 16{,}7°$, $f' = (0{,}15,\ 0{,}5)$, skewness 0,24: faccia accettabile (soglie 70° e 4), con una correzione non ortogonale modesta e un errore di skewness trascurabile.

Commento. La skewness di `checkMesh` normalizza lo scostamento con un quinto della distanza fra i centri (non con la dimensione della faccia): per questo le soglie sono 4 e 20, molto più grandi di 1.

B.4 Esercizio 4 – Il flusso diffusivo con la correzione non ortogonale

Testo

Per la faccia dell'esercizio precedente, con $\Gamma = 1$, $\phi_C = 1$ e un campo il cui gradiente, uniforme, vale $\nabla\phi = (0{,}2,\ 0{,}9)$, calcolare il flusso diffusivo esatto $\Gamma\nabla\phi \cdot \mathbf{S}_f$ e la sua approssimazione con lo schema `corrected` (approccio sovra-rilassato: $\mathbf{E} = \mathbf{S}_f^2/(\mathbf{d}_{CF} \cdot \mathbf{S}_f)\,\mathbf{d}_{CF}$, $\mathbf{T} = \mathbf{S}_f - \mathbf{E}$), con `uncorrected` e con `orthogonal`.

Impostazione. $\phi_F = \phi_C + \nabla\phi \cdot \mathbf{d}_{CF}$; la parte implicita è $\Gamma(\phi_F - \phi_C)|\mathbf{E}|/|\mathbf{d}_{CF}|$, la correzione esplicita $\Gamma(\nabla\phi)_f \cdot \mathbf{T}$; `uncorrected` omette la correzione; `orthogonal` usa $|\mathbf{S}_f|$ al posto di $|\mathbf{E}|$.

Svolgimento. $\phi_F = 1 + 0{,}06 + 0{,}9 = 1{,}96$. $\mathbf{E} = 0{,}25/0{,}5\,(0{,}3,\ 1) = (0{,}15,\ 0{,}5)$, $|\mathbf{E}| = 0{,}522$; $\mathbf{T} = (-0{,}15,\ 0)$. Implicito: $0{,}96 \cdot 0{,}522/1{,}044 = 0{,}48$; correzione: $(0{,}2)(-0{,}15) = -0{,}03$; `corrected` $= 0{,}45$. Esatto: $(0{,}2,\ 0{,}9) \cdot (0,\ 0{,}5) = 0{,}45$. `uncorrected`: 0,48 (errore +7%). `orthogonal`: $0{,}96 \cdot 0{,}5/1{,}044 = 0{,}46$ (errore +2%, ma per caso: con un gradiente diverso sarebbe maggiore).

Risultato

Esatto 0,45; `corrected` 0,45 (esatto per un campo lineare, come deve essere); `uncorrected` 0,48; `orthogonal` 0,46.

Commento. Lo schema corretto riproduce esattamente i campi lineari su qualunque mesh; la parte implicita è però più grande del flusso vero (0,48 contro 0,45) e la correzione la riduce: è il segno dell'approccio sovra-rilassato, che rende la matrice più diagonale e il calcolo più stabile.

B.5 Esercizio 5 – Péclet di cella e scelta dello schema

Testo

Calcolare il numero di Péclet di cella $Pe = U\Delta x/\Gamma$ (a) nella cavità del Cap. 8 ($U = 1$, 128 celle su $L = 1$, $\nu = 0{,}01$) e (b) nel gradino del Cap. 9 lontano dalle pareti ($U = 44$ m/s, $\Delta x = 5$ mm, $\nu_{\text{eff}} = \nu + \nu_t \approx 1{,}5 \times 10^{-5} + 2 \times 10^{-3}$ m^2/s), e dire in quale dei due casi lo schema `linear` è ammissibile.

Impostazione. Lo schema centrato è limitato per $Pe < 2$; sopra, produce oscillazioni (Cap. 5), e serve `upwind` o uno schema limitato.

Svolgimento. (a) $\Delta x = 1/128 = 0{,}0078$, $Pe = 1 \cdot 0{,}0078/0{,}01 = 0{,}78 < 2$: `linear` ammissibile, e infatti nel Cap. 8 su 32×32 ($Pe = 3{,}1$) ha dato oscillazioni leggere ma errore minore di `upwind`. (b) $Pe = 44 \cdot 0{,}005/2 \times 10^{-3} = 110$: `linear` inammissibile; `linearUpwind` o `limitedLinear`.

Risultato

$Pe = 0{,}78$ (cavità, `linear` possibile) e $Pe \approx 110$ (gradino, schema upwind di ordine superiore obbligatorio).

Commento. Nei flussi turbolenti ad alto Re il Péclet di cella è quasi sempre $\gg 2$: lo schema centrato puro si usa solo nelle LES, dove la mesh è fine abbastanza da avere Pe di cella moderato e la dissipazione numerica è vietata.

B.6 Esercizio 6 – Il passo temporale per la scia del cilindro

Testo

Per il cilindro a $Re = 100$ (Cap. 14, $St \approx 0{,}166$, 160 celle sulla circonferenza, $D = U = 1$) scegliere il passo temporale in modo da avere almeno 100 passi per periodo di distacco e verificare il numero di Courant nelle celle a parete; dire se si può usare PISO.

Impostazione. $T = 1/(St\,U/D)$; $\Delta t = T/100$; $\mathrm{Co} = U\Delta t/\Delta x$ con $\Delta x = \pi D/160$ la dimensione azimutale della prima cella (la velocità a parete è però nulla: il Courant effettivo è calcolato con la velocità locale, e il massimo si trova a mezza altezza dello strato limite e nella scia).

Svolgimento. $T = 6{,}0$, $\Delta t = 0{,}060$, $\Delta x = 0{,}0196$, $\mathrm{Co} = 3{,}1$ con $U = 1$: troppo per PISO. Con $\Delta t = 0{,}005$ (il valore del caso) $\mathrm{Co} = 0{,}25$ nominale e 0,42 effettivo, 1200 passi per periodo.

Risultato

$\Delta t = 0{,}06$ darebbe 100 passi per periodo ma $\mathrm{Co} \approx 3$: servirebbe PIMPLE con 2–3 correttori esterni; con $\Delta t = 0{,}005$ e PISO si ha $\mathrm{Co}_{\max} = 0{,}42$ e 1200 passi per periodo, più che sufficienti.

Commento. Il vincolo stringente è il numero di Courant nelle celle piccole a parete, non la risoluzione del periodo: è la ragione per cui nei transitori con strati limite risolti si usa PIMPLE a Co di qualche unità, o il passo locale (`localEuler`) nei calcoli pseudo-transitori.

B.7 Esercizio 7 – Il rilassamento implicito su un'equazione

Testo

L'equazione discretizzata di una cella è $2\phi_P = \phi_W + \phi_E + 1$ con $\phi_W = 1$, $\phi_E = 0$ fissi. Partendo da $\phi_P^{(0)} = 0$, eseguire sei iterazioni con rilassamento implicito $\alpha = 0{,}7$ e confrontare con la soluzione; ripetere con $\alpha = 1$ e con $\alpha = 0{,}3$ e commentare.

Impostazione. Il rilassamento implicito (Cap. 6) scrive $(a_P/\alpha)\phi_P = \sum a_N\phi_N + b + \frac{1-\alpha}{\alpha}a_P\phi_P^{\text{old}}$, con $a_P = 2$, $\sum a_N\phi_N + b = 1 + 0 + 1 = 2$.

Svolgimento. Soluzione: $\phi_P = 1$. Con $\alpha = 0{,}7$: $\phi_P^{(k)} = \left(2 + \frac{0{,}3}{0{,}7}\cdot 2\phi_P^{(k-1)}\right)/(2/0{,}7) = 0{,}7 + 0{,}3\phi_P^{(k-1)}$: 0,7; 0,91; 0,973; 0,992; 0,998; 0,999. Con $\alpha = 1$: $\phi_P = 1$ alla prima iterazione (l'equazione è lineare e i vicini fissi). Con $\alpha = 0{,}3$: $\phi_P^{(k)} = 0{,}3 + 0{,}7\phi_P^{(k-1)}$: 0,3; 0,51; 0,66; 0,76; 0,83; 0,88.

Risultato

L'errore si riduce di un fattore $1 - \alpha$ a ogni iterazione: 0,3 con $\alpha = 0{,}7$ (sei iterazioni per 10^{-3}), 0,7 con $\alpha = 0{,}3$ (venti iterazioni).

Commento. Su un'equazione lineare con vicini fissi il rilassamento serve solo a rallentare; nei sistemi accoppiati e non lineari rallenta *per* stabilizzare, perché i vicini cambiano a ogni iterazione e la pressione risponde alla velocità. Il Cap. 8 ha mostrato che il numero di iterazioni di SIMPLE scala con $1/\alpha_U$ nello stesso modo.

B.8 Esercizio 8 – Il residuo normalizzato di OpenFOAM

Testo

Per il sistema $A\mathbf{x} = \mathbf{b}$ con $A = \left(\begin{smallmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{smallmatrix}\right)$, $\mathbf{b} = (1,\ 0{,}5,\ 1)$, calcolare il residuo normalizzato di OpenFOAM per $\mathbf{x} = (0{,}9,\ 1{,}3,\ 0{,}8)$ e per $\mathbf{x} = (1{,}2,\ 1{,}4,\ 1{,}2)$, e confrontarlo con il residuo assoluto $\|\mathbf{b} - A\mathbf{x}\|_1$.

Impostazione. $r = \sum |b_i - (A\mathbf{x})_i|/n$ con $n = \sum |(A\mathbf{x})_i - (A\bar{\mathbf{x}})_i| + |b_i - (A\bar{\mathbf{x}})_i|$ e $\bar{\mathbf{x}}$ il vettore uniforme con la media di $\mathbf{x}$ (Cap. 6). La soluzione esatta è $(1{,}25,\ 1{,}5,\ 1{,}25)$.

Svolgimento. Primo vettore: $A\mathbf{x} = (0{,}5,\ 0{,}9,\ 0{,}3)$, $\bar{x} = 1$, $A\bar{\mathbf{x}} = (1,\ 0,\ 1)$; $n = |-0{,}5| + |0{,}9| + |-0{,}7| + |0| + |0{,}5| + |0| = 2{,}6$; $\|\mathbf{b} - A\mathbf{x}\|_1 = 0{,}5 + 0{,}4 + 0{,}7 = 1{,}6$; $r = 0{,}615$. Secondo: $A\mathbf{x} = (1,\ 0{,}4,\ 1)$, $\bar{x} = 1{,}267$, $A\bar{\mathbf{x}} = (1{,}267,\ 0,\ 1{,}267)$; $n = 0{,}267 + 0{,}4 + 0{,}267 + 0{,}267 + 0{,}5 + 0{,}267 = 1{,}967$; $\|\mathbf{b} - A\mathbf{x}\|_1 = 0{,}1$; $r = 0{,}051$.

Risultato

$r = 0{,}62$ e $r = 0{,}051$; i residui assoluti sono 1,6 e 0,1. La normalizzazione rende il residuo adimensionale e confrontabile fra campi, ma un campo quasi uniforme (piccolo n) dà residui normalizzati grandi anche se vicino alla soluzione: è il caso della componente U_z nel condotto assialsimmetrico del Cap. 11.

Commento. La trappola opposta è un campo con grandi variazioni (n grande) che dà residui normalizzati piccoli pur con errori locali non trascurabili: il residuo è una misura *relativa*, e le grandezze integrali restano il giudice finale.

B.9 Esercizio 9 – Scegliere il solutore lineare

Testo

Dai dati del Cap. 7 (cavità 128×128: `PCG/DIC` 75 iterazioni per passo, `GAMG` 2, tempi 3,6 e 2,3 s per 100 passi SIMPLE) stimare il numero di iterazioni e i tempi su una mesh 512×512, sapendo che le iterazioni di PCG crescono come $\sqrt{\kappa}$ con $\kappa \propto N^2$ (N celle per lato) e quelle di GAMG restano circa costanti, e che il costo per iterazione è proporzionale al numero di celle.

Impostazione. Celle $\times 16$; PCG: iterazioni $\times 4$, costo $\times 64$; GAMG: iterazioni $\times 1$, costo $\times 16$ (più un sovraccosto per i livelli aggiuntivi, $\sim 20\%$). Il tempo di PCG a 128^2 comprende anche la parte comune (quantità di moto, assemblaggio), che si stima dal tempo di GAMG: ≈ 2 s comune, 1,6 s per p con PCG, 0,3 s con GAMG.

Svolgimento. A 512^2: parte comune $2 \cdot 16 = 32$ s; p con PCG $1{,}6 \cdot 64 = 102$ s (300 iterazioni per passo); con GAMG $0{,}3 \cdot 16 \cdot 1{,}2 \approx 6$ s. Totale: 134 s contro 38 s.

Risultato

A 512×512 il PCG impiega circa 300 iterazioni per passo e il calcolo è 3,5 volte più lento che con GAMG; a 128^2 il rapporto era 1,6.

Commento. Il vantaggio del multigriglia cresce con la mesh, ed è decisivo in 3D (10^6–10^8 celle); GAMG resta la scelta per la pressione in tutti i casi, e PCG serve come riferimento di robustezza quando GAMG fallisce su mesh di pessima qualità.

B.10 Esercizio 10 – Un passo di SIMPLE su tre celle

Testo

Un condotto 1D ha tre celle di lunghezza $\Delta x = 1$, area unitaria, velocità di ingresso $u_{\text{in}} = 1$ imposta sulla faccia sinistra e pressione $p = 0$ sulla faccia destra. Dopo il predittore della quantità di moto le celle hanno $u^* = (1{,}2,\ 0{,}8,\ 1{,}0)$ e coefficienti diagonali $a_P = (2,\ 2,\ 2)$ (senza il termine di pressione). Costruire l'equazione della pressione di SIMPLE (con $\mathbf{H}/a_P = u^*$ e $1/a_P$ interpolati linearmente sulle facce), risolverla e calcolare le velocità corrette; verificare la continuità.

Impostazione. Sulle facce interne f_{12}, f_{23} e sulla faccia d'uscita: $\phi_f^* = (u^*)_f$ (media), $(1/a_P)_f = 0{,}5$; faccia d'ingresso: flusso imposto 1. Equazione: $\sum_f (1/a_P)_f (\partial p/\partial x)_f = \sum_f \phi_f^*$ per cella; $(\partial p/\partial x)_f = (p_F - p_C)/\Delta x$; all'uscita $p = 0$ con distanza $\Delta x/2$; all'ingresso gradiente nullo.

Svolgimento. Flussi previsti: $\phi_{12}^* = 1{,}0$, $\phi_{23}^* = 0{,}9$, $\phi_{\text{out}}^* = 1{,}0$ (zeroGradient su u all'uscita). Divergenze: cella 1: $1{,}0 - 1 = 0$; cella 2: $0{,}9 - 1{,}0 = -0{,}1$; cella 3: $1{,}0 - 0{,}9 = 0{,}1$. Equazioni (coefficiente 0,5 sulle facce interne, $0{,}5/0{,}5 = 1$ all'uscita): cella 1: $0{,}5(p_2 - p_1) = 0$; cella 2: $0{,}5(p_1 - p_2) + 0{,}5(p_3 - p_2) = -0{,}1$; cella 3: $0{,}5(p_2 - p_3) + 1 \cdot (0 - p_3) = 0{,}1$. Da 1: $p_1 = p_2$; da 2: $0{,}5(p_3 - p_2) = -0{,}1 \Rightarrow p_3 = p_2 - 0{,}2$; da 3: $0{,}5 \cdot 0{,}2 - p_3 = 0{,}1 \Rightarrow p_3 = 0$, $p_2 = p_1 = 0{,}2$. Flussi corretti: $\phi_{12} = 1{,}0 - 0{,}5(0{,}2 - 0{,}2) = 1{,}0$; $\phi_{23} = 0{,}9 - 0{,}5(0 - 0{,}2) = 1{,}0$; $\phi_{\text{out}} = 1{,}0 - 1 \cdot (0 - 0) = 1{,}0$. Velocità di cella: $u = u^* - (1/a_P)(\partial p/\partial x)_C$ con il gradiente di Gauss: cella 1: $1{,}2 - 0{,}5 \cdot 0 = 1{,}2$; cella 2: $0{,}8 - 0{,}5\,(p_{23} - p_{12}) = 0{,}8 - 0{,}5(0{,}1 - 0{,}2) = 0{,}85$; cella 3: $1{,}0 - 0{,}5(0 - 0{,}1) = 1{,}05$.

Risultato

$p = (0{,}2,\ 0{,}2,\ 0)$; i flussi di faccia sono tutti uguali a 1 (continuità esatta); le velocità di cella $(1{,}2,\ 0{,}85,\ 1{,}05)$ non sono ancora uniformi: lo diventeranno nelle iterazioni successive, quando la quantità di moto vedrà il gradiente di pressione.

Commento. È il nocciolo della griglia collocata con Rhie–Chow: la continuità è imposta sui *flussi di faccia*, esattamente, a ogni iterazione; le velocità di cella sono ricostruite e convergono più lentamente. Nel codice (`correctPressure.C`, Cap. 8) le tre righe sono `phiHbyA`, `pEqn.solve(),phi = phiHbyA - pEqn.flux()`.

B.11 Esercizio 11 – La prima cella per un dato y^+

Testo

Una lastra piana in aria ($\nu = 2 \times 10^{-7}$ in unità con $U = 1$, cioè $Re_x = 5 \times 10^6$ a $x = 1$) deve avere $y^+ = 1$ nel primo centro cella a $x = 1$. Calcolare lo sforzo di parete con la correlazione di White, la velocità d'attrito, l'altezza della prima cella, e l'altezza che darebbe $y^+ = 30$ per una mesh con funzioni di parete.

Impostazione. $C_f = 0{,}455/\ln^2(0{,}06 Re_x)$, $u_\tau = U\sqrt{C_f/2}$, $y^+ = y\,u_\tau/\nu$ con y la distanza del centro cella (metà altezza).

Svolgimento. $\ln(0{,}06 \cdot 5\times10^6) = 12{,}61$, $C_f = 0{,}455/159 = 2{,}86\times10^{-3}$; $u_\tau = \sqrt{1{,}43\times10^{-3}} = 0{,}0378$; $y(y^+=1) = \nu/u_\tau = 5{,}3\times10^{-6}$, cella alta $1{,}06\times10^{-5}$; $y^+ = 30$: $y = 1{,}6\times10^{-4}$, cella alta $3{,}2\times10^{-4}$.

Risultato

$C_f = 2{,}86\times10^{-3}$, $u_\tau/U = 0{,}038$; prima cella alta $1{,}1\times10^{-5}L$ per $y^+=1$, $3{,}2\times10^{-4}L$ per $y^+=30$: un fattore 30 che vale in tutte le direzioni normali e spiega perché le mesh con funzioni di parete hanno 3–5 volte meno celle negli strati.

Commento. La stima va fatta nel punto di u_τ massimo (vicino al bordo d'attacco, dove C_f è più alto) se si vuole $y^+ < 1$ ovunque; sul profilo del Cap. 13 il massimo di y^+ è al bordo d'attacco, con un rapporto 2–3 rispetto alla media.

B.12 Esercizio 12 – I valori turbolenti d'ingresso

Testo

Il gradino del Cap. 9 ha $U = 44{,}2$ m/s, altezza del canale a monte $8h = 0{,}1016$ m, $\nu = 1{,}5\times10^{-5}$. Con intensità di turbolenza $I = 1\%$ e lunghezza di mescolamento $\ell = 0{,}07\times$ altezza, calcolare k, ε, ω e ν_t/ν all'ingresso, e confrontare con i valori usati nel caso ($k = 0{,}293$, $\omega = 1953$, $\varepsilon = 51{,}5$).

Impostazione. $k = \frac{3}{2}(IU)^2$, $\varepsilon = C_\mu^{3/4}k^{3/2}/\ell$, $\omega = \sqrt{k}/(C_\mu^{1/4}\ell)$, $\nu_t = C_\mu k^2/\varepsilon$ con $C_\mu = 0{,}09$ (Cap. 9).

Svolgimento. $k = 1{,}5\cdot(0{,}442)^2 = 0{,}293$ m^2/s^2; $\ell = 0{,}00711$ m; $\varepsilon = 0{,}1643\cdot0{,}1586/0{,}00711 = 3{,}67$ m^2/s^3; $\omega = 0{,}541/(0{,}548\cdot0{,}00711) = 139$ s^{-1}; $\nu_t = 0{,}09\cdot0{,}0858/3{,}67 = 2{,}1\times10^{-3}$, $\nu_t/\nu = 140$.

Risultato

$k = 0{,}293$ (uguale al caso); $\varepsilon = 3{,}7$ e $\omega = 139$ contro 51,5 e 1953 del caso, che corrispondono a una lunghezza di mescolamento 14 volte più piccola ($\ell \approx 0{,}5$ mm, $\nu_t/\nu \approx 10$): il caso usa i valori di Driver–Seegmiller, misurati, che descrivono una turbolenza di strato limite più che di canale.

Commento. I valori d'ingresso di ε e ω sono incerti di un ordine di grandezza e la soluzione a valle, qui, lo è poco (lo strato limite ricostruisce i suoi valori a parete); dove il risultato dipende sensibilmente da essi (getti, scie di corpi vicini all'ingresso), vanno presi dall'esperimento o da un calcolo precursore.

B.13 Esercizio 13 – L'urto obliquo a Mach 3

Testo

Preparare il caso `c12_cuneo` per $Ma_1 = 3$ e un cuneo di 10°: calcolare l'angolo d'urto, il rapporto di pressione e il Mach a valle (soluzione debole), la velocità da imporre all'ingresso con $T_1 = 300$ K, e stimare il tempo fisico di `endTime` per quattro attraversamenti del dominio (0,46 m).

Impostazione. Relazione θ–β–Ma (Cap. 10) risolta per β fra $\arcsin(1/Ma_1)$ e la soluzione forte; $a_1 = \sqrt{\gamma R T_1}$.

Svolgimento. Per $\theta = 10°$, $Ma_1 = 3$: $\beta = 27{,}4°$; $Ma_{n1} = 3\sin\beta = 1{,}380$; $p_2/p_1 = 1 + \frac{2\gamma}{\gamma+1}(Ma_{n1}^2 - 1) = 2{,}05$; $Ma_{n2} = 0{,}748$, $Ma_2 = Ma_{n2}/\sin(\beta - \theta) = 2{,}50$. $a_1 = \sqrt{1{,}4 \cdot 287 \cdot 300} = 347$ m/s, $U_1 = 1041$ m/s. Un attraversamento: $0{,}46/1041 = 4{,}4 \times 10^{-4}$ s; quattro: $1{,}8 \times 10^{-3}$ s.

Risultato

$\beta = 27{,}4°$, $p_2/p_1 = 2{,}05$, $Ma_2 = 2{,}50$, $U_1 = 1041$ m/s, `endTime 0.0018`; il vertice del cuneo va spostato o il dominio allungato perché l'urto, più inclinato, esce dal contorno superiore più a valle ($x = 0{,}152/\tan 27{,}4° = 0{,}29$ m).

Commento. Il numero di passi, con `maxCo 0.5` e celle di 1,3 mm, è circa $4 \times 10^{-4} \cdot 1041 \cdot 4/(0{,}5 \cdot 0{,}0013) \approx 2600$: lo stesso ordine del caso a $Ma = 2{,}5$, perché il Courante acustico domina.

B.14 Esercizio 14 – Un ingresso pulsante con `codedFixedValue`

Testo

Scrivere la condizione al contorno per l'ingresso del condotto del Cap. 11 che imponga il profilo parabolico con velocità media $U_m(t) = U_0(1 + 0{,}2\sin 2\pi f t)$, $U_0 = 0{,}1$ m/s, $f = 2$ Hz, (a) con `codedFixedValue`, (b) con la classe `profiloParabolico` del Cap. 12 e una `Function1`; dire quale passo temporale e quale schema si userebbero.

Impostazione. (a) nel blocco `code` il tempo è `db().time().value()`; (b) la classe accetta `fattore` come `Function1` `sine` con `amplitude`, `frequency`, `level`.

Svolgimento. (a):

```
code
#{
    const scalar t = db().time().value();
    const scalar Um = 0.1*(1 + 0.2*sin(2*constant::mathematical::pi*2*t));
    const vectorField& Cf = patch().Cf();
    vectorField Uf(Cf.size(), vector::zero);
    forAll(Cf, i) { const scalar r = sqrt(sqr(Cf[i].y()) + sqr(Cf[i].z()));
                    Uf[i] = vector(2*Um*(1 - sqr(r/0.01)), 0, 0); }
    operator==(Uf);
```

```
#};
```

(b): `type profiloParabolico; Um 0.1; R 0.01; fattore sine; fattoreCoeffs { amplitude 0.2; frequency 2; level 1; }`. Transitorio: periodo 0,5 s, $\Delta t = 0{,}005$ s (100 passi per periodo, Co $\approx 0{,}2 \cdot 0{,}005/0{,}002 = 0{,}5$ con celle di 2 mm), PISO, `backward`, `incompressibleFluid`.

Risultato

Le due versioni danno lo stesso campo; la seconda è riutilizzabile e dichiarativa. Il numero di Womersley $R\sqrt{2\pi f/\nu} = 0{,}01\sqrt{12{,}6/10^{-5}} = 11$ dice che il profilo *non* resta parabolico nel condotto: lo strato di Stokes ha spessore $\sqrt{\nu/(\pi f)} = 1{,}3$ mm e la soluzione a valle è quella di Womersley, con il massimo di velocità vicino alla parete in alcune fasi.

Commento. Imporre un profilo parabolico pulsante all'ingresso è una scelta legittima (è ciò che si farebbe con una pompa a pistone) ma non è la soluzione sviluppata: il condotto va reso lungo abbastanza da lasciarla svilupparsi, o si impone direttamente la soluzione di Womersley, che è un buon esercizio per `codedFixedValue`.

B.15 Esercizio 15 – Uno studio parametrico con `foamDictionary`

Testo

Scrivere lo script di shell che esegue il caso `c09_lastra` con i modelli `kOmegaSST`, `kEpsilon`, `SpalartAllmaras` e `realizableKE`, salvando i registri e lo y^+ medio di ciascuno in una tabella, senza modificare i dizionari a mano.

Impostazione. Il modello si cambia con `foamDictionary -entry RAS/model -set <modello>` sul file `constant/momentumTransport`; la *function object* `yPlus` scrive il valore medio nel registro; i risultati si estraggono con `grep`.

```
#!/bin/bash
cd c09_lastra || exit 1
printf "%-18s %8s\n" modello "y+ medio" > tabella.txt
for m in kOmegaSST kEpsilon SpalartAllmaras realizableKE; do
    ./Allclean && blockMesh > /dev/null
    foamDictionary -entry RAS/model -set $m constant/momentumTransport > /dev/null
    foamRun > log.$m 2>&1
    yp=$(grep "patch lastra y+" log.$m | tail -n 1 | awk '{print $NF}')
    printf "%-18s %8s\n" $m $yp >> tabella.txt
done
cat tabella.txt
```

Risultato

Quattro registri e una tabella; con la mesh con funzioni di parete i quattro modelli danno y^+ medio fra 18 e 20 e C_f entro il 7% dalla correlazione; `SpalartAllmaras` richiede `0/nuTilda` (già presente nel caso) e `realizableKE` gli stessi campi di `kEpsilon`.

Commento. Lo script di shell basta per quattro varianti; per decine, con estrazione di profili e figure, lo schema Python di `genera.py` (Cap. 16) è più solido: controlla gli errori, salva i risultati in JSON e rende le figure riproducibili.

B.16 Esercizio 16 – `forceCoeffs` con l'incidenza

Testo

Scrivere le direzioni `liftDir` e `dragDir` per un profilo a $\alpha = 8°$ ottenuta ruotando la corrente (profilo fisso), e per lo stesso profilo ruotato di 8° in una corrente orizzontale; calcolare poi C_L e C_D dalle forze nel sistema degli assi della mesh $F_x = -0{,}0512$, $F_y = 0{,}4202$ (per unità di apertura, $\rho = 1$, $U = 1$, $c = 1$) nel primo caso, e dire che cosa si otterrebbe usando per errore `dragDir (1 0 0)`.

Impostazione. Portanza e resistenza sono perpendicolare e parallela alla *corrente*: con la corrente ruotata, $\hat{\mathbf{e}}_L = (-\sin\alpha, \cos\alpha)$, $\hat{\mathbf{e}}_D = (\cos\alpha, \sin\alpha)$; con il profilo ruotato e corrente orizzontale, $(0, 1)$ e $(1, 0)$. $C_L = 2L/(\rho U^2 c)$.

Svolgimento. $\sin 8° = 0{,}1392$, $\cos 8° = 0{,}9903$: `liftDir (-0.1392 0.9903 0)`, `dragDir (0.9903 0.1392 0)`; nel secondo caso `(0 1 0)` e `(1 0 0)`. $L = -0{,}1392 \cdot (-0{,}0512) + 0{,}9903 \cdot 0{,}4202 = 0{,}0071 + 0{,}4161 = 0{,}4232$, $D = 0{,}9903 \cdot (-0{,}0512) + 0{,}1392 \cdot 0{,}4202 = -0{,}0507 + 0{,}0585 = 0{,}0078$; $C_L = 2L = 0{,}846$, $C_D = 2D = 0{,}0156$. Con `dragDir (1 0 0)` si otterrebbe $C_D = 2F_x = -0{,}102$.

Risultato

$C_L = 0{,}846$ e $C_D = 0{,}0156$ (i valori del caso `c15_naca0012` a 8°). La forza lungo x è *negativa*: la risultante della pressione sul profilo in incidenza è quasi normale alla corda e punta in avanti (la "spinta" di bordo d'attacco), e la resistenza nasce dalla cancellazione fra due termini di segno opposto e di grandezza sei-sette volte maggiore di lei ($-0{,}0507$ e $+0{,}0585$). Per questo `dragDir` deve essere allineata alla corrente con la precisione dell'angolo stesso: una `dragDir` orizzontale dà una resistenza negativa, e un errore di un solo grado sull'incidenza sposta C_D di $2L\sin 1° \approx 0{,}015$, cioè quanto la resistenza stessa. La stessa cancellazione spiega perché C_D converge più lentamente di C_L con la mesh (Cap. 13).

Commento. È l'errore più frequente nelle polari calcolate ruotando la corrente, ed è silenzioso: i valori sembrano plausibili. Lo script del Cap. 13 ruota `liftDir` e `dragDir` insieme alla velocità e ricalcola le forze in Python come verifica indipendente.

B.17 Esercizio 17 – L'indice di convergenza di griglia

Testo

Tre mesh 2D di 6400, 12 800 e 25 600 celle danno $C_L = 1{,}060,\ 1{,}075,\ 1{,}082$. Calcolare il rapporto di raffinamento, l'ordine osservato, il valore estrapolato e il GCI della mesh fine; dire se le soluzioni sono nell'intervallo asintotico.

Impostazione. $h \propto N^{-1/2}$, $r = \sqrt{2}$ uniforme; le formule (16.1)–(16.2) con $f_1 = 1{,}082$ (fine), $f_2 = 1{,}075$, $f_3 = 1{,}060$; con r uniforme $q = 0$ e $p = \ln|\varepsilon_{32}/\varepsilon_{21}|/\ln r$. Lo script `foamgci.py` fa i conti.

Svolgimento. $\varepsilon_{21} = -0{,}007$, $\varepsilon_{32} = -0{,}015$; $p = \ln(2{,}143)/\ln(1{,}414) = 2{,}20$; $f_{\text{ext}} = (r^p f_1 - f_2)/(r^p - 1) = (2{,}143 \cdot 1{,}082 - 1{,}075)/1{,}143 = 1{,}0881$; $e_a = 0{,}00647$; $\text{GCI}_{\text{fine}} = 1{,}25 \cdot 0{,}00647/1{,}143 = 0{,}0071$; rapporto $\text{GCI}_{23}/(r^p \text{GCI}_{12}) = 1{,}007$.

Risultato

$p = 2{,}2$, $C_{L,\text{ext}} = 1{,}088$, $\text{GCI}_{\text{fine}} = 0{,}71\%$ (cioè $C_L = 1{,}082 \pm 0{,}008$); rapporto $1{,}007 \approx 1$: intervallo asintotico, convergenza monotona.

Commento. Un ordine osservato di 2,2 con schemi del secondo ordine è un buon segno; valori di 4–5 o negativi indicano che le mesh grosse non sono ancora nell'intervallo asintotico o che fra una mesh e l'altra è cambiato altro (y^+, convergenza iterativa). Nel Cap. 13 i tre valori reali del profilo danno risultati diversi per C_L e per C_D, e la resistenza è sempre la grandezza che converge peggio.

B.18 Esercizio 18 – Frequenza da un segnale

Testo

Il segnale di C_L del cilindro mostra 12 periodi completi in 72 unità D/U. Stimare il numero di Strouhal e la sua incertezza contando i periodi; dire quale risoluzione in frequenza ha la trasformata di Fourier della stessa finestra, quanto vale l'incertezza relativa su St, e che cosa cambia raddoppiando la finestra o il passo di campionamento.

Impostazione. $St = n/T_{\text{fin}}$; contando a occhio l'incertezza è di circa mezzo periodo su 12; la FFT ha passo $\Delta f = 1/T_{\text{fin}}$ e, senza interpolazione, l'incertezza sulla posizione del picco è $\pm\Delta f/2$; il passo di campionamento fissa solo la frequenza massima (Nyquist).

Svolgimento. $St = 12/72 = 0{,}1667$; $\pm 0{,}5/72 = \pm 0{,}007$ (4%). $\Delta f = 1/72 = 0{,}0139\ D/U$: $\Delta f/St = 8\%$, $\pm 4\%$ senza interpolazione; con l'interpolazione parabolica del picco (Cap. 14) $\pm 0{,}5\%$. Raddoppiare la finestra dimezza Δf; dimezzare il passo di campionamento (già 0,005, cioè 1200 campioni per periodo) non cambia nulla.

Risultato

$St = 0{,}167 \pm 0{,}007$ contando; $\pm 0{,}007$ anche con la FFT grezza, $\pm 0{,}001$ con il picco interpolato; la precisione viene dalla *durata* del segnale, non dalla sua densità.

Commento. Un calcolo transitorio costoso va pianificato per la durata necessaria alla precisione in frequenza richiesta: per St all'1% senza interpolazione servono 100 periodi, con interpolazione una decina.

B.19 Esercizio 19 – Quanti processori

Testo

Un caso di 2×10^6 celle impiega 10 ore su un processore; la frazione del tempo non parallelizzabile (lettura, scrittura, assemblaggio seriale) è il 5%. Stimare con la legge di Amdahl il tempo su 2, 4, 8, 16 processori; dire quanti processori conviene usare sapendo che l'efficienza crolla sotto 50 000 celle per processore, e come scrivere `decomposeParDict`.

Impostazione. $S(N) = 1/(s + (1-s)/N)$ con $s = 0{,}05$; il vincolo di celle per processore dà $N \leq 40$.

Svolgimento. $S = 1{,}90,\ 3{,}48,\ 5{,}93,\ 9{,}14$: tempi 5,3, 2,9, 1,7, 1,1 ore. A 16 processori l'efficienza è 57%; con 125 000 celle per processore si è ancora sopra la soglia. `decomposeParDict`: `numberOfSubdomains 16; decomposer scotch; distributor ptscotch;`.

Risultato

16 processori riducono le 10 ore a poco più di 1; oltre, il guadagno è marginale ($S(32) = 12{,}5$) e la frazione seriale domina. In pratica la frazione seriale di OpenFOAM è minore del 5% se si evita la scrittura frequente dei campi, e il limite viene dalle comunicazioni di GAMG.

Commento. La legge di Amdahl è ottimistica (ignora le comunicazioni) e pessimistica (ignora che su più processori i dati stanno nelle cache): la misura su 2, 4, 8 processori di un caso rappresentativo vale più di ogni stima.

B.20 Esercizio 20 – Dal problema al caso

Testo

Si deve calcolare la polare di un profilo di elica (NACA 4412) a $Re_c = 3 \times 10^5$, $Ma < 0{,}3$, per incidenze da $-4°$ a $12°$, con precisione del 5% su C_L e del 10% su C_D, confrontando con i dati di galleria. Compilare la lista di controllo del Cap. 16: modello, dominio, mesh, condizioni al contorno, schemi, algoritmo, *function object*, verifica e validazione.

Impostazione. Il numero di Reynolds è basso per un calcolo completamente turbolento: a 3×10^5 la transizione avviene a metà corda e influenza C_D del 30–50%; un modello di transizione (`kOmegaSSTLM`) è necessario per la precisione richiesta su C_D, oppure si impone la transizione dove l'esperimento la misura.

Svolgimento. *Modello*: `incompressibleFluid`, RAS `kOmegaSSTLM` (o `kOmegaSST` con `nutLowReWallFunction` se si accetta l'errore su C_D), mesh risolta ($y^+ < 1$) perché la transizione richiede lo strato limite risolto. *Dominio*: mesh a C con `nacamesh.py` (il generatore accetta la sigla 4412), $R = 20c$, `freestream`. *Mesh*: prima cella da $y^+ = 1$ con C_f a $Re_c = 3 \times 10^5$ ($u_\tau/U \approx 0{,}05$: $y_1 \approx 3 \times 10^{-5} c$), 100–150 celle per lato, tre livelli per il GCI a un'incidenza. *Condizioni*: `freestreamVelocity`/`freestreamPressure`, k e ω da $I = 0{,}1\%$ (galleria a bassa turbolenza) e $\nu_t/\nu \approx 1$, `ReThetat` e `gammaInt` d'ingresso secondo il modello. *Schemi*: `bounded Gauss linearUpwind grad(U)`, `limitedLinear 1` per gli scalari, `Gauss linear corrected`. *Algoritmo*: SIMPLE con `potentialFoam`, avvio `upwind` e rilassamento 0,2/0,5 (Cap. 13), poi 0,3/0,7; 5000 iterazioni con `residualControl` e controllo di C_L, C_D sulle ultime 500. *Function object*: `forceCoeffs` con le direzioni ruotate, `yPlus`, `wallShearStress` (posizione della transizione dal C_f), `residuals`. *Verifica*: GCI su C_L e C_D a 4°; confronto `upwind`/`linearUpwind`; forze dalla *function object* e da `foamcase.py`. *Validazione*: polare contro la galleria con l'incertezza sperimentale; posizione della transizione contro le misure; oltre 10° il profilo si avvicina allo stallo e un calcolo stazionario RANS può non convergere: si dichiara il limite.

Risultato

Un piano di lavoro di una decina di calcoli 2D (polare a 9 incidenze, tre mesh, due schemi) di qualche minuto ciascuno, con il risultato espresso come $C_L \pm$ GCI e $C_D \pm$ GCI e il confronto con l'esperimento; e la dichiarazione esplicita che oltre lo stallo il modello non è affidabile.

Commento. La parte difficile non è lanciare i calcoli ma scrivere, prima, che cosa si vuole sapere e con quale precisione: è ciò che decide il modello di transizione (che un calcolo "standard" avrebbe ignorato) e la mesh risolta a parete, e che permette di dire, alla fine, se il risultato è buono.

Bibliografia

Testo di riferimento del modulo

[1] F. Moukalled, L. Mangani, M. Darwish, *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM® and Matlab®*, Fluid Mechanics and Its Applications vol. 113, Springer, Cham, 2016. Il testo di riferimento del modulo: la Parte I (fondamenti: Capp. 1–6) corrisponde ai nostri Capp. 2–3, la Parte II (discretizzazione: Capp. 7–13) ai Capp. 4–7, la Parte III (algoritmi: Capp. 14–17) ai Capp. 8–9; ogni capitolo del testo si chiude con la lettura del codice OpenFOAM corrispondente, che qui è stata ripresa e aggiornata alla versione 12.

OpenFOAM

[2] The OpenFOAM Foundation, *OpenFOAM v12 User Guide*, 2024, `https://doc.cfd.direct/openfoam/user-guide-v12/`. Il manuale ufficiale: installazione, struttura dei casi, `blockMesh`, `snappyHexMesh`, dizionari, *function object*, post-processing.

[3] The OpenFOAM Foundation, *OpenFOAM-12 source code*, `https://github.com/OpenFOAM/OpenFOAM-12`; documentazione del codice (Doxygen) `https://cpp.openfoam.org/v12/`.

[4] ESI Group, *OpenFOAM v2406 User Guide*, `https://www.openfoam.com/documentation/`. L'altra linea di sviluppo; la Tabella A.5 dà le corrispondenze.

[5] C. J. Greenshields, H. G. Weller, *Notes on Computational Fluid Dynamics: General Principles*, CFD Direct, Reading, 2022, `https://doc.cfd.direct/notes/cfd-general-principles/`. Dagli autori di OpenFOAM: la discretizzazione e gli algoritmi come sono implementati nel codice, con la notazione dei dizionari.

[6] H. Jasak, *Error Analysis and Estimation for the Finite Volume Method with Applications to Fluid Flows*, tesi di dottorato, Imperial College, London, 1996. L'origine di OpenFOAM: la correzione non ortogonale, la skewness, lo schema `corrected` e il metodo dei volumi finiti su mesh poliedriche.

[7] H. G. Weller, G. Tabor, H. Jasak, C. Fureby, *A tensorial approach to computational continuum mechanics using object-oriented techniques*, Computers in Physics **12**(6) (1998) 620–631. L'articolo che presenta FOAM e la sintassi `fvm::/fvc::`.

Metodi numerici e volumi finiti

[8] J. H. Ferziger, M. Perić, R. L. Street, *Computational Methods for Fluid Dynamics*, 4ª ed., Springer, Cham, 2020. Volumi finiti, SIMPLE e varianti, griglie collocate, turbolenza: il complemento classico al testo di riferimento.

[9] H. K. Versteeg, W. Malalasekera, *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*, 2ª ed., Pearson, Harlow, 2007. L'introduzione più accessibile a schemi convettivi, TVD e SIMPLE.

[10] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere, Washington, 1980. L'origine di SIMPLE, del rilassamento e delle regole di Patankar per i coefficienti.

[11] C. M. Rhie, W. L. Chow, *Numerical study of the turbulent flow past an airfoil with trailing edge separation*, AIAA Journal **21**(11) (1983) 1525–1532. L'interpolazione di Rhie–Chow (Cap. 8).

[12] R. I. Issa, *Solution of the implicitly discretised fluid flow equations by operator-splitting*, Journal of Computational Physics **62** (1986) 40–65. PISO.

[13] J. P. Van Doormaal, G. D. Raithby, *Enhancements of the SIMPLE method for predicting incompressible fluid flows*, Numerical Heat Transfer **7** (1984) 147–163. SIMPLEC.

[14] P. K. Sweby, *High resolution schemes using flux limiters for hyperbolic conservation laws*, SIAM Journal on Numerical Analysis **21**(5) (1984) 995–1011. Il diagramma di Sweby e la condizione TVD (Cap. 5).

[15] B. P. Leonard, *The ULTIMATE conservative difference scheme applied to unsteady one-dimensional advection*, Computer Methods in Applied Mechanics and Engineering **88** (1991) 17–74. Il diagramma NVD e lo schema QUICK.

[16] A. Kurganov, E. Tadmor, *New high-resolution central schemes for nonlinear conservation laws and convection-diffusion equations*, Journal of Computational Physics **160** (2000) 241–282. Il flusso di `shockFluid` (Cap. 10).

[17] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*, Cambridge University Press, Cambridge, 2002. Il problema di Riemann, i limitatori e la soluzione esatta del tubo d'urto usata nel Cap. 10.

[18] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2ª ed., SIAM, Philadelphia, 2003. Gradiente coniugato, BiCGStab, precondizionatori, multigriglia (Cap. 7).

Turbolenza

[19] D. C. Wilcox, *Turbulence Modeling for CFD*, 3ª ed., DCW Industries, La Cañada, 2006. I modelli RANS, le funzioni di parete, la legge di parete.

[20] F. R. Menter, *Two-equation eddy-viscosity turbulence models for engineering applications*, AIAA Journal **32**(8) (1994) 1598–1605; F. R. Menter, M. Kuntz, R. Langtry, *Ten years of industrial experience with the SST turbulence model*, in *Turbulence, Heat and Mass Transfer 4*, Begell House, 2003, 625–632. Il modello k–ω SST nella forma implementata in OpenFOAM.

[21] P. R. Spalart, S. R. Allmaras, *A one-equation turbulence model for aerodynamic flows*, La Recherche Aérospatiale **1** (1994) 5–21 (AIAA Paper 92-0439).

[22] B. E. Launder, D. B. Spalding, *The numerical computation of turbulent flows*, Computer Methods in Applied Mechanics and Engineering **3** (1974) 269–289. Il modello k–ε standard e le funzioni di parete.

[23] D. B. Spalding, *A single formula for the law of the wall*, Journal of Applied Mechanics **28** (1961) 455–458. La legge usata da `nutUSpaldingWallFunction`.

[24] C. L. Rumsey (a cura di), *NASA Langley Turbulence Modeling Resource*, `https://turbmodels.larc.nasa.gov/`. Le definizioni dei modelli come implementati nei codici di riferimento, i casi di verifica (lastra piana, NACA 0012 con i risultati CFL3D e FUN3D usati nel Cap. 13) e le griglie.

Dati sperimentali e di riferimento

[25] U. Ghia, K. N. Ghia, C. T. Shin, *High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method*, Journal of Computational Physics **48** (1982) 387–411. La cavità (Cap. 8).

[26] D. M. Driver, H. L. Seegmiller, *Features of a reattaching turbulent shear layer in divergent channel flow*, AIAA Journal **23**(2) (1985) 163–171. Il gradino a valle (Cap. 9).

[27] Flexcompute, *Flow360 Documentation: NACA 0012 Low Speed Airfoil*, `https://docs.flexcompute.com/projects/flow360/en/latest/caseStudies/NACA0012/NACA0012.html`. Le tabelle riportano i risultati dei codici della NASA TMR e la ripartizione della resistenza in attrito e pressione sulla griglia 897×257.

[28] C. L. Ladson, *Effects of independent variation of Mach and Reynolds numbers on the low-speed aerodynamic characteristics of the NACA 0012 airfoil section*, NASA TM 4074, 1988. La polare del NACA 0012 (Cap. 13).

[29] N. Gregory, C. L. O'Reilly, *Low-speed aerodynamic characteristics of NACA 0012 aerofoil section, including the effects of upper-surface roughness simulating hoar frost*, NPL Aero Report 1308, 1970 (ARC R&M 3726, 1973). Le distribuzioni di pressione sul NACA 0012.

[30] I. H. Abbott, A. E. von Doenhoff, *Theory of Wing Sections*, Dover, New York, 1959. Le coordinate dei profili NACA e le polari classiche.

[31] C. H. K. Williamson, *Defining a universal and continuous Strouhal–Reynolds number relationship for the laminar vortex shedding of a circular cylinder*, Physics of Fluids **31**(10) (1988) 2742–2744; A. Roshko, *On the development of turbulent wakes from vortex streets*, NACA Report 1191, 1954. Le correlazioni del numero di Strouhal (Cap. 14).

[32] G. A. Sod, *A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws*, Journal of Computational Physics **27** (1978) 1–31. Il tubo d'urto.

[33] F. M. White, *Viscous Fluid Flow*, 3ª ed., McGraw-Hill, New York, 2006. Le correlazioni di attrito della lastra piana (Blasius, Schultz–Grunow, White), il condotto di Poiseuille e di Womersley.

[34] H. Schlichting, K. Gersten, *Boundary-Layer Theory*, 9ª ed., Springer, Berlin, 2017.

[35] J. D. Anderson, *Modern Compressible Flow with Historical Perspective*, 3ª ed., McGraw-Hill, New York, 2003. Le relazioni dell'urto obliquo e le tabelle (Cap. 10).

Verifica e validazione

[36] P. J. Roache, *Verification and Validation in Computational Science and Engineering*, Hermosa, Albuquerque, 1998; P. J. Roache, *Perspective: a method for uniform reporting of grid refinement studies*, Journal of Fluids Engineering **116** (1994) 405–413. Il GCI.

[37] I. B. Celik, U. Ghia, P. J. Roache, C. J. Freitas, H. Coleman, P. E. Raad, *Procedure for estimation and reporting of uncertainty due to discretization in CFD applications*, Journal of Fluids Engineering **130** (2008) 078001. La procedura implementata in `foamgci.py`.

[38] ASME, *Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer*, ASME V&V 20-2009 (R2021).

[39] M. Casey, T. Wintergerste (a cura di), *ERCOFTAC Best Practice Guidelines for Industrial Computational Fluid Dynamics*, ERCOFTAC, 2000. Le buone pratiche del Cap. 16.

I moduli precedenti del corso

[40] B. Raucci, *Python per la fluidodinamica numerica* (Modulo 0); *Problemi ai valori iniziali in fluidodinamica* (Modulo 1); *Flussi non viscosi* (Modulo 2); *Flussi viscosi* (Modulo 3); *Soluzione numerica delle equazioni di Navier–Stokes* (Modulo 4), `https://braucci.github.io/CFD/`. I prerequisiti di questa monografia: Python e NumPy, l'integrazione nel tempo, il problema di Riemann e la formazione dell'urto, la linea portante e lo strato limite, le differenze upwind e la cavità in variabili primitive.